
Modern C++ for Python Programmers

From Absolute Beginner to Systems, Graphics, and Game Development

2026

Contents

How to Use This Book	3
Table of Contents	3
Part I – Foundations	5
Chapter 1: The Compilation Model and Your First Program	7
What Does “Running a Program” Actually Mean?	7
How Python Runs Your Code	7
How C++ Runs Your Code	9
Stage 1: The Preprocessor in Detail	9
Stage 2: The Compiler in Detail	10
Stage 3: The Linker in Detail	10
The Core Difference	10
Your First C++ Program	10
#include <iostream>	11
int main()	11
std::cout << "Hello, world!\n";	12
return 0;	12
Compiling: What the Flags Mean	12
Two Build Modes You Will Use	13
What Is in the Compiled Binary?	13
Multi-File Programs	14
Reading Compiler Errors	14
Missing Semicolon	14
Missing #include	15
Undefined Reference (Linker Error)	15
Type Mismatch	15
Common Mistakes in This Chapter	16
Mistake 1: Forgetting the semicolon	16
Mistake 2: Writing cout without std::	16
Mistake 3: Confusing #include <...> with #include "...".	16
Exercises	17
Chapter 2: Variables, Types, and the Static Type System	19
What IS a Variable?	19
How Integers Are Stored: Binary and Two’s Complement	20
How Negative Numbers Work: Two’s Complement	21
The Fundamental Types	21
bool	21
char	22

Integer Types	22
float and double	22
std::string	23
Fixed-Width Integer Types	23
Declaring and Initializing Variables	24
Never Leave a Variable Uninitialized	24
The Four Initialization Forms	24
Memory Layout of Variables	25
auto – Type Deduction	26
const and constexpr	26
const – Constant After Initialization	26
constexpr – Must Be Computed at Compile Time	27
Type Conversion	27
Safe Widening (Automatic)	27
Narrowing (Dangerous Without Braces)	27
Explicit Conversion with static_cast	28
Checking Sizes with sizeof	28
Common Mistakes in This Chapter	28
Mistake 1: Uninitialized Variable	28
Mistake 2: Signed Integer Overflow	29
Mistake 3: Integer Division When Expecting Decimal	29
Mistake 4: Comparing Floats with ==	29
Exercises	30
Chapter 3: Operators and Expressions	33
Arithmetic Operators	33
The Classic Division Trap	33
Modulo With Negative Numbers	34
Increment and Decrement Operators	34
Compound Assignment	35
Comparison Operators	35
The Floating-Point Comparison Problem	35
Logical Operators	36
Short-Circuit Evaluation	36
Bitwise Operators	37
Practical Use: Boolean Flags Packed in an Integer	38
Operator Precedence	38
The Ternary Operator	39
Common Mistakes in This Chapter	39
Mistake 1: Assignment Instead of Comparison	39
Mistake 2: Integer Division Surprise	40
Mistake 3: == on Floats	40
Mistake 4: Bitwise vs Logical Operators	40
Exercises	40

Chapter 4: Control Flow: Branching and Loops	43
if/else if/else	43
Always Use Curly Braces – A Real Cautionary Tale	43
if with Initializer (C++17)	44
switch	44
Fallthrough: The break Requirement	45
while Loop	45
do-while Loop	46
for Loop: C-Style	46
Range-Based for Loop (C++11)	48
The Reference vs Copy Decision	48
break and continue	49
Loop Execution Visualization	50
Common Mistakes in This Chapter	50
Mistake 1: Off-By-One Error	50
Mistake 2: Forgetting break in switch	51
Mistake 3: Modifying a Container While Iterating Over It	51
Mistake 4: Using = Instead of == in a Condition	51
Exercises	52
Chapter 5: Functions, Overloading, and Declarations vs Definitions	55
What Happens When You Call a Function	55
Function Syntax	56
Declarations vs Definitions	57
The Three Solutions	57
Pass by Value: What Is Copied and Why It Matters	58
Default Parameters	59
Function Overloading	60
What Overloading Is NOT	60
Compiler Messages for Function Errors	60
No Matching Function	61
Ambiguous Overload	61
Wrong Number of Arguments	61
Common Mistakes in This Chapter	62
Mistake 1: Expecting Pass-by-Value to Modify the Original	62
Mistake 2: Missing Return Statement	62
Mistake 3: Defining a Function Twice	62
Mistake 4: Forgetting to Compile the File Containing the Definition	63
Exercises	63
Part II – Core C++ (the part that is not like Python)	67
Chapter 6: References – Aliases for Variables	69
The Problem References Solve	69
What Is a Reference?	69
Pass by Reference	70
Pass by const Reference	71
The Decision Table for Parameter Style	71

References Cannot Be Null	72
Returning References	72
Common Mistakes in This Chapter	72
Mistake 1: Expecting pass-by-reference from a non-reference parameter	72
Mistake 2: Forgetting <code>const</code> on read-only reference parameters	73
Mistake 3: Returning a reference to a local variable	73
Exercises	73
Chapter 7: Pointers and Memory Addresses	75
The Address of a Variable	75
Modifying Through a Pointer	76
<code>nullptr</code> : The Null Pointer	76
Pointer Arithmetic	77
Pointers vs Python References	77
The <code>-></code> Operator	78
<code>const</code> and Pointers	78
Common Mistakes in This Chapter	79
Mistake 1: Dereferencing a Null or Uninitialized Pointer	79
Mistake 2: Using a Pointer After the Pointed-To Variable is Destroyed	79
Mistake 3: Confusing <code>&</code> as Address-Of vs <code>&</code> as Reference Type	80
Exercises	80
Chapter 8: The Stack and the Heap	83
Two Regions of Memory	83
The Stack in Detail	83
The Heap in Detail	85
Heap Arrays	85
Why Python Uses the Heap for Everything	86
Memory Leaks	86
Stack vs Heap: Which to Use	87
Common Mistakes in This Chapter	87
Mistake 1: <code>delete</code> Instead of <code>delete[]</code> for Arrays	87
Mistake 2: Double Delete	87
Mistake 3: Using After Delete (Use-After-Free)	88
Exercises	88
Chapter 9: <code>const</code> Correctness	91
Why <code>const</code> Matters	91
<code>const</code> Variables	91
<code>const</code> Function Parameters	91
<code>const</code> Member Functions	92
<code>constexpr</code> – Compile-Time Constants	93
Cascading <code>const</code>	93
Common Mistakes in This Chapter	94
Mistake 1: Omitting <code>const</code> on Member Functions That Should Be <code>const</code>	94
Mistake 2: Trying to Modify Through a <code>const</code> Reference	94
Exercises	94

Chapter 10: Arrays, <code>std::vector</code>, and <code>std::string</code>	97
C-Style Arrays (Understand, then Avoid)	97
Critical Weakness: No Bounds Checking	97
Size Is Not Carried With the Array	98
<code>std::vector</code> – The Right Way to Do Dynamic Arrays	98
What's Inside <code>std::vector</code>	98
Key Operations	99
Iterating Over a Vector	99
2D Vectors	99
<code>std::string</code> – The Right Way to Handle Text	100
Key Operations	100
String Comparison	101
Raw C-Style Strings	101
<code>std::array</code> – Fixed-Size Array With Safety	101
Common Mistakes in This Chapter	101
Mistake 1: Off-By-One / Out-of-Bounds Access	101
Mistake 2: Modifying a Vector While Iterating With Indices	102
Mistake 3: Using <code>+</code> to Concatenate Non-String Literals	102
Mistake 4: Accessing <code>.c_str()</code> Pointer After Modifying the String	102
Exercises	103
Chapter 11: Scope, Lifetime, and Organizing Code into Files	105
Scope	105
Scope in Loops and Conditionals	105
Lifetime	105
Organizing Code: The Header/Source Split	106
Why Headers Exist	106
The Compilation Model With Multiple Files	107
What Goes In Headers vs Source Files	107
Include Guards (Alternative to <code>#pragma once</code>)	108
Namespaces	108
Writing Your Own Namespace	109
Static Local Variables	109
Common Mistakes in This Chapter	110
Mistake 1: Defining a Non-Inline Function in a Header	110
Mistake 2: Missing <code>#pragma once</code> (or Include Guard)	110
Mistake 3: using <code>namespace std;</code> in a Header	110
Exercises	111
Part III – Ownership and Memory Management	115
Chapter 12: RAI – The Core Idea That Replaces Garbage Collection	117
The Problem: Resources Need Cleanup	117
What RAI Is	117
The Destructor	118
RAI for File Handling	119
RAI for Mutex Locks	120
Python's Equivalent: Context Managers	120

The Lifetime Guarantee Visualized	120
Common Mistakes in This Chapter	121
Mistake 1: Not Writing a Destructor for Resource-Owning Classes	121
Mistake 2: Destructors That Throw	121
Exercises	121
Chapter 13: Dynamic Allocation: new, delete, and Why You Avoid Them	123
new and delete Revisited	123
What new Actually Does	123
What delete Actually Does	123
The Problems With Raw new/delete	124
Problem 1: Exception Safety	124
Problem 2: Ownership Ambiguity	124
Problem 3: Paired Delete is Fragile	124
When Raw new/delete Is Acceptable	124
std::bad_alloc – Out of Memory	125
Common Mistakes in This Chapter	125
Mistake 1: delete on Stack Memory	125
Mistake 2: Accessing Memory After delete	125
Exercises	126
Chapter 14: Smart Pointers: unique_ptr, shared_ptr, weak_ptr	127
The Core Idea	127
std::unique_ptr – Exclusive Ownership	127
Unique Ownership Enforced at Compile Time	128
Unique Pointer to a Custom Class	128
unique_ptr as a Function Parameter	128
std::shared_ptr – Shared Ownership	129
When to Use shared_ptr	130
std::weak_ptr – Non-Owning Observer	130
Choosing the Right Smart Pointer	131
Common Mistakes in This Chapter	131
Mistake 1: Creating a shared_ptr From a Raw Pointer Twice	131
Mistake 2: Calling .get() and Storing the Result	131
Mistake 3: Passing unique_ptr by Value When Borrowing	132
Exercises	132
Chapter 15: Move Semantics, lvalues, and rvalues	135
The Performance Problem With Copies	135
lvalues and rvalues	135
The Move Operation	136
std::move – Casting to rvalue	136
Move Semantics and Returned Values	137
Move Constructor and Move Assignment	137
std::string and std::vector Move in Action	138
When C++ Moves Automatically	138
Common Mistakes in This Chapter	139
Mistake 1: Using a Moved-From Object	139

Mistake 2: <code>return std::move(local)</code> Disabling NRVO	139
Exercises	139
Chapter 16: The Rule of 0, 3, and 5	141
The Problem: Special Member Functions	141
The Rule of Zero	141
The Rule of Three	142
The Rule of Five	142
<code>= delete</code> and <code>= default</code>	143
Compiler-Generated Functions Are Suppressed When You Define Others	144
The Practical Takeaway	144
Common Mistakes in This Chapter	145
Mistake 1: Shallow Copy of Owing Pointer (Forgetting Rule of Three)	145
Mistake 2: Forgetting the Self-Assignment Guard	145
Exercises	145
Part IV – Object-Oriented Programming	147
Chapter 17: Classes, Objects, and Encapsulation	149
Python Classes vs C++ Classes	149
Defining a Class: Full Example	149
<code>struct</code> vs <code>class</code>	151
Memory Layout of a Class	151
<code>this</code> Pointer	152
Static Members	152
Common Mistakes in This Chapter	153
Mistake 1: Forgetting the <code>::</code> Scope Resolution in the <code>.cpp</code> File	153
Mistake 2: Calling a Non-const Method on a Const Object	153
Mistake 3: Forgetting to Define Static Members	154
Exercises	154
Chapter 18: Constructors, Destructors, and Initialization	155
The Member Initializer List	155
Initialization Order	156
Delegating Constructors (C++11)	156
Constructor Kinds	156
Default Constructor	156
Explicit Constructor (Prevent Implicit Conversion)	157
Destructor Revisited: The Full Rules	157
In-Class Member Initializers (C++11)	158
Common Mistakes in This Chapter	158
Mistake 1: Initializer List Order Different From Declaration Order	158
Mistake 2: Missing <code>explicit</code> on Converting Constructor	159
Exercises	159
Chapter 19: Inheritance and Composition	161
Two Kinds of Reuse	161

Inheritance Syntax	161
The <code>public</code> Inheritance Keyword	162
Constructors in Derived Classes	162
Memory Layout With Inheritance	163
Slicing: The Object-Oriented Trap	163
Composition vs Inheritance	164
Common Mistakes in This Chapter	164
Mistake 1: Forgetting to Call the Base Constructor	164
Mistake 2: Object Slicing	165
Exercises	165
Chapter 20: Virtual Functions and Polymorphism	167
The Problem Without Virtual	167
<code>virtual</code> Enables Dynamic Dispatch	167
How Virtual Dispatch Works: The vtable	168
<code>override</code> and <code>final</code> (C++11)	169
Virtual Destructor: Why It Is Required	169
Polymorphism With <code>unique_ptr</code>	170
Python Polymorphism vs C++ Polymorphism	170
Common Mistakes in This Chapter	171
Mistake 1: Non-Virtual Base Destructor	171
Mistake 2: Forgetting <code>override</code>	171
Mistake 3: Calling Virtual Functions in Constructors/Destructors	171
Exercises	172
Chapter 21: Abstract Classes and Interfaces	173
Pure Virtual Functions	173
Concrete Derived Classes	173
Interfaces in C++	174
Partial Implementations (Partially Abstract)	175
Checking the Type at Runtime: <code>dynamic_cast</code>	175
Common Mistakes in This Chapter	176
Mistake 1: Instantiating an Abstract Class	176
Mistake 2: Missing <code>override</code> in Concrete Class	176
Exercises	176
Chapter 22: Operator Overloading	179
What Operator Overloading Is	179
Which Operators to Overload	179
Member vs Non-Member Operators	181
The Spaceship Operator (C++20)	181
<code>operator()</code> – Making Objects Callable	181
Rules and Guidelines for Operator Overloading	182
Common Mistakes in This Chapter	182
Mistake 1: Returning <code>*this</code> by Value Instead of Reference From <code>+=</code>	182
Mistake 2: Missing the Const Overload for <code>operator[]</code>	182
Exercises	183

Part V – Generic Programming	185
Chapter 23: Function and Class Templates	187
The Problem Without Templates	187
Function Templates	187
Template Type Deduction	188
Multiple Template Parameters	188
How Templates Are Compiled	188
Why Templates Must Live in Headers	189
Class Templates	189
Non-Type Template Parameters	190
Template Functions in the Standard Library	190
Common Mistakes in This Chapter	191
Mistake 1: Putting Template Definitions in a .cpp File	191
Mistake 2: Mixed Types Without Explicit Instantiation	191
Mistake 3: Confusing Template Errors With Logic Errors	191
Exercises	192
Chapter 24: Template Specialization and Variadic Templates	195
Template Specialization	195
Partial Specialization	195
if constexpr – Compile-Time Branching	196
Type Traits	197
Variadic Templates	197
Recursive Variadic Template (Classic Style)	198
Fold Expressions (C++17)	198
Common Mistakes in This Chapter	199
Mistake 1: Full Specialization After the Template Is Already Instantiated	199
Mistake 2: Forgetting template <> for Full Specialization	199
Exercises	199
Chapter 25: Concepts (C++20) – Compile-Time Duck Typing	201
The Problem With Unconstrained Templates	201
Defining and Using Concepts	201
Standard Library Concepts (<concepts>)	202
Writing Your Own Concepts	203
Abbreviated Function Templates (C++20)	204
Concepts vs Runtime Type Checks	204
Common Mistakes in This Chapter	204
Mistake 1: Using requires in the Wrong Place	204
Mistake 2: Concept Is Too Strict or Too Loose	205
Exercises	205
Chapter 26: An Introduction to Template Metaprogramming	207
What Is Template Metaprogramming?	207
Compile-Time Recursion	207
std::enable_if – Conditional Compilation (Old Style)	208
Type Lists – Compile-Time Lists of Types	208

Compile-Time Lookup Tables	209
<code>std::tuple</code> – Compile-Time Heterogeneous Collections	209
<code>std::conditional</code> – Type Selection at Compile Time	210
The Difference Between <code>TMP</code> and <code>constexpr</code>	210
Common Mistakes in This Chapter	210
Mistake 1: Trying to Use a Runtime Value as a Template Argument	210
Mistake 2: Recursive <code>TMP</code> Instead of <code>constexpr</code>	211
Exercises	211
Part VI – The Standard Library	213
Chapter 27: Containers: <code>vector</code>, <code>map</code>, <code>set</code>, <code>array</code>, and Friends	215
The Container Taxonomy	215
<code>std::vector<T></code> – Your Default Container	215
Reservation and Capacity Management	216
<code>emplace_back</code> vs <code>push_back</code>	216
<code>std::deque<T></code> – Fast at Both Ends	216
<code>std::map<K, V></code> – Sorted Key-Value Store	217
Key Difference: <code>std::map</code> vs Python <code>dict</code>	217
Safe Lookup: <code>at()</code> vs <code>operator[]</code>	217
Iterating Over a Map	218
Inserting and Checking Simultaneously	218
<code>std::unordered_map<K, V></code> – Hash Map	219
<code>std::set<K></code> – Sorted Unique Keys	219
Choosing the Right Container	219
Common Mistakes in This Chapter	220
Mistake 1: <code>map[key]</code> to Check Existence	220
Mistake 2: Invalidating Iterators by Modifying the Container	220
Mistake 3: Linear Search on an Unsorted <code>vector</code> When a <code>set</code> Would Be $O(\log n)$	220
Exercises	221
Chapter 28: Iterators	223
What Is an Iterator?	223
Iterator Categories	223
The Iterator-Pair Pattern	224
<code>begin</code> / <code>end</code> Free Functions	224
Reverse Iterators	225
Insert Iterators	225
Stream Iterators	225
Common Mistakes in This Chapter	226
Mistake 1: Dereferencing <code>end()</code>	226
Mistake 2: Iterator Invalidation	226
Exercises	226
Chapter 29: Algorithms: <code>sort</code>, <code>find</code>, <code>transform</code>, and the Rest	229
Why Algorithms	229
The Most Useful Algorithms	229
Searching	229

Sorting and Ordering	230
Transforming	231
The Erase-Remove Idiom	231
Numeric Algorithms (<numeric>)	231
Common Mistakes in This Chapter	232
Mistake 1: Forgetting That <code>std::remove</code> Does Not Actually Remove	232
Mistake 2: Sorting Before a Linear Search (Mismatch of Algorithm)	232
Exercises	232
Chapter 30: Lambdas and Function Objects	235
What Is a Lambda?	235
Lambda Syntax	235
Captures: Accessing the Enclosing Scope	236
Lambda Memory Model	236
Generic Lambdas (C++14)	237
Storing Lambdas: <code>std::function</code>	237
Immediately Invoked Lambdas	237
Common Mistakes in This Chapter	238
Mistake 1: Capturing a Local Variable by Reference After It Goes Out of Scope	238
Mistake 2: <code>[=]</code> Capturing <code>this</code> by Value	238
Exercises	239
Chapter 31: Ranges and Views (C++20)	241
The Problem With Iterator Pairs	241
Ranges: The C++20 Solution	241
Views: Lazy, Composable Transformations	242
Common Views	242
Materializing a View	243
Common Mistakes in This Chapter	243
Mistake 1: Storing a View to a Destroyed Container	243
Exercises	244
Chapter 32: Utility Types: <code>optional</code>, <code>variant</code>, <code>any</code>, <code>tuple</code>	245
<code>std::optional<T></code> – A Value That May Not Exist	245
<code>std::variant<Types...></code> – Type-Safe Union	245
Practical Use: Error Handling	246
<code>std::any</code> – Type-Erased Storage	247
<code>std::tuple</code> Revisited: Structured Bindings and Helpers	247
<code>std::pair<A, B></code> – Two Values	247
Choosing the Right Utility Type	248
Common Mistakes in This Chapter	248
Mistake 1: Accessing Empty <code>optional</code>	248
Mistake 2: <code>std::get</code> with Wrong Type on <code>variant</code>	249
Exercises	249

Chapter 33: auto, Type Deduction, and Structured Bindings	253
Type Deduction: The Full Picture	253
auto Strips Top-Level Qualifiers	253
decltype – Deduce the Type of an Expression	253
decltype(auto) – Perfect Return Type Deduction	254
auto for Function Return Types	254
Structured Bindings (C++17)	255
if and switch With Initializers (C++17)	255
Common Mistakes in This Chapter	256
Mistake 1: auto Dropping const or Reference	256
Mistake 2: decltype(x) Returning Reference to Local	256
Exercises	256
Chapter 34: constexpr and Compile-Time Computation	259
Beyond Simple Constants	259
constexpr Variables	259
constexpr – Must Be Compile-Time (C++20)	260
constexpr – Initialized at Compile Time, Mutable After (C++20)	260
if constexpr – Compile-Time Branch Selection	260
Compile-Time Strings and Algorithms (C++20)	261
static_assert – Compile-Time Assertions	261
Common Mistakes in This Chapter	261
Mistake 1: Calling constexpr With a Runtime Argument and Expecting Compile-Time	261
Mistake 2: constexpr Function That Cannot Actually Be Evaluated at Compile Time	262
Exercises	262
Chapter 35: std::format and Modern String Handling	265
The String Formatting History in C++	265
std::format Basics	265
Format Specifiers	266
std::format vs std::to_string vs std::stringstream	266
std::format for Building Strings	267
std::string_view – Non-Owning String Reference	267
Common Mistakes in This Chapter	268
Mistake 1: Using std::format But Forgetting to #include <format>	268
Mistake 2: string_view Outliving the Source String	268
Exercises	268
Chapter 36: Coroutines and Generators	271
What Is a Coroutine?	271
How Coroutines Work (Conceptually)	271
Generators With co_yield	272
Infinite Sequences	272
co_await – Asynchronous Operations	272
co_return – Returning a Value	273
When to Use Coroutines	273
Common Mistakes in This Chapter	274
Mistake 1: Using a Coroutine Return Value as a Regular Value	274

Mistake 2: Dangling References in Coroutine Locals	274
Exercises	274
Chapter 37: Modules (C++20)	277
The Problem With Headers	277
Writing a Module	277
Module Partitions	278
Modules vs Headers: Side-by-Side	278
Toolchain Support (as of 2025/2026)	278
Common Mistakes in This Chapter	279
Mistake 1: Including Headers Inside a Module That Leaks Macros	279
Mistake 2: Exporting Everything	279
Exercises	279
Part VIII – Performance	281
Chapter 38: Value vs Reference Semantics	283
The Two Semantic Models	283
Why Value Semantics Makes Reasoning Easier	283
When Value Semantics Is Expensive	284
The Slice Problem and Polymorphism	285
Small Object Optimization (SOO)	285
std::span<T> – Non-Owning View Over Contiguous Data	286
Common Mistakes in This Chapter	286
Mistake 1: Passing Large Objects by Value When Only Reading	286
Mistake 2: Returning const Value (Disables Move)	286
Exercises	287
Chapter 39: How Memory Layout Affects Speed	289
The CPU and the Memory Hierarchy	289
Contiguous Arrays vs Linked Structures	289
Array of Structs vs Struct of Arrays	290
False Sharing: The Multi-Core Cache Problem	291
Memory Alignment	291
Branch Prediction	292
[[likely]] and [[unlikely]] (C++20)	293
Common Mistakes in This Chapter	293
Mistake 1: Using std::list When std::vector Would Do	293
Mistake 2: Struct Members Ordered Arbitrarily	293
Exercises	293
Chapter 40: Data-Oriented Design	295
What Is Data-Oriented Design?	295
The OOP Game Entity Problem	295
Entity-Component-System (ECS)	296
Hot/Cold Data Splitting	296
The Cost of Indirection	297
SIMD: Processing Multiple Values at Once	298

Common Mistakes in This Chapter	298
Mistake 1: Premature DOD	298
Mistake 2: Pointer-to-Pointer Collections for “Flexibility”	298
Exercises	299
Chapter 41: Profiling and Flamegraphs	301
Measure First, Optimize Second	301
Compiler Optimization Levels	301
Quick Timing With <code>std::chrono</code>	301
Google Benchmark	302
<code>perf</code> – Linux System Profiler	303
Flamegraphs	303
<code>valgrind --tool=callgrind</code> – Instruction-Level Profiling	304
Address Sanitizer, UBSan, and ThreadSanitizer	304
The Optimization Checklist	304
Common Mistakes in This Chapter	305
Mistake 1: Optimizing Before Profiling	305
Mistake 2: Benchmarking Debug Builds	305
Mistake 3: Letting the Compiler Optimize Away the Benchmark	305
Exercises	305
Part IX – Concurrency	307
Chapter 42: Threads and <code>std::jthread</code>	309
What Is a Thread?	309
<code>std::thread</code> – Creating a Thread	309
<code>std::jthread</code> – The Safe Thread (C++20)	310
Thread Arguments and Return Values	311
Thread Local Storage	311
Common Thread Patterns	311
Parallel For (Manual)	311
Common Mistakes in This Chapter	312
Mistake 1: Destroying a Joined/Unjoined <code>std::thread</code>	312
Mistake 2: Accessing Shared Data Without Synchronization	312
Mistake 3: Capturing Local Variables by Reference in a Detached Thread	313
Exercises	313
Chapter 43: Mutexes, Locks, and Race Conditions	315
What Is a Race Condition?	315
<code>std::mutex</code> – Mutual Exclusion	316
RAII Mutex Locking: <code>std::lock_guard</code> and <code>std::unique_lock</code>	316
<code>std::shared_mutex</code> – Multiple Readers, One Writer	317
Deadlocks	317
Deadlock Prevention	318
Condition Variables – Waiting for a Condition	318
Common Mistakes in This Chapter	319
Mistake 1: Forgetting to Lock Before Checking a Shared Variable	319
Mistake 2: Deadlock From Recursive Locking	319

Mistake 3: Condition Variable Without a Predicate	320
Exercises	320
Chapter 44: Atomics and the C++ Memory Model	323
When Mutexes Are Too Heavy	323
What Operations Are Atomic?	323
std::atomic<bool> – Flags and Stop Signals	324
The C++ Memory Model: Why It Matters	324
The Acquire-Release Pattern (Publish-Subscribe)	325
std::atomic_flag – The Simplest Atomic	325
When to Use Atomic vs Mutex	326
Common Mistakes in This Chapter	326
Mistake 1: Thinking Non-Atomic Operations on std::atomic Are Atomic	326
Mistake 2: Using memory_order_relaxed When Order Matters	326
Mistake 3: Busy-Waiting With Atomics in a Long Wait	327
Exercises	327
Chapter 45: Async, Futures, and Tasks	329
The Problem With Raw Threads for Result Delivery	329
std::async – The Easiest Async Pattern	329
Launch Policies	329
Parallel Async Example	330
std::promise and std::future – Manual Wiring	330
std::shared_future – Multiple Waiters	331
Parallel STL (C++17) – Automatic Parallelism	331
Exception Handling Across Threads	332
Choosing Concurrency Tools	332
Common Mistakes in This Chapter	332
Mistake 1: Forgetting to Call future.get()	332
Mistake 2: Storing std::future in a Container and Not Getting	333
Exercises	333
Part X – Graphics and Game Development	335
Chapter 46: Math for Graphics – Vectors, Matrices, and Quaternions	337
Why Math Matters	337
Vectors	337
Matrices and Transformations	338
The Transform Pipeline	339
Quaternions – Rotations Without Gimbal Lock	340
Common Mistakes in This Chapter	341
Mistake 1: Column-Major vs Row-Major Confusion	341
Mistake 2: Normalizing Before Cross Product	341
Mistake 3: Gimbal Lock With Euler Angles	342
Exercises	342
Chapter 47: How the GPU Works	343
CPU vs GPU: Two Different Machines	343

The Rendering Pipeline	343
GLSL Shader Basics	344
GPU Memory: Where Things Live	345
The CPU-GPU Handoff	345
Common Mistakes in This Chapter	346
Mistake 1: Uploading Data to GPU Every Frame	346
Mistake 2: Reading Back Data From GPU to CPU	346
Exercises	346
Chapter 48: OpenGL Fundamentals	347
What Is OpenGL?	347
Setting Up: GLFW + GLAD	347
Drawing a Triangle: The Full Pipeline	348
Textures	349
Uniforms: Sending Data to Shaders	350
Depth Testing	350
Common Mistakes in This Chapter	350
Mistake 1: Wrong Attribute Pointer Stride	350
Mistake 2: Not Binding VAO Before Drawing	351
Mistake 3: Forgetting to Enable Depth Test	351
Exercises	351
Chapter 49: Vulkan – Explicit GPU Control	353
Why Vulkan Exists	353
Core Vulkan Concepts	353
Vulkan Initialization Sequence	353
The Vulkan Render Loop	354
Vulkan Memory Management	355
OpenGL vs Vulkan: When to Use Each	355
Common Mistakes in This Chapter	356
Mistake 1: Not Enabling Validation Layers in Debug	356
Mistake 2: Wrong Semaphore Usage	356
Exercises	356
Chapter 50: Game Loop, ECS, and Engine Design	357
The Game Loop	357
Fixed Timestep Loop (The Standard Solution)	357
Entity-Component System (ECS)	358
Simple ECS Implementation	359
Scene Graph vs ECS	360
Resource Management in a Game Engine	360
Common Mistakes in This Chapter	361
Mistake 1: Variable Timestep for Physics	361
Mistake 2: One Component = One Allocation	361
Exercises	362
Part XI – Systems Programming	365

Chapter 51: The Machine – Registers, Memory, and Syscalls	367
The Execution Model: What the CPU Actually Does	367
From C++ to Machine Code	367
Virtual Memory: How the OS Lies About Memory	368
System Calls: Asking the OS to Do Things	369
Calling Conventions and ABI	369
Inspecting the Stack Frame	370
mmap – Direct Memory Mapping	370
Common Mistakes in This Chapter	371
Mistake 1: Reading Uninitialized Stack Variables	371
Mistake 2: Stack Overflow From Large Stack Allocations	371
Exercises	371
Chapter 52: Working With OS and Linux APIs	373
POSIX: The Portable Interface	373
Processes	373
Signals	374
File Descriptors and POSIX I/O	374
Pipes: Inter-Process Communication	375
epoll – Scalable I/O Multiplexing	375
Common Mistakes in This Chapter	376
Mistake 1: Not Handling EINTR	376
Mistake 2: Forgetting O_CLOEXEC on Forked Programs	376
Exercises	377
Chapter 53: Networking From the Ground Up	379
The Network Stack	379
TCP Sockets: A Server	379
TCP Sockets: A Client	380
UDP Sockets	380
Non-Blocking I/O and epoll	381
Address Resolution: getaddrinfo	381
Common Mistakes in This Chapter	382
Mistake 1: Assuming recv Returns the Full Message	382
Mistake 2: Not Converting Byte Order	382
Exercises	383
Chapter 54: Where C++ Meets C, eBPF, and Go	385
Calling C From C++	385
Calling C++ From Python (ctypes / cffi)	385
Calling C++ From Go (cgo)	386
eBPF: Running C++ Code in the Linux Kernel	386
std::bit_cast and Bitwise Reinterpretation (C++20)	388
Inline Assembly	388
Common Mistakes in This Chapter	389
Mistake 1: Name Mangling Mismatch	389
Mistake 2: Type Mismatch Across FFI Boundary	389
Exercises	389

Appendices	391
Appendix A: Setting Up Your Toolchain	393
Linux (Ubuntu/Debian)	393
macOS	394
Windows	394
Your First CMake Project	395
IDE Setup	396
VS Code	396
CLion (JetBrains)	396
Appendix B: Compiler Flags Reference	397
GCC / Clang Flags	397
Warning Flags	397
Optimization Flags	397
Debug Flags	397
Sanitizer Flags	398
Architecture and Feature Flags	398
Hardening Flags (Production Security)	398
MSVC Flags (Windows)	398
Recommended Flag Sets	399
Appendix C: Python to C++ Cheat Sheet	401
Types	401
Variables and Assignment	401
Functions	402
Classes	402
Containers	403
String Formatting	403
Error Handling	404
Lambdas / Closures	404
Concurrency	405
Memory Model Comparison	405
Common Python Patterns and Their C++ Equivalents	405
Appendix D: Common Mistakes and Debugging Guide	407
The Thirty Most Common C++ Bugs	407
Memory Bugs	407
Uninitialized Variables	408
Integer and Arithmetic Bugs	408
Object Lifetime Bugs	409
Concurrency Bugs	409
Template and Type Bugs	410
Type System and Casting Bugs	411
I/O and Format Bugs	411
Build and Linking Bugs	412
Debugging Workflow	412
Step 1: Read the Error Message	412

Step 2: Reproduce Minimally	412
Step 3: Use the Debugger	412
Step 4: Add Logging	413
Step 5: Static Analysis	413
Quick Diagnostic Decision Tree	413

Target Standard: C++23 | **Prerequisites:** Comfortable Python, zero C++ assumed

How to Use This Book

Each chapter teaches one concept from scratch. Every concept is first shown in Python (which you know), then explained in C++, then built up with multiple examples, memory diagrams where helpful, common mistakes, and exercises with answers.

Work through Part I completely before moving on. The foundations – types, memory, functions – underpin everything else.

Table of Contents

Part I – Foundations 1. [The Compilation Model and Your First Program](#) 2. [Variables, Types, and the Static Type System](#) 3. [Operators and Expressions](#) 4. [Control Flow: Branching and Loops](#) 5. [Functions, Overloading, and Declarations vs Definitions](#)

Part II – Core C++ (the part that is not like Python) 6. [References – Aliases for Variables](#) 7. [Pointers and Memory Addresses](#) 8. [The Stack and the Heap](#) 9. [const Correctness](#) 10. [Arrays, std::vector, and std::string](#) 11. [Scope, Lifetime, and Organizing Code into Files](#)

Part III – Ownership and Memory Management 12. [RAII – The Core Idea That Replaces Garbage Collection](#) 13. [Dynamic Allocation: new, delete, and Why You Avoid Them](#) 14. [Smart Pointers: unique_ptr, shared_ptr, weak_ptr](#) 15. [Move Semantics, lvalues and rvalues](#) 16. [The Rule of 0, 3, and 5](#)

Part IV – Object-Oriented Programming 17. [Classes, Objects, and Encapsulation](#) 18. [Constructors, Destructors, and Initialization](#) 19. [Inheritance and Composition](#) 20. [Virtual Functions and Polymorphism](#) 21. [Abstract Classes and Interfaces](#) 22. [Operator Overloading](#)

Part V – Generic Programming 23. [Function and Class Templates](#) 24. [Template Specialization and Variadic Templates](#) 25. [Concepts \(C++20\) – Compile-Time Duck Typing](#) 26. [An Introduction to Template Metaprogramming](#)

Part VI – The Standard Library 27. [Containers: vector, map, set, array, and friends](#) 28. [Iterators](#) 29. [Algorithms: sort, find, transform, and the rest](#) 30. [Lambdas and Function Objects](#) 31. [Ranges and Views \(C++20\)](#) 32. [Utility Types: optional, variant, any, tuple](#)

Part VII – Modern C++ (C++11 to C++23) 33. [auto, type deduction, and structured bindings](#) 34. [constexpr and compile-time computation](#) 35. [std::format and modern string handling](#) 36. [Coroutines and Generators](#) 37. [Modules \(C++20\)](#)

Part VIII – Performance 38. [Value vs Reference Semantics](#) 39. [How Memory Layout Affects Speed](#) 40. [Data-Oriented Design](#) 41. [Profiling and Flamegraphs](#)

Part IX – Concurrency 42. [Threads and `std::jthread`](#) 43. [Mutexes, Locks, and Race Conditions](#) 44. [Atomics and the C++ Memory Model](#) 45. [Async, Futures, and Tasks](#)

Part X – Graphics and Game Development 46. [The Math: vectors, matrices, quaternions](#) 47. [How the GPU Works](#) 48. [OpenGL Fundamentals](#) 49. [Moving to Vulkan](#) 50. [Game Loop, ECS Architecture, and Engine Design](#)

Part XI – Systems Programming 51. [The Machine: registers, memory, syscalls](#) 52. [Working with the OS and Linux APIs](#) 53. [Networking from the Ground Up](#) 54. [Where C++ Meets C, eBPF, and Go](#)

Appendices - A. [Setting Up Your Toolchain](#) - B. [Compiler Flags Reference](#) - C. [Python to C++ Cheat Sheet](#) - D. [Common Mistakes and How to Debug Them](#)

Part I – Foundations

Chapter 1: The Compilation Model and Your First Program

What Does “Running a Program” Actually Mean?

Before you write a single line of C++, you need to understand something Python completely hides: what a computer actually does to run your code.

When you type `python hello.py`, a lot of invisible machinery kicks in. When you compile and run a C++ program, that machinery becomes visible – and you control it. That visibility is both the source of C++’s power and the main source of beginner confusion.

How Python Runs Your Code

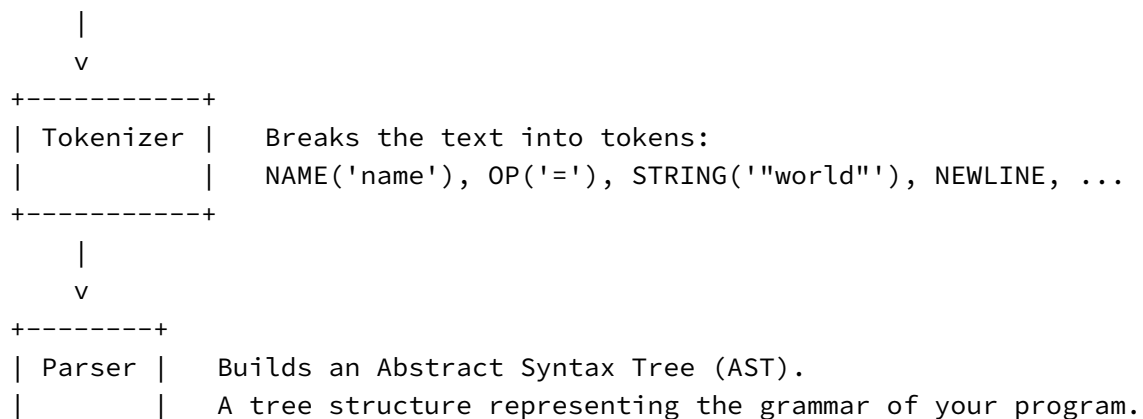
Let’s trace exactly what happens when you run a Python script.

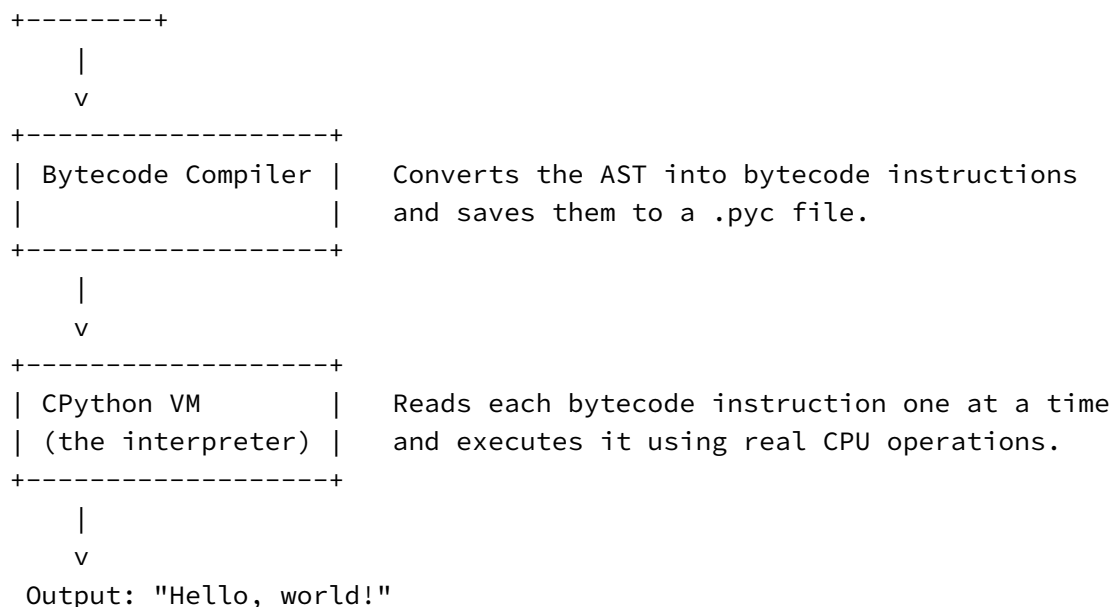
```
# hello.py
name = "world"
print(f"Hello, {name}!")
```

```
$ python hello.py
Hello, world!
```

CPython (the standard Python interpreter) runs your code through five stages:

hello.py (plain text file)





The `.pyc` file contains instructions for a **virtual machine** – a software CPU that Python invented. These are NOT instructions your real CPU understands. The CPython VM translates each bytecode instruction into real CPU instructions as the program runs.

You can actually see the bytecode:

```
import dis

def greet(name):
    return f"Hello, {name}!"

dis.dis(greet)
```

Output:

2	0 RESUME	0
3	2 LOAD_CONST	1 ('Hello, ')
	4 LOAD_FAST	0 (name)
	6 FORMAT_VALUE	0
	8 BUILD_STRING	2
	10 RETURN_VALUE	

LOAD_FAST, FORMAT_VALUE, BUILD_STRING – your CPU has never heard of these. The CPython VM reads each one and does the corresponding real work.

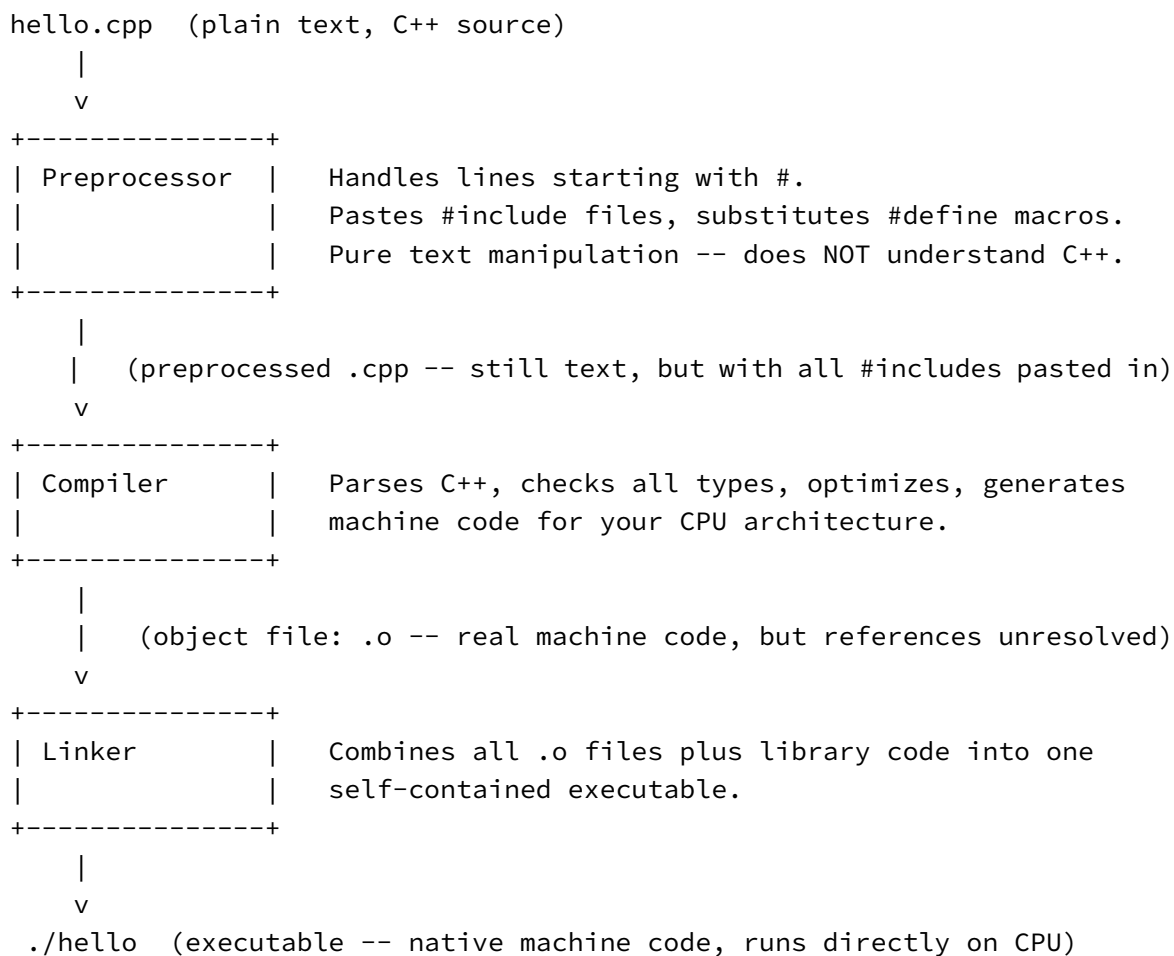
The consequence: There is always a middleman between your code and the CPU. Every Python operation passes through the interpreter loop. This is why Python is typically 10 to 100 times slower than C++ for CPU-bound work.

How C++ Runs Your Code

C++ uses a completely different model. Your source code is **translated directly into CPU instructions** before the program runs. There is no interpreter, no virtual machine, no middleman. When you run the program, your CPU executes your instructions directly.

This is called **ahead-of-time compilation**.

The translation is done by a chain of three programs:



Stage 1: The Preprocessor in Detail

The preprocessor is a dumb text tool. It knows only three things:

- `#include <file>` or `#include "file"` – paste the entire text of that file here
- `#define NAME text` – replace every occurrence of NAME with text
- `#ifdef` / `#ifndef` / `#else` / `#endif` – include or exclude blocks of text

It does not parse C++. It does not understand types. It just manipulates text.

When you write `#include <iostream>`, the preprocessor pastes thousands of lines into your file before the compiler sees anything. You can see how much:

```
$ g++ -E hello.cpp | wc -l  
45231
```

One line expands to 45,231 lines. That is why large projects are slow to compile – every .cpp file re-expands every header it includes.

Stage 2: The Compiler in Detail

The compiler reads the preprocessed text and does the real work:

1. **Parsing:** builds an AST (same concept as Python’s parser)
2. **Semantic analysis:** checks types, resolves names, catches errors
3. **Optimization** (with `-O2`): rewrites your code to run faster
4. **Code generation:** emits machine code for your target CPU

The output is an **object file** (.o). It contains real machine code, but it has gaps – references to functions defined in other files (like `std::cout`) that haven’t been connected yet.

Stage 3: The Linker in Detail

The linker takes all object files and the standard library and combines them into one executable. It resolves every function call to an actual memory address. When `main.cpp` calls `std::cout`, the linker finds `cout`’s implementation in the standard library and wires them together.

The Core Difference

Python:

source --> bytecode (at first run) --> CPU (via interpreter, every run)

C++:

source --> machine code (once, at compile time) --> CPU (directly, every run)

After you compile once, running the program has zero translation overhead. The CPU just executes instructions.

Your First C++ Program

Create a file called `hello.cpp`:

```
#include <iostream>

int main() {
    std::cout << "Hello, world!\n";
    return 0;
}
```

Compile and run:

```
$ g++ -std=c++23 -o hello hello.cpp
$ ./hello
Hello, world!
```

Now let's understand every token in this program.

#include <iostream>

A preprocessor directive. Pastes the `iostream` header (thousands of lines) into your file.

`iostream` declares `std::cout`, the standard output stream. Without it:

```
error: 'cout' is not a member of 'std'
  4 |     std::cout << "Hello, world!\n";
    |     ^~~~~~
note: 'std::cout' is defined in header '<iostream>';
    did you forget to '#include <iostream>'?
```

Angle brackets `<iostream>` mean “find this in the compiler’s system include path.” Double quotes `"my-file.h"` mean “look in my project directory first.”

int main()

The **entry point** of every C++ program. When the OS launches your executable, it calls `main()`. Everything begins here.

`int` is the **return type**. The function returns an integer to the OS as an exit code: `-0` = success (by convention) - Non-zero = some kind of failure

Unlike Python where top-level code runs directly, C++ requires exactly one function called `main` as the entry point. You can’t rename it or have two of them.

The `{}` braces delimit the function body, playing the same role as `:` and indentation in Python.


```
std::cout << "Hello, world!\n";
```

This is how you print to the terminal.

std is a **namespace** – a named container for related identifiers. The entire C++ standard library lives inside `std`. The `::` operator means “look inside.” So `std::cout` means “the `cout` that lives inside the `std` namespace.”

cout stands for “character output.” It is an object representing your terminal’s standard output.

`<<` is the **stream insertion operator**. It sends the value on the right into the stream on the left. You can chain it:

```
std::cout << "Hello" << ", " << "world" << "!\n";
```

Each `<<` sends one more thing into the stream.

`"Hello, world!\n"` is a string literal. The `\n` is a newline character. Unlike Python’s `print()`, which adds a newline automatically, C++ streams do not add newlines unless you include them.

`;` terminates the statement. Every statement ends with a semicolon. This is the single most common beginner error. Forgetting it produces an error on the *next* line because the compiler doesn’t notice until it sees an unexpected token.

```
return 0;
```

Returns the exit code 0 (success) to the OS. In `main` specifically, you can omit this and the compiler inserts it, but writing it is explicit and clear.

Compiling: What the Flags Mean

```
$ g++ -std=c++23 -Wall -Wextra -o hello hello.cpp
```

Flag	What it does
<code>g++</code>	The GNU C++ compiler. Also try <code>clang++</code> (often better error messages).
<code>-std=c++23</code>	Use the C++23 standard. Without this you get an older, less capable standard.
<code>-Wall</code>	Enable all common warnings. The W stands for Warning.
<code>-Wextra</code>	Enable even more warnings.
<code>-o hello</code>	Name the output executable <code>hello</code> . Default without <code>-o</code> is <code>a.out</code> .
<code>hello.cpp</code>	The source file to compile.

Two Build Modes You Will Use

```
# Development (use this while writing code):
$ g++ -std=c++23 -Wall -Wextra -g -fsanitize=address,undefined -o hello hello.cpp
#
#                               |   |
#                               |   +-- catches memory bugs and UB at runtime
#                               +----- adds debug symbols for stack traces

# Release (use this for final builds):
$ g++ -std=c++23 -O2 -o hello hello.cpp
#
#                               |
#                               +-- enables optimization (2x-10x faster)
```

The sanitizers (`-fsanitize=address,undefined`) catch entire classes of bugs that would otherwise produce silent wrong answers or random crashes. Always use them during development.

What Is in the Compiled Binary?

The executable file is organized into sections:

`./hello` (ELF executable on Linux, Mach-O on macOS, PE on Windows)

Section	Contents
-----	-----
<code>.text</code>	Your compiled machine code instructions
<code>.rodata</code>	Read-only data: string literals like <code>"Hello, world!\n"</code>
<code>.data</code>	Initialized global variables
<code>.bss</code>	Uninitialized global variables (zeroed at program start)

You can disassemble the executable to see what your code became:

```
$ objdump -d -C hello | grep -A 15 "<main>"
```

```
000000000001149 <main>:
 1149: push    rbp
 114a: mov     rbp, rsp
 114d: lea     rdi, [rip+0xeb0]    <- loads address of "Hello, world!\n"
 1154: call    1060 <puts@plt>    <- calls puts() (compiler optimized cout to this)
 1159: mov     eax, 0x0
 115e: pop     rbp
 115f: ret
```

Seven CPU instructions. No interpreter, no type checking, no reference counting. This is what runs when you execute `./hello`.

Multi-File Programs

Once programs grow beyond one file, you split them:

```
main.cpp  --compile-->  main.o  --+
                                           +--> linker --> program
greet.cpp --compile--> greet.o  --+
```

```
// greet.h -- declaration (the interface, what other files need to know)
#pragma once
#include <string>
void greet(const std::string& name);

// greet.cpp -- definition (the actual implementation)
#include <iostream>
#include "greet.h"
void greet(const std::string& name) {
    std::cout << "Hello, " << name << "!\n";
}

// main.cpp -- uses the function
#include "greet.h"
int main() {
    greet("world");
    return 0;
}
```

```
$ g++ -std=c++23 -o program main.cpp greet.cpp
```

Chapter 11 covers the full rationale for this split.

Reading Compiler Errors

The compiler is your most important tool. It catches errors before your program runs. Learning to read its output quickly is a core skill.

Missing Semicolon

```
int main() {
    std::cout << "Hello"    // missing semicolon here
    return 0;
}
```

```
hello.cpp:4:5: error: expected ';' before 'return'
```

```
4 |     std::cout << "Hello"
  |                               ^
```

```

    |
5 |     return 0;

```

The caret ^ marks where the problem was detected – the end of line 4. The message says “before ‘return’” because the compiler only noticed something was wrong when it hit return.

Missing #include

```

int main() {
    std::cout << "Hello\n";
}

```

hello.cpp:2:5: error: 'cout' is not a member of 'std'

```

2 |     std::cout << "Hello\n";
  |     ^~~~~~

```

note: 'std::cout' is defined in header '<iostream>';
did you forget to '#include <iostream>'?

Read every note: line. Modern compilers often tell you exactly what to do.

Undefined Reference (Linker Error)

```

// main.cpp
void greet(const std::string& name); // declared but never defined anywhere

int main() { greet("world"); }

```

```

/usr/bin/ld: main.o: undefined reference to `greet(std::string const&)'
collect2: error: ld returned 1 exit status

```

“Undefined reference” means the compiler accepted the code (it saw the declaration), but the linker couldn’t find the function’s body. Either you forgot to define it, or forgot to include its .cpp file in the build command.

Type Mismatch

```

int x = "hello";

```

error: invalid conversion from 'const char*' to 'int'

```

1 | int x = "hello";
  |     ^~~~~~

```

The static type system caught this before the program ran. Python would allow `x = 5; x = "hello"` – C++ refuses.

Common Mistakes in This Chapter

Mistake 1: Forgetting the semicolon

The bug: `std::cout << "Hello"` with no `;`

The symptom: Error message appears on the *next* line, not the line where you forgot it. The compiler doesn't notice until it encounters an unexpected token.

The fix: Every statement ends with `;`. After `return 0`. After every `cout`. After every variable declaration.

Mistake 2: Writing `cout` without `std::`

The bug:

```
cout << "Hello\n"; // 'cout' -- where is it?
```

The symptom:

```
error: 'cout' was not declared in this scope
```

The fix: Write `std::cout`. You can also add `using std::cout;` at the top of a function to avoid typing `std::` everytime. Do not use `using namespace std;` – it pulls in hundreds of names that may conflict with your own.

Mistake 3: Confusing `#include <...>` with `#include "..."`

The bug: Writing `#include "iostream"` instead of `#include <iostream>`

What happens: The preprocessor looks for `iostream` in the current directory first. It may find nothing and fail, or find the wrong file.

The fix: Use `<...>` for system and library headers. Use `"..."` for your own headers.

Exercises

Exercise 1.1 – Predict the output

What does this print? Work it out before running it.

```
#include <iostream>
int main() {
    std::cout << "Line 1\n";
    std::cout << "Line " << 2 << "\n";
    std::cout << "Line " << 1 + 2 << "\n";
    return 0;
}
```

Answer:

Line 1
Line 2
Line 3

1 + 2 is evaluated to 3 before being inserted into the stream.

Exercise 1.2 – Find all three bugs

This program has exactly three syntax errors. Find them all.

```
#include <iostream>
int main() {
    std::cout << "Hello, world!"
    std::cout << "How are you?"
    return 0
}
```

Answer: 1. Missing ; after "Hello, world!" 2. Missing ; after "How are you?" 3. Missing ; after return 0

Exercise 1.3 – Write from scratch

Write a program that prints this output exactly (copy the spacing):

```
=== My C++ Program ===
Author: Your Name
Version: 1.0
```

Answer:

```
#include <iostream>
int main() {
    std::cout << "=== My C++ Program ===\n";
    std::cout << "Author: Your Name\n";
    std::cout << "Version: 1.0\n";
    return 0;
}
```

Exercise 1.4 – Explain the difference

In your own words: what is the difference between compiling with `-O0` and `-O2`? Why would you use each?

Answer: `-O0` (no optimization) translates your code nearly literally into machine code. The machine code closely mirrors your source, making it easier to debug – variables are in the places you’d expect, and the program steps through code the way you wrote it. Use this during development.

`-O2` enables aggressive optimization: the compiler may inline functions, eliminate redundant computations, reorder instructions, and more. The program runs significantly faster. The machine code may look very different from your source. Use this for release builds.

Chapter 2: Variables, Types, and the Static Type System

What IS a Variable?

In Python, a variable is a **name tag** attached to an object. The object lives on the heap; the name is just a reference to it.

```
x = 42          # 'x' is a label pointing to an int object on the heap
y = x          # 'y' is another label pointing to the SAME object
x = "hello"     # 'x' now points to a different object; y is unchanged
print(y)       # 42
```

The object carries its own type:

Python memory:

Variable	Points to
-----	-----
x ----->	[type: int refcount: 2 value: 42]
y ----->	[same object]

After `x = "hello"`:

x ----->	[type: str refcount: 1 value: "hello"]
y ----->	[type: int refcount: 1 value: 42]

Every Python variable is a pointer. The object it points to carries its type, a reference count, and metadata.

In C++, a variable is a **named region of memory with a fixed type**. The variable IS the storage. There is no separate heap object (unless you explicitly create one), no reference count, no type tag at runtime.

```
int x = 42;    // 'x' is 4 bytes of memory containing the bit pattern for 42
               // The type 'int' exists only in the compiler's mind
```

C++ memory:

Address	Variable	Bytes stored
-----	-----	-----
0x7fff10	x (int)	[00][00][00][2A] <- that is 42 in hex

No type tag. No reference count. No metadata. Four bytes. That is it.

The type `int` tells the **compiler** (not the CPU, not the runtime): - Reserve 4 bytes - Treat those bytes as a signed two's complement integer - Allow operations like `+`, `-`, `*`, `/` - Reject operations like string concatenation

At runtime, there are only bytes. Types are a compile-time concept.

This is the most fundamental difference between Python and C++. Almost every other difference you will encounter flows from this one.

How Integers Are Stored: Binary and Two's Complement

To understand C++ types, you need to understand how integers are stored in memory. If you already know two's complement, skim this section.

A computer stores everything as bits: 0 or 1. Eight bits make one byte.

The number 42 in binary:

$$42 = 32 + 8 + 2 = 2^5 + 2^3 + 2^1$$

Bit positions (0 = rightmost):

7	6	5	4	3	2	1	0
0	0	1	0	1	0	1	0
		32		8		2	
0							

$$0 \times 128 + 0 \times 64 + 1 \times 32 + 0 \times 16 + 1 \times 8 + 0 \times 4 + 1 \times 2 + 0 \times 1 = 42$$

An `int` uses 4 bytes = 32 bits. So 42 stored as a 32-bit int:

Byte 3	Byte 2	Byte 1	Byte 0
00000000	00000000	00000000	00101010
			= 42

No type metadata anywhere. The CPU interprets these bytes according to which instruction operates on them.

How Negative Numbers Work: Two's Complement

For signed integers, the highest bit is the sign bit: 0 = positive, 1 = negative.

For a 32-bit signed int: - Maximum positive: 0111 1111 ... 1111 = 2,147,483,647 - Maximum negative: 1000 0000 ... 0000 = -2,147,483,648 - -1: 1111 1111 ... 1111 (all bits set)

To negate a number: flip all bits, then add 1.

```
42 in binary:  0000 0000 0000 0000 0000 0000 0010 1010
Flip all bits: 1111 1111 1111 1111 1111 1111 1101 0101
Add 1:         1111 1111 1111 1111 1111 1111 1101 0110  = -42
```

The critical implication: **signed integer overflow is undefined behavior in C++.**

```
int max = 2147483647;    // the largest possible int
int bad = max + 1;       // UNDEFINED BEHAVIOR
                        // The standard does not define what happens.
                        // In practice: wraps to -2147483648 on most systems.
                        // But the compiler is allowed to assume overflow
                        // never happens and optimize in ways that break you.
```

Python integers are arbitrary precision – they never overflow. C++ `int` has fixed size and hard limits. Always think about whether your values can exceed the range.

The Fundamental Types

`bool`

```
bool yes = true;
bool no  = false;

std::cout << yes << "\n";    // prints 1
std::cout << no  << "\n";    // prints 0
```

Stores true/false. Takes 1 byte (even though 1 bit would suffice – byte is the smallest addressable unit on most hardware). `true` is stored as 1, `false` as 0.

Any non-zero integer converts to `true`, zero converts to `false`:

```
bool b1 = 42;    // true  (42 != 0)
bool b2 = 0;     // false
bool b3 = -1;    // true  (-1 != 0)
```

char

```
char letter = 'A';    // single character, single quotes
char newline = '\n';  // escape sequences work like Python
char tab = '\t';
char null_c = '\0';   // null character (value 0)
```

One byte. Holds one ASCII character. The character is stored as its ASCII code: 'A' = 65, 'Z' = 90, '0' = 48, '\n' = 10.

You can do arithmetic on chars because they are just numbers:

```
char c = 'A';
std::cout << c + 1;           // 66    (int arithmetic)
std::cout << static_cast<char>(c + 1); // 'B'    (cast back to char)
std::cout << (char)('a' + 2);    // 'c'    (C-style cast, avoid)
```

Integer Types

Type	Typical size	Signed range	Notes
short	2 bytes	-32,768 to 32,767	Rarely used
int	4 bytes	-2.1B to 2.1B	Default integer type
long	4 or 8 bytes	platform-dependent	Avoid – use long long
long long	8 bytes	-9.2e18 to 9.2e18	Use when int overflows

To make a type unsigned (no negatives, larger maximum):

```
unsigned int u = 4000000000u;    // 'u' suffix = unsigned literal
unsigned long long big = 18446744073709551615ull; // 'ull' suffix
```

An unsigned type with N bits holds 0 to $2^N - 1$. A signed type with N bits holds $-2^{(N-1)}$ to $2^{(N-1)} - 1$.

float and double

```
float f = 3.14f;    // 32-bit floating point, 'f' suffix is required
double d = 3.14;    // 64-bit floating point -- use this by default
```

double is what Python calls float. Use double unless you have a specific reason (memory, GPU/SIMD code).

Why prefer double? Precision: - float: approximately 7 significant decimal digits - double: approximately 15 significant decimal digits

```
float f = 1.23456789f;
double d = 1.23456789;
std::cout << std::setprecision(10) << f << "\n"; // 1.234567881 (wrong at 7th digit)
std::cout << std::setprecision(10) << d << "\n"; // 1.23456789 (correct)
```

Floating point is not exact – same in Python and C++:

```
# Python
print(0.1 + 0.2) # 0.30000000000000004
```

```
// C++
#include <iomanip>
double x = 0.1 + 0.2;
std::cout << x << "\n"; // 0.3 (default rounds nicely)
std::cout << std::setprecision(17) << x << "\n"; // 0.30000000000000004
std::cout << std::setprecision(17) << 0.3 << "\n"; // 0.29999999999999999
```

0.1, 0.2, and 0.3 cannot be represented exactly in binary floating point – the same way 1/3 cannot be represented exactly in decimal. This is not a C++ bug; it is IEEE 754 floating point. The implication: **never compare floating-point numbers with ==**.

std::string

```
#include <string>
std::string name = "Alice";
std::string greeting = "Hello, " + name + "!"; // concatenation with +
std::cout << greeting.size() << "\n"; // 13
```

Not a primitive type. `std::string` is a class from the standard library that manages heap-allocated character data. We cover it in depth in Chapter 10.

Fixed-Width Integer Types

The sizes of `short`, `int`, `long` depend on the platform and compiler. For code that must work correctly regardless of platform – networking, file formats, hardware interfaces – use exact-size types from `<cstdint>`:

```
#include <cstdint>

int8_t a = -128; // exactly 8 bits, signed (-128 to 127)
uint8_t b = 255; // exactly 8 bits, unsigned (0 to 255)
int16_t c = 32767; // exactly 16 bits, signed
uint16_t d = 65535; // exactly 16 bits, unsigned
int32_t e = -1; // exactly 32 bits, signed
uint32_t f = 4294967295u; // exactly 32 bits, unsigned
int64_t g = -1LL; // exactly 64 bits, signed
uint64_t h = 0xFFFFFFFFFFFFFFFFull; // exactly 64 bits, unsigned, all bits set
```

Use these whenever the exact bit width matters. Use `int` when it does not.

Declaring and Initializing Variables

Never Leave a Variable Uninitialized

This is the most important rule in this chapter:

```
int sum; // BAD: sum contains whatever bytes were
for (int i = 0; i < 10; ++i) // in that memory location before.
    sum += i; // Adding to garbage = garbage.
std::cout << sum << "\n"; // Prints something unpredictable.

int sum{0}; // GOOD: explicitly initialized to zero.
for (int i = 0; i < 10; ++i)
    sum += i;
std::cout << sum << "\n"; // 45, always.
```

On some runs, the garbage value might happen to be 0, making the bug invisible. On other runs, it will be -858993460 or similar. Undefined behavior is unpredictable.

The Four Initialization Forms

C++ has four ways to initialize a variable. They have subtle differences.

Form 1: Default initialization (leaves value unset – dangerous)

```
int x; // x has garbage value
double d; // d has garbage value
```

Do not do this for primitive types.

Form 2: Copy initialization (C-style, from C)

```
int x = 5;
double d = 3.14;
std::string s = "hello";
```

Looks like assignment but is initialization (happens at construction, not after). Fine for simple cases.

Form 3: Direct initialization

```
int x(5);
double d(3.14);
std::string s("hello");
```

More verbose for primitives but is the natural form for class objects with constructors.

Form 4: Brace (uniform) initialization – prefer this

```
int x{5};
double d{3.14};
std::string s{"hello"};
```

Why prefer brace initialization? It prevents **narrowing conversions** – type truncations that silently discard data:

```
// With = initialization, narrowing is SILENT:
int a = 3.7;           // truncates to 3. No error. Potential bug hiding.
int b = 300000;        // fits in int, fine
int c = 300000LL;      // long long to int, may truncate on 16-bit systems, no warning

// With {} initialization, narrowing is a COMPILE ERROR:
int a{3.7};           // error: narrowing conversion of '3.7e+0' from 'double' to 'int'
int b{300000};        // fine
int c{300000LL};      // fine (value fits in int)
```

The compiler tells you about the truncation at compile time instead of letting it silently corrupt your data at runtime.

You can value-initialize (set to zero/false/null) with empty braces:

```
int    n{};           // 0
double d{};           // 0.0
bool   b{};           // false
char   c{};           // '\0'
```

Memory Layout of Variables

```
bool   a{true};       // 1 byte
char   b{'X'};        // 1 byte
int     c{42};         // 4 bytes
double d{3.14};       // 8 bytes
```

Stack memory (addresses are illustrative):

Addr	Var	Size	Raw bytes (hex)	
0x1000	a (bool)	1 byte	[01]	
0x1001	b (char)	1 byte	[58]	'X' = ASCII 88 = 0x58
0x1004	c (int)	4 bytes	[00][00][00][2A]	42 = 0x2A
0x1008	d (double)	8 bytes	[40][09][1E][B8][51][EB][85][1F]	

(The actual layout may differ due to alignment, but the sizes are as shown.)

There is no type information stored at runtime. The CPU knows to treat `c` as a signed integer because the compiled instructions operating on it are integer instructions. The type system is entirely a compile-time construct.

auto – Type Deduction

When the type is obvious from the right-hand side, `auto` lets the compiler deduce it:

```
auto x = 42;           // int      (integer literals default to int)
auto y = 42L;          // long     (L suffix)
auto z = 42LL;         // long long (LL suffix)
auto d = 3.14;          // double   (decimal literals default to double)
auto f = 3.14f;         // float    (f suffix)
auto b = true;          // bool
auto s = std::string{"hi"}; // std::string
```

`auto` does NOT make C++ dynamically typed. The type is deduced once at compile time and then fixed permanently:

```
auto x = 42;           // x is int, fixed forever
x = 3.14;              // fine: double 3.14 truncates to int 3 (with = init)
x = "hello";           // COMPILE ERROR: cannot assign const char* to int
```

`auto` strips top-level `const` and references from the deduced type. Add them explicitly if you need them:

```
int n = 5;
auto a = n;           // int      (copy)
auto& b = n;           // int&     (reference to n)
const auto& c = n;     // const int& (const reference, no copy, no modify)
```

Where `auto` really shines is avoiding verbose type names:

```
// Without auto:
std::vector<std::pair<std::string, int>>::iterator it = scores.begin();

// With auto (same type, more readable):
auto it = scores.begin();
```

const and constexpr

const – Constant After Initialization

```
const int MAX = 8;
MAX = 10;           // COMPILE ERROR: assignment of read-only variable

const int n = get_input(); // value known only at runtime -- still valid const
```

Use `const` for values that are initialized once and never change. The compiler enforces this – it’s not just a convention.

constexpr – Must Be Computed at Compile Time

```
constexpr int BOARD = 8;
constexpr double PI = 3.14159265358979;

int grid[BOARD][BOARD]; // OK -- array size must be a compile-time constant

constexpr int square(int x) { return x * x; }
constexpr int area = square(BOARD); // 64, computed at compile time, zero runtime cost
```

`constexpr` is stronger than `const`. The value must be computable without running the program. If it isn’t, you get a compile error – which is what you want.

Rule: use `constexpr` for mathematical constants, array sizes, and configuration values. Use `const` for values set at runtime that shouldn’t change afterward.

Type Conversion

Safe Widening (Automatic)

```
int    i = 42;
double d = i; // int to double: no data loss, happens automatically
long long l = i; // int to long long: always safe
```

Narrowing (Dangerous Without Braces)

```
double d = 3.99;
int    i = d; // silently truncates to 3 (with = initialization)
int    j{d};  // COMPILE ERROR: narrowing (with {} initialization)
```


Explicit Conversion with `static_cast`

When you intend to convert, be explicit about it:

```
double d = 3.99;
int i = static_cast<int>(d);    // 3, explicit truncation, intent is documented

int a = 5, b = 2;
// Bug: division happens as int first, then converts to double
double wrong = a / b;          // 2.0

// Fix: cast one operand to double before dividing
double correct = static_cast<double>(a) / b; // 2.5
```

`static_cast<NewType>(expr)` is the C++ way to convert. It is checked at compile time. Do not use C-style casts like `(int)x` – they bypass some safety checks and are harder to search for in code.

Checking Sizes with `sizeof`

```
#include <iostream>
int main() {
    std::cout << "bool:      " << sizeof(bool)      << " byte(s)\n"; // 1
    std::cout << "char:      " << sizeof(char)       << " byte(s)\n"; // 1
    std::cout << "int:       " << sizeof(int)        << " byte(s)\n"; // 4
    std::cout << "long long: " << sizeof(long long) << " byte(s)\n"; // 8
    std::cout << "float:     " << sizeof(float)      << " byte(s)\n"; // 4
    std::cout << "double:    " << sizeof(double)     << " byte(s)\n"; // 8
    return 0;
}
```

`sizeof` is evaluated at compile time – zero runtime overhead. It works on both types (`sizeof(int)`) and variables (`sizeof(x)`).

Common Mistakes in This Chapter

Mistake 1: Uninitialized Variable

The bug:

```
int total;
for (int i = 1; i <= 5; ++i)
    total += i;    // total starts with garbage, not 0
std::cout << total;
```

The symptom: Wrong or unpredictable output. Sometimes appears correct (when garbage happens to be 0).

The fix: `int total{0};`

How to detect: Compile with `-Wall` (warns about “total may be used uninitialized”). Run with `-fsanitize=undefined` for runtime detection.

Mistake 2: Signed Integer Overflow

The bug:

```
int seconds_per_year = 365 * 24 * 60 * 60;           // 31,536,000 -- fits in int
int seconds_per_century = seconds_per_year * 100;    // 3,153,600,000 -- OVERFLOW!
// int max is ~2.1 billion; 3.15 billion overflows
```

The symptom: Wrong answer (often a negative number). No compile error.

The fix: `long long seconds_per_century = (long long)seconds_per_year * 100;`

How to detect: `-fsanitize=undefined` reports “signed integer overflow.”

Mistake 3: Integer Division When Expecting Decimal

The bug:

```
double average = (100 + 75 + 90) / 3;    // 88 -- not 88.333...
```

All literals are `int`. The division is integer division before the result is stored in `double`.

The fix: `double average = (100 + 75 + 90) / 3.0;` or use `static_cast<double>`.

Mistake 4: Comparing Floats with ==

The bug:

```
double x = 0.1 + 0.2;
if (x == 0.3) {
    std::cout << "equal\n";    // never prints
}
```

The fix:

```
#include <cmath>
if (std::abs(x - 0.3) < 1e-9) {
    std::cout << "equal\n";    // works
}
```

Exercises

Exercise 2.1 – Type sizes

Without looking anything up, predict the output of this program on a 64-bit Linux system, then verify:

```
#include <iostream>
int main() {
    std::cout << sizeof(bool)      << "\n";
    std::cout << sizeof(char)      << "\n";
    std::cout << sizeof(short)     << "\n";
    std::cout << sizeof(int)       << "\n";
    std::cout << sizeof(long)      << "\n";
    std::cout << sizeof(long long) << "\n";
    std::cout << sizeof(float)     << "\n";
    std::cout << sizeof(double)    << "\n";
}
```

Answer: 1 1 2 4 8 8 4 8 on 64-bit Linux. Note that `long` is 8 bytes on 64-bit Linux but 4 bytes on 64-bit Windows – another reason to use `long long` when you need 64 bits.

Exercise 2.2 – Brace initialization rejection

Which of these lines would brace initialization reject with a compile error?

```
int a{3.0};      // (a)
int b{3};        // (b)
int c{3LL};      // (c) LL = long long literal
double d{3};     // (d)
float e{3.14};   // (e)
```

Answer: - (a): double -> int narrows (loses the decimal). **ERROR.** - (b): int -> int, exact. Fine. - (c): long long -> int may narrow if value doesn't fit. **ERROR** (even though 3 fits, the types differ). - (d): int -> double widens (no data loss). Fine. - (e): double -> float narrows (loses precision). **ERROR.**

Exercise 2.3 – Fix the division

Fix this code so it prints 2.5 instead of 2:

```
#include <iostream>
int main() {
    int a = 5, b = 2;
    std::cout << a / b << "\n";
}
```

Answer:

```
std::cout << static_cast<double>(a) / b << "\n";
// or:
std::cout << a / static_cast<double>(b) << "\n";
// or:
std::cout << a / 2.0 << "\n";
```

Exercise 2.4 – Max value calculation

What is the maximum value of `uint32_t`? Show the calculation. Do the same for `int32_t`.

Answer: - `uint32_t`: 32 bits, unsigned. Range = 0 to $2^{32} - 1 = \mathbf{4,294,967,295}$. - `int32_t`: 32 bits, signed. Range = -2^{31} to $2^{31} - 1 = \mathbf{-2,147,483,648 \text{ to } 2,147,483,647}$. One bit is used for the sign.

Chapter 3: Operators and Expressions

Arithmetic Operators

Most arithmetic operators work the same as Python. Division is the exception that trips everyone up.

```
int a = 17, b = 5;

std::cout << a + b << "\n"; // 22 -- addition
std::cout << a - b << "\n"; // 12 -- subtraction
std::cout << a * b << "\n"; // 85 -- multiplication
std::cout << a / b << "\n"; // 3 -- WATCH OUT: integer division
std::cout << a % b << "\n"; // 2 -- remainder (modulo)
```

The division rule: **when both operands are integers, / discards the remainder.**

```
# Python: / always returns a float
print(17 / 5) # 3.4
print(17 // 5) # 3 (floor division, rounds toward negative infinity)
print(-17 // 5) # -4 (floor: -3.4 rounds down to -4)
```

```
// C++: / on integers truncates toward zero (not floor)
std::cout << 17 / 5 << "\n"; // 3 (truncated toward zero)
std::cout << -17 / 5 << "\n"; // -3 (truncated toward zero, NOT -4)
std::cout << 17.0 / 5 << "\n"; // 3.4 (float division: 17.0 is a double)
```

The key distinction: C++ truncates toward zero. Python's // floors toward negative infinity. For positive numbers they agree. For negative numbers they differ.

The Classic Division Trap

```
int numerator = 7;
int denominator = 2;

double result = numerator / denominator; // 3.0, NOT 3.5
```

What happened? The division `numerator / denominator` is evaluated first – both are `int`, so integer division gives 3. Then 3 is converted to 3.0 for the `double` assignment.

The fix: make at least one operand a `double` before the division:

```
double result = static_cast<double>(numerator) / denominator; // 3.5
// or:
double result = numerator / static_cast<double>(denominator); // 3.5
// or (if one is a literal):
double result = numerator / 2.0; // 3.5
```

Modulo With Negative Numbers

```
std::cout << 7 % 3 << "\n"; // 1 (7 = 2*3 + 1)
std::cout << -7 % 3 << "\n"; // -1 (sign follows dividend in C++)
std::cout << 7 % -3 << "\n"; // 1 (sign follows dividend)
```

Python's % always returns non-negative when the divisor is positive (e.g., $-7 \% 3 = 2$ in Python). C++'s % sign follows the dividend.

This matters for “wrap around” patterns. To get Python-like non-negative modulo:

```
int python_mod(int a, int b) {
    return ((a % b) + b) % b;
}
// python_mod(-7, 3) = ((-1) + 3) % 3 = 2 % 3 = 2 -- same as Python
```

Increment and Decrement Operators

C++ has ++ (add 1) and -- (subtract 1) as standalone operators. Python does not.

```
int x = 5;
++x; // pre-increment: x becomes 6
x++; // post-increment: x becomes 7
--x; // pre-decrement: x becomes 6
x--; // post-decrement: x becomes 5
```

Used as standalone statements, ++x and x++ are identical in effect. The difference appears when used inside a larger expression:

```
int x = 5;
int a = ++x; // pre: x becomes 6 FIRST, then a = 6 (new value)
int b = x++; // post: b = 6 (old value), then x becomes 7
```

Pre-increment (++x):

Step 1: increment x (x = 7)

Step 2: expression evaluates to new x (7)

Post-increment (x++):

- Step 1: save current x (6)
- Step 2: increment x (x = 7)
- Step 3: expression evaluates to saved old x (6)

Always prefer ++x (pre-increment) as a habit. It is never slower than x++, and for iterator objects (Chapter 28) it can be significantly faster because post-increment must save a copy of the old value.

Compound Assignment

These modify a variable in place:

```
int x = 20;
x += 5;    // x = 25    (same as x = x + 5)
x -= 3;    // x = 22
x *= 2;    // x = 44
x /= 4;    // x = 11    (integer division if x is int)
x %= 3;    // x = 2     (remainder)
x <<= 2;   // x = 8     (left shift by 2: same as x * 4)
x >>= 1;   // x = 4     (right shift by 1: same as x / 2 for non-negative)
```

Comparison Operators

All return bool:

```
int a = 5, b = 10;
bool r1 = (a == b);    // false -- equal
bool r2 = (a != b);    // true  -- not equal
bool r3 = (a < b);     // true  -- less than
bool r4 = (a > b);     // false -- greater than
bool r5 = (a <= b);    // true  -- less than or equal
bool r6 = (a >= b);    // false -- greater than or equal
```

The Floating-Point Comparison Problem

```
double x = 0.1 + 0.2;
double y = 0.3;
std::cout << (x == y) << "\n";    // 0 (false!)
```

0.1 cannot be represented exactly in binary. Neither can 0.2 or 0.3. The sum 0.1 + 0.2 comes out as 0.30000000000000004... Stored 0.3 is 0.29999999999999998... They are not equal, even though mathematically they should be.

This is not a C++ problem – the same thing happens in Python and every other language that uses IEEE 754 floating point (which is all of them).

Rule: Never compare floating-point numbers with ==. Use a tolerance.

```
#include <cmath>    // for std::abs

// Returns true if a and b are within tolerance of each other
bool nearly_equal(double a, double b, double tolerance = 1e-9) {
    return std::abs(a - b) < tolerance;
}

if (nearly_equal(0.1 + 0.2, 0.3)) {
    std::cout << "equal (within tolerance)\n"; // this prints
}
```

Choose tolerance based on the scale of your values: - For values around 1.0: 1e-9 is reasonable - For values around 1,000,000: 1e-3 may be needed - For values that went through many operations: use larger tolerances

Logical Operators

```
bool x = true, y = false;

bool and_result = x && y;    // false (both must be true) -- Python: x and y
bool or_result  = x || y;    // true  (either must be true) -- Python: x or y
bool not_result = !x;        // false (flip)                -- Python: not x
```

Short-Circuit Evaluation

&& stops evaluating as soon as it finds a false. || stops as soon as it finds a true. This is identical to Python and is critically important for safety:

```
int* p = nullptr;

// Safe: if p is null, the right side is NEVER evaluated
if (p != nullptr && *p > 0) {
    std::cout << "positive pointee\n";
}
// Without short-circuit: *p when p is null = crash

// Common optimization: cheap check first, expensive check second
if (is_in_cache(key) || compute_and_cache(key)) {
    // compute_and_cache only called when not in cache
}
```

Short-circuit evaluation is relied upon for correctness, not just optimization.

Bitwise Operators

These operate on individual bits within integers. They appear constantly in systems programming, graphics, embedded code, and game engines.

Op	Name	Meaning
&	AND	Output bit is 1 only if BOTH input bits are 1
	OR	Output bit is 1 if EITHER input bit is 1
^	XOR	Output bit is 1 if the two input bits are DIFFERENT
~	NOT	Flips all bits
<<	Left shift	Shifts bits left, filling with 0 on the right
>>	Right shift	Shifts bits right (fills with 0 for unsigned, sign bit for signed)

Example: $a = 5 = 0101$ in binary
 $b = 3 = 0011$ in binary

```
a & b:  0101
        0011
        ----
        0001  = 1  (only bit 0 is set in both)
```

```
a | b:  0101
        0011
        ----
        0111  = 7  (any bit set in either)
```

```
a ^ b:  0101
        0011
        ----
        0110  = 6  (bits that differ)
```

```
~a:      ~0101 = 1111...11111010 = -6 (for 32-bit signed int)
```

Left shift by 1 = multiply by 2. Left shift by n = multiply by 2^n :

```
1 << 0 = 1
1 << 1 = 2
1 << 2 = 4
1 << 3 = 8
1 << 4 = 16
1 << 10 = 1024
```

Practical Use: Boolean Flags Packed in an Integer

Instead of a separate `bool` for each property, pack them into one integer – each bit represents one flag:

```
const uint32_t VISIBLE    = 1u << 0; // bit 0: 00000001
const uint32_t SOLID     = 1u << 1; // bit 1: 00000010
const uint32_t ACTIVE    = 1u << 2; // bit 2: 00000100
const uint32_t HAS_HEALTH = 1u << 3; // bit 3: 00001000

uint32_t entity_flags = 0;

// Set a flag (turn bit ON):
entity_flags |= VISIBLE; // flags = 00000001
entity_flags |= ACTIVE;  // flags = 00000101

// Test a flag (check if bit is set):
if (entity_flags & VISIBLE) { // nonzero = true
    render(entity);
}
if (entity_flags & ACTIVE) {
    update(entity);
}

// Clear a flag (turn bit OFF):
entity_flags &= ~ACTIVE; // ~ACTIVE = 11111011; AND clears bit 2
                        // flags = 00000001

// Toggle a flag (flip):
entity_flags ^= SOLID; // XOR flips bit 1
```

This pattern is everywhere: OpenGL uses `GL_COLOR_BUFFER_BIT` | `GL_DEPTH_BUFFER_BIT`, Vulkan uses `VkImageUsageFlags`, Linux uses `O_RDONLY` | `O_CREAT` for file open flags.

Operator Precedence

When operators are mixed, precedence determines the order of evaluation:

Highest priority (evaluated first)

<code>() [] -> .</code>	grouping, subscript, member access
<code>! ~ ++x --x (unary)</code>	unary operators (right-to-left)
<code>* / %</code>	multiplicative
<code>+ -</code>	additive
<code><< >></code>	bitwise shift
<code>< <= > >=</code>	relational comparison
<code>== !=</code>	equality comparison
<code>&</code>	bitwise AND
<code>^</code>	bitwise XOR
<code> </code>	bitwise OR

&&	logical AND
	logical OR
?:	ternary (right-to-left)
= += -= ...	assignment (right-to-left)

 Lowest priority (evaluated last)

Common traps:

```
// Trap 1: bitwise AND vs equality
if (flags & MASK == 1) { }    // parsed as: flags & (MASK == 1)  -- WRONG
if ((flags & MASK) == 1) { }  // correct

// Trap 2: shift and arithmetic
int x = 2 + 3 << 1;    // parsed as: 2 + (3 << 1) = 2 + 6 = 8
int y = (2 + 3) << 1;  // = 5 << 1 = 10

// Rule: when in doubt, parenthesize
```

The Ternary Operator

```
# Python
label = "even" if n % 2 == 0 else "odd"
```

```
// C++
std::string label = (n % 2 == 0) ? "even" : "odd";
//                condition   true    false
```

The ternary is an *expression* that produces a value. Good for simple one-liners. Do not nest them – `a ? b ? c : d : e` is unreadable.

Common Mistakes in This Chapter

Mistake 1: Assignment Instead of Comparison

The bug:

```
if (x = 5) { ... }    // assigns 5 to x, condition is always true (5 != 0)
```

The symptom: The `if` branch always runs; the `else` never runs; `x` silently changed. **The fix:** `if (x == 5)`. Compile with `-Wall` to get a warning about this. **Defensive coding:** Some people write `if (5 == x)` (“Yoda conditions”). Then a typo `if (5 = x)` is a compile error (can’t assign to a literal).

Mistake 2: Integer Division Surprise

The bug:

```
double bmi = weight / (height * height); // all ints -- integer division!
```

The fix: Cast one operand: `static_cast<double>(weight) / (height * height)`

Mistake 3: == on Floats

The bug: `if (result == expected_value)` where both are doubles **The fix:** `if (std::abs(result - expected_value) < 1e-9)`

Mistake 4: Bitwise vs Logical Operators

The bug:

```
bool a = is_ready();
bool b = can_proceed();
if (a & b) { ... } // bitwise AND -- works only if a and b are 0 or 1
// breaks for general integers
if (a && b) { ... } // logical AND -- correct for boolean conditions
```

The fix: Use `&&` and `||` for boolean logic. Use `&` and `|` for bit manipulation.

Exercises

Exercise 3.1 – Predict the output

```
int a = 17, b = 5;
std::cout << a / b << "\n";
std::cout << a % b << "\n";
std::cout << -a / b << "\n";
std::cout << -a % b << "\n";
```

Answer: 3, 2, -3, -2. C++ truncates toward zero; the sign of % follows the dividend.

Exercise 3.2 – Bit flag operations

Starting from `uint8_t flags = 0b00000000`: 1. Set bit 2 (value 4) 2. Set bit 5 (value 32) 3. Check if bit 2 is set (should be true) 4. Clear bit 2

Write the four operations and show the final value of `flags`.

Answer:

```
uint8_t flags = 0b00000000;

flags |= (1u << 2);           // 1. set bit 2: flags = 0b000000100
flags |= (1u << 5);           // 2. set bit 5: flags = 0b00100100

bool bit2_set = (flags & (1u << 2)) != 0; // 3. check bit 2: true

flags &= ~(1u << 2);          // 4. clear bit 2: flags = 0b00100000
// ~(1u << 2) = ~0b000000100 = 0b11111011
// 0b00100100 & 0b11111011 = 0b00100000 = 32
```

Exercise 3.3 – Floating-point tolerance

Write a function `are_equal(double a, double b)` that returns true when `a` and `b` are within 0.0001 of each other. Test it with `0.1 + 0.2` vs `0.3`.

Answer:

```
#include <cmath>
bool are_equal(double a, double b) {
    return std::abs(a - b) < 0.0001;
}
// are_equal(0.1 + 0.2, 0.3) returns true
// are_equal(0.1 + 0.2, 0.4) returns false
```


Chapter 4: Control Flow: Branching and Loops

if/else if/else

```
# Python
score = 85
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
else:
    grade = "C"
```

```
// C++
int score = 85;
std::string grade;

if (score >= 90) {
    grade = "A";
} else if (score >= 80) {
    grade = "B";
} else {
    grade = "C";
}
```

Three syntax differences: 1. The condition must be in **parentheses**: `if (score >= 90)` not `if score >= 90` 2. The body uses **curly braces** `{}` instead of `:` and indentation 3. **else if** (two words, with a space) not `elif`

Always Use Curly Braces – A Real Cautionary Tale

Braces are technically optional when the body is a single statement:

```
if (error)
    std::cout << "Error occurred\n"; // technically fine
```

But do not do this. The Apple “goto fail” bug (2014) used this pattern and allowed attackers to bypass SSL certificate verification on millions of Apple devices. The vulnerable code looked like:


```
if ((err = SSLHashSHA1.update(&hashCtx, &signedParams)) != 0)
    goto fail;
goto fail; // This ALWAYS executes -- attacker bypasses verification
```

The second `goto fail` is not inside the `if`. The indentation lies. Always use braces.

if with Initializer (C++17)

Declare a variable scoped only to the `if/else` block:

```
// C++ before 2017:
int value = compute();
if (value > 0) {
    use(value);
}
// 'value' still exists here (wasted scope, potential confusion)

// C++17 and later:
if (int value = compute(); value > 0) {
    // ^-- initializer      ^-- condition
    use(value);
} else {
    handle_error(value);
}
// 'value' does not exist here -- scope is clean
```

Use this when the variable is only meaningful within the `if/else`.

switch

`switch` tests one integer or enum value against a list of constants:

```
int day = 3;

switch (day) {
    case 1:
        std::cout << "Monday\n";
        break;
    case 2:
        std::cout << "Tuesday\n";
        break;
    case 3:
        std::cout << "Wednesday\n";
        break;
    case 6:
    case 7:
        std::cout << "Weekend\n"; // cases 6 and 7 share the same body
        break;
    default:
        std::cout << "Weekday\n";
}
```

```
    break;
}
```

Fallthrough: The break Requirement

Without break, execution falls through into the next case:

```
int x = 1;
switch (x) {
    case 1:
        std::cout << "one\n";
        // no break!
    case 2:
        std::cout << "two\n";
        break;
    case 3:
        std::cout << "three\n";
        break;
}
```

Output when `x == 1`: one then two. The control falls through from case 1 into case 2.

This is almost always a bug. Always add break. If fallthrough is intentional, use `[[fallthrough]]` (C++17) to document it and silence the warning:

```
case 'A':
    [[fallthrough]]; // intentional: A and a are treated the same
case 'a':
    process_letter_a();
    break;
```

`switch` only works on integral types (int, char, enum). It does not work on strings or floating-point numbers.

while Loop

Checks the condition before each iteration. If the condition is false initially, the body never runs.

```
# Python
x = 0
while x < 5:
    print(x)
    x += 1
```

```
// C++
int x = 0;
while (x < 5) {
    std::cout << x << "\n";
    ++x;
}
// prints 0, 1, 2, 3, 4
```

do-while Loop

The body runs at least once. The condition is checked after the body.

```
int x = 100;
do {
    std::cout << x << "\n"; // prints 100 even though 100 >= 5
    ++x;
} while (x < 5);
// body ran once; condition was false; loop ends
```

When to use it: Input validation – you always want to ask once, then check.

```
#include <iostream>

int main() {
    int age;
    do {
        std::cout << "Enter your age (1-120): ";
        std::cin >> age;
    } while (age < 1 || age > 120);
    // Guaranteed: asked at least once; keeps asking until valid input

    std::cout << "Age accepted: " << age << "\n";
    return 0;
}
```

Python has no do-while. The equivalent Python idiom:

```
while True:
    age = int(input("Enter your age (1-120): "))
    if 1 <= age <= 120:
        break
```

for Loop: C-Style

```
for (initializer; condition; increment) {
    body;
}
```

```
for (int i = 0; i < 10; ++i) {
    std::cout << i << " ";    // 0 1 2 3 4 5 6 7 8 9
}
// 'i' does not exist after the loop -- scoped to the for
```

Execution order:

```
for (int i = 0; i < 10; ++i)

    int i = 0      <-- runs ONCE before anything
        |
        v
    [check: i < 10?]  ----NO----> exit loop
        |
        YES
        |
        v
    [body: cout << i]
        |
        v
    [++i]
        |
        +-----> [check: i < 10?] (repeat)
```

Key point: the variable declared in the initializer (`int i`) exists only inside the `for` loop. This is better than Python, where loop variables leak out:

```
# Python: loop variable persists after loop
for i in range(5):
    pass
print(i)    # 4 -- still accessible
```

```
// C++: loop variable is gone after loop
for (int i = 0; i < 5; ++i) { }
std::cout << i;    // COMPILE ERROR: 'i' was not declared in this scope
```

Useful patterns:

```
// Count down:
for (int i = 10; i >= 0; --i) {
    std::cout << i << " ";    // 10 9 8 7 6 5 4 3 2 1 0
}
```

```
// Step by 3 (like range(0, 30, 3)):
for (int i = 0; i < 30; i += 3) {
    std::cout << i << " ";    // 0 3 6 9 12 15 18 21 24 27
}

// Multiple variables:
for (int i = 0, j = 10; i < j; ++i, --j) {
    std::cout << i << " " << j << "\n";
}
// 0 10, 1 9, 2 8, 3 7, 4 6
```

Range-Based for Loop (C++11)

The C++ equivalent of Python's `for x in collection`:

```
numbers = [10, 20, 30, 40, 50]
for n in numbers:
    print(n)
```

```
#include <vector>
std::vector<int> numbers = {10, 20, 30, 40, 50};
for (int n : numbers) {
    std::cout << n << "\n";
}
```

The Reference vs Copy Decision

This is the most important thing to understand about range-based `for` in C++.

By value (copy): Each iteration creates a copy. Modifying the loop variable does NOT change the container.

```
std::vector<int> v = {1, 2, 3};
for (int n : v) {
    n *= 10;    // modifies the copy, not the element in v
}
// v is still {1, 2, 3}
```

By reference (&): The loop variable IS the element. Modifying it DOES change the container.

```
for (int& n : v) {
    n *= 10;    // modifies the actual element in v
}
// v is now {10, 20, 30}
```

By const reference (const&): Read-only access to the element, no copy made. Use this for large objects you only need to read.

```
for (const int& n : v) {
    std::cout << n << "\n";    // read-only, no copy
    // n = 0;    // COMPILE ERROR: n is const
}
```

When to use which:

Type is small (int, double, bool, char, pointer):

```
for (T x : v)            -- copy is cheap, fine
for (const T& x : v)    -- also fine
```

Type is large (string, struct, vector, custom class):

```
for (const T& x : v)    -- ALWAYS prefer: no copy, no modify
for (T& x : v)         -- when you need to modify in place
```

You don't know the type or it's complex:

```
for (const auto& x : v) -- safe universal choice
```

break and continue

```
for (int i = 0; i < 10; ++i) {
    if (i == 5) {
        break;        // exit the loop immediately
    }
    if (i % 2 == 0) {
        continue;    // skip to the next iteration
    }
    std::cout << i << " ";    // prints: 1 3
}
// loop ends because of break when i == 5
```

break and continue only affect the **innermost** enclosing loop. For nested loops, use a function return or a flag:

```
// Method: refactor into a function, use return
bool find_target(const std::vector<std::vector<int>>& grid, int target) {
    for (int row = 0; row < (int)grid.size(); ++row) {
        for (int col = 0; col < (int)grid[row].size(); ++col) {
            if (grid[row][col] == target) {
                return true;    // exits both loops
            }
        }
    }
    return false;
}
```

Loop Execution Visualization

For `for (int i = 0; i < 3; ++i) { body; }:`

Start:

`int i = 0` (initialization -- runs once)

Iteration 1:

`i < 3?` YES (`0 < 3`)

body executes

`++i` -> `i = 1`

Iteration 2:

`i < 3?` YES (`1 < 3`)

body executes

`++i` -> `i = 2`

Iteration 3:

`i < 3?` YES (`2 < 3`)

body executes

`++i` -> `i = 3`

Check:

`i < 3?` NO (`3` is not `< 3`)

Loop exits

Common Mistakes in This Chapter

Mistake 1: Off-By-One Error

The bug:

```
std::vector<int> v = {10, 20, 30};
for (int i = 0; i <= v.size(); ++i) {    // <= instead of <
    std::cout << v[i] << "\n";          // accesses v[3] -- does not exist!
}
```

The symptom: Crash, garbage output, or silent memory corruption on the last iteration.

The fix: Use `i < v.size()` (strictly less than). Or better: use range-based `for`, which cannot have this bug.

Mistake 2: Forgetting break in switch

The bug:

```
switch (command) {
    case 'q':
        prepare_to_quit();
        // forgot break -- falls through to 'h'!
    case 'h':
        show_help();
        break;
}
// When command == 'q': prepare_to_quit() runs, THEN show_help() runs
```

The fix: Add break after every case.

Mistake 3: Modifying a Container While Iterating Over It

The bug:

```
std::vector<int> v = {1, 2, 3, 4, 5};
for (int n : v) {
    if (n == 3) v.erase(v.begin() + 2); // modifies v while iterating!
}
// undefined behavior -- the range-based for's internal iterator is now invalid
```

The fix: Collect things to remove first, then remove them after the loop. Or use `std::remove_if` (Chapter 29).

Mistake 4: Using = Instead of == in a Condition

The bug:

```
int x = 5;
while (x = 10) { // assigns 10 to x, condition is always true (infinite loop!)
    std::cout << x << "\n";
}
```

The symptom: Infinite loop.

The fix: `while (x == 10)`. Compile with `-Wall` to get a warning.

Exercises

Exercise 4.1 – Predict the output

```
for (int i = 1; i <= 10; ++i) {  
    if (i % 3 == 0) continue;  
    if (i > 7) break;  
    std::cout << i << " ";  
}
```

Answer: 1 2 4 5 7 – skips multiples of 3 (3, 6), breaks when $i > 7$ (stops before 8).

Exercise 4.2 – FizzBuzz

Print numbers 1 to 20. Multiples of 3: print “Fizz”. Multiples of 5: print “Buzz”. Multiples of both 3 and 5: print “FizzBuzz”.

Answer:

```
#include <iostream>  
int main() {  
    for (int i = 1; i <= 20; ++i) {  
        if (i % 15 == 0) std::cout << "FizzBuzz\n";  
        else if (i % 3 == 0) std::cout << "Fizz\n";  
        else if (i % 5 == 0) std::cout << "Buzz\n";  
        else std::cout << i << "\n";  
    }  
}
```

Check % 15 first. If you check % 3 first, 15 would print “Fizz” instead of “FizzBuzz”.

Exercise 4.3 – Sum with while

Use a `while` loop to compute the sum of integers from 1 to 100.

Answer:

```
int sum = 0, i = 1;  
while (i <= 100) {  
    sum += i;  
    ++i;  
}  
std::cout << sum << "\n"; // 5050
```

Exercise 4.4 – Triangle

Print this triangle with nested loops:

```
*
* *
* * *
* * * *
* * * * *
```

Answer:

```
for (int row = 1; row <= 5; ++row) {
    for (int col = 1; col <= row; ++col) {
        std::cout << "* ";
    }
    std::cout << "\n";
}
```

Exercise 4.5 – do-while validation

Write a program that asks the user to enter a password. Keep asking until they type “secret”. (Use `std::cin >> password` to read.)

Answer:

```
#include <iostream>
#include <string>

int main() {
    std::string password;
    do {
        std::cout << "Enter password: ";
        std::cin >> password;
    } while (password != "secret");
    std::cout << "Access granted.\n";
    return 0;
}
```


Chapter 5: Functions, Overloading, and Declarations vs Definitions

What Happens When You Call a Function

Before the syntax, understand what a function call does to memory. This mental model makes pass-by-value, scope, and the declaration/definition split all click at once.

Your program uses a region of memory called the **call stack**. It operates like a stack of plates: each function call pushes a new **frame** (plate) on top; when the function returns, that frame is popped off and gone.

Each frame holds: - The function's parameters (copies of the arguments) - The function's local variables - The return address (where to continue after returning)

```
int multiply(int a, int b) {
    int result = a * b;
    return result;
}

int square(int x) {
    int s = multiply(x, x);
    return s;
}

int main() {
    int answer = square(5);
    std::cout << answer << "\n"; // 25
}
```

When `square(5)` is called, then `multiply(5, 5)` is called inside `square`:

CALL STACK (top = currently executing function)

+-----+	
multiply's frame	<-- currently executing
a = 5	COPY of square's x
b = 5	COPY of square's x
result = 25	
+-----+	
square's frame	
x = 5	COPY of main's argument 5
s = ?	not yet assigned

```

+-----+
| main's frame          |
|   answer = ?         |   not yet assigned
+-----+

```

When `multiply` finishes: 1. It returns 25 2. Its entire frame is popped – `a`, `b`, `result` are gone from memory 3. Back in `square`, `s` receives the returned 25 4. `square` returns 25, its frame is popped – `x`, `s` are gone 5. In `main`, `answer` receives 25

Critical insight: `a` and `b` inside `multiply` are COPIES of `x` from `square`. They live at different memory addresses. Changing `a` inside `multiply` has absolutely no effect on `x` in `square`. This is **pass by value**.

Function Syntax

```

# Python -- no return type, no parameter types
def add(a, b):
    return a + b

```

```

// C++ -- return type required, parameter types required
int add(int a, int b) {
    return a + b;
}

```

What you must provide: 1. **Return type** before the function name. `int` means the function returns an `int`. 2. **Type for each parameter**. Each parameter is `type name`. 3. A `return` statement if the return type is not `void`.

```

// Returns nothing:
void print_banner() {
    std::cout << "=====\n";
} // no return needed for void

// Returns bool:
bool is_palindrome(std::string s) {
    std::string rev = s;
    std::reverse(rev.begin(), rev.end());
    return s == rev;
}

// Multiple parameters:
double distance(double x1, double y1, double x2, double y2) {
    double dx = x2 - x1;
    double dy = y2 - y1;
    return std::sqrt(dx*dx + dy*dy);
}

// No parameters:
int get_screen_width() {
    return 1920;
}

```

Declarations vs Definitions

This is a concept Python does not have. It exists because C++ compiles each .cpp file independently, top-to-bottom.

The problem: If you call a function before the compiler has seen it, the compiler does not know: - Does this function exist? - How many parameters does it take? - What types are the parameters? - What type does it return?

Without this information, the compiler cannot type-check the call. It refuses.

A declaration (also called a forward declaration or prototype) gives the compiler the function's signature without the body:

```
int add(int a, int b); // declaration: signature only, ends with semicolon
```

A definition is the full function with its body:

```
int add(int a, int b) { // definition: has the body in braces
    return a + b;
}
```

Every definition is implicitly also a declaration. A declaration without a body is not a definition.

The Three Solutions

Problem: function used before it is defined

```
int main() {
    int r = add(3, 4); // ERROR: 'add' not declared yet
}

int add(int a, int b) { return a + b; }
```

Solution 1: Move the definition above the call

```
int add(int a, int b) { return a + b; } // defined first

int main() {
    int r = add(3, 4); // OK -- compiler has seen add
}
```

Works for simple, single-file programs. Becomes unmanageable with circular dependencies (function A calls B, B calls A).

Solution 2: Forward declaration

```
int add(int a, int b);    // declaration -- tells compiler the signature

int main() {
    int r = add(3, 4);    // OK -- compiler knows signature, trusts you to define it
}

int add(int a, int b) { return a + b; }    // definition can appear anywhere after
```

Works in a single file. For multi-file projects, use headers.

Solution 3: Header file (the real solution for multi-file projects)

```
// math.h -- declarations only
#pragma once
int add(int a, int b);
int subtract(int a, int b);

// math.cpp -- definitions
#include "math.h"
int add(int a, int b)    { return a + b; }
int subtract(int a, int b) { return a - b; }

// main.cpp -- uses the functions
#include "math.h"    // gives main.cpp the declarations it needs
int main() {
    int r = add(3, 4);    // compiler checks types via the declaration in math.h
    int s = subtract(10, 3);
}
```

We cover headers fully in Chapter 11.

Pass by Value: What Is Copied and Why It Matters

When you pass a variable to a function, C++ copies it by default. The function gets its own independent copy.

```
void zero_it(int x) {
    x = 0;    // modifies the local copy only
    std::cout << "Inside: " << x << "\n";
}

int main() {
    int n = 42;
    zero_it(n);
    std::cout << "Outside: " << n << "\n";
}
```

Output:

```
Inside: 0
Outside: 42    <-- n was NOT changed
```

The stack during the call:

```

main's frame:                                zero_it's frame:
+-----+                                     +-----+
| n = 42      |                               | x = 42      | <-- a COPY at a different address
| (at 0x7fff1000) |                           | (at 0x7fff0ff0) |
+-----+                                     +-----+
                                         |
                                         | x = 0 (modifies copy)
                                         |
                                         | zero_it returns, frame popped
main's frame:
+-----+
| n = 42      | <-- unchanged
+-----+

```

This applies to every type: `int`, `double`, `std::string`, structs, classes. Passing a `std::string` with 10,000 characters by value copies all 10,000 characters. This is why Chapter 6 (References) and `const&` parameters matter.

Default Parameters

```

# Python
def connect(host, port=8080, timeout=30):
    print(f"Connecting to {host}:{port} (timeout={timeout}s)")

```

```

// C++
void connect(std::string host, int port = 8080, int timeout = 30) {
    std::cout << "Connecting to " << host << ":" << port
               << " (timeout=" << timeout << "s)\n";
}

```

```

connect("localhost");           // port=8080, timeout=30
connect("localhost", 9000);     // port=9000, timeout=30
connect("localhost", 9000, 60); // port=9000, timeout=60

```

Rules: 1. Default parameters must be at the **end** of the parameter list. 2. You cannot skip a middle argument: `connect("host", , 60)` is illegal. 3. Specify defaults in the **declaration** (header file), not in the definition:

```

// math.h -- put defaults here
void connect(std::string host, int port = 8080, int timeout = 30);

// math.cpp -- no defaults here (would be a redefinition error)
void connect(std::string host, int port, int timeout) {
    // implementation
}

```


Function Overloading

C++ lets you define multiple functions with the same name, as long as they have different parameter types. The compiler picks the right one at each call site.

```
int    abs_val(int x)    { return x < 0 ? -x : x; }
double abs_val(double x) { return x < 0.0 ? -x : x; }
float  abs_val(float x)  { return x < 0.0f ? -x : x; }

abs_val(5);           // calls int version
abs_val(-3.14);       // calls double version
abs_val(-2.0f);       // calls float version (f suffix = float literal)
```

The compiler resolves overloads at compile time using these rules (simplified): 1. **Exact match** – preferred 2. **Trivial conversion** (e.g., non-const to const) 3. **Promotion** (e.g., float to double, short to int) 4. **Standard conversion** (e.g., int to double) 5. **Ambiguous** (two overloads equally good) – compile error

```
void foo(int x)      { std::cout << "int\n"; }
void foo(double x)   { std::cout << "double\n"; }

foo(5);             // "int"    -- exact match
foo(5.0);           // "double" -- exact match
foo(5.0f);          // "double" -- float promotes to double (closer than int)
foo('A');           // "int"    -- char promotes to int
// foo(true);       -- ambiguous: bool can promote to int OR double equally well
```

Python does not have overloading. You achieve similar behavior with type checks (`isinstance`) or default parameters. The C++ approach is resolved entirely at compile time with zero runtime cost.

What Overloading Is NOT

Overloading is based on **parameter types only**. You cannot overload on return type:

```
int    get() { return 5; }
double get() { return 5.0; } // ERROR: same parameters, only return type differs
```

The compiler uses the call arguments to pick the overload. If two functions take the same arguments, there is no way to distinguish them.

Compiler Messages for Function Errors

Understanding compiler error messages makes debugging much faster. Here are the most common function-related errors:

No Matching Function

```
void greet(std::string name) { }
```

```
int main() {  
    greet(42);    // passing int, but greet takes string  
}
```

```
error: no matching function for call to 'greet(int)'  
    note: candidate: 'void greet(std::string)'  
    note:   no known conversion from 'int' to 'std::string'
```

The compiler tells you what it found and why it didn't match.

Ambiguous Overload

```
void process(int x) { }
```

```
void process(long x) { }
```

```
process(42);    // 42 is int -- exact match for int version, fine  
process(42L);   // 42L is long -- exact match for long version, fine  
// process(true); -- ambiguous: bool can convert to int OR long
```

```
error: call to 'process' is ambiguous  
    note: candidate: 'void process(int)'  
    note: candidate: 'void process(long)'
```

Wrong Number of Arguments

```
void foo(int a, int b) { }
```

```
foo(1, 2, 3);    // too many
```

```
error: too many arguments to function 'void foo(int, int)'  
    note: declared here: void foo(int a, int b)
```

Common Mistakes in This Chapter

Mistake 1: Expecting Pass-by-Value to Modify the Original

The bug:

```
void set_to_zero(int x) { x = 0; }

int count = 100;
set_to_zero(count);
std::cout << count;    // 100 -- NOT 0
```

Why it fails: `x` is a copy. Setting `x = 0` changes only the copy.

The fix: Use a reference (Chapter 6): `void set_to_zero(int& x) { x = 0; }`

Mistake 2: Missing Return Statement

The bug:

```
int max(int a, int b) {
    if (a > b) return a;
    // forgot: return b;
}
```

The symptom: Compiler warning “control may reach end of non-void function.” Calling `max` when `a <= b` returns garbage.

The fix: All code paths must return a value.

Mistake 3: Defining a Function Twice

The bug:

```
int add(int a, int b) { return a + b; }
int add(int a, int b) { return a + b; } // second definition
```

error: redefinition of 'int add(int, int)'

The fix: Declarations can appear multiple times (they’re just type information). Definitions can appear exactly once.

Mistake 4: Forgetting to Compile the File Containing the Definition

The bug:

```
$ g++ -std=c++23 -o program main.cpp # forgot greet.cpp
```

undefined reference to `greet(std::string const&)'

The fix: Include all .cpp files that contain definitions you need:

```
$ g++ -std=c++23 -o program main.cpp greet.cpp
```

Exercises

Exercise 5.1 – Declaration or definition?

For each, say whether it is a declaration, a definition, or both:

```
double sqrt(double x); // (a)
double square(double x) { return x*x; } // (b)
void print(); // (c)
void print() { std::cout << "!\n"; } // (d)
```

Answer: - (a): declaration only (no body) - (b): definition (has body) – and also implicitly a declaration - (c): declaration only - (d): definition (and implicitly a declaration)

Exercise 5.2 – Stack trace

Draw the call stack when `inner` is executing:

```
int inner(int x)    { return x * 2; }
int middle(int x)   { return inner(x + 1); }
int outer(int x)    { return middle(x + 1); }
int main()          { int r = outer(5); }
```

Answer:

inner's frame: x = 7 (outer 5 -> middle 6 -> inner 7)
middle's frame: x = 6
outer's frame: x = 5
main's frame: r = ?

inner returns $7 * 2 = 14$. Pops. middle returns 14. Pops. outer returns 14. Pops. main's $r = 14$.

Exercise 5.3 – Overloaded describe

Write three overloaded functions named describe that: - Take int and print "Integer: N" - Take double and print "Double: N.NN" - Take std::string and print "String: S (length L)"

Answer:

```
#include <iostream>
#include <string>
#include <iomanip>

void describe(int n) {
    std::cout << "Integer: " << n << "\n";
}

void describe(double d) {
    std::cout << "Double: " << std::fixed << std::setprecision(2) << d << "\n";
}

void describe(std::string s) {
    std::cout << "String: " << s << " (length " << s.size() << ")\n";
}

int main() {
    describe(42);
    describe(3.14159);
    describe(std::string{"hello"});
}
```

Output:

Integer: 42

Double: 3.14

String: hello (length 5)

Exercise 5.4 – power with default

Write power (base, exponent=2) that computes $\text{base}^{\text{exponent}}$ using a loop. It should work for non-negative integer exponents. Verify: power (3) returns 9, power (2, 10) returns 1024.

Answer:

```
double power(double base, int exponent = 2) {
    double result = 1.0;
    for (int i = 0; i < exponent; ++i) {
        result *= base;
    }
    return result;
}
```

Part I is complete. You now have the foundations: the compilation model, types and memory, operators, control flow, and functions.

Part II begins with the concepts that most distinguish C++ from Python: references, pointers, the stack vs the heap, and const correctness. These are where C++ becomes its own language. Ask to continue.

Part II – Core C++ (the part that is not like Python)

The next six chapters cover the concepts that have no real equivalent in Python. Python hides all of this behind its object model and garbage collector. C++ exposes it – and once you understand it, you understand why both languages make the choices they do.

Chapter 6: References – Aliases for Variables

The Problem References Solve

At the end of Chapter 5 you saw that passing a variable to a function copies it. That is usually fine for `int` and `double`. It is expensive for large objects and sometimes flat-out wrong when you need the function to modify the original.

```
// Costs nothing to copy:
void print_count(int n) { std::cout << n << "\n"; }

// Copying a 50,000-byte buffer is painful:
void compress(HugeBuffer data) { ... } // copies 50,000 bytes on every call

// Needs to modify the original -- copy is useless:
void swap(int a, int b) {
    int tmp = a;
    a = b;      // modifies the copy, not main's variables
    b = tmp;
}
int x = 10, y = 20;
swap(x, y);
// x is still 10, y is still 20 -- nothing swapped
```

References solve all three problems.

What Is a Reference?

A reference is an **alias** – another name for an existing variable. It is not a copy. It is not a pointer (though it is implemented as one). It is a second name that refers to the exact same memory location.

```
int x = 42;
int& ref = x; // ref is a reference to x. & after the type = reference.

std::cout << x << "\n"; // 42
std::cout << ref << "\n"; // 42 -- same memory, same value

ref = 100; // modifying ref modifies x (they ARE the same thing)
std::cout << x << "\n"; // 100
std::cout << ref << "\n"; // 100
```

Memory picture:

Before: `ref = x` declared

Address	Name	Value
0x1000	x	42
	\	
	+-- ref is another name for this same location	
		No new memory is allocated for ref itself.
		(The compiler may use a pointer internally,
		but you never see it.)

Rules for references: 1. A reference **must** be initialized when declared. `int& r;` is a compile error. 2. A reference **cannot be reseated** – once bound to a variable, it refers to that variable for its entire life. 3. After initialization, using `ref` is exactly like using the original variable.

```
int a = 1, b = 2;
int& r = a;      // r refers to a
r = b;           // this does NOT make r refer to b
                  // it assigns b's value (2) TO a via r
std::cout << a; // 2
// r still refers to a, not b
```

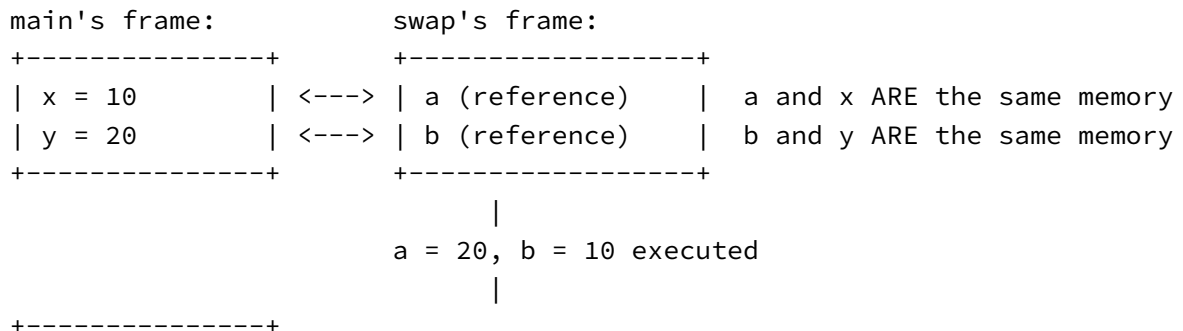
Pass by Reference

To let a function modify its argument, pass it by reference:

```
void swap(int& a, int& b) {    // & in parameter = reference parameter
    int tmp = a;
    a = b;
    b = tmp;
}

int x = 10, y = 20;
swap(x, y);
std::cout << x << " " << y; // 20 10 -- actually swapped
```

Inside `swap`, `a` IS `x` and `b` IS `y`. They share the same memory.



x = 20	x changed through a
y = 10	y changed through b
+-----+	

Contrast with pass-by-value from Chapter 5:

```
void swap_broken(int a, int b) { // no &, copies made
    int tmp = a; a = b; b = tmp; // swaps the copies only
}
swap_broken(x, y); // x and y unchanged
```

Pass by const Reference

When you want to avoid copying a large object but do NOT need to modify it, use const&:

```
// BAD: copies the entire string (potentially many bytes)
void print_name(std::string name) {
    std::cout << name << "\n";
}

// GOOD: no copy, but cannot modify (const ensures this)
void print_name(const std::string& name) {
    std::cout << name << "\n";
    // name = "Bob"; // COMPILE ERROR -- const reference, cannot assign
}
```

const std::string& name means: - & - reference (no copy) - const - cannot be modified through this reference

The caller's string is not copied. The function gets direct read-only access to the original. This is the most common parameter style for anything larger than a few bytes.

The Decision Table for Parameter Style

Parameter type	When to use
int x	Small types (int, double, bool, char, pointer) that are cheap to copy and you don't need to modify.
int& x	Small or large type that the function MUST modify. (signals to the caller: "I will change your variable")
const std::string& x	Large type (string, vector, struct) that you only need to read, not modify. No copy. Fastest.
std::string& x	Large type that the function must modify. (rare -- usually return a new value instead)

References Cannot Be Null

Unlike pointers (next chapter), a reference is guaranteed to refer to a valid object. You cannot have a “null reference” in well-formed C++.

```
int& bad_ref = *nullptr;    // undefined behavior -- DO NOT DO THIS
                          // dereferencing null pointer gives "reference to nothing"
```

If validity is uncertain at the time of binding, use a pointer instead. References are for when you know the object exists.

Returning References

A function can return a reference to give the caller direct access to something inside the function... but only if that thing outlives the function call.

```
// CORRECT: returning a reference to a member of an object that outlives the call
int& get_element(std::vector<int>& v, int index) {
    return v[index];    // v exists outside this function; safe
}

std::vector<int> nums = {10, 20, 30};
get_element(nums, 1) = 99;    // assigns through the returned reference
// nums is now {10, 99, 30}
```

```
// WRONG: returning a reference to a local variable
int& bad() {
    int x = 5;
    return x;    // x is destroyed when bad() returns -- dangling reference!
}

int& r = bad();    // r refers to memory that no longer exists
r = 10;           // undefined behavior -- corrupts memory
```

The compiler often warns about this:

```
warning: reference to local variable 'x' returned [-Wreturn-local-addr]
   7 |     return x;
     |         ^
```

Common Mistakes in This Chapter

Mistake 1: Expecting pass-by-reference from a non-reference parameter

The bug:

```
void double_it(int n) { n *= 2; }    // n is a copy
int x = 5;
double_it(x);
std::cout << x;    // 5, not 10
```

The fix: `void double_it(int& n) { n *= 2; }`

Mistake 2: Forgetting `const` on read-only reference parameters

The bug:

```
void print_sum(std::string& s) {    // non-const reference
    std::cout << s << "\n";
}
print_sum("hello");    // COMPILE ERROR: cannot bind non-const reference to rvalue
```

Why it fails: Temporary values (like string literals) cannot bind to non-const references because the compiler cannot take their address in a meaningful way. **The fix:** `void print_sum(const std::string& s)`

Mistake 3: Returning a reference to a local variable

The bug: As shown above – the local variable is destroyed on return, leaving a dangling reference. **The fix:** Only return references to objects that outlive the function (parameters, class members, statics).

Exercises

Exercise 6.1 – Trace the output

```
void increment(int& x) { ++x; }

int a = 5;
int& b = a;
increment(b);
std::cout << a << " " << b;
```

Answer: 6 6. `b` is a reference to `a`, so they are the same variable. `increment(b)` increments `a` (via `b` via the parameter `x`). Both names show the new value.

Exercise 6.2 – Fix the swap

```
void swap(double a, double b) {  
    double tmp = a;  
    a = b;  
    b = tmp;  
}  
double x = 1.5, y = 2.5;  
swap(x, y);  
// x and y are unchanged -- fix swap so they actually swap
```

Answer:

```
void swap(double& a, double& b) {  
    double tmp = a;  
    a = b;  
    b = tmp;  
}
```

Exercise 6.3 – const& parameter

Write a function `print_stats(const std::vector<int>& v)` that prints the size, first element, and last element of a vector. It should not copy the vector.

Answer:

```
#include <iostream>  
#include <vector>  
  
void print_stats(const std::vector<int>& v) {  
    std::cout << "Size: " << v.size() << "\n";  
    std::cout << "First: " << v.front() << "\n";  
    std::cout << "Last: " << v.back() << "\n";  
}  
  
int main() {  
    std::vector<int> nums = {10, 20, 30, 40, 50};  
    print_stats(nums); // no copy of nums made  
}
```

Chapter 7: Pointers and Memory Addresses

The Address of a Variable

Every variable lives at a specific location in memory – a numbered address. On a 64-bit system, addresses are 64-bit numbers, usually displayed in hexadecimal.

```
int x = 42;
std::cout << &x << "\n"; // prints something like: 0x7ffee4b3c5ac
```

The `&` operator (when used in an expression, not a declaration) is the **address-of** operator. It gives you the memory address where a variable lives.

Variable `x`:

Address	Value
0x7fff10	[00][00][00][2A] <- <code>&x</code> is 0x7fff10; <code>*(&x)</code> is 42

A **pointer** is a variable that stores an address. The `*` in the type means “pointer to”:

```
int x = 42;           // x is an int
int* p = &x;          // p is a pointer to int, stores x's address

std::cout << p << "\n"; // 0x7fff10 -- the address (the pointer value)
std::cout << *p << "\n"; // 42      -- the value at that address (dereference)
std::cout << &p << "\n"; // 0x7fff08 -- address of the pointer itself
```

The `*` when used with an existing pointer is the **dereference** operator – it follows the address to get the value stored there.

Memory picture:

Address	Variable	Value
0x7fff10	<code>x (int)</code>	42
0x7fff08	<code>p (int*)</code>	0x7fff10 <- <code>p</code> stores <code>x</code> 's address

`p` is the address 0x7fff10.

`*p` follows that address and gives 42.

Modifying Through a Pointer

```
int x = 42;
int* p = &x;

*p = 100; // write 100 to the memory location p points to
          // same as: x = 100

std::cout << x; // 100
std::cout << *p; // 100
```

This is the lower-level version of references. Pointers and references both allow indirect modification, but:

	Reference	Pointer
Syntax	<code>int& r = x</code>	<code>int* p = &x</code>
Must be initialized	Yes	No (but you should)
Can be null	No	Yes (<code>nullptr</code>)
Can be reseated	No	Yes
Dereference syntax	just use <code>r</code>	<code>*p</code>
When to use	Aliases, function parameters	Optional relationships, arrays, dynamic memory

nullptr: The Null Pointer

A pointer that points to nothing is called a null pointer. Always use `nullptr` (C++11), never `NULL` (old C macro) or `0`:

```
int* p = nullptr; // p points to nothing

if (p != nullptr) {
    std::cout << *p; // safe -- only dereference if not null
}

// Dereferencing a null pointer is undefined behavior (usually a crash):
int* q = nullptr;
std::cout << *q; // CRASH -- segmentation fault
```

Null pointer:

Address	Variable	Value
0x7fff08	p (int*)	0x0000000000000000 <- nullptr (address zero)

Dereferencing: *p means "go to address 0 and read from there"
 OS protects address 0 -- your program gets SIGSEGV (segfault)

Always check pointers before dereferencing if they might be null.

Pointer Arithmetic

Pointers support arithmetic that advances by the size of the pointed-to type:

```
int arr[5] = {10, 20, 30, 40, 50};
int* p = arr;    // p points to arr[0]

std::cout << *p      << "\n";    // 10
std::cout << *(p + 1) << "\n";    // 20 (moves 4 bytes forward, one int)
std::cout << *(p + 2) << "\n";    // 30

p++;              // p now points to arr[1]
std::cout << *p      << "\n";    // 20
```

Memory (each int is 4 bytes):

Address	Value
0x1000	10 <- arr[0], p points here initially
0x1004	20 <- arr[1], p+1 points here
0x1008	30 <- arr[2]
0x100C	40 <- arr[3]
0x1010	50 <- arr[4]

p + 1 moves by sizeof(int) = 4 bytes, landing on the next integer. This is why arrays are contiguous in memory – pointer arithmetic only works correctly because of that contiguity.

Pointers vs Python References

In Python, every variable is a reference (pointer) to an object. Python programmers are already using pointers – they just don't see the mechanics.

```
# Python: all "variables" are pointers
a = [1, 2, 3]
b = a          # b and a point to the SAME list
b.append(4)
print(a)       # [1, 2, 3, 4] -- a was affected through b

# Python explicitly shows this with id():
print(id(a) == id(b)) # True -- same memory address
```

```
// C++: copying by default, pointer explicitly
std::vector<int> a = {1, 2, 3};
std::vector<int> b = a;          // b is a COPY -- different memory
b.push_back(4);
std::cout << a.size();          // 3 -- a is unaffected

std::vector<int>* p = &a;        // p points to a
p->push_back(4);                  // modifies a through the pointer
std::cout << a.size();          // 4
```

In Python, `b = a` for a list makes two labels for one object. In C++, `b = a` copies everything. To share, you explicitly use a pointer or reference.

The `->` Operator

When you have a pointer to an object and want to access its members, `->` is shorthand for `(*p).member`:

```
struct Point {
    double x, y;
};

Point pt{3.0, 4.0};
Point* p = &pt;

std::cout << (*p).x << "\n"; // 3.0 -- dereference then member access
std::cout << p->x << "\n";   // 3.0 -- same thing, cleaner syntax
p->x = 10.0;                  // modifies pt.x through the pointer
```

`->` is the idiomatic way to access members through a pointer. You will see it constantly in C++ code.

const and Pointers

There are two things that can be `const` with a pointer: the pointer itself (where it points) and the value it points to.

```

int a = 1, b = 2;

// Pointer to const int: cannot change the pointed-to value
const int* p1 = &a;
*p1 = 10;    // ERROR: *p1 is const
p1 = &b;     // OK: p1 itself can be changed (points to b now)

// Const pointer to int: cannot change where it points
int* const p2 = &a;
*p2 = 10;    // OK: the int it points to can be changed
p2 = &b;     // ERROR: p2 is const (cannot reset)

// Const pointer to const int: cannot change either
const int* const p3 = &a;
*p3 = 10;    // ERROR
p3 = &b;     // ERROR

// Memory trick: read the type right-to-left
// const int*      --> pointer to const int
// int* const      --> const pointer to int
// const int* const --> const pointer to const int

```

Common Mistakes in This Chapter

Mistake 1: Dereferencing a Null or Uninitialized Pointer

The bug:

```

int* p;    // uninitialized -- contains garbage address
*p = 5;    // writes to garbage address -- undefined behavior

```

The symptom: Immediate crash (segfault), or silent memory corruption that crashes later.

The fix: Always initialize pointers: `int* p = nullptr;` or `int* p = &some_variable;`

Mistake 2: Using a Pointer After the Pointed-To Variable is Destroyed

The bug:

```

int* get_ptr() {
    int local = 42;
    return &local; // local is destroyed when function returns
}
int* p = get_ptr(); // p now points to freed stack memory
std::cout << *p;    // undefined behavior -- dangling pointer

```

The fix: Never return a pointer to a local variable. Return the value, or make the variable static, or allocate on the heap (Chapter 8).

Mistake 3: Confusing & as Address-Of vs & as Reference Type

The confusion:

```
int x = 5;
int& r = x;    // & in declaration = reference type
int* p = &x;    // & in expression = address-of operator
```

The position matters: & after a type in a declaration means “reference.” & before a variable in an expression means “take the address of.”

Exercises

Exercise 7.1 – Trace the pointer

```
int a = 10, b = 20;
int* p = &a;
*p = 30;
p = &b;
*p += 5;
std::cout << a << " " << b;
```

Answer: 30 25. Step by step: `*p = 30` sets a to 30. `p = &b` makes p point to b. `*p += 5` adds 5 to b (20 + 5 = 25).

Exercise 7.2 – Pointer parameter

Rewrite `double_it` to take a pointer parameter instead of a reference:

```
void double_it(int* p) {
    *p *= 2;
}
int x = 7;
double_it(&x);    // must pass address explicitly
std::cout << x;    // 14
```

Which style (reference or pointer) is more idiomatic in modern C++ for this use case? Why?

Answer: Reference is more idiomatic. It is simpler (`double_it(x)` vs `double_it(&x)`), cannot be null, and cannot be reseated. Pointers are used when nullability or reseating is needed.

Exercise 7.3 – const correctness with pointers

Declare a pointer p to `double` such that: - The value at p cannot be changed through p - p itself can be pointed at a different `double`

Then declare q such that: - q is fixed to always point to the same `double` - The value at q CAN be changed through q

Answer:

```
double a = 1.0, b = 2.0;
const double* p = &a;    // ptr to const double
p = &b;                  // OK -- p can be reseated
// *p = 5.0;             // ERROR -- cannot modify through p

double* const q = &a;    // const pointer to double
*q = 5.0;                // OK -- can modify value
// q = &b;               // ERROR -- cannot resear q
```


Chapter 8: The Stack and the Heap

Two Regions of Memory

Every running program has two main regions where variables can live:

+-----+	
Stack	
- Fast (just moves a pointer)	
- Automatic (compiler manages lifetime)	
- Limited size (~1-8 MB)	
- Local variables, function parameters	
+-----+	
(other memory: code, globals...)	
+-----+	
Heap	
- Slower (OS/allocator involved)	
- Manual (YOU manage lifetime in C++)	
- Huge (limited by RAM)	
- Dynamic allocation: new, malloc	
+-----+	

Python hides this entirely – all objects live on the heap, managed by the garbage collector. C++ exposes both regions and lets you choose where memory comes from.

The Stack in Detail

The stack is a LIFO (Last In, First Out) structure. Local variables and function parameters live here.

```
void bar() {
    int z = 30;
    // z lives on the stack while bar() is running
}

void foo() {
    int y = 20;
    bar();    // bar's frame pushed on top of foo's
    // z is gone (bar's frame popped); y is back on top
}
```



```
int main() {
    int x = 10;
    foo();
    // y is gone; x is still here
}
```

Call stack during bar():

High addresses	+-----+	
	main's frame	
	x = 10	
	+-----+	
	foo's frame	
	y = 20	
	+-----+	
	bar's frame	
	z = 30	
Low addresses	+-----+	<- stack pointer (SP) is here

After bar() returns:

+-----+
main's frame
x = 10
+-----+
foo's frame
y = 20
+-----+

(bar's memory freed -- SP moved up)

Allocating stack memory is nearly free: the CPU just decrements the stack pointer register by the number of bytes needed. Freeing stack memory is also free: the CPU increments the stack pointer back.

Stack overflow: If you allocate too much on the stack (e.g., declaring a 10MB array locally, or infinitely deep recursion), the stack runs into other memory and the OS kills your program.

```
// Stack overflow example:
void recurse() { recurse(); } // infinite recursion
recurse(); // segfault when stack space exhausted

// Stack overflow with large local array:
void dangerous() {
    int huge[2000000]; // ~8 MB on the stack -- likely crashes
}
```

The Heap in Detail

The heap is a large pool of memory managed by the OS and your allocator. You request chunks with `new` and release them with `delete`.

```
int* p = new int{42};           // allocates 4 bytes on the heap; p stores the address
std::cout << *p << "\n";      // 42
delete p;                       // releases the 4 bytes back to the allocator
p = nullptr;                   // good practice: prevents accidental reuse
```

After `int* p = new int{42}`:

Stack:		Heap:
+-----+		+-----+
p = 0x555f2a1b10	----->	42 (4 bytes)
+-----+		+-----+

After `delete p`:

Stack:		Heap:
+-----+		(memory returned to heap pool)
p = 0x555f2a1b10	---X-->	(no longer valid to access)
+-----+		

After `p = nullptr`:

Stack:		Heap:
+-----+		
p = 0x0		(safe -- cannot accidentally dereference)
+-----+		

Heap Arrays

```
int n = 1000;
int* arr = new int[n]{};           // allocate n ints on the heap, zero-initialized
arr[0] = 10;
arr[n-1] = 99;

delete[] arr; // MUST use delete[] for arrays (not delete)
arr = nullptr;
```

The size does not need to be a compile-time constant – you can compute it at runtime. This is the main use case for heap allocation.

Why Python Uses the Heap for Everything

Python allocates every object on the heap and uses reference counting to track when objects can be freed. This is why Python is flexible but slower:

```
x = [1, 2, 3]    # list allocated on heap
y = x           # y is another reference to the same heap object
                # reference count goes from 1 to 2
del x           # reference count goes from 2 to 1 -- NOT freed
del y           # reference count goes from 1 to 0 -- freed!
```

C++ gives you the choice: - Stack: fast, automatic, size known at compile time - Heap: flexible, manual (or smart pointers), any size

```
// Stack -- automatic lifetime, fast
std::vector<int> v = {1, 2, 3};    // v is on the stack
// vector's CONTENTS are on the heap (internally), but you don't manage that
// v is automatically destroyed when it goes out of scope

// Heap -- manual lifetime (rare in modern C++ -- use smart pointers instead)
std::vector<int>* vp = new std::vector<int>{1, 2, 3};
delete vp;    // you must remember to do this
```

Memory Leaks

A memory leak is heap memory that was allocated but never freed. In C++, the OS reclaims all memory when the process exits, so short-lived programs don't suffer from leaks permanently. But long-running programs (servers, games) slowly consume more and more memory until they crash or the machine runs out.

```
void leaky() {
    int* p = new int{42};    // allocates heap memory
    // forgot: delete p;
    // function returns -- p (the pointer) is gone
    // the heap memory at p's address is still allocated, now unreachable
}

for (int i = 0; i < 1000000; ++i)
    leaky();    // leaks 4 bytes per call = 4 MB leaked
```

Detection: Run with `valgrind ./program` or compile with `-fsanitize=address`:

```
==12345== LEAK SUMMARY:
==12345==    definitely lost: 4 bytes in 1 blocks
==12345==    at 0x4C2FB0F: operator new(unsigned long) (vg_replace_malloc.c:334)
==12345==    by 0x10868B: leaky() (example.cpp:2)
```

The real fix is to never use raw `new/delete`. Use `std::vector`, `std::string`, and smart pointers (Chapter 14), which manage heap memory automatically through RAII.

Stack vs Heap: Which to Use

Use the stack (local variables) when:

- Size is known at compile time
- Lifetime matches the current scope
- Performance is critical (tight loops, small structures)

Examples: `int`, `double`, small structs, `std::array<int, 10>`

Use the heap (via containers or smart pointers) when:

- Size is determined at runtime (user input, file content)
- Lifetime must extend beyond the creating function
- Data is very large (millions of elements)

Examples: `std::vector`, `std::string`, objects stored in maps

In modern C++:

- You almost never write `new/delete` directly
- `std::vector` and `std::string` manage their heap memory internally
- smart pointers (`unique_ptr`, `shared_ptr`) handle ownership automatically
- Raw `new/delete` is a code smell in modern C++

Common Mistakes in This Chapter

Mistake 1: `delete` Instead of `delete[]` for Arrays

The bug:

```
int* arr = new int[100];  
delete arr;    // undefined behavior! Should be delete[]
```

The symptom: Memory corruption or crash – the allocator uses the wrong size to free. **The fix:** `delete[] arr;`

Mistake 2: Double Delete

The bug:

```
int* p = new int{5};  
delete p;  
delete p;    // undefined behavior -- freeing already-freed memory
```

The symptom: Crash or silent heap corruption. **The fix:** Set pointer to `nullptr` after deleting. Deleting `nullptr` is a no-op.

Mistake 3: Using After Delete (Use-After-Free)

The bug:

```
int* p = new int{5};
delete p;
std::cout << *p;    // reads freed memory -- undefined behavior
```

Detection: `-fsanitize=address` reports “heap-use-after-free” with a stack trace.

Exercises

Exercise 8.1 – Where does it live?

For each variable, say whether it lives on the stack or heap:

```
int a = 5;
int* b = new int{10};
std::string s = "hello";
std::vector<int> v = {1, 2, 3};
```

Answer: - a: stack - b: stack (the pointer). The `int` it points to: heap. - s: stack (the `std::string` object itself). The character data it manages: heap (internally). - v: stack (the `std::vector` object). The `int` elements it manages: heap (internally).

Exercise 8.2 – Spot the leak

How many memory leaks does this code have?

```
void process() {
    int* a = new int{1};
    int* b = new int{2};
    if (*a > 0) return;    // early return!
    delete a;
    delete b;
}
```

Answer: Two leaks when `*a > 0` (which is always true here, since `a = 1`). The early return bypasses both `delete` calls. This is exactly why RAII (Chapter 12) and smart pointers (Chapter 14) exist – they clean up even when exceptions or early returns happen.

Exercise 8.3 – Heap array

Allocate an array of 5 doubles on the heap, set them to 1.1, 2.2, 3.3, 4.4, 5.5, print them, then free the memory correctly.

Answer:

```
double* arr = new double[5]{1.1, 2.2, 3.3, 4.4, 5.5};
for (int i = 0; i < 5; ++i)
    std::cout << arr[i] << "\n";
delete[] arr;
arr = nullptr;
```


Chapter 9: const Correctness

Why const Matters

`const` is not just a safety net for your own mistakes. It is a contract: you tell the compiler “this value will not change,” and the compiler enforces that contract everywhere the value is used. This contract propagates through the codebase, letting the compiler catch bugs at compile time that would otherwise be silent data corruption at runtime.

In Python, there is no `const`. You use naming conventions (ALL_CAPS) to signal “don’t change this,” but nothing enforces it. C++’s `const` is enforced.

const Variables

```
const int MAX_PLAYERS = 8;
MAX_PLAYERS = 10;    // COMPILE ERROR: assignment of read-only variable
```

The compiler rejects any assignment to `MAX_PLAYERS` after initialization. Every future reader of the code knows with certainty: this value never changes.

```
const double PI = 3.14159265358979;
const std::string GREETING = "Hello";
```

Prefer `const` for anything that does not need to change. It is documentation that is enforced.

const Function Parameters

A `const` parameter promises not to modify the argument:

```
void print_length(const std::string& s) {
    std::cout << s.size() << "\n";
    s = "modified";    // COMPILE ERROR -- s is const reference
}
```

This is important for `const&` parameters: you can pass temporaries (rvalues) to them:


```

void print_length(const std::string& s) { ... }

print_length("hello");           // OK: string literal binds to const&
print_length(std::string{"hi"}); // OK: temporary binds to const&

void modify(std::string& s) { ... }
modify("hello");                 // ERROR: non-const reference cannot bind to temporary

```

const Member Functions

When you write a class, member functions that do not modify the object's state should be marked `const`. This is the `const` at the end of the function signature:

```

class Rectangle {
public:
    Rectangle(double w, double h) : width{w}, height{h} {}

    double area() const {           // const: does not modify this object
        return width * height;
    }

    void scale(double factor) {     // non-const: modifies this object
        width *= factor;
        height *= factor;
    }

private:
    double width, height;
};

const Rectangle r{3.0, 4.0};
std::cout << r.area();           // OK -- area() is const, usable on const objects
r.scale(2.0);                    // ERROR -- scale() is non-const, not usable on const objects

```

The `const` after the parameter list means: “this function can be called on `const` objects, and it promises not to modify the object.”

Rules: - On a `const` object, only `const` member functions can be called - A non-`const` object can call both `const` and non-`const` member functions - Inside a `const` member function, you cannot assign to member variables

```

double area() const {
    width = 0; // COMPILE ERROR -- modifying member in const function
    return width * height;
}

```

constexpr – Compile-Time Constants

constexpr goes further than const – the value must be computable at compile time:

```
constexpr int BOARD_SIZE = 8;
constexpr double TAX_RATE = 0.085;
constexpr int CELLS = BOARD_SIZE * BOARD_SIZE; // 64, computed at compile time

// constexpr functions:
constexpr int factorial(int n) {
    return n <= 1 ? 1 : n * factorial(n - 1);
}

constexpr int FACT_10 = factorial(10); // 3628800, computed at compile time
// The result is embedded in the binary as a constant -- zero runtime cost
```

```
// Cannot be constexpr if value comes from runtime:
int n;
std::cin >> n;
constexpr int x = n; // ERROR: n is not a compile-time constant
const int y = n; // OK: const, but runtime value
```

constexpr is useful for: - Mathematical constants (PI, E, conversion factors) - Array sizes (int grid[BOARD_SIZE][BOARD_SIZE]; – array size must be compile-time) - Lookup tables computed at compile time - Performance: zero-cost named constants

Cascading const

const propagates: once you have a const object, everything you do with it must also be const.

```
std::vector<int> v1 = {1, 2, 3};
const std::vector<int> v2 = {4, 5, 6};

v1.push_back(4); // OK -- v1 is not const
v2.push_back(7); // ERROR -- v2 is const; push_back is non-const

int x = v1[0]; // OK
int y = v2[0]; // OK -- operator[] has a const overload (returns const ref)
v2[0] = 99; // ERROR -- v2[0] returns const int&, cannot assign
```

When you pass by const&, you can only call const member functions on the object. The compiler checks the entire call chain.

Common Mistakes in This Chapter

Mistake 1: Omitting `const` on Member Functions That Should Be `const`

The bug:

```
class Circle {
    double radius;
public:
    double area() { // forgot const
        return 3.14159 * radius * radius;
    }
};

const Circle c{5.0};
c.area(); // ERROR: 'this' argument discards qualifiers (area is non-const)
```

The fix: `double area() const { ... }` – add `const` after the parameter list.

Mistake 2: Trying to Modify Through a `const` Reference

The bug:

```
void process(const std::string& s) {
    s += " world"; // ERROR: s is const, cannot modify
}
```

The fix: Take by value (`std::string s`) if you need a modifiable copy, or by non-const reference (`std::string& s`) if you intend to modify the original.

Exercises

Exercise 9.1 – Mark `const` correctly

Which of these should be `const` members?

```
class Counter {
    int count = 0;
public:
    void increment() { ++count; }
    int get_count() { return count; }
    void reset() { count = 0; }
    bool is_zero() { return count == 0; }
};
```

Answer: `get_count()` and `is_zero()` should be `const` – they read but do not modify `count`. `increment()` and `reset()` modify `count`, so they cannot be `const`.

```
void increment()      { ++count; }  
int  get_count() const { return count; }  
void reset()          { count = 0; }  
bool is_zero()  const { return count == 0; }
```

Exercise 9.2 – constexpr table

Write a constexpr function `celsius_to_fahrenheit(double c)` and use it to create a compile-time constant for the boiling point of water (100°C).

Answer:

```
constexpr double celsius_to_fahrenheit(double c) {  
    return c * 9.0 / 5.0 + 32.0;  
}  
constexpr double WATER_BOILING_F = celsius_to_fahrenheit(100.0); // 212.0
```


Chapter 10: Arrays, `std::vector`, and `std::string`

C-Style Arrays (Understand, then Avoid)

The C language had arrays built in. C++ inherited them. They are the underlying model behind everything else, so understand them, then use `std::vector` instead.

```
int scores[5];           // array of 5 ints (uninitialized -- garbage!)
int primes[5] = {2, 3, 5, 7, 11}; // initialized
int zeros[100] = {};      // all zeros (zero-initialized with {})

std::cout << primes[0] << "\n"; // 2 -- zero-indexed
std::cout << primes[4] << "\n"; // 11
```

Memory layout of `primes[5]`:

Address	Value	
0x1000	2	<- primes[0]
0x1004	3	<- primes[1]
0x1008	5	<- primes[2]
0x100C	7	<- primes[3]
0x1010	11	<- primes[4]

The name 'primes' in expressions decays to a pointer to `primes[0]`.

Critical Weakness: No Bounds Checking

C++ does not check if your index is valid at runtime:

```
int arr[3] = {1, 2, 3};
std::cout << arr[5]; // reads memory past the array -- undefined behavior
arr[5] = 99;        // writes past the array -- corrupts other data!
```

This is one of the leading causes of security vulnerabilities. The compiler doesn't catch it and there is no runtime exception – it just reads or writes whatever is at that address.

Size Is Not Carried With the Array

```
int arr[5] = {1, 2, 3, 4, 5};
sizeof(arr);    // 20 -- size in bytes (only works if arr is in scope as an array)

void bad_print(int arr[]) {    // array decays to pointer when passed to function
    sizeof(arr);              // 8 -- size of pointer, NOT the array!
}
```

You must pass the size separately, which is error-prone. `std::vector` solves this.

`std::vector` – The Right Way to Do Dynamic Arrays

`std::vector` is C++'s resizable array. It is what Python's `list` most closely corresponds to:

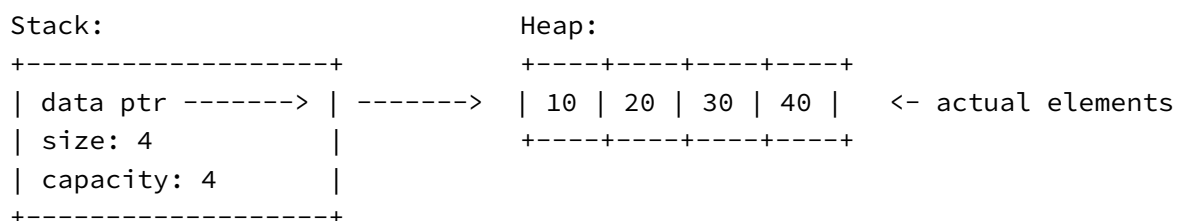
```
# Python list
numbers = [10, 20, 30]
numbers.append(40)
print(len(numbers))    # 4
print(numbers[1])      # 20
```

```
// C++ vector
#include <vector>
std::vector<int> numbers = {10, 20, 30};
numbers.push_back(40);
std::cout << numbers.size() << "\n";    // 4
std::cout << numbers[1] << "\n";        // 20
```

What's Inside `std::vector`

A `std::vector` is a small object (24 bytes typically) that internally manages a heap-allocated array:

```
std::vector<int> numbers = {10, 20, 30, 40};
```



- `size`: number of elements currently in the vector
- `capacity`: how many elements fit in the currently allocated heap space
- When `size == capacity` and you `push_back`, the vector allocates a bigger heap array (typically doubles), copies all elements, and frees the old array.

You never see any of this. The vector manages it transparently.

Key Operations

```
std::vector<int> v;           // empty vector
v.push_back(10);             // append 10: v = {10}
v.push_back(20);             // append 20: v = {10, 20}
v.push_back(30);             // append 30: v = {10, 20, 30}

v.size();                    // 3 -- number of elements
v.empty();                   // false -- is it empty?
v.front();                   // 10 -- first element
v.back();                    // 30 -- last element
v.pop_back();                // removes last: v = {10, 20}

v[0];                        // 10 -- no bounds check (unsafe, fast)
v.at(0);                     // 10 -- bounds-checked (throws if out of range)

v.insert(v.begin() + 1, 99); // insert 99 at index 1: v = {10, 99, 20}
v.erase(v.begin() + 1);      // remove element at index 1: v = {10, 20}

v.clear();                   // remove all elements (size = 0, capacity kept)
v.resize(5, 0);              // resize to 5 elements, new ones initialized to 0

std::vector<int> w(10, 42);   // 10 elements all set to 42
```

Iterating Over a Vector

```
std::vector<int> v = {1, 2, 3, 4, 5};

// Range-based for (preferred):
for (int n : v)          std::cout << n << " "; // copies each element
for (int& n : v)          n *= 2;                // modifies in place
for (const int& n : v)    std::cout << n << " "; // read-only, no copy

// Index-based (when you need the index):
for (int i = 0; i < (int)v.size(); ++i) {
    std::cout << i << ": " << v[i] << "\n";
}

// Note: v.size() returns size_t (unsigned). Comparing int i < size_t is
// technically a warning. Cast to (int)v.size() or use size_t i.
```

2D Vectors

```
// 3 rows, 4 columns, all initialized to 0:
std::vector<std::vector<int>> grid(3, std::vector<int>(4, 0));

grid[1][2] = 99;
std::cout << grid[1][2]; // 99
```


std::string – The Right Way to Handle Text

```
# Python strings
s = "hello"
s += " world"
print(len(s))      # 11
print(s.upper())   # HELLO WORLD
print(s[1:5])      # ello
```

```
// C++ strings
#include <string>
std::string s = "hello";
s += " world";
std::cout << s.size() << "\n";      // 11
// (no built-in upper() -- use a loop or std::transform)

std::string sub = s.substr(1, 4);     // "ello" (start=1, length=4)
```

std::string internally manages a heap-allocated buffer of characters, similar to std::vector<char>.

Key Operations

```
std::string s = "Hello, World!";

s.size();           // 13 -- number of characters
s.empty();          // false
s[0];               // 'H' -- no bounds check
s.at(0);            // 'H' -- bounds-checked
s.front();          // 'H'
s.back();           // '!'

s.find("World");    // 7 -- index where found
s.find("xyz");      // std::string::npos -- not found

s.substr(7, 5);     // "World" (start=7, length=5)

s.replace(7, 5, "C++"); // "Hello, C++!"

std::string t = "hello";
s.compare(t);       // negative (s < t lexicographically)

// Concatenation:
std::string a = "foo", b = "bar";
std::string c = a + b;      // "foobar"
a += "!";                  // "foo!"

// Convert to/from numbers:
#include <string>
std::string num_str = std::to_string(42); // "42"
int n = std::stoi("123");                // 123
double d = std::stod("3.14");             // 3.14
```

String Comparison

Unlike Python where `==` compares value, C++ `std::string` also uses `==` for value comparison:

```
std::string a = "hello", b = "hello";  
if (a == b) std::cout << "equal\n";    // equal -- compares content
```

Raw C-Style Strings

You will see C-style strings (`const char*`) in older code and C interfaces:

```
const char* cs = "hello";    // points to read-only character array  
std::string s = cs;          // convert: C-string to std::string (fine)  
  
// std::string to C-string (for C library functions):  
const char* c = s.c_str();   // null-terminated char array  
// valid only as long as s is alive and unmodified
```

Always use `std::string` for your own code. Use `const char*` only when a C API requires it, and immediately pass it through – do not store it.

std::array – Fixed-Size Array With Safety

When the size is known at compile time, use `std::array` instead of a C-style array:

```
#include <array>  
std::array<int, 5> primes = {2, 3, 5, 7, 11};  
  
primes.size();    // 5 -- size is always available  
primes[2];        // 5  
primes.at(10);    // throws std::out_of_range -- bounds checked  
primes.front();   // 2  
primes.back();    // 11
```

`std::array` knows its own size, can be passed to functions without separately passing the size, and supports all the standard iteration patterns.

Common Mistakes in This Chapter

Mistake 1: Off-By-One / Out-of-Bounds Access

The bug:

```
std::vector<int> v = {1, 2, 3};
for (int i = 0; i <= (int)v.size(); ++i)    // <= instead of <
    std::cout << v[i] << "\n";           // v[3] is past the end
```

Detection: –fsanitize=address reports “heap-buffer-overflow.” **The fix:** Use < not <=. Better: use range-based for.

Mistake 2: Modifying a Vector While Iterating With Indices

The bug:

```
std::vector<int> v = {1, 2, 3, 4, 5};
for (int i = 0; i < (int)v.size(); ++i) {
    if (v[i] % 2 == 0) v.erase(v.begin() + i);
    // After erasing index 1 (value 2), index 1 now holds 3 (shifted)
    // The loop increments i to 2, skipping 3
}
```

The fix: Iterate backward, or use the erase-remove idiom (Chapter 29).

Mistake 3: Using + to Concatenate Non-String Literals

The bug:

```
std::string result = "Count: " + 42;    // ERROR: no + between const char* and int
```

The fix:

```
std::string result = "Count: " + std::to_string(42);
// or better (C++23):
std::string result = std::format("Count: {}", 42);
```

Mistake 4: Accessing .c_str() Pointer After Modifying the String

The bug:

```
std::string s = "hello";
const char* p = s.c_str();
s += " world";    // may reallocate the internal buffer
printf("%s\n", p);    // p may now be dangling
```

The fix: Call .c_str() immediately before passing it to the C function, never store it.

Exercises

Exercise 10.1 – Vector operations

Starting from an empty `std::vector<int>`: 1. Push back 5, 10, 15, 20, 25 2. Print the size (should be 5) 3. Remove the last element 4. Insert the value 99 at index 2 5. Print all elements

Answer:

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v;
    v.push_back(5);
    v.push_back(10);
    v.push_back(15);
    v.push_back(20);
    v.push_back(25);

    std::cout << v.size() << "\n";    // 5

    v.pop_back();                      // removes 25; v = {5,10,15,20}

    v.insert(v.begin() + 2, 99);       // v = {5,10,99,15,20}

    for (int n : v)
        std::cout << n << " ";       // 5 10 99 15 20
    std::cout << "\n";
}
```

Exercise 10.2 – String manipulation

Given `std::string sentence = "the quick brown fox"`: 1. Print its length 2. Extract and print the substring “quick” (starts at index 4, length 5) 3. Find where “brown” appears 4. Replace “fox” with “cat” 5. Print the final string

Answer:

```
std::string sentence = "the quick brown fox";
std::cout << sentence.size() << "\n";    // 19
std::cout << sentence.substr(4, 5) << "\n"; // quick
std::cout << sentence.find("brown") << "\n"; // 10

size_t pos = sentence.find("fox");
sentence.replace(pos, 3, "cat");
std::cout << sentence << "\n";           // the quick brown cat
```

Exercise 10.3 – 2D grid

Create a 4x4 grid (vector of vectors), fill position `[row][col]` with the value `row * 4 + col`, then print it as a square.

Answer:

```
std::vector<std::vector<int>> grid(4, std::vector<int>(4));  
for (int r = 0; r < 4; ++r)  
    for (int c = 0; c < 4; ++c)  
        grid[r][c] = r * 4 + c;  
  
for (const auto& row : grid) {  
    for (int n : row)  
        std::cout << std::setw(3) << n;  
    std::cout << "\n";  
}
```

Output:

```
0  1  2  3  
4  5  6  7  
8  9 10 11  
12 13 14 15
```

Chapter 11: Scope, Lifetime, and Organizing Code into Files

Scope

Scope is the region of code where a name is visible. In C++, every pair of `{ }` creates a new scope.

```
int x = 1;           // x in outer scope

{
    int x = 2;       // x in inner scope -- SHADOWS the outer x
    std::cout << x;  // 2 -- inner x
}                   // inner x destroyed here

std::cout << x;      // 1 -- outer x, back in scope
```

Shadowing (declaring an inner variable with the same name as an outer one) is legal but confusing. Avoid it. Compile with `-Wshadow` to get warnings.

Scope in Loops and Conditionals

```
for (int i = 0; i < 10; ++i) {
    // i only visible inside the loop body
}
// i does NOT exist here

if (int n = get_value(); n > 0) { // C++17 initializer
    // n only visible inside the if/else
} else {
    // n also visible here
}
// n does NOT exist here
```

Lifetime

Lifetime is the period during which a variable's storage is valid. For stack variables, it matches scope. For heap variables, it starts at `new` and ends at `delete`.

```
{
    std::string s = "hello";    // s constructed, storage allocated
    // ... use s ...
}                               // s destructed, storage freed
// using s here is undefined behavior (it no longer exists)
```

The key insight: in C++, when a variable goes out of scope, its **destructor** is called automatically. For `std::string`, the destructor frees the heap-allocated character buffer. For `std::vector`, it frees the element array. This automatic cleanup is called **RAII** (Chapter 12) and is C++'s answer to garbage collection.

Organizing Code: The Header/Source Split

As programs grow, you split code into multiple files. The organization:

```
projectname/
  include/
    math_utils.h      <- declarations (the public interface)
  src/
    math_utils.cpp    <- definitions (the implementation)
    main.cpp          <- uses the functions
```

Why Headers Exist

C++ compiles each `.cpp` file separately. When `main.cpp` calls `multiply(3, 4)`, the compiler must know (at compile time) what `multiply` looks like – its parameter types and return type – to type-check the call.

The linker resolves where `multiply`'s actual code is. But the compiler must see the declaration.

The solution: put declarations in a header file. Every `.cpp` that needs to call `multiply` includes the header.

```
// math_utils.h -- declarations only
#pragma once

int add(int a, int b);
int subtract(int a, int b);
int multiply(int a, int b);
double divide(double a, double b);
```

`#pragma once` tells the preprocessor: include this file at most once per compilation unit, even if it is `#include`d multiple times. It prevents duplicate declaration errors from diamond-shaped include chains.

```
// math_utils.cpp -- definitions
#include "math_utils.h" // include own header to verify declarations match

int add(int a, int b)      { return a + b; }
int subtract(int a, int b) { return a - b; }
int multiply(int a, int b)  { return a * b; }
double divide(double a, double b) { return a / b; }
```

```
// main.cpp -- uses the functions
#include <iostream>
#include "math_utils.h" // gets the declarations; tells compiler the signatures

int main() {
    std::cout << add(3, 4)      << "\n"; // 7
    std::cout << multiply(6, 7)  << "\n"; // 42
    std::cout << divide(10.0, 4.0) << "\n"; // 2.5
}
```

Build:

```
$ g++ -std=c++23 -Wall -o program src/main.cpp src/math_utils.cpp -I include/
```

The `-I include/` tells the compiler to look in the `include/` directory for headers.

The Compilation Model With Multiple Files

```
math_utils.cpp --> [compiler] --> math_utils.o  --+
main.cpp       --> [compiler] --> main.o        --+--> [linker] --> program
```

Each `.cpp` is compiled independently. The compiler only needs to see the header (declarations) to compile `main.cpp`. It trusts that `math_utils.cpp` will provide the actual definitions. The linker connects everything.

What Goes In Headers vs Source Files

Headers (`.h`):

- Function declarations
- Class declarations (but not the bodies of non-inline methods)
- Type definitions (structs, enums, using aliases)
- `#include` directives needed by the declarations
- inline functions (small utility functions that are called often)
- `constexpr` and `const` variable definitions

Source (`.cpp`):

- Function definitions (the actual code)
- Class method definitions

- Global variable definitions
- #include of their own header + other headers needed for the implementation
- Implementation details not exposed to users

Do NOT put in headers: - using namespace std; - it pollutes every file that includes your header - Non-inline function definitions - causes “multiple definition” linker errors if included in multiple .cpp files - Definitions of non-const global variables - also causes linker errors

Include Guards (Alternative to #pragma once)

Older code uses include guards instead of #pragma once:

```
#ifndef MATH_UTILS_H
#define MATH_UTILS_H

// ... declarations ...

#endif // MATH_UTILS_H
```

If math_utils.h is included twice in the same compilation unit, the second inclusion sees that MATH_UTILS_H is already defined and skips everything inside. #pragma once does the same thing more concisely and is supported by all modern compilers.

Namespaces

Namespaces prevent name collisions between libraries. When two libraries both define Matrix, they can put it in different namespaces.

```
namespace geometry {
    struct Point { double x, y; };
    double distance(Point a, Point b);
}

namespace graphics {
    struct Point { float x, y, z; }; // different Point, no conflict
    void draw(Point p);
}

// Use with ::
geometry::Point p1{1.0, 2.0};
graphics::Point p2{3.0f, 4.0f, 5.0f};
```

The standard library lives in std. That is why everything is std::cout, std::vector, std::string.

You can pull specific names into scope:

```
using std::cout;    // just cout, not everything
using std::vector;

cout << "hello\n"; // OK
vector<int> v;      // OK
```

Do this inside functions, not at file scope (it affects the rest of the file, which may cause surprises in headers).

Writing Your Own Namespace

```
// mylib.h
#pragma once

namespace mylib {

int clamp(int value, int lo, int hi);
double lerp(double a, double b, double t);

} // namespace mylib

// mylib.cpp
#include "mylib.h"

namespace mylib {

int clamp(int value, int lo, int hi) {
    if (value < lo) return lo;
    if (value > hi) return hi;
    return value;
}

double lerp(double a, double b, double t) {
    return a + t * (b - a);
}

} // namespace mylib
```

Static Local Variables

A static local variable is initialized once and persists across calls:

```
int counter() {
    static int count = 0; // initialized only once (first call)
    ++count;
    return count;
}

std::cout << counter() << "\n"; // 1
```

```
std::cout << counter() << "\n"; // 2
std::cout << counter() << "\n"; // 3
```

The variable `count` lives for the entire program duration (like a global), but is only accessible inside `counter`. This is useful for functions that need to remember state between calls without using a class.

Common Mistakes in This Chapter

Mistake 1: Defining a Non-Inline Function in a Header

The bug:

```
// utils.h
int add(int a, int b) { return a + b; } // DEFINITION in header

// main.cpp includes utils.h
// utils.cpp also includes utils.h
// Both .cpp files define add() -- linker error!
```

linker error: multiple definition of `'add(int, int)'`

The fix: Declarations in headers, definitions in `.cpp` files. Or mark the function `inline` (which tells the linker to allow multiple identical definitions).

Mistake 2: Missing `#pragma once` (or Include Guard)

The bug:

```
// a.h includes b.h and c.h
// b.h includes c.h
// c.h has no include guard

// When a.h is processed: c.h is included twice, all declarations in c.h appear twice
error: redefinition of 'class Foo'
```

The fix: Start every header with `#pragma once`.

Mistake 3: using `namespace std;` in a Header

The bug:

```
// utils.h
#include <string>
using namespace std;    // pollutes every file that includes utils.h

// Now every file including utils.h gets all of std:: imported
// Causes conflicts with user-defined names like 'vector', 'string', 'max', 'min'
```

The fix: Never put `using namespace X;` in a header. In `.cpp` files it is acceptable (though `using std::cout;` for specific names is better style).

Exercises

Exercise 11.1 – Trace scope

Predict the output:

```
int x = 10;
{
    int x = 20;
    {
        int x = 30;
        std::cout << x << "\n";
    }
    std::cout << x << "\n";
}
std::cout << x << "\n";
```

Answer: 30, 20, 10. Each inner `x` shadows the outer one. When the inner scope ends, the outer `x` becomes visible again.

Exercise 11.2 – Split into files

Split this single-file program into `main.cpp`, `greeting.h`, and `greeting.cpp`:

```
#include <iostream>
#include <string>

std::string make_greeting(const std::string& name) {
    return "Hello, " + name + "!";
}

int main() {
    std::cout << make_greeting("Alice") << "\n";
    std::cout << make_greeting("Bob") << "\n";
}
```

Answer:

```
// greeting.h
#pragma once
#include <string>

std::string make_greeting(const std::string& name);
```

```
// greeting.cpp
#include "greeting.h"

std::string make_greeting(const std::string& name) {
    return "Hello, " + name + "!";
}
```

```
// main.cpp
#include <iostream>
#include "greeting.h"

int main() {
    std::cout << make_greeting("Alice") << "\n";
    std::cout << make_greeting("Bob") << "\n";
}
```

Build: `g++ -std=c++23 -o program main.cpp greeting.cpp`

Exercise 11.3 – Static local counter

Write a function `next_id()` that returns 1 on the first call, 2 on the second, and so on, without using any global variable.

Answer:

```
int next_id() {
    static int id = 0;
    return ++id;
}
// next_id() == 1, next_id() == 2, next_id() == 3 ...
```

Exercise 11.4 – Namespace

Write a namespace `convert` with two functions: `km_to_miles(double km)` and `miles_to_km(double miles)`. 1 km = 0.621371 miles.

Answer:

```
// convert.h
#pragma once

namespace convert {
    double km_to_miles(double km);
    double miles_to_km(double miles);
}
```

```
// convert.cpp
#include "convert.h"

namespace convert {
    double km_to_miles(double km)    { return km * 0.621371; }
    double miles_to_km(double miles) { return miles / 0.621371; }
}
```

Usage:

```
std::cout << convert::km_to_miles(100.0) << "\n";    // 62.1371
std::cout << convert::miles_to_km(62.0)  << "\n";    // 99.79...
```

Part II is complete. You now understand the concepts that have no Python equivalent: references as aliases, pointers and memory addresses, the stack vs heap memory model, const correctness enforced by the compiler, the standard library containers, and how C++ organizes multi-file projects.

Part III covers ownership and memory management – the RAII pattern, smart pointers, and move semantics. These are what make modern C++ safe while staying fast. Ask to continue.

Part III – Ownership and Memory Management

This part covers what separates experienced C++ programmers from beginners: understanding *who owns a resource* and *when it gets cleaned up*. Python’s garbage collector answers both questions automatically. C++ makes you – or your types – answer them explicitly.

The central pattern is RAII. Everything in this part flows from it.

Chapter 12: RAI – The Core Idea That Replaces Garbage Collection

The Problem: Resources Need Cleanup

Some things you acquire must eventually be released:

Resource	Acquire	Release
Heap memory	<code>new</code>	<code>delete</code>
File	<code>fopen / open</code>	<code>fclose / close</code>
Mutex lock	<code>lock()</code>	<code>unlock()</code>
Network socket	<code>socket() / connect()</code>	<code>close()</code>
Database connection	<code>connect()</code>	<code>disconnect()</code>

If you forget to release, you get leaks, deadlocks, or corruption. If an exception or early return happens between acquire and release, the release never runs.

```
void risky() {
    int* data = new int[1000];

    if (something_failed()) {
        return;           // LEAK: data never deleted
    }

    if (something_else_failed()) {
        throw std::runtime_error("oops"); // LEAK: data never deleted
    }

    delete[] data; // only reached on the happy path
}
```

This is the problem RAI solves.

What RAI Is

RAI stands for **Resource Acquisition Is Initialization**. The name is cryptic; the idea is simple:

Tie a resource's lifetime to the lifetime of an object. Acquire the resource in the constructor. Release the resource in the destructor.

When the object goes out of scope, C++ automatically calls its destructor. The destructor releases the resource. This happens even if an exception is thrown, even if there is an early return. The cleanup is guaranteed.

```
class IntArray {
    int* data;
    int size;
public:
    IntArray(int n) : data{new int[n]{}}, size{n} {
        // constructor: acquires the resource (heap memory)
    }

    ~IntArray() {                // destructor: ~ prefix, no return type
        delete[] data;          // releases the resource
    }

    int& operator[](int i) { return data[i]; }
    int get_size() const { return size; }
};

void safe() {
    IntArray arr{1000};          // constructor runs: allocates 1000 ints

    if (something_failed()) {
        return;                  // destructor runs: delete[] data -- NO LEAK
    }

    if (something_else_failed()) {
        throw std::runtime_error("oops"); // destructor runs -- NO LEAK
    }

    // destructor runs at end of scope -- NO LEAK
}
```

The destructor runs at scope exit **no matter how the scope exits**. Return, exception, fall-through – the destructor always runs.

The Destructor

A destructor is a special member function:

```
class Foo {
public:
    Foo() { std::cout << "constructed\n"; } // constructor
    ~Foo() { std::cout << "destroyed\n"; } // destructor (~ prefix)
};

{
    Foo a;
```

```

    Foo b;
    std::cout << "inside block\n";
} // b destructs first (LIFO), then a

```

Output:

```

constructed
constructed
inside block
destructed    <- b (last in, first out)
destructed    <- a

```

Destructors run in **reverse order of construction** (last in, first out – like the stack). This matters when objects depend on each other.

RAII for File Handling

Without RAII:

```

void write_data_bad(const std::string& filename) {
    FILE* f = fopen(filename.c_str(), "w");
    if (!f) return; // ok, no file to close

    // ... write stuff ...

    if (error_condition) {
        return; // LEAK: file never closed!
    }

    fclose(f); // only reached on happy path
}

```

With RAII (using `std::fstream` from the standard library):

```

#include <fstream>

void write_data_good(const std::string& filename) {
    std::ofstream f{filename}; // constructor opens the file
    if (!f) return;

    // ... write stuff ...

    if (error_condition) {
        return; // destructor closes the file -- always
    }

} // destructor closes the file -- always

```

`std::ofstream`'s destructor closes the file handle. You never call `fclose`. The file is always closed, regardless of how the function exits.

RAII for Mutex Locks

```
#include <mutex>

std::mutex mtx;

void bad_worker() {
    mtx.lock();
    if (error) {
        return;           // DEADLOCK: mutex never unlocked!
    }
    mtx.unlock();
}

void good_worker() {
    std::lock_guard<std::mutex> guard{mtx}; // constructor: locks mtx
    if (error) {
        return;           // destructor: unlocks mtx -- always
    }
} // destructor: unlocks mtx -- always
```

`std::lock_guard` is a tiny RAII wrapper. It locks on construction and unlocks on destruction. The mutex is always released, even if an exception is thrown.

Python's Equivalent: Context Managers

Python's `with` statement provides similar guarantees:

```
# Python RAII equivalent: context manager
with open("file.txt", "w") as f:
    f.write("hello")
# f.__exit__() called here -- file closed even on exception
```

C++'s RAII is more general and automatic: it applies to every object with a destructor, with no special syntax at the call site. In Python you must explicitly write `with`.

The Lifetime Guarantee Visualized

Scope:

```
{
    IntArray arr{1000};    <- constructor: allocates memory
    process(arr);
    if (fail) return;      <- early return?  destructor still runs!
    if (err) throw ...;    <- exception?      destructor still runs!
}                          <- normal exit:   destructor runs
```

Stack unwind on exception:

When an exception propagates, C++ calls the destructor of every local object as it unwinds each stack frame. No resource is leaked.

Common Mistakes in This Chapter

Mistake 1: Not Writing a Destructor for Resource-Owning Classes

The bug:

```
class Buffer {
    int* data;
public:
    Buffer(int n) { data = new int[n]; }
    // forgot: ~Buffer() { delete[] data; }
};
// Every Buffer that goes out of scope leaks its allocation.
```

The fix: If a class owns a raw pointer acquired with `new`, it needs a destructor with `delete`.

Mistake 2: Destructors That Throw

The bug:

```
~MyClass() {
    if (!cleanup()) throw std::runtime_error("cleanup failed"); // DANGEROUS
}
```

Why it's dangerous: If a destructor throws while the stack is already unwinding from another exception, `std::terminate` is called and the program crashes with no recovery. **The fix:** Never let destructors throw. Swallow or log errors inside destructors.

Exercises

Exercise 12.1 – Trace the destructor order

```
struct Log {
    std::string name;
    Log(std::string n) : name{n} { std::cout << "create " << name << "\n"; }
    ~Log() { std::cout << "destroy " << name << "\n"; }
};

int main() {
```

```
Log a{"A"};
{
    Log b{"B"};
    Log c{"C"};
}
Log d{"D"};
}
```

What is the output?

Answer:

```
create A
create B
create C
destroy C
destroy B
create D
destroy D
destroy A
```

C and B are destroyed in reverse order when the inner block ends. D is created after the block. Then A and D are destroyed in reverse order when main ends.

Exercise 12.2 – Design a RAII wrapper

Design (declarations only, no need to implement) an RAII class `FileHandle` that wraps `FILE*`. What members does it need? What should the constructor and destructor do?

Answer:

```
class FileHandle {
    FILE* file;
public:
    FileHandle(const char* path, const char* mode); // opens: file = fopen(path, mode)
    ~FileHandle(); // closes: if (file) fclose(file)
    bool is_open() const; // returns file != nullptr
    FILE* get() const; // returns the raw FILE* for C API use

    // Disable copy (two wrappers closing the same file = double-close bug):
    FileHandle(const FileHandle&) = delete;
    FileHandle& operator=(const FileHandle&) = delete;
};
```

Chapter 13: Dynamic Allocation: new, delete, and Why You Avoid Them

new and delete Revisited

We introduced new and delete in Chapter 8. Here is the complete picture.

```
// Single object:
int* p = new int{42};    // allocate one int on heap, initialize to 42
delete p;                // free it
p = nullptr;             // prevent accidental reuse

// Array:
int* arr = new int[100]{}; // allocate 100 ints, zero-initialized
delete[] arr;              // MUST use delete[] for arrays
arr = nullptr;

// Default-initialized (no value specified):
int* q = new int;          // value is garbage (same as uninitialized local)
delete q;

// Value-initialized (zero):
int* r = new int{};        // value is 0
delete r;
```

What new Actually Does

1. Calls operator new to ask the allocator for N bytes of heap memory
2. Constructs the object in that memory (runs the constructor)
3. Returns a typed pointer to the constructed object

What delete Actually Does

1. Calls the destructor of the object
2. Calls operator delete to return the memory to the allocator

If you use delete on an array (allocated with new[]), the destructor for each element is called, and then the memory is freed. If you use delete instead of delete[], only one destructor is called and the allocator is given the wrong size – undefined behavior.

The Problems With Raw new/delete

Problem 1: Exception Safety

```
int* a = new int{1};
int* b = new int{2};    // if this throws (out of memory), a leaks
process(a, b);          // if this throws, a and b leak
delete a;
delete b;
```

Problem 2: Ownership Ambiguity

```
int* create() { return new int{42}; }    // who must call delete?

void use(int* p) {
    // Am I supposed to delete p? Is it heap-allocated? Who owns it?
}
```

When raw pointers are passed around, ownership becomes unclear. This leads to either leaks (nobody deletes) or double-frees (two places both delete).

Problem 3: Paired Delete is Fragile

Every new must be matched by exactly one delete. This is an invariant you must maintain manually across hundreds or thousands of lines of code, through exceptions and early returns. One mistake = leak or crash.

When Raw new/delete Is Acceptable

Almost never in modern C++. The standard library provides better alternatives:

Old pattern	Modern replacement
<code>new int[n]</code>	<code>std::vector<int></code>
<code>new SomeClass(...)</code>	<code>std::make_unique<SomeClass>(...)</code>
Shared ownership	<code>std::make_shared<SomeClass>(...)</code>
Fixed-size array	<code>std::array<T, N></code>

The only legitimate uses of raw new/delete are: - Writing a custom allocator or container - Interfacing with a C API that expects you to free() memory it returned - Writing operator new/operator delete for a class

If you find yourself writing new in application code, stop and use a smart pointer or container.

std::bad_alloc – Out of Memory

When new cannot get memory (system is out of RAM), it throws std::bad_alloc. Most programs do not handle this because there is little useful you can do, but for critical systems:

```
#include <new>
try {
    int* p = new int[1'000'000'000'000]; // 1 trillion ints ~ 4TB
} catch (const std::bad_alloc& e) {
    std::cerr << "Out of memory: " << e.what() << "\n";
}
```

If you want new to return nullptr instead of throwing:

```
int* p = new(std::nothrow) int[1'000'000'000'000];
if (p == nullptr) {
    // handle out of memory
}
```

Common Mistakes in This Chapter

Mistake 1: delete on Stack Memory

The bug:

```
int x = 5;
int* p = &x;
delete p; // undefined behavior -- x is on the stack, not heap
```

The symptom: Crash or heap corruption. **The rule:** Only delete memory that came from new.

Mistake 2: Accessing Memory After delete

```
int* p = new int{10};
delete p;
std::cout << *p; // use-after-free: undefined behavior
```

Detection: –fsanitize=address reports “heap-use-after-free.” **The fix:** Set p = nullptr immediately after delete.

Exercises

Exercise 13.1 – Manual heap string

Without using `std::string`, allocate a `char` array on the heap large enough to hold “Hello, C++!” (including the null terminator), fill it using `strcpy`, print it, then free it correctly.

Answer:

```
#include <cstring>
const char* src = "Hello, C++!";
int len = strlen(src) + 1;           // +1 for null terminator
char* buf = new char[len];
strcpy(buf, src);
std::cout << buf << "\n";
delete[] buf;
buf = nullptr;
```

Exercise 13.2 – Identify the errors

How many bugs does this code have? Classify each as leak, double-free, or use-after-free:

```
int* p = new int{5};
int* q = p;
delete p;
delete q;           // (a)
std::cout << *p;    // (b)
int x = 10;
delete &x;          // (c)
```

Answer: - (a): double-free – `p` and `q` point to the same memory; freeing both is double-free. - (b): use-after-free – `p` was deleted; reading `*p` is undefined behavior. - (c): delete on stack memory – `x` is a local variable, not heap-allocated.

Chapter 14: Smart Pointers: `unique_ptr`, `shared_ptr`, `weak_ptr`

The Core Idea

Smart pointers are RAII wrappers around raw pointers. They manage ownership: when the smart pointer is destroyed, it automatically deletes the object it owns.

```
#include <memory>

// Instead of:
int* raw = new int{42};
// ... possibly forget to delete ...
delete raw;

// Use:
auto smart = std::make_unique<int>(42);
// ... no delete needed -- automatically freed when smart goes out of scope ...
```

There are three kinds, each for a different ownership pattern.

`std::unique_ptr` – Exclusive Ownership

`unique_ptr` represents **sole ownership**: exactly one `unique_ptr` owns the object at any time. When the `unique_ptr` is destroyed, it deletes the owned object.

```
#include <memory>

auto p = std::make_unique<int>(42); // allocates int{42} on heap
                                   // p is the sole owner

std::cout << *p << "\n";           // 42 -- dereference like a raw pointer
std::cout << p.get() << "\n";       // raw address (for C API use)

// p is automatically deleted when it goes out of scope
// No manual delete. No leaks.
```

`make_unique<T>(args...)` is the right way to create a `unique_ptr`. It: 1. Allocates the object on the heap 2. Constructs it with the given arguments 3. Returns a `unique_ptr` owning it

Unique Ownership Enforced at Compile Time

```
auto p = std::make_unique<int>(42);
auto q = p;    // COMPILER ERROR: unique_ptr cannot be copied
```

You cannot copy a `unique_ptr` – that would create two owners for one object (a contradiction of unique ownership). The compiler enforces this.

You CAN move it (transfer ownership):

```
auto p = std::make_unique<int>(42);
auto q = std::move(p);    // ownership transferred from p to q
// p is now empty (nullptr); q owns the int
std::cout << *q << "\n";    // 42
// *p would be undefined behavior -- p is now empty
```

`std::move` is covered in Chapter 15. For now, understand that `std::move(p)` says “I am done with p; transfer what it owns to q.”

Unique Pointer to a Custom Class

```
struct Player {
    std::string name;
    int health;
    Player(std::string n, int h) : name{n}, health{h} {}
    ~Player() { std::cout << name << " destroyed\n"; }
};

{
    auto hero = std::make_unique<Player>("Alice", 100);
    hero->health -= 20;    // -> accesses members (same as raw pointer)
    std::cout << hero->name << " has " << hero->health << " HP\n";
}    // hero goes out of scope -- Player destructor called, memory freed
// prints: "Alice has 80 HP" then "Alice destroyed"
```

unique_ptr as a Function Parameter

```
// Takes ownership (caller cannot use the pointer after this):
void consume(std::unique_ptr<Player> p) {
    std::cout << "consuming " << p->name << "\n";
}    // p destroyed here

// Borrows (caller keeps ownership, function just uses the object):
void use(const Player& p) {    // prefer this
    std::cout << "using " << p.name << "\n";
}

void use_ptr(const Player* p) {    // raw ptr = borrow, no ownership
```

```

    if (p) std::cout << "using " << p->name << "\n";
}

auto hero = std::make_unique<Player>("Bob", 80);
use(*hero);           // dereference to get Player&
use_ptr(hero.get());  // .get() returns raw pointer, no ownership transfer
consume(std::move(hero)); // hero is empty after this

```

The guideline: pass `unique_ptr` only when you intend to transfer ownership. For “just using” the object, pass a reference or raw pointer (raw pointer = borrow, no ownership implied).

std::shared_ptr – Shared Ownership

`shared_ptr` uses **reference counting**: multiple `shared_ptr`s can all own the same object. The object is deleted when the last `shared_ptr` to it is destroyed.

```

# Python reference counting -- the same idea Python uses for all objects
a = [1, 2, 3]    # ref count = 1
b = a            # ref count = 2
del a            # ref count = 1
del b            # ref count = 0 --> freed

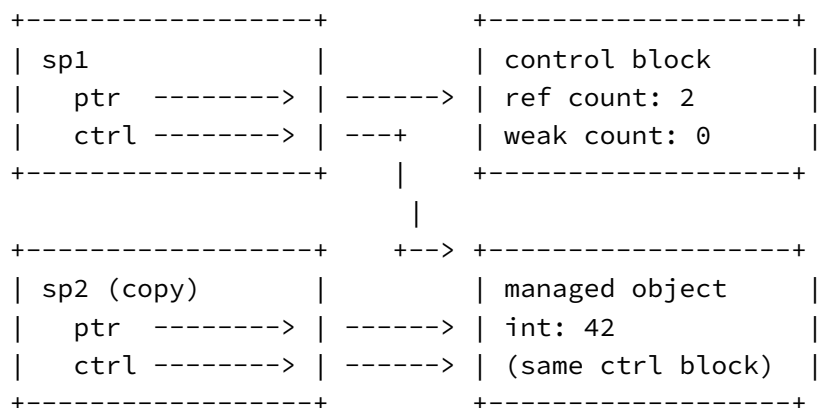
```

```

auto sp1 = std::make_shared<int>(42); // ref count = 1
{
    auto sp2 = sp1; // ref count = 2 (COPY IS ALLOWED for shared_ptr)
    std::cout << *sp2 << "\n"; // 42
    std::cout << sp1.use_count() << "\n"; // 2
} // sp2 destroyed, ref count = 1
// int still alive (sp1 holds it)
std::cout << sp1.use_count() << "\n"; // 1
// sp1 goes out of scope, ref count = 0, int deleted

```

Memory layout of `shared_ptr`:



When to Use `shared_ptr`

Use `shared_ptr` when multiple objects genuinely need to share ownership and you cannot determine statically which one will outlive the others:

- Scene graph nodes that can be referenced from multiple places
- Cache entries referenced by many users
- Callbacks registered with multiple event systems

Do NOT use `shared_ptr` by default. It has overhead (atomic ref-count increment/decrement on every copy and destroy). If one owner makes sense, use `unique_ptr`. If no ownership is needed (just borrowing), use a reference or raw pointer.

`std::weak_ptr` – Non-Owning Observer

A `weak_ptr` holds a non-owning reference to an object managed by `shared_ptr`. It does not affect the ref count. You must convert it to a `shared_ptr` to actually use the object, and that conversion can fail if the object was already deleted.

The main use: breaking reference cycles.

```
// Without weak_ptr: a cycle keeps both objects alive forever (memory leak)
struct Node {
    std::shared_ptr<Node> next;    // strong reference
};
auto a = std::make_shared<Node>();
auto b = std::make_shared<Node>();
a->next = b;
b->next = a;    // cycle: a holds b, b holds a -- neither ever freed

// With weak_ptr: the cycle is broken
struct Node {
    std::weak_ptr<Node> next;    // weak reference (non-owning)
};
// Now b->next does not keep a alive. When a goes out of scope, it is freed.
```

Using a `weak_ptr`:

```
auto sp = std::make_shared<int>(99);
std::weak_ptr<int> wp = sp;

if (auto locked = wp.lock()) {    // lock() returns shared_ptr if alive, empty if dead
    std::cout << *locked << "\n";    // 99
} else {
    std::cout << "object was deleted\n";
}

sp.reset();    // delete the object

if (auto locked = wp.lock()) {
    std::cout << *locked << "\n";
}
```

```
} else {  
    std::cout << "object was deleted\n";    // this prints  
}
```

Choosing the Right Smart Pointer

Is there exactly one owner that will definitely outlive all users?

--> `std::unique_ptr` (default choice)

Do multiple owners genuinely need to share the object's lifetime?

--> `std::shared_ptr`

Do you need to observe a `shared_ptr`-managed object without affecting its lifetime?

--> `std::weak_ptr`

Are you just borrowing -- the object's owner is clear and nearby?

--> `T&` (reference) or `const T&` (const reference)

--> `T*` (raw pointer) if nullability is needed

90% of cases: use `unique_ptr`. 9%: `shared_ptr`. 1%: `weak_ptr`.

Common Mistakes in This Chapter

Mistake 1: Creating a `shared_ptr` From a Raw Pointer Twice

The bug:

```
int* raw = new int{42};  
auto sp1 = std::shared_ptr<int>(raw);  
auto sp2 = std::shared_ptr<int>(raw); // two independent shared_ptrs own raw!  
// When both are destroyed: double-free
```

The fix: Always use `std::make_shared`. If you must wrap an existing pointer, do it once and copy the `shared_ptr`.

Mistake 2: Calling `.get()` and Storing the Result

The bug:


```

auto sp = std::make_shared<int>(5);
int* raw = sp.get();    // raw is a non-owning pointer -- fine so far
sp.reset();             // sp destroys the int
std::cout << *raw;      // use-after-free

```

The fix: Never store the result of `.get()` beyond the lifetime of the smart pointer.

Mistake 3: Passing `unique_ptr` by Value When Borrowing

The bug:

```

void display(std::unique_ptr<Player> p) { ... } // takes ownership!
auto hero = std::make_unique<Player>("Alice", 100);
display(hero);           // ERROR: cannot copy unique_ptr
display(std::move(hero)); // compiles, but hero is now empty -- likely a bug

```

The fix: For borrowing, pass `const Player&` or `Player*` (via `.get()`).

Exercises

Exercise 14.1 – `unique_ptr` basics

Rewrite this raw-pointer code using `unique_ptr`:

```

double* compute() {
    double* result = new double{3.14};
    return result;    // caller must delete
}

int main() {
    double* r = compute();
    std::cout << *r << "\n";
    delete r;
}

```

Answer:

```

#include <memory>
#include <iostream>

std::unique_ptr<double> compute() {
    return std::make_unique<double>(3.14);
}

int main() {
    auto r = compute();    // unique_ptr takes ownership
    std::cout << *r << "\n"; // 3.14
    // r automatically deleted at end of main
}

```

Exercise 14.2 – shared_ptr ref count

Predict the ref count at each comment:

```
auto a = std::make_shared<int>(10); // (1)
{
    auto b = a;                      // (2)
    auto c = a;                      // (3)
    c.reset();                       // (4)
}                                    // (5)
// (6)
```

Answer: - (1): 1 - (2): 2 - (3): 3 - (4): 2 (c . reset () releases c's ownership) - (5): 1 (b goes out of scope) - (6): 1 (a is still alive outside the block)

Exercise 14.3 – Choose the smart pointer

For each scenario, say whether to use `unique_ptr`, `shared_ptr`, `weak_ptr`, or a raw reference:

- a. A game entity that owns a weapon (weapon lives exactly as long as the entity)
- b. A texture loaded into a cache, referenced by hundreds of sprites
- c. A parent node in a tree that wants to observe its own child (child is managed by parent via `shared_ptr`)
- d. A function that needs to read-only access an object whose lifetime is certain to outlast the function

Answer: - a: `unique_ptr<Weapon>` – one owner, no sharing. - b: `shared_ptr<Texture>` – many owners, object lives until last sprite is done. - c: `weak_ptr<Node>` – parent already holds a `shared_ptr` to child; back-reference via `weak_ptr` avoids cycle. - d: `const T&` (reference) – just borrowing, no ownership needed.

Chapter 15: Move Semantics, lvalues, and rvalues

The Performance Problem With Copies

Consider what happens when you return a `std::vector` from a function:

```
std::vector<int> make_million() {  
    std::vector<int> v(1'000'000, 0); // 1 million ints on heap: ~4 MB  
    return v;  
}  
  
std::vector<int> result = make_million();
```

Naively, this would copy 4 MB from `v` into `result`. That is a lot of work. But modern C++ avoids this copy entirely. To understand how, you need lvalues and rvalues.

lvalues and rvalues

An **lvalue** is an expression that has a stable memory address – you can take its address with `&`, and it persists beyond the current expression. Named variables are lvalues.

An **rvalue** is a temporary – it has no persistent address. Literals, arithmetic results, and return values are rvalues.

```
int x = 5;  
int y = x + 3;  
  
// x is an lvalue: it has an address, it persists  
// 5 is an rvalue: it is a temporary value with no address of its own  
// x + 3 is an rvalue: the result is a temporary  
  
int* p = &x; // OK: x is an lvalue, can take its address  
int* q = &(x + 3); // ERROR: x+3 is an rvalue, no persistent address
```

The shorthand: if you can put it on the left side of an assignment, it is an lvalue. Rvalues cannot be assigned to.

```
x = 42;           // OK: x is an lvalue
x + 3 = 42;       // ERROR: x+3 is an rvalue, you cannot assign to it
```

The Move Operation

An rvalue is a temporary that is about to be discarded. If you are initializing a new object from a temporary, there is no need to copy the temporary's data – you can just steal it.

This is the **move operation**: instead of copying data from source to destination and then destroying the source, move hands ownership of the source's resources directly to the destination, then puts the source in a valid-but-empty state.

Copy:

```
[Source] --> makes a duplicate --> [Destination]
[Source] still valid and full      [Destination] is a new copy
```

Move:

```
[Source] --> hands over resources --> [Destination]
[Source] is now empty (valid but empty) [Destination] owns the data
```

For a `std::vector` with 1 million elements:

Copy: allocate 4MB, copy 1M integers, now two 4MB allocations exist

Move: copy 3 pointers (data, size, capacity), set source to empty state
-- essentially free

std::move – Casting to rvalue

By default, named variables are lvalues and are copied. To tell C++ “treat this lvalue as a temporary so it can be moved,” use `std::move`:

```
std::vector<int> a(1'000'000, 42); // a: 4MB

std::vector<int> b = a;           // COPY: 4MB allocated, 1M ints copied
std::vector<int> c = std::move(a); // MOVE: 3 pointers copied, a is now empty
// a is empty after the move -- do not use a anymore
```

`std::move` does NOT move anything. It just casts a to an rvalue reference, signaling that the move constructor (instead of the copy constructor) should be used.

Move Semantics and Returned Values

```
std::vector<int> make_million() {
    std::vector<int> v(1'000'000, 0);
    return v;    // v is a local variable, about to be destroyed
                // compiler applies NRVO (named return value optimization)
                // or the implicit move -- either way, no copy
}

auto result = make_million();    // no copy of 4MB
```

The compiler applies **NRVO** (Named Return Value Optimization) or uses the implicit move. In practice, returning a local variable from a function is always efficient in modern C++. Do not write `std::move(v)` – that actually disables NRVO.

Move Constructor and Move Assignment

You can define how your own class moves:

```
class Buffer {
    int* data;
    int size;

public:
    // Constructor
    Buffer(int n) : data{new int[n]{}}, size{n} {}

    // Destructor
    ~Buffer() { delete[] data; }

    // Copy constructor (deep copy -- expensive)
    Buffer(const Buffer& other) : data{new int[other.size]{}}, size{other.size} {
        std::copy(other.data, other.data + size, data);
    }

    // Move constructor (steal the pointer -- cheap)
    Buffer(Buffer&& other) noexcept // && = rvalue reference
        : data{other.data}, size{other.size} {
        other.data = nullptr; // source is now empty
        other.size = 0;
    }

    // Move assignment
    Buffer& operator=(Buffer&& other) noexcept {
        if (this != &other) {
            delete[] data; // free current data
            data = other.data; // steal other's data
            size = other.size;
            other.data = nullptr; // leave other empty
            other.size = 0;
        }
    }
}
```

```

        return *this;
    }
};

```

The `&&` in `Buffer(Buffer&& other)` is an **rvalue reference** – a reference that binds only to temporaries (rvalues). This overload is chosen when moving from a temporary.

`noexcept` tells the compiler “this function will not throw.” Move operations should be `noexcept` whenever possible – some standard library algorithms (like `std::vector` reallocation) can only use the move constructor if it is `noexcept`.

std::string and std::vector Move in Action

```

std::string s1 = "This is a long string that lives on the heap";
std::string s2 = s1;           // copy: s2 gets its own copy of the chars
std::string s3 = std::move(s1); // move: s3 steals the char buffer from s1
                               // s1 is now an empty string ""

std::cout << s2 << "\n"; // "This is a long string..."
std::cout << s3 << "\n"; // "This is a long string..."
std::cout << s1 << "\n"; // "" (empty -- moved-from state)

```

When C++ Moves Automatically

You do not always need `std::move` explicitly. Moves happen automatically:

1. **Returning a local variable from a function** (NRVO or implicit move)
2. **Initializing from a temporary** (rvalue): `auto s = get_string();`
3. **Passing a temporary to a function**: `process(std::vector<int>{1,2,3});`

You need explicit `std::move` when: 1. You want to transfer ownership from one named variable to another 2. You want to move-insert into a container

```

std::vector<std::string> words;
std::string word = "hello";
words.push_back(word);           // copy: word still valid
words.push_back(std::move(word)); // move: word is now empty, faster

```

Common Mistakes in This Chapter

Mistake 1: Using a Moved-From Object

The bug:

```
std::vector<int> src = {1, 2, 3};
auto dst = std::move(src);
for (int n : src) { ... } // src is empty -- loop runs zero times
                          // (well-defined but probably wrong)
```

The rule: After `std::move(x)`, treat `x` as if it were default-constructed (empty). Reassign before using.

Mistake 2: `return std::move(local)` Disabling NRVO

The bug:

```
std::vector<int> make() {
    std::vector<int> v = {1, 2, 3};
    return std::move(v); // PESSIMIZATION: disables NRVO
}
```

The fix: Just write `return v;`. The compiler already knows to move or elide.

Exercises

Exercise 15.1 – lvalue or rvalue?

Classify each expression as lvalue or rvalue:

```
int x = 5;
int arr[3] = {1,2,3};

x           // (a)
x + 1       // (b)
arr[0]      // (c)
42          // (d)
std::string{"hello"} // (e)
```

Answer: - (a) lvalue – `x` is a named variable with a persistent address - (b) rvalue – `x + 1` is a temporary result - (c) lvalue – `arr[0]` refers to a specific array element with an address - (d) rvalue – `42` is a literal, no persistent address - (e) rvalue – a temporary `std::string` constructed inline

Exercise 15.2 – Move vs copy performance

Explain in your own words why moving a `std::vector<int>` with 1 million elements is faster than copying it.

Answer: A `std::vector` consists of three things: a pointer to the heap-allocated element array, a size, and a capacity. Copying requires allocating a new heap array and copying all 1 million integers – $O(N)$ work proportional to the number of elements. Moving just copies the three pointer/size/capacity values and sets the source to empty – $O(1)$ work, regardless of the number of elements. The move does not touch the elements at all; it transfers ownership of the existing heap array.

Exercise 15.3 – Efficient string collection

Write a function that takes a `std::string` and pushes it into a `std::vector<std::string>`. Write two versions: one that copies, one that moves. When would you use each?

Answer:

```
std::vector<std::string> words;

// Version 1: copy (use when you need to keep the original)
void add_copy(const std::string& s) {
    words.push_back(s);           // copies s into the vector
}

// Version 2: move (use when you are done with the original)
void add_move(std::string s) {    // takes by value (already a copy or move from caller)
    words.push_back(std::move(s)); // moves into the vector
}

std::string word = "hello";
add_copy(word);           // word still valid ("hello")
add_move(std::move(word)); // word is now empty
```

Chapter 16: The Rule of 0, 3, and 5

The Problem: Special Member Functions

C++ classes have six **special member functions** that the compiler can generate automatically:

Function	Signature	What it does
Default constructor	<code>T()</code>	Creates an object with no arguments
Copy constructor	<code>T(const T&)</code>	Creates a copy of another object
Copy assignment	<code>T& operator=(const T&)</code>	Overwrites this object with a copy
Destructor	<code>~T()</code>	Cleans up when the object goes out of scope
Move constructor	<code>T(T&&)</code>	Creates object by stealing from a temporary
Move assignment	<code>T& operator=(T&&)</code>	Overwrites this object by stealing a temporary

The compiler generates these automatically if you do not write them. The compiler-generated versions do **memberwise** operations – they copy/move/destroy each member in turn.

The problem: the compiler-generated versions are wrong when your class **owns a resource** (raw pointer, file handle, socket, etc.).

The Rule of Zero

If your class does **not** own any raw resources, do not define any special member functions. Let the compiler generate them all. The compiler's versions will correctly copy/move/destroy each member.

```
// Rule of Zero: no raw resources, no user-defined special members
struct PlayerStats {
    std::string name;           // std::string manages its own memory
    int score{0};
    std::vector<int> history;    // std::vector manages its own memory
    // No raw pointers, no new/delete -- compiler generates correct defaults
};

PlayerStats a{"Alice", 100, {90, 95, 100}};
```

```
PlayerStats b = a;           // correct memberwise copy
PlayerStats c = std::move(a); // correct memberwise move
```

This is the best outcome. Use `std::string`, `std::vector`, and smart pointers as members, so the Rule of Zero applies.

The Rule of Three

If you define **any** of: destructor, copy constructor, or copy assignment – define all three.

Why: If you need a custom destructor, your class probably manages a resource. If it manages a resource, the compiler’s copy operations (which do a shallow copy of the pointer) are wrong.

```
class Buffer {
    int* data;
    int size;

public:
    Buffer(int n) : data{new int[n]{}}, size{n} {}

    // 1. Destructor: free the resource
    ~Buffer() { delete[] data; }

    // 2. Copy constructor: deep copy
    Buffer(const Buffer& other) : data{new int[other.size]{}}, size{other.size} {
        std::copy(other.data, other.data + size, data);
    }

    // 3. Copy assignment: free old, deep copy new
    Buffer& operator=(const Buffer& other) {
        if (this == &other) return *this; // self-assignment guard
        delete[] data;
        size = other.size;
        data = new int[size]{};
        std::copy(other.data, other.data + size, data);
        return *this;
    }
};
```

Without copy constructor and copy assignment, the compiler generates shallow copies – both objects’ data pointer points to the same heap memory. When both destructors run, `delete[]` is called twice on the same pointer. Crash.

The Rule of Five

C++11 added move semantics. If you are defining the Rule of Three, also define the move constructor and move assignment for efficiency: the **Rule of Five**.

```

class Buffer {
    int* data;
    int size;

public:
    Buffer(int n) : data{new int[n]{}}, size{n} {}

    // 1. Destructor
    ~Buffer() { delete[] data; }

    // 2. Copy constructor (deep copy)
    Buffer(const Buffer& other) : data{new int[other.size]{}}, size{other.size} {
        std::copy(other.data, other.data + size, data);
    }

    // 3. Copy assignment (free old, deep copy)
    Buffer& operator=(const Buffer& other) {
        if (this == &other) return *this;
        delete[] data;
        size = other.size;
        data = new int[size]{};
        std::copy(other.data, other.data + size, data);
        return *this;
    }

    // 4. Move constructor (steal pointer)
    Buffer(Buffer&& other) noexcept
        : data{other.data}, size{other.size} {
        other.data = nullptr;
        other.size = 0;
    }

    // 5. Move assignment (free old, steal pointer)
    Buffer& operator=(Buffer&& other) noexcept {
        if (this == &other) return *this;
        delete[] data;
        data = other.data;
        size = other.size;
        other.data = nullptr;
        other.size = 0;
        return *this;
    }
};

```

= delete and = default

You can explicitly suppress or request the default implementation:

```

class Unique {
public:
    Unique() = default;                // use compiler's default constructor

    Unique(const Unique&) = delete;     // copying is forbidden

```

```

Unique& operator=(const Unique&) = delete; // copy assignment is forbidden

Unique(Unique&&) = default;                // move is fine, use default
Unique& operator=(Unique&&) = default;
};

Unique a;
Unique b = a; // COMPILE ERROR: copy constructor is deleted
Unique c = std::move(a); // OK: move is allowed

```

= `delete` is how `std::unique_ptr` prevents copying. It causes a clear compile error rather than a silent wrong-copy.

= `default` explicitly requests the compiler-generated version, which is useful when you need to declare one special member (which inhibits some compiler-generated ones) but still want the defaults for others.

Compiler-Generated Functions Are Suppressed When You Define Others

The rules for when the compiler generates special members are complex. Key interactions:

You define	Compiler suppresses
Destructor	Move constructor, move assignment (still generates copy ops)
Copy constructor	Default constructor, move constructor, move assignment
Copy assignment	Move constructor, move assignment
Move constructor	Copy constructor, copy assignment
Move assignment	Copy constructor, copy assignment

This is why the Rule of Five says: if you define any one, define all five. Defining just a destructor silently disables move semantics, forcing expensive copies everywhere.

The Practical Takeaway

In priority order:

1. **Rule of Zero** (best): use standard containers and smart pointers as members. The compiler generates correct defaults. No special member functions needed.
2. **Rule of Five** (when you must own raw resources): write all five explicitly. This is mainly for implementing containers and RAII wrappers, not typical application code.
3. Never write only a destructor without the others. The compiler's default copy does a shallow copy of raw pointers, leading to double-free crashes.

Common Mistakes in This Chapter

Mistake 1: Shallow Copy of Owing Pointer (Forgetting Rule of Three)

The bug:

```
class Buffer {
    int* data;
public:
    Buffer(int n) { data = new int[n]{}; }
    ~Buffer() { delete[] data; }
    // forgot copy constructor and copy assignment
};

Buffer a{10};
Buffer b = a;    // compiler shallow-copies: b.data == a.data (same pointer!)
// both destructors run: double-free crash
```

The fix: Implement copy constructor and copy assignment (or use `= delete` to forbid copying).

Mistake 2: Forgetting the Self-Assignment Guard

The bug:

```
Buffer& operator=(const Buffer& other) {
    delete[] data;          // frees data
    size = other.size;
    data = new int[size]{};
    std::copy(other.data, ...); // if other IS *this, other.data was just deleted!
}
// buf = buf; --> crash
```

The fix: if (`this == &other`) return `*this`; at the top of copy assignment.

Exercises

Exercise 16.1 – Rule of Zero or Three?

For each class, say whether the Rule of Zero (no user-defined specials needed) or Rule of Three/Five applies:

```
struct Point { double x, y; };           // (a)
class Socket { int fd; public: Socket(int f):fd{f}{} // (b)
    ~Socket() { close(fd); } };
struct Config { std::string name; std::vector<int> v; }; // (c)
class RawArr { int* p; int n; public: RawArr(int n):p{new int[n]{}},n{n}{}
    ~RawArr(){delete[]p;} };           // (d)
```

Answer: - (a): Rule of Zero – double members, no resource ownership. Compiler generates correct copy/move. - (b): Rule of Five – owns a file descriptor (OS resource). Needs explicit copy/move handling (probably = delete for copy, custom move that sets fd = -1). - (c): Rule of Zero – std::string and std::vector manage their own resources. Compiler generates correct defaults. - (d): Rule of Five – owns a raw int*. Needs destructor (done), copy constructor (deep copy), copy assignment, move constructor (steal pointer), move assignment.

Exercise 16.2 – Complete the Rule of Five

Complete RawArr from the exercise above with all five special members. The copy should make a deep copy; the move should steal the pointer.

Answer:

```
class RawArr {
    int* p;
    int n;
public:
    RawArr(int sz) : p{new int[sz]{}}, n{sz} {}

    ~RawArr() { delete[] p; }

    RawArr(const RawArr& o) : p{new int[o.n]{}}, n{o.n} {
        std::copy(o.p, o.p + n, p);
    }

    RawArr& operator=(const RawArr& o) {
        if (this == &o) return *this;
        delete[] p;
        n = o.n;
        p = new int[n]{};
        std::copy(o.p, o.p + n, p);
        return *this;
    }

    RawArr(RawArr&& o) noexcept : p{o.p}, n{o.n} {
        o.p = nullptr; o.n = 0;
    }

    RawArr& operator=(RawArr&& o) noexcept {
        if (this == &o) return *this;
        delete[] p;
        p = o.p; n = o.n;
        o.p = nullptr; o.n = 0;
        return *this;
    }
};
```

Part III is complete. You now understand the ownership model that makes C++ both safe and fast: RAII ties resource lifetime to object lifetime, smart pointers automate ownership, move semantics avoid unnecessary copies, and the Rule of Zero/Five ensures correct copy and move behavior for your own types.

Part IV covers object-oriented programming – classes, constructors, inheritance, virtual functions, and polymorphism. Ask to continue.

Part IV – Object-Oriented Programming

C++ OOP looks similar to Python OOP on the surface – both have classes, inheritance, and virtual dispatch. The differences are in where they live (stack vs heap), how they are initialized (constructor member initializer lists), what you must declare explicitly (virtual, override, abstract), and the performance implications of each choice.

Chapter 17: Classes, Objects, and Encapsulation

Python Classes vs C++ Classes

```
# Python class
class Rectangle:
    def __init__(self, width, height):
        self.width = width      # no access control -- everything is public
        self.height = height

    def area(self):
        return self.width * self.height

r = Rectangle(3, 4)
r.width = -5                # Python cannot stop this
print(r.area())             # -20 -- corrupt state
```

```
// C++ class
class Rectangle {
public:                        // public interface
    Rectangle(double w, double h); // constructor declaration
    double area() const;         // method declaration

private:                     // hidden implementation
    double width;              // cannot be accessed from outside
    double height;
};
```

The key difference: **access specifiers**. C++ divides class members into:

- `public`: accessible by anyone
- `private`: accessible only by the class's own member functions
- `protected`: accessible by the class and its derived classes (Chapter 19)

In Python everything is public by convention (with `_` as a hint). In C++, `private` is enforced by the compiler.

Defining a Class: Full Example

```
// Rectangle.h
#pragma once
#include <string>

class Rectangle {
public:
    // Constructor
    Rectangle(double width, double height);

    // Const methods (do not modify the object)
    double area() const;
    double perimeter() const;
    std::string describe() const;

    // Getters and setters with validation
    double get_width() const;
    double get_height() const;
    void set_width(double w);
    void set_height(double h);

private:
    double width;
    double height;
};
```

```
// Rectangle.cpp
#include "Rectangle.h"
#include <stdexcept>
#include <string>

// Constructor: ClassName::method_name(params) { body }
Rectangle::Rectangle(double w, double h) {
    if (w <= 0 || h <= 0)
        throw std::invalid_argument("Dimensions must be positive");
    width = w;
    height = h;
}

double Rectangle::area() const { return width * height; }
double Rectangle::perimeter() const { return 2.0 * (width + height); }

std::string Rectangle::describe() const {
    return "Rectangle(" + std::to_string(width) + " x "
        + std::to_string(height) + ")";
}

double Rectangle::get_width() const { return width; }
double Rectangle::get_height() const { return height; }

void Rectangle::set_width(double w) {
    if (w <= 0) throw std::invalid_argument("Width must be positive");
    width = w;
}

void Rectangle::set_height(double h) {
    if (h <= 0) throw std::invalid_argument("Height must be positive");
    height = h;
}
```

```
// main.cpp
#include <iostream>
#include "Rectangle.h"

int main() {
    Rectangle r{3.0, 4.0};           // calls constructor
    std::cout << r.area()           << "\n"; // 12
    std::cout << r.perimeter()      << "\n"; // 14
    std::cout << r.describe()       << "\n"; // Rectangle(3.000000 x 4.000000)

    r.set_width(5.0);
    std::cout << r.area() << "\n"; // 20

    // r.width = -1; // COMPILE ERROR: 'width' is private
    // r.set_width(-1); // throws std::invalid_argument at runtime
}
```

struct vs class

In C++, `struct` and `class` are almost identical. The only difference: `struct` members are public by default; `class` members are private by default.

```
struct Point {           // members are public by default
    double x, y;
};

class Point2 {           // members are private by default
    double x, y;         // these are private
};

Point p{1.0, 2.0};       // p.x accessible
Point2 q{1.0, 2.0};      // q.x NOT accessible (private)
```

Convention: - Use `struct` for simple data aggregates with no invariants to protect (Point, Color, Size) - Use `class` when you need access control and invariants (Rectangle must have positive dimensions)

Memory Layout of a Class

```
class Vec2 {
    float x; // 4 bytes
    float y; // 4 bytes
};

Vec2 v{1.0f, 2.0f};
```

Stack (or wherever `v` lives):

Address	Member	Bytes	
0x1000	x (float)	[00][00][80][3F]	<- 1.0f in IEEE 754
0x1004	y (float)	[00][00][00][40]	<- 2.0f in IEEE 754

`sizeof(Vec2) == 8` (same as two separate floats)

There is no hidden per-object overhead from member functions. Functions exist once in the `.text` section of the executable. The object just holds data.

Code (compiled once, shared by all `Vec2` instances):

```
.text:
Vec2::area():
    mov eax, [this+0]    <- reads this->x
    ...
```

The `this` pointer (implicit first parameter of every member function) tells the function which object to operate on.

this Pointer

Inside any non-static member function, `this` is a pointer to the current object:

```
class Counter {
    int count{0};
public:
    Counter& increment() {
        ++count;          // same as: ++(this->count)
        return *this;     // return reference to self (enables chaining)
    }
    int get() const { return count; }
};

Counter c;
c.increment().increment().increment(); // method chaining
std::cout << c.get(); // 3
```

`return *this` returns a reference to the object itself, allowing `.method()` to be called immediately on the result. This pattern (fluent interface / method chaining) is common in builder classes.

Static Members

static data members and methods belong to the **class**, not to any individual object:

```

class Player {
    std::string name;
    static int player_count;    // shared across ALL Player objects

public:
    Player(std::string n) : name{n} { ++player_count; }
    ~Player()                { --player_count; }

    static int get_count() { return player_count; }
    // static methods have no 'this' -- cannot access non-static members
};

int Player::player_count = 0;    // definition outside the class (required)

Player p1{"Alice"};
Player p2{"Bob"};
std::cout << Player::get_count() << "\n";    // 2 (call via class name)
{
    Player p3{"Carol"};
    std::cout << Player::get_count() << "\n";    // 3
}    // p3 destroyed
std::cout << Player::get_count() << "\n";    // 2

```

Common Mistakes in This Chapter

Mistake 1: Forgetting the :: Scope Resolution in the .cpp File

The bug:

```

// Rectangle.cpp
double area() const {    // ERROR: this defines a FREE function named area,
    return width * height;    // not Rectangle::area. 'width' is not in scope.
}

```

The fix: `double Rectangle::area() const { return width * height; }`

Mistake 2: Calling a Non-const Method on a Const Object

The bug:

```

const Rectangle r{3.0, 4.0};
r.set_width(5.0);    // ERROR: set_width is non-const (it modifies the object)

```

Compiler message:

error: passing 'const Rectangle' as 'this' argument discards qualifiers

The fix: Either remove `const` from the object, or don't call mutating methods on `const` objects.

Mistake 3: Forgetting to Define Static Members

The bug:

```
class Foo { static int count; };
// Forgot: int Foo::count = 0;
// Linker error: undefined reference to 'Foo::count'
```

The fix: Static data members must be defined exactly once in a .cpp file.

Exercises

Exercise 17.1 – Design a class

Design a `BankAccount` class with: - Private balance (double, starts at 0) - `deposit(double amount)` - adds to balance (reject negative amounts) - `withdraw(double amount)` - subtracts (reject if insufficient funds) - `get_balance()` const - returns balance

Answer:

```
class BankAccount {
    double balance{0.0};
public:
    void deposit(double amount) {
        if (amount <= 0) throw std::invalid_argument("Amount must be positive");
        balance += amount;
    }
    void withdraw(double amount) {
        if (amount <= 0) throw std::invalid_argument("Amount must be positive");
        if (amount > balance) throw std::runtime_error("Insufficient funds");
        balance -= amount;
    }
    double get_balance() const { return balance; }
};
```

Exercise 17.2 – Const correctness

Which of these methods should be const?

```
class Circle {
    double radius;
public:
    void set_radius(double r) { radius = r; }
    double get_radius() { return radius; }
    double area() { return 3.14159 * radius * radius; }
    bool is_unit_circle() { return radius == 1.0; }
};
```

Answer: `get_radius()`, `area()`, and `is_unit_circle()` should all be const – none of them modify `radius`. `set_radius()` cannot be const because it assigns to `radius`.

Chapter 18: Constructors, Destructors, and Initialization

The Member Initializer List

C++ constructors have a special syntax for initializing members **before the constructor body runs**. This is the **member initializer list**, the `:` clause between the parameter list and `{`:

```
class Particle {
    double x, y;
    double vx, vy;
    std::string name;

public:
    // WITHOUT member initializer list (suboptimal):
    Particle(double px, double py, std::string n) {
        x = px;    // assignment (not initialization)
        y = py;    // members are default-constructed first, then assigned
        vx = 0.0;
        vy = 0.0;
        name = n;  // name is default-constructed to "" then copy-assigned
    }

    // WITH member initializer list (correct and efficient):
    Particle(double px, double py, std::string n)
        : x{px}, y{py}, vx{0.0}, vy{0.0}, name{std::move(n)}
    { // body runs after all members are initialized
    }
};
```

The member initializer list directly constructs each member with the given value. The assignment version first default-constructs each member, then overwrites it – two operations instead of one.

For `std::string`, `std::vector`, and any class type, always use the initializer list. For `const` members and reference members, the initializer list is not optional – they **cannot** be assigned after construction:

```
class Fixed {
    const int id;
    int& ref;
public:
    Fixed(int i, int& r) : id{i}, ref{r} {} // MUST use initializer list
    // id = i; // COMPILE ERROR: cannot assign to const member
};
```


Initialization Order

Members are initialized in the **order they are declared in the class**, not the order they appear in the initializer list:

```
class Tricky {
    int a;
    int b;
public:
    Tricky(int x) : b{x}, a{b} // WARNING: a is initialized before b!
    {}
    // a is initialized with b's value, but b hasn't been initialized yet
    // 'a' gets garbage
};
```

Rule: Put initializers in the same order as the declarations. Most compilers warn about order mismatches with `-Wall`.

Delegating Constructors (C++11)

One constructor can call another constructor of the same class:

```
class Vec3 {
    double x, y, z;
public:
    Vec3(double x, double y, double z) : x{x}, y{y}, z{z} {}

    // Delegate to the main constructor:
    Vec3() : Vec3{0.0, 0.0, 0.0} {} // zero vector
    Vec3(double uniform) : Vec3{uniform, uniform, uniform} {}
};

Vec3 origin; // (0, 0, 0)
Vec3 ones{1.0}; // (1, 1, 1)
Vec3 point{1.0, 2.0, 3.0};
```

Constructor Kinds

Default Constructor

Called when no arguments are provided:

```
class Foo {
public:
    Foo() { std::cout << "default constructed\n"; }
};
Foo f;           // calls Foo()
Foo g{};         // also calls Foo()
std::vector<Foo> v(5); // calls Foo() five times
```

If you declare **any** constructor, the compiler no longer generates a default constructor automatically. Add `Foo() = default;` to get it back.

Explicit Constructor (Prevent Implicit Conversion)

```
class Radius {
    double value;
public:
    Radius(double v) : value{v} {} // implicit: Radius r = 5.0; works
};

class SafeRadius {
    double value;
public:
    explicit SafeRadius(double v) : value{v} {} // explicit: no implicit conversion
};

Radius    r1 = 5.0;           // OK: implicit conversion double -> Radius
SafeRadius r2 = 5.0;          // ERROR: explicit constructor cannot convert implicitly
SafeRadius r3{5.0};          // OK: direct initialization always works
SafeRadius r4(5.0);          // OK: direct initialization

void process(Radius r) {}
process(5.0);                 // OK for Radius (implicit conversion allowed)
process(SafeRadius{5.0});     // OK: explicit construction then pass
```

Use `explicit` for single-argument constructors to prevent surprising implicit conversions. `std::vector<int>` is `explicit` – `std::vector<int> v = 5;` would be confusing.

Destructor Revisited: The Full Rules

```
class Resource {
    int* data;
public:
    Resource() : data{new int[100]} {} { std::cout << "acquired\n"; }
    ~Resource() { delete[] data; std::cout << "released\n"; }
};
```

Destructors are called: 1. **When a local variable goes out of scope** (stack unwinding) 2. **When delete is called** on a heap-allocated object 3. **When a container element is removed** (vector : : pop_back, etc.) 4. **When a member variable's owner is destroyed** (member destructors run after the owner's destructor body)

The destructor call order for members is the **reverse of construction order**.

In-Class Member Initializers (C++11)

Members can have default values directly in the class definition:

```
class Config {
    int    width{1920};        // default: 1920
    int    height{1080};       // default: 1080
    bool   fullscreen{false};  // default: false
    std::string title{"My App"};

public:
    Config() = default;        // uses all defaults above

    Config(int w, int h) // overrides width and height, keeps other defaults
        : width{w}, height{h} {};

};

Config default_cfg;           // 1920x1080, windowed, "My App"
Config custom{800, 600};     // 800x600, windowed, "My App"
```

This is the cleanest way to set defaults. Prefer it over setting values in the constructor body.

Common Mistakes in This Chapter

Mistake 1: Initializer List Order Different From Declaration Order

The bug:

```
class Pair {
    int second;    // declared first
    int first;     // declared second
public:
    Pair(int f, int s) : first{f}, second{s} {}
    // first is initialized with f -- but second is initialized FIRST (declaration order)
    // and second{s} is fine since s is not dependent on first
    // If instead: Pair(int x) : first{x}, second{first*2}{}
    // then second = first*2 but first hasn't been initialized yet!
};
```

The fix: Keep initializer list order matching declaration order.

Mistake 2: Missing `explicit` on Converting Constructor

The bug:

```
class Seconds { double val; public: Seconds(double v):val{v}{} };
void wait(Seconds s) { ... }
wait(100); // Did you mean 100 seconds? Or was 100 an int you forgot to convert?
```

The fix: `explicit Seconds(double v)` forces `wait(Seconds{100})` – the intent is clear.

Exercises

Exercise 18.1 – Rewrite with initializer list

Rewrite this constructor to use a member initializer list:

```
class Player {
    std::string name;
    int hp;
    int max_hp;
public:
    Player(std::string n, int health) {
        name = n;
        hp = health;
        max_hp = health;
    }
};
```

Answer:

```
Player(std::string n, int health)
: name{std::move(n)}, hp{health}, max_hp{health} {}
```

`std::move(n)` avoids copying the string (since `n` is a value parameter, it's a local copy we can move from).

Exercise 18.2 – Delegating constructors

Write a `Color` class with `r`, `g`, `b` components (0.0-1.0 doubles). Provide: - A full 3-argument constructor - A grayscale constructor taking one value (sets `r=g=b`) - A default constructor (black: 0,0,0)

Use delegating constructors.

Answer:

```
class Color {  
    double r, g, b;  
public:  
    Color(double r, double g, double b) : r{r}, g{g}, b{b} {}  
    Color(double gray) : Color{gray, gray, gray} {}  
    Color() : Color{0.0} {}  
};
```

Chapter 19: Inheritance and Composition

Two Kinds of Reuse

When one class needs functionality from another, you have two options:

- **Inheritance (“is-a”)**: Dog is-a Animal. Dog inherits from Animal.
- **Composition (“has-a”)**: Car has-a Engine. Car has an Engine as a member.

Python programmers often overuse inheritance. Prefer composition in C++ (and generally): it is more flexible and avoids the tight coupling that inheritance creates.

Inheritance Syntax

```
# Python
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        return "..."
```

```
class Dog(Animal):
    def speak(self):
        return "Woof"
```

```
// C++
class Animal {
protected:           // accessible by Animal and derived classes
    std::string name;
public:
    Animal(const std::string& n) : name{n} {}
    std::string get_name() const { return name; }
    virtual std::string speak() const { return "..."; }
    virtual ~Animal() {} // virtual destructor -- explained in Chapter 20
};

class Dog : public Animal { // Dog inherits publicly from Animal
public:
    Dog(const std::string& n) : Animal{n} {} // must call base constructor

    std::string speak() const override { return "Woof"; }
    // ^--- override keyword (C++11): verifies this
```

```
// actually overrides a virtual base method
};
```

The public Inheritance Keyword

class Dog : public Animal – the public here controls how the base class’s access specifiers are inherited:

Inheritance type	public becomes	protected becomes	private becomes
public	public	protected	inaccessible
protected	protected	protected	inaccessible
private	private	private	inaccessible

Almost always use public inheritance. private and protected inheritance are rare, specialized tools.

Constructors in Derived Classes

A derived class constructor **must** call the base class constructor:

```
class Shape {
    std::string color;
public:
    Shape(const std::string& c) : color{c} {}
    std::string get_color() const { return color; }
};

class Circle : public Shape {
    double radius;
public:
    Circle(double r, const std::string& c)
        : Shape{c}           // must explicitly call base constructor
        , radius{r}         // then initialize own members
    {}
    double area() const { return 3.14159 * radius * radius; }
};
```

If the base class has no default constructor, the derived class **must** explicitly call a base constructor in its initializer list. The base part is always constructed first, before the derived members.

Memory Layout With Inheritance

```
class Base {
    int x;    // 4 bytes
    int y;    // 4 bytes
};

class Derived : public Base {
    int z;    // 4 bytes
};

Derived d;
```

Memory layout of d (Derived):

Address	Member	Size	
0x1000	x (Base)	4 bytes	<- Base part comes first
0x1004	y (Base)	4 bytes	
0x1008	z (Derived)	4 bytes	<- Derived's own members after

`sizeof(Derived) == 12`

The base class subobject lives at the beginning of the derived object's memory. A pointer to `Derived` is also a valid pointer to `Base` (just points to the same address – the `Base` subobject is right there).

Slicing: The Object-Oriented Trap

```
Animal a = Dog{"Rex"};    // BAD: Dog is sliced into an Animal!
a.speak();                // calls Animal::speak, not Dog::speak -- "..."
```

When you copy a `Dog` into an `Animal` variable, only the `Animal` part is copied. The `Dog`-specific data is sliced off. The object is now truly just an `Animal`.

Before slice:		After slice:
+-----+		+-----+
Dog		Animal
name: "Rex"	-->	name: "Rex"
(Dog data)		(Dog data GONE)
+-----+		+-----+

To avoid slicing, use pointers or references to the base class:


```
// Correct: pointer to Animal, no slicing
std::unique_ptr<Animal> a = std::make_unique<Dog>("Rex");
a->speak(); // "Woof" -- polymorphic dispatch (Chapter 20)

// Correct: reference to Animal, no slicing
Dog dog{"Rex"};
Animal& ref = dog;
ref.speak(); // "Woof" -- polymorphic dispatch
```

Composition vs Inheritance

Most “is-a” relationships in the real world are actually “can-do” or “has-a”. Prefer composition:

```
// Inheritance: Engine IS-A Vehicle? No. Engine is part of Vehicle.
class Engine { void start(); void stop(); };
class CarBad : public Engine {}; // wrong: Car is NOT an Engine

// Composition: Car HAS-A Engine
class CarGood {
    Engine engine; // Engine is a component
public:
    void start() { engine.start(); }
};
```

Prefer composition when: - You need the functionality of another class but are not that class - You want to change the implementation later without affecting callers - You want to combine multiple behaviors (C++ has single inheritance)

Use inheritance when: - There is a genuine “is-a” relationship - You need polymorphic dispatch (Chapter 20) through a common base pointer

Common Mistakes in This Chapter

Mistake 1: Forgetting to Call the Base Constructor

The bug:

```
class Derived : public Base {
public:
    Derived(int x) { ... } // Base's constructor never called!
    // If Base has no default constructor: compile error
    // If Base has a default constructor: it is called, possibly not what you want
};
```

The fix: `Derived(int x) : Base{...} { ... }`

Mistake 2: Object Slicing

The bug:

```
void process(Animal a) { a.speak(); } // takes by value -- slices!
process(Dog{"Rex"}); // Dog is sliced to Animal; speak() returns "..."
```

The fix: `void process(const Animal& a)` or `void process(Animal* a)` – reference/pointer avoids slicing.

Exercises

Exercise 19.1 – Class hierarchy

Design a small hierarchy: `Shape` (base), `Rectangle` and `Circle` (derived). Each should have `area()` `const` and `perimeter()` `const`. Use composition or inheritance as appropriate.

Answer (abbreviated):

```
class Shape {
public:
    virtual double area() const = 0; // pure virtual (Chapter 21)
    virtual double perimeter() const = 0;
    virtual ~Shape() {}
};

class Rectangle : public Shape {
    double w, h;
public:
    Rectangle(double w, double h) : w{w}, h{h} {}
    double area() const override { return w * h; }
    double perimeter() const override { return 2*(w+h); }
};

class Circle : public Shape {
    double r;
public:
    Circle(double r) : r{r} {}
    double area() const override { return 3.14159265 * r * r; }
    double perimeter() const override { return 2 * 3.14159265 * r; }
};
```


Chapter 20: Virtual Functions and Polymorphism

The Problem Without Virtual

```
class Animal {
public:
    std::string speak() const { return "..."; } // not virtual
};

class Dog : public Animal {
public:
    std::string speak() const { return "Woof"; } // hides, not overrides
};

Animal* a = new Dog{};
std::cout << a->speak(); // "..." -- calls Animal::speak, not Dog::speak!
```

Without `virtual`, the function called is determined by the **type of the pointer** at compile time. `a` is `Animal*`, so `Animal::speak` is called. The fact that the object is actually a `Dog` is ignored.

This is called **static dispatch** (compile-time resolution).

`virtual` Enables Dynamic Dispatch

Adding `virtual` tells the compiler to resolve the call at **runtime** based on the actual type of the object:

```
class Animal {
public:
    virtual std::string speak() const { return "..."; }
    virtual ~Animal() {} // always virtual destructor in polymorphic base classes
};

class Dog : public Animal {
public:
    std::string speak() const override { return "Woof"; }
};

class Cat : public Animal {
public:
    std::string speak() const override { return "Meow"; }
};
```

```
};

Animal* a1 = new Dog{};
Animal* a2 = new Cat{};
std::cout << a1-> speak() << "\n"; // "Woof" -- dynamic dispatch to Dog::speak
std::cout << a2-> speak() << "\n"; // "Meow" -- dynamic dispatch to Cat::speak
delete a1;
delete a2;
```

The same pointer type (`Animal*`), the same call (`->speak()`), different behavior based on what the pointer actually points to. This is **runtime polymorphism**.

How Virtual Dispatch Works: The vtable

For each class with virtual functions, the compiler creates a **vtable** (virtual function table): an array of function pointers, one per virtual function.

Each object of a polymorphic class has a hidden **vp**tr (vtable pointer) pointing to its class's vtable:

Memory layout of a Dog object (with virtual speak):

```
+-----+
| vp
```

tr -----> Dog's vtable:
| | [0] --> Dog::speak()
| name (string) | [1] --> Dog::~Dog()
+-----+

Memory layout of a Cat object:

```
+-----+
| vp
```

tr -----> Cat's vtable:
| | [0] --> Cat::speak()
| name (string) | [1] --> Cat::~Cat()
+-----+

When you call `a->speak()`: 1. Load `a` (the pointer) 2. Follow `a` to the object 3. Load `vp`tr from the object (first bytes of the object) 4. Index into the vtable: `vp`tr[0] 5. Call the function pointer found there

The overhead: one pointer dereference and one indirect call. Tiny but non-zero. For hot inner loops, this matters (Chapter 38 covers alternatives).

override and final (C++11)

`override` tells the compiler to verify that this function actually overrides a virtual base function. Without it, a typo silently creates a new function instead:

```
class Animal {
    virtual std::string speak() const { return "..."; }
};

class Dog : public Animal {
    std::string speak() const { return "Woof"; } // typo! No error without override.
    // Does not override speak(). Dog::speak is a new function.
    // Animal::speak is still called for Dog objects via base pointer.
};

class DogCorrect : public Animal {
    std::string speak() const override { return "Woof"; }
    // ERROR: 'speak' does not override any virtual function
    // Compiler catches the typo.
};
```

Always use `override`. No downside, catches bugs.

`final` prevents further overriding (for a method) or prevents inheritance (for a class):

```
class SafeAnimal : public Animal {
    std::string speak() const override final { return "safe"; }
    // no derived class can override speak() anymore
};

class Terminal final : public Animal { ... };
// class Sub : public Terminal {}; // ERROR: Terminal is final
```

Virtual Destructor: Why It Is Required

```
class Base {
public:
    ~Base() { std::cout << "Base destroyed\n"; } // NOT virtual
};

class Derived : public Base {
    int* data;
public:
    Derived() : data{new int[100]} {}
    ~Derived() { delete[] data; std::cout << "Derived destroyed\n"; }
};

Base* p = new Derived{};
delete p; // calls Base::~~Base ONLY -- Derived::~~Derived never called!
          // 'data' is leaked every time
```

When you delete through a base class pointer and the destructor is not virtual, only the base class destructor is called. The derived class's destructor (and its cleanup code) is bypassed.

```
class Base {
public:
    virtual ~Base() { std::cout << "Base destroyed\n"; } // virtual
};
// Now: delete p; calls Derived::~Derived first (via vtable), then Base::~Base
```

Rule: If a class has any virtual functions, make its destructor virtual. The overhead is one vtable slot – negligible.

Polymorphism With `unique_ptr`

Modern C++ polymorphism uses smart pointers, not raw pointers:

```
#include <memory>
#include <vector>

std::vector<std::unique_ptr<Animal>> animals;
animals.push_back(std::make_unique<Dog>("Rex"));
animals.push_back(std::make_unique<Cat>("Whiskers"));
animals.push_back(std::make_unique<Dog>("Buddy"));

for (const auto& a : animals) {
    std::cout << a->get_name() << " says: " << a->speak() << "\n";
}
// Rex says: Woof
// Whiskers says: Meow
// Buddy says: Woof
// All Animals deleted automatically when vector is destroyed
```

No manual delete. No memory leaks. Full polymorphism.

Python Polymorphism vs C++ Polymorphism

Python uses **duck typing**: no inheritance required. Any object with a `speak()` method works.

```
class Dog: def speak(self): return "Woof"
class Cat: def speak(self): return "Meow"
class Rock: pass # no speak

animals = [Dog(), Cat()]
for a in animals:
    print(a.speak()) # works because both have speak()
```

C++ polymorphism requires: 1. A common base class with `virtual` functions 2. The object accessed through a base pointer or reference 3. `override` in derived classes

C++ offers **static polymorphism** via templates (Chapter 23/25) for zero-overhead duck typing.

Common Mistakes in This Chapter

Mistake 1: Non-Virtual Base Destructor

Already covered above. Always make the destructor `virtual` in a polymorphic base class.

Mistake 2: Forgetting `override`

The bug:

```
class Derived : public Base {
    void render() { ... }    // Base has: virtual void Render() { ... }
    // typo: render vs Render -- silent new function, not an override
};
```

The fix: Always write `override`. The compiler will tell you if the name doesn't match.

Mistake 3: Calling Virtual Functions in Constructors/Destructors

The bug:

```
class Base {
public:
    Base() { initialize(); }    // calls virtual -- WRONG
    virtual void initialize() { std::cout << "Base\n"; }
};
class Derived : public Base {
public:
    void initialize() override { std::cout << "Derived\n"; }
};
Derived d; // prints "Base", not "Derived" -- vtable not fully set up yet
```

Why: During the base constructor, the object is still a Base. The vtable points to Base's functions. Dynamic dispatch is suspended during construction and destruction. **The fix:** Never call virtual functions in constructors or destructors. Use a factory function or a two-phase initialization if needed.

Exercises

Exercise 20.1 – Virtual dispatch trace

```
struct A {
    virtual void f() { std::cout << "A\n"; }
    void g()         { std::cout << "A::g\n"; }
};
struct B : A {
    void f() override { std::cout << "B\n"; }
    void g()         { std::cout << "B::g\n"; }
};
A* p = new B{};
p->f();    // (a)
p->g();    // (b)
B* q = static_cast<B*>(p);
q->f();    // (c)
q->g();    // (d)
delete p;
```

Answer: - (a): B – f () is virtual, dynamic dispatch to B : : f - (b): A : : g – g () is not virtual, compile-time resolution via A* - (c): B – f () is virtual, q is B*, dispatch to B : : f - (d): B : : g – g () is not virtual, compile-time resolution via B*

Exercise 20.2 – Polymorphic zoo

Create a vector of `unique_ptr<Animal>`, add at least 3 different animal types, and print each one's `speak()` result using a range-based loop.

Answer:

```
#include <memory>
#include <vector>
#include <iostream>

// Using the Animal/Dog/Cat hierarchy from this chapter:
std::vector<std::unique_ptr<Animal>> zoo;
zoo.push_back(std::make_unique<Dog>("Rex"));
zoo.push_back(std::make_unique<Cat>("Whiskers"));
zoo.push_back(std::make_unique<Dog>("Buddy"));

for (const auto& animal : zoo) {
    std::cout << animal->get_name() << ": " << animal->speak() << "\n";
}
```

Chapter 21: Abstract Classes and Interfaces

Pure Virtual Functions

A **pure virtual function** is a virtual function declared with `= 0`. It has no implementation in the base class. Any class with at least one pure virtual function is an **abstract class** – it cannot be instantiated.

```
class Shape {
public:
    virtual double area() const = 0; // pure virtual
    virtual double perimeter() const = 0; // pure virtual
    virtual void draw() const = 0; // pure virtual
    virtual ~Shape() {}
};

Shape s; // COMPILE ERROR: cannot instantiate abstract class 'Shape'
```

A class is abstract to express: “I define the interface; subclasses provide the implementation.”

Concrete Derived Classes

A derived class that provides implementations for all pure virtual functions is **concrete** (can be instantiated):

```
class Circle : public Shape {
    double radius;
public:
    Circle(double r) : radius{r} {}

    double area() const override { return 3.14159265 * radius * radius; }
    double perimeter() const override { return 2 * 3.14159265 * radius; }
    void draw() const override { std::cout << "0\n"; }
};

class Square : public Shape {
    double side;
public:
    Square(double s) : side{s} {}

    double area() const override { return side * side; }
    double perimeter() const override { return 4 * side; }
    void draw() const override { std::cout << "1\n"; }
};
```

```
// Now concrete:
Circle c{5.0};
Square s{3.0};
std::cout << c.area() << "\n"; // 78.5...
std::cout << s.area() << "\n"; // 9
```

If a derived class leaves any pure virtual functions unimplemented, it is also abstract:

```
class PartialShape : public Shape {
    double area() const override { return 0; }
    // perimeter() and draw() not implemented -- PartialShape is still abstract
};
PartialShape p; // COMPILE ERROR: still abstract
```

Interfaces in C++

C++ does not have a built-in interface keyword (unlike Java or C#). An **interface** is a convention: an abstract class with only pure virtual functions and a virtual destructor – no data members, no implementation.

```
# Python "interface" via ABC
from abc import ABC, abstractmethod

class Drawable(ABC):
    @abstractmethod
    def draw(self) -> None: ...

    @abstractmethod
    def bounding_box(self) -> tuple: ...
```

```
// C++ interface convention
class IDrawable {
public:
    virtual void draw() const = 0;
    virtual std::pair<double,double> bounding_box() const = 0;
    virtual ~IDrawable() {}
};

class ISerializable {
public:
    virtual std::string serialize() const = 0;
    virtual void deserialize(const std::string&) = 0;
    virtual ~ISerializable() {}
};

// A class can implement multiple interfaces:
class Sprite : public IDrawable, public ISerializable {
    // must implement all pure virtuals from both interfaces
    void draw() const override { ... }
```

```
std::pair<double,double> bounding_box() const override { ... }
std::string serialize() const override { ... }
void deserialize(const std::string&) override { ... }
};
```

Multiple inheritance from multiple **pure-interface** base classes is safe and common. Multiple inheritance from classes with data members is tricky (diamond problem) and usually avoided.

Partial Implementations (Partially Abstract)

A class can implement some pure virtuals and leave others:

```
class AbstractRenderer : public IDrawable {
public:
    // Provides a default implementation for draw:
    void draw() const override {
        set_up_context();
        do_draw(); // calls another pure virtual -- template method pattern
        tear_down_context();
    }
    // Subclasses must provide do_draw():
    virtual void do_draw() const = 0;
protected:
    void set_up_context() const { ... }
    void tear_down_context() const { ... }
};

class OpenGLRenderer : public AbstractRenderer {
    void do_draw() const override { ... } // only override what is necessary
};
```

This is the **Template Method** pattern: the base class defines the algorithm skeleton, derived classes fill in the specific steps.

Checking the Type at Runtime: `dynamic_cast`

When you have a base class pointer and need to find out the real type:

```
void process(Shape* s) {
    if (Circle* c = dynamic_cast<Circle*>(s)) {
        // s points to a Circle (or derived from Circle)
        std::cout << "radius: " << c->radius << "\n";
    } else if (Square* sq = dynamic_cast<Square*>(s)) {
        std::cout << "side: " << sq->side << "\n";
    }
}
```

`dynamic_cast<Circle*>(s)` returns a `Circle*` if `s` actually points to a `Circle`, or `nullptr` otherwise. It works via the vtable (requires at least one virtual function).

Caveat: Frequent `dynamic_cast` is usually a design smell. It means the code is asking “what type is this?” at runtime instead of using virtual dispatch. Prefer redesigning with virtual functions. `dynamic_cast` is legitimate for occasional introspection or when interacting with external code.

Common Mistakes in This Chapter

Mistake 1: Instantiating an Abstract Class

The bug:

```
Shape s{};    // or: Shape* p = new Shape{};
```

Compiler error:

```
error: cannot declare variable 's' to be of abstract type 'Shape'
note: because the following virtual functions are pure within 'Shape':
note: virtual double Shape::area() const
```

The fix: Instantiate a concrete derived class, not the abstract base.

Mistake 2: Missing `override` in Concrete Class

The bug:

```
class Rect : public Shape {
    double area() const { return w * h; } // forgot override
    // If Shape::area() signature changes, this silently stops overriding it
};
```

The fix: Always write `override`. If the signature in the base changes, the compiler will tell you.

Exercises

Exercise 21.1 – Design an interface

Design a `ILogger` interface with methods `log_info(std::string)`, `log_warning(std::string)`, `log_error(std::string)`. Then write two implementations: `ConsoleLogger` (prints to cout) and `NullLogger` (does nothing – useful for disabling logging in tests).

Answer:

```
class ILogger {
public:
    virtual void log_info(const std::string& msg)    = 0;
    virtual void log_warning(const std::string& msg) = 0;
    virtual void log_error(const std::string& msg)   = 0;
    virtual ~ILogger() {}
};

class ConsoleLogger : public ILogger {
public:
    void log_info(const std::string& msg)    override {
        std::cout << "[INFO] " << msg << "\n"; }
    void log_warning(const std::string& msg) override {
        std::cout << "[WARN] " << msg << "\n"; }
    void log_error(const std::string& msg)   override {
        std::cout << "[ERROR] " << msg << "\n"; }
};

class NullLogger : public ILogger {
public:
    void log_info(const std::string&)    override {}
    void log_warning(const std::string&) override {}
    void log_error(const std::string&)   override {}
};

// Usage: inject the logger as a pointer to interface
void do_work(ILogger& log) {
    log.log_info("Starting work");
    // ...
    log.log_warning("Something unusual");
}

ConsoleLogger cl;
do_work(cl);    // prints

NullLogger nl;
do_work(nl);    // prints nothing (testing/silent mode)
```


Chapter 22: Operator Overloading

What Operator Overloading Is

C++ lets you define what the built-in operators (+, -, ==, <, <<, [], etc.) do for your own types. This is called **operator overloading**.

Python calls these **dunder methods** (double underscore):

```
class Vec2:
    def __init__(self, x, y): self.x, self.y = x, y
    def __add__(self, other): return Vec2(self.x+other.x, self.y+other.y)
    def __repr__(self):      return f"Vec2({self.x}, {self.y})"

a = Vec2(1, 2)
b = Vec2(3, 4)
print(a + b)    # Vec2(4, 6)
```

```
struct Vec2 {
    double x, y;

    Vec2 operator+(const Vec2& other) const {
        return Vec2{x + other.x, y + other.y};
    }
};

// Non-member: output stream operator
std::ostream& operator<<(std::ostream& os, const Vec2& v) {
    return os << "Vec2(" << v.x << ", " << v.y << ")";
}

Vec2 a{1.0, 2.0};
Vec2 b{3.0, 4.0};
Vec2 c = a + b;
std::cout << c << "\n";    // Vec2(4, 6)
```

Which Operators to Overload

Not all operators make sense for all types. Overload only what has a clear, unsurprising meaning for your type.


```

struct Vec2 {
    double x, y;

    // Arithmetic (member: left operand is *this)
    Vec2 operator+(const Vec2& o) const { return {x+o.x, y+o.y}; }
    Vec2 operator-(const Vec2& o) const { return {x-o.x, y-o.y}; }
    Vec2 operator*(double s) const { return {x*s, y*s }; }
    Vec2 operator-( ) const { return {-x, -y }; } // unary minus

    // Compound assignment
    Vec2& operator+=(const Vec2& o) { x+=o.x; y+=o.y; return *this; }
    Vec2& operator-(const Vec2& o) { x-=o.x; y-=o.y; return *this; }
    Vec2& operator*(double s) { x*=s; y*=s; return *this; }

    // Comparison (C++20 spaceship operator generates all 6 at once)
    bool operator==( const Vec2& o) const { return x==o.x && y==o.y; }
    bool operator!=(const Vec2& o) const { return !(*this == o); }

    // Array subscript
    double& operator[(int i) ] {
        if (i == 0) return x;
        if (i == 1) return y;
        throw std::out_of_range{"Vec2 index out of range"};
    }
    const double& operator[(int i) ] const { // const version
        if (i == 0) return x;
        if (i == 1) return y;
        throw std::out_of_range{"Vec2 index out of range"};
    }
};

// Non-member: allows s * v as well as v * s
Vec2 operator*(double s, const Vec2& v) { return v * s; }

// Non-member: stream output (cannot be member because left operand is ostream)
std::ostream& operator<<(std::ostream& os, const Vec2& v) {
    return os << "(" << v.x << ", " << v.y << ")";
}

```

Usage:

```

Vec2 a{1.0, 2.0};
Vec2 b{3.0, 4.0};
Vec2 c = a + b;           // (4, 6)
Vec2 d = 2.0 * a;         // (2, 4)
Vec2 e = -a;              // (-1, -2)
a += b;                   // a = (4, 6)
std::cout << c << "\n"; // (4, 6)
std::cout << c[0] << "\n"; // 4

```

Member vs Non-Member Operators

Some operators must be members (`=`, `[]`, `()`, `->`). Others should be non-members when the left operand may not be your type.

Rule of thumb: - If the operator modifies the object (`+=`, `-=`, etc.): **member** - If the left operand might not be your type (`<<`, `*`, `+` with mixed types): **non-member** - Symmetric binary operators (`+`, `-`, `==`): **non-member** (for symmetry)

For `operator<<` specifically: the left operand is `std::ostream`, which is not your type. You cannot add a member to `std::ostream`. Therefore `operator<<` is always a non-member.

The Spaceship Operator (C++20)

C++20 introduced `operator<=>` (the three-way comparison operator), which generates all six comparison operators at once:

```
#include <compare>

struct Point {
    double x, y;

    auto operator<=>(const Point& o) const = default;
    // Generates: ==, !=, <, <=, >, >= all with memberwise comparison
};

Point a{1.0, 2.0};
Point b{1.0, 3.0};
bool less = (a < b);    // true: a.x == b.x, a.y < b.y
bool eq   = (a == a);   // true
```

`= default` tells the compiler to generate memberwise comparison in declaration order. For most value types, this is exactly right.

operator() – Making Objects Callable

```
struct Adder {
    int n;
    Adder(int n) : n{n} {}
    int operator()(int x) const { return x + n; }
};

Adder add5{5};
std::cout << add5(10) << "\n";    // 15
std::cout << add5(20) << "\n";    // 25
```

An object with operator () is called a **functor** or **function object**. They can store state (unlike raw function pointers) and are the foundation of lambdas (Chapter 30).

Rules and Guidelines for Operator Overloading

DO:

- Overload only when the semantics are obvious and unsurprising
- Keep operators consistent with each other (if + is defined, += should be too)
- Make operators non-throwing when possible
- Return *this from compound-assignment operators (enables a += b += c)

DON'T:

- Overload operators for unrelated meanings (e.g., using << for bit-shift on a list)
- Overload &&, ||, or comma (,) -- they lose short-circuit/sequencing guarantees
- Make operator+ modify the left operand (it should return a new value)
- Overload more operators than your type needs

Common Mistakes in This Chapter

Mistake 1: Returning *this by Value Instead of Reference From +=

The bug:

```
Vec2 operator+=(const Vec2& o) { // returns by value
    x += o.x; y += o.y;
    return *this; // returns a COPY
}
(a += b) = c; // modifies the temporary copy, not a -- probably wrong
```

The fix: Return Vec2& (reference to *this): Vec2& operator+=(const Vec2& o) { ... return *this; }

Mistake 2: Missing the Const Overload for operator []

The bug:

```
double& operator[](int i) { ... } // only non-const version
const Vec2 v{1.0, 2.0};
v[0]; // ERROR: no operator[] for const Vec2
```

The fix: Provide both double& operator[](int i) and const double& operator[](int i) const.

Exercises

Exercise 22.1 – Complete the vector class

Add the following to Vec2: - `length()` `const` returns `sqrt(x*x + y*y)` - `normalized()` `const` returning a unit vector (length 1.0) - `dot(Vec2)` `const` returning the dot product

Answer:

```
#include <cmath>

double length() const { return std::sqrt(x*x + y*y); }

Vec2 normalized() const {
    double len = length();
    if (len < 1e-12) throw std::runtime_error("Cannot normalize zero vector");
    return {x/len, y/len};
}

double dot(const Vec2& o) const { return x*o.x + y*o.y; }
```

Exercise 22.2 – Implement a Matrix2x2

Write a Mat2 class (2x2 matrix of doubles) with: - `operator*` for matrix-matrix multiplication - `operator*` for matrix-vector multiplication (using Vec2) - `operator==`

Answer:

```
struct Mat2 {
    double a, b, c, d; // [a b; c d]

    Mat2 operator*(const Mat2& o) const {
        return {a*o.a + b*o.c, a*o.b + b*o.d,
                c*o.a + d*o.c, c*o.b + d*o.d};
    }

    Vec2 operator*(const Vec2& v) const {
        return {a*v.x + b*v.y, c*v.x + d*v.y};
    }

    bool operator==(const Mat2& o) const {
        return a==o.a && b==o.b && c==o.c && d==o.d;
    }
};
```

Part IV is complete. You can now design a full object-oriented C++ system: classes with proper encapsulation, constructors with initializer lists, inheritance hierarchies with virtual dispatch, abstract interfaces, and expressive operator overloading.

Part V covers generic programming – templates and concepts, which let you write type-safe code that works for any type that meets your requirements, with zero runtime overhead. Ask to continue.

Part V – Generic Programming

Python uses duck typing: if an object has the right methods, any algorithm works on it. C++ achieves the same expressiveness at compile time through **templates** – code that is parameterized over types. The key difference: Python resolves method calls at runtime; C++ generates specialized code for each type at compile time, producing no overhead.

Chapter 23: Function and Class Templates

The Problem Without Templates

You want a max function. You need it for int, double, float, long long – every numeric type:

```
int    max(int a,    int b)    { return a > b ? a : b; }
double max(double a, double b) { return a > b ? a : b; }
float  max(float a,  float b)  { return a > b ? a : b; }
// ... and so on for every type
```

The logic is identical for every type. Only the type changes. This is what templates solve.

Function Templates

A function template is a pattern. You write it once with a placeholder type, and the compiler generates concrete functions for each type you use it with.

```
# Python: works for any type automatically (duck typing)
def my_max(a, b):
    return a if a > b else b

my_max(3, 5)           # int
my_max(3.14, 2.7)      # float
my_max("b", "a")       # str
```

```
// C++: template -- compiler generates a version for each type
template <typename T>
T my_max(T a, T b) {
    return a > b ? a : b;
}

my_max(3, 5);           // compiler generates: int my_max(int, int)
my_max(3.14, 2.7);      // compiler generates: double my_max(double, double)
my_max('b', 'a');       // compiler generates: char my_max(char, char)
// my_max("b", "a");    // works too: const char* comparison (pointer comparison -- careful)
```

template <typename T> declares a **type parameter** T. Everywhere T appears in the function, the compiler substitutes the actual type when instantiating the template.

Template Type Deduction

The compiler deduces T from the arguments. You usually do not need to specify it explicitly:

```
my_max(3, 5);           // T deduced as int
my_max<double>(3, 5);   // T explicitly specified as double (3 and 5 converted)
my_max(3, 5.0);         // ERROR: ambiguous -- is T int or double?
my_max<double>(3, 5.0); // OK: explicit T=double
```

When arguments have different types, deduction fails (the compiler cannot pick one T). Specify T explicitly, or cast one argument.

Multiple Template Parameters

```
template <typename T, typename U>
auto add(T a, U b) {      // auto return type: deduced from the expression
    return a + b;
}

add(1, 2.0);             // returns double (int + double = double)
add(1, 2);               // returns int
```

How Templates Are Compiled

Templates are compiled differently from regular functions. This is why template errors look strange and why templates must be in headers.

When you write `my_max(3, 5)`, the compiler: 1. Finds the template definition 2. Substitutes `T = int` throughout 3. Compiles the resulting `int my_max(int, int)` as if you wrote it by hand 4. Places it in the object file

Source:

```
template<typename T> T my_max(T a, T b) { ... }
my_max(3, 5);      --> generates: int    my_max(int, int)    { ... }
my_max(3.0, 5.0);  --> generates: double my_max(double, double) { ... }
my_max('a', 'b'); --> generates: char   my_max(char, char)   { ... }
```

Each instantiation is a separate compiled function. No runtime type dispatch. Zero overhead compared to writing three functions by hand.

Why Templates Must Live in Headers

When the compiler processes `main.cpp` and sees `my_max(3, 5)`, it needs the template definition to generate the instantiation. It cannot use a declaration alone (unlike regular functions where the linker handles it later).

Therefore: **template definitions go in header files**, not `.cpp` files.

```
// math_utils.h -- template definitions here, not in a .cpp
#pragma once

template <typename T>
T my_max(T a, T b) {
    return a > b ? a : b;
}
```

If you put a template definition in a `.cpp` file and try to use it from another file, you get “undefined reference” at link time – the compiler in the other `.cpp` had no template definition to instantiate from.

Class Templates

The standard library containers (`std::vector<T>`, `std::map<K,V>`) are class templates. You can write your own:

```
// A simple fixed-size stack
template <typename T, int MaxSize = 16>    // T: type, MaxSize: non-type parameter
class Stack {
    T data[MaxSize];
    int top{0};

public:
    void push(const T& value) {
        if (top >= MaxSize) throw std::overflow_error{"Stack full"};
        data[top++] = value;
    }

    T pop() {
        if (top == 0) throw std::underflow_error{"Stack empty"};
        return data[--top];
    }

    const T& peek() const {
        if (top == 0) throw std::underflow_error{"Stack empty"};
        return data[top - 1];
    }

    bool empty() const { return top == 0; }
    int size() const { return top; }
};
```

Usage:

```

Stack<int>    int_stack;           // Stack of ints, max 16
Stack<double, 32> big_stack;      // Stack of doubles, max 32

int_stack.push(1);
int_stack.push(2);
int_stack.push(3);
std::cout << int_stack.pop() << "\n"; // 3
std::cout << int_stack.pop() << "\n"; // 2

Stack<std::string> str_stack;
str_stack.push("hello");
str_stack.push("world");
std::cout << str_stack.peek() << "\n"; // "world" (not popped)

```

Each combination of type arguments generates a completely separate class. `Stack<int>` and `Stack<double>` are two distinct types with no relationship to each other.

Non-Type Template Parameters

Templates can be parameterized by values (not just types):

```

template <int N>
struct Factorial {
    static constexpr int value = N * Factorial<N-1>::value;
};

template <>                                // specialization for base case
struct Factorial<0> {
    static constexpr int value = 1;
};

constexpr int f5 = Factorial<5>::value; // 120, computed entirely at compile time
constexpr int f10 = Factorial<10>::value; // 3628800

```

Array sizes, buffer sizes, and algorithm tuning parameters can all be non-type template parameters. This is how `std::array<int, 10>` works – the size 10 is a non-type template parameter.

Template Functions in the Standard Library

The standard library is built on templates. Some functions you will use constantly:

```

#include <algorithm>
#include <vector>

std::vector<int> v = {5, 3, 1, 4, 2};

// All work for any type that supports the required operations:
std::sort(v.begin(), v.end()); // {1,2,3,4,5}

```

```
std::reverse(v.begin(), v.end());           // {5,4,3,2,1}
auto it = std::find(v.begin(), v.end(), 3); // iterator to 3
int mx = *std::max_element(v.begin(), v.end()); // 5

std::vector<double> dv = {1.0, 2.0, 3.0};
std::sort(dv.begin(), dv.end());           // same template, different type
```

Common Mistakes in This Chapter

Mistake 1: Putting Template Definitions in a .cpp File

The bug:

```
// math.cpp:
template <typename T>
T square(T x) { return x * x; }

// main.cpp:
template <typename T> T square(T x); // declaration only
int main() { square(5); }           // linker error: no instantiation found
```

Linker error: undefined reference to 'int square<int>(int)' **The fix:** Move the template definition into the header.

Mistake 2: Mixed Types Without Explicit Instantiation

The bug:

```
template <typename T>
T add(T a, T b) { return a + b; }

add(1, 2.0); // ERROR: deduction fails -- is T int or double?
```

The fix: `add<double>(1, 2.0)` or `add(1.0, 2.0)` or `add(1, 2)`.

Mistake 3: Confusing Template Errors With Logic Errors

Template errors are notorious for long, cryptic messages because the error is reported after instantiation. Always look at the first error and the “required from” line that shows where you used the template.

Exercises

Exercise 23.1 – Write a swap template

Write `template_swap(a, b)` that swaps two values of any type. Test with `int`, `double`, and `std::string`.

Answer:

```
template <typename T>
void template_swap(T& a, T& b) {
    T tmp = std::move(a);
    a = std::move(b);
    b = std::move(tmp);
}

int a = 1, b = 2;
template_swap(a, b);    // a=2, b=1

double x = 1.5, y = 2.5;
template_swap(x, y);    // x=2.5, y=1.5

std::string s = "hello", t = "world";
template_swap(s, t);    // s="world", t="hello"
```

Exercise 23.2 – min/max template

Write `clamp<T>(value, lo, hi)` that returns `lo` if `value < lo`, `hi` if `value > hi`, otherwise `value`. The standard library has `std::clamp` – write your own.

Answer:

```
template <typename T>
T clamp(T value, T lo, T hi) {
    if (value < lo) return lo;
    if (value > hi) return hi;
    return value;
}

clamp(5, 0, 10);    // 5
clamp(-3, 0, 10);   // 0
clamp(15, 0, 10);   // 10
clamp(0.5, 0.0, 1.0); // 0.5
```

Exercise 23.3 – Class template Pair

Write a class template `Pair<A, B>` that stores two values of potentially different types. Provide: - Constructor `Pair(A, B)` - `first()` and `second()` getters (const) - Non-member `make_pair(a, b)` that deduces the types

Answer:

```
template <typename A, typename B>
class Pair {
    A first_val;
    B second_val;
public:
    Pair(A a, B b) : first_val{std::move(a)}, second_val{std::move(b)} {}
    const A& first() const { return first_val; }
    const B& second() const { return second_val; }
};

template <typename A, typename B>
Pair<A,B> make_pair(A a, B b) {
    return Pair<A,B>{std::move(a), std::move(b)};
}

auto p = make_pair(42, std::string{"hello"});
std::cout << p.first() << " " << p.second() << "\n"; // 42 hello
```


Chapter 24: Template Specialization and Variadic Templates

Template Specialization

Sometimes the generic template is wrong or inefficient for a specific type. **Full specialization** provides a completely custom implementation for a concrete type:

```
// Primary template: works for any T
template <typename T>
struct TypeInfo {
    static std::string name() { return "unknown"; }
};

// Full specialization for int:
// <> means: no template parameters -- this is a specialization
template <>
struct TypeInfo<int> {
    static std::string name() { return "int"; }
};

// Full specialization for double:
template <>
struct TypeInfo<double> {
    static std::string name() { return "double"; }
};

std::cout << TypeInfo<int>::name() << "\n"; // "int"
std::cout << TypeInfo<double>::name() << "\n"; // "double"
std::cout << TypeInfo<char>::name() << "\n"; // "unknown" (uses primary template)
```

Partial Specialization

Partial specialization provides a custom implementation for a subset of type combinations, keeping some template parameters generic:

```
// Primary template
template <typename T, typename U>
struct IsSame { static constexpr bool value = false; };

// Partial specialization: both types are the same
template <typename T>
struct IsSame<T, T> { static constexpr bool value = true; };
```



```
IsSame<int, int>::value    // true
IsSame<int, double>::value // false
```

Another common example – specializing for pointers:

```
// Primary template: general case
template <typename T>
class Storage {
    T data;
public:
    void set(T v) { data = v; }
    T get()      { return data; }
};

// Partial specialization for pointer types
template <typename T>
class Storage<T*> {           // specialization matches any T*
    T* data{nullptr};
public:
    void set(T* p) { data = p; }
    T* get()       { return data; }
    bool valid()   { return data != nullptr; }
    // Extra: null check makes sense for pointers
};

Storage<int>    si;    // uses primary template
Storage<int*>   sp;    // uses pointer specialization -- has valid() method
```

if constexpr – Compile-Time Branching

Before concepts (next chapter), `if constexpr` lets you branch based on compile-time conditions inside templates:

```
#include <type_traits>

template <typename T>
void describe(T value) {
    if constexpr (std::is_integral_v<T>) {
        std::cout << "integer: " << value << "\n";
    } else if constexpr (std::is_floating_point_v<T>) {
        std::cout << "float:    " << value << "\n";
    } else {
        std::cout << "other:    " << value << "\n";
    }
}

describe(42);           // "integer: 42"
describe(3.14);         // "float:    3.14"
describe("hello");      // "other:    hello"
```

Unlike a regular `if`, only the matching branch is compiled. The other branches can contain code that would fail to compile for this type – they are simply discarded.

```
template <typename T>
void print_info(T v) {
    if constexpr (std::is_integral_v<T>) {
        std::cout << "bits: " << sizeof(T) * 8 << "\n";
        std::cout << "max: " << std::numeric_limits<T>::max() << "\n";
    } else {
        std::cout << "not an integer\n";
    }
}
```

Type Traits

`<type_traits>` provides compile-time predicates about types:

```
#include <type_traits>

std::is_integral_v<int>           // true
std::is_integral_v<double>       // false
std::is_floating_point_v<float>  // true
std::is_pointer_v<int*>          // true
std::is_reference_v<int&>        // true
std::is_const_v<const int>       // true
std::is_same_v<int, int>         // true
std::is_same_v<int, long>        // false (on most platforms!)
std::is_base_of_v<Animal, Dog>   // true
std::is_convertible_v<int, double> // true
```

These are the building blocks of generic code that adapts to the type it receives.

Variadic Templates

A variadic template accepts **any number of type arguments** (including zero). This is how `std::tuple`, `std::make_unique`, and `std::format` are implemented.

```
# Python: *args handles any number of arguments
def print_all(*args):
    for arg in args:
        print(arg)

print_all(1, "hello", 3.14)
```

```
// C++: variadic template with parameter pack
template <typename... Args> // Args is a "parameter pack" -- zero or more types
void print_all(Args... args) {
    // Expand the pack using fold expression (C++17):
    (std::cout << ... << args); // prints each arg with no separator
    std::cout << "\n";
}

print_all(1, " hello ", 3.14); // "1 hello 3.14"
```

The ... is the pack expansion operator. Args... unpacks the types; args... unpacks the values.

Recursive Variadic Template (Classic Style)

Before C++17 fold expressions, variadic templates were handled recursively:

```
// Base case: zero arguments
void print_all() {}

// Recursive case: print first, then recurse on the rest
template <typename First, typename... Rest>
void print_all(First first, Rest... rest) {
    std::cout << first << " ";
    print_all(rest...); // recurse with one fewer argument
}

print_all(1, "hello", 3.14, true);
// Output: 1 hello 3.14 1
```

Fold Expressions (C++17)

More concise for common patterns:

```
// Sum of any number of values:
template <typename... Ts>
auto sum(Ts... args) {
    return (... + args); // left fold: ((arg1 + arg2) + arg3) + ...
}

sum(1, 2, 3, 4, 5); // 15
sum(1.0, 2.5, 0.5); // 4.0
sum(std::string{"a"}, std::string{"b"}, std::string{"c"}); // "abc"
```

Fold expression forms:

```
(... op pack) // left fold: (((a op b) op c) op d)
(pack op ...) // right fold: (a op (b op (c op d)))
(init op ... op pack) // left fold with initial value
(pack op ... op init) // right fold with initial value
```

Common Mistakes in This Chapter

Mistake 1: Full Specialization After the Template Is Already Instantiated

The bug:

```
template <typename T> void foo(T) { std::cout << "generic\n"; }
foo(5);                               // instantiates foo<int> as generic
template <> void foo<int>(int) { std::cout << "int\n"; }
// Too late: the specialization after the call point may not be seen by the compiler
```

The fix: Declare all specializations before any instantiations – usually by putting everything in the header, with specializations after the primary template.

Mistake 2: Forgetting `template <>` for Full Specialization

The bug:

```
template <typename T> struct Foo { };
struct Foo<int> { }; // ERROR: looks like a redefinition of a non-template class
```

The fix: `template <> struct Foo<int> { };`

Exercises

Exercise 24.1 – Specialize for `bool`

The `Stack<T>` from Chapter 23 stores `bool` values as full `T` data[MaxSize] which wastes space. Write a full specialization `Stack<bool>` that uses a `uint64_t` as a bitset (packing 64 bools into one `uint64_t`). Only implement `push`, `pop`, and `size`.

Answer (simplified):

```
template <>
class Stack<bool, 64> {
    uint64_t bits{0};
    int      top{0};
public:
    void push(bool v) {
        if (top >= 64) throw std::overflow_error{"Stack full"};
        if (v) bits |= (1ULL << top);
        else bits &= ~(1ULL << top);
        ++top;
    }
    bool pop() {
        if (top == 0) throw std::underflow_error{"Stack empty"};
        --top;
        return (bits >> top) & 1;
    }
};
```

```
    }  
    int size() const { return top; }  
};
```

Exercise 24.2 – Variadic min

Write a variadic `vmin(args...)` that returns the minimum of any number of values.

Answer:

```
template <typename T>  
T vmin(T only) { return only; }  
  
template <typename T, typename... Rest>  
T vmin(T first, Rest... rest) {  
    T rest_min = vmin(rest...);  
    return first < rest_min ? first : rest_min;  
}  
  
vmin(5, 3, 8, 1, 6); // 1  
vmin(3.14, 2.71, 1.41); // 1.41
```

Chapter 25: Concepts (C++20) – Compile-Time Duck Typing

The Problem With Unconstrained Templates

```
template <typename T>
T my_max(T a, T b) {
    return a > b ? a : b;
}

struct Foo {};
my_max(Foo{}, Foo{}); // ERROR -- but the error is inside the template, very confusing
```

The error message:

```
error: no match for 'operator>' (operand types are 'Foo' and 'Foo')
   3 |         return a > b ? a : b;
       |                ~~~^~~
note: in instantiation of function template specialization 'my_max<Foo>' requested here
   8 |         my_max(Foo{}, Foo{});
```

The error tells you the problem, but for complex templates it can span dozens of lines. More importantly, the error message talks about the template's internals – not about what the caller did wrong.

Concepts let you state requirements on template parameters up front, producing clear error messages at the call site.

Defining and Using Concepts

A **concept** is a named compile-time predicate over types:

```
# Python 3.12+ type hints (still runtime duck typing)
from typing import Protocol

class Comparable(Protocol):
    def __gt__(self, other) -> bool: ...
```

```
// C++20 concept: compile-time, enforced by compiler
#include <concepts>

template <typename T>
concept Comparable = requires(T a, T b) {
    { a > b } -> std::convertible_to<bool>;    // expression 'a > b' must be valid
    { a < b } -> std::convertible_to<bool>;    // and convertible to bool
};

// Use the concept to constrain the template:
template <Comparable T>    // shorthand constraint syntax
T my_max(T a, T b) {
    return a > b ? a : b;
}

// Or equivalent with 'requires' clause:
template <typename T>
requires Comparable<T>
T my_max(T a, T b) { return a > b ? a : b; }
```

Now trying `my_max(Foo{}, Foo{})` gives:

```
error: no matching function for call to 'my_max(Foo, Foo)'
note: constraints not satisfied
note: 'Comparable<Foo>' evaluated to false
note: 'a > b' is not a valid expression for type 'Foo'
```

The error is at the call site and says exactly what is missing.

Standard Library Concepts (<concepts>)

C++20 provides many ready-made concepts:

```
#include <concepts>

std::integral<int>           // true: int is integral
std::integral<double>       // false
std::floating_point<double> // true
std::signed_integral<int>   // true
std::same_as<int, int>      // true
std::derived_from<Dog, Animal> // true (Dog derived from Animal)
std::convertible_to<int, double> // true
std::equality_comparable<int> // true (int has == and !=)
std::totally_ordered<int>    // true (int has <, >, <=, >=)
std::copyable<std::vector<int>> // true
std::movable<std::unique_ptr<int>> // true
std::callable<std::function<int()>> // true
```

Use these in your own templates:

```
#include <concepts>

template <std::integral T>           // T must be an integral type
T next(T n) { return n + 1; }

template <std::floating_point T>     // T must be a float type
T reciprocal(T n) { return T{1} / n; }

next(5);           // OK
next(3.14);        // ERROR: double does not satisfy 'integral'
reciprocal(2.0);   // OK
reciprocal(2);     // ERROR: int does not satisfy 'floating_point'
```

Writing Your Own Concepts

A concept is defined with `requires` expressions that describe what operations must be valid:

```
// A type T is Printable if it can be inserted into an ostream:
template <typename T>
concept Printable = requires(T v, std::ostream& os) {
    { os << v };    // this expression must compile
};

// A type T is Container if it has begin(), end(), and size():
template <typename T>
concept Container = requires(T c) {
    { c.begin() } -> std::input_iterator;
    { c.end()   } -> std::input_iterator;
    { c.size()  } -> std::convertible_to<std::size_t>;
};

// A type T is Numeric if it supports +, -, *, / with itself:
template <typename T>
concept Numeric = requires(T a, T b) {
    { a + b } -> std::same_as<T>;
    { a - b } -> std::same_as<T>;
    { a * b } -> std::same_as<T>;
    { a / b } -> std::same_as<T>;
};

// Use all three:
template <Container C, Printable E>
requires std::same_as<typename C::value_type, E>
void print_container(const C& c) {
    for (const E& e : c)
        std::cout << e << " ";
    std::cout << "\n";
}
```


Abbreviated Function Templates (C++20)

The cleanest concept syntax uses `auto` with concept constraints:

```
// Instead of:
template <std::integral T>
T double_it(T n) { return n * 2; }

// Write:
std::integral auto double_it(std::integral auto n) { return n * 2; }
//           ^-- abbreviated function template: auto = template parameter
//   std::integral auto means "auto, but constrained to integral types"
```

For simple cases this is much shorter. For complex cases with multiple interdependent parameters, the explicit `template <typename T>` form is clearer.

Concepts vs Runtime Type Checks

Concepts check types at **compile time**. The entire constraint system is resolved before any code runs. Failed constraints are compiler errors, not runtime exceptions.

Python (duck typing):

- Checked at runtime when the method is called
- `TypeError` raised if method missing
- Only the failing call path is caught

C++ concepts:

- Checked at compile time when the template is instantiated
- Compile error if constraint not satisfied
- All call sites checked, not just the ones tested
- Zero runtime overhead

This is what C++ means by “you don’t pay for what you don’t use” – concepts are checked once at compile time and then disappear from the runtime.

Common Mistakes in This Chapter

Mistake 1: Using `requires` in the Wrong Place

The bug:

```
template <typename T>
T foo(T a) requires std::integral<T> // requires clause at end of function signature
{ return a; }
// Fine syntactically, but easy to confuse with body. Prefer:
template <std::integral T> T foo(T a) { return a; }
```

Both are legal; the second is cleaner for simple cases.

Mistake 2: Concept Is Too Strict or Too Loose

The bug: Writing a concept that requires `operator<` but your algorithm actually needs `operator<=`:

```
template <typename T>
concept Ordered = requires(T a, T b) { a < b; }; // missing <=

template <Ordered T>
bool in_range(T val, T lo, T hi) {
    return lo <= val && val <= hi; // uses <= -- not checked by concept!
}
```

The fix: List every operation your template actually uses in the concept.

Exercises

Exercise 25.1 – Constrain `my_max`

Rewrite `my_max` from Chapter 23 with a concept that requires `operator>`. Test that it rejects `Foo{}` with a clear error.

Answer:

```
template <typename T>
concept HasGreaterThan = requires(T a, T b) {
    { a > b } -> std::convertible_to<bool>;
};

template <HasGreaterThan T>
T my_max(T a, T b) { return a > b ? a : b; }

my_max(3, 5); // OK
my_max(3.0, 5.0); // OK
// my_max(Foo{}, Foo{}); // Compile error with clear message
```

Exercise 25.2 – Numeric concept

Write a `sum` function that computes the sum of a `std::vector<T>` where `T` must satisfy `std::integral` or `std::floating_point`. Use a combined concept.

Answer:

```
template <typename T>
concept Arithmetic = std::integral<T> || std::floating_point<T>;

template <Arithmetic T>
T sum(const std::vector<T>& v) {
    T total{};
    for (const T& x : v) total += x;
    return total;
}

sum(std::vector<int>{1, 2, 3, 4, 5}); // 15
sum(std::vector<double>{1.1, 2.2, 3.3}); // 6.6
// sum(std::vector<std::string>{"a", "b"}); // ERROR: string not Arithmetic
```

Chapter 26: An Introduction to Template Metaprogramming

What Is Template Metaprogramming?

Template metaprogramming (TMP) is using the C++ template system to compute values and make decisions at **compile time**. The template system is Turing-complete – in principle you can compute anything at compile time.

In practice, TMP is used for: - Computing compile-time constants (sizes, masks, lookup tables) - Selecting code paths based on types without runtime overhead - Generating optimized code for specific type configurations - Implementing type traits and concepts

Modern C++ has made most raw TMP unnecessary. `constexpr` functions, `if constexpr`, and concepts replace many old TMP patterns more readably. But understanding TMP helps you read library code and write efficient generic code.

Compile-Time Recursion

TMP was originally done with recursive struct templates (before `constexpr`):

```
// Fibonacci at compile time (old style):
template <int N>
struct Fib {
    static constexpr int value = Fib<N-1>::value + Fib<N-2>::value;
};
template <> struct Fib<0> { static constexpr int value = 0; };
template <> struct Fib<1> { static constexpr int value = 1; };

constexpr int f10 = Fib<10>::value; // 55, computed at compile time
```

Modern equivalent with `constexpr` function (far more readable):

```
constexpr int fib(int n) {
    if (n <= 1) return n;
    return fib(n-1) + fib(n-2);
}

constexpr int f10 = fib(10); // 55, computed at compile time
```

Always prefer `constexpr` functions over struct-recursion TMP for computations.

std::enable_if – Conditional Compilation (Old Style)

Before concepts, `std::enable_if` was used to enable or disable template instantiations based on type traits:

```
#include <type_traits>

// Only enabled for integral types (old style, pre-C++20):
template <typename T,
         typename = std::enable_if_t<std::is_integral_v<T>>>
T next(T n) { return n + 1; }

// With concepts (C++20, prefer this):
template <std::integral T>
T next(T n) { return n + 1; }
```

You will encounter `enable_if` in older code and library documentation. Understanding it: `std::enable_if_t<condition>` is void when condition is true (template is valid) and causes substitution failure when condition is false (template is removed from consideration). This is called **SFINAE** (Substitution Failure Is Not An Error).

Type Lists – Compile-Time Lists of Types

Variadic templates let you create lists of types that exist purely at compile time:

```
// A type list:
template <typename... Ts>
struct TypeList {};

using MyTypes = TypeList<int, double, std::string, bool>;

// Count the types in a list:
template <typename List>
struct Length;

template <typename... Ts>
struct Length<TypeList<Ts...>> {
    static constexpr int value = sizeof...(Ts);
};

constexpr int n = Length<MyTypes>::value; // 4
```

Type lists are the foundation of tuple implementations, plugin systems, and serialization frameworks that need to enumerate types at compile time.

Compile-Time Lookup Tables

A powerful pattern: compute a lookup table at compile time so it is a static constant array in the binary:

```
#include <array>

// Generate a table of squares at compile time:
constexpr std::array<int, 10> squares = []() {
    std::array<int, 10> arr{};
    for (int i = 0; i < 10; ++i) arr[i] = i * i;
    return arr;
}(); // immediately invoked lambda

// At runtime, squares is a pre-computed constant array in .rodata
// No runtime computation at all:
std::cout << squares[7] << "\n"; // 49 -- just a memory read
```

This pattern works because: 1. The lambda is `constexpr`-evaluable 2. `std::array` is a literal type 3. The entire computation happens at compile time

For game engines, look-up tables for sin/cos, byte-reversal, CRC, and Huffman trees are often pre-computed this way.

std::tuple – Compile-Time Heterogeneous Collections

`std::tuple` is a variadic class template that holds values of different types:

```
# Python tuple
t = (42, "hello", 3.14)
print(t[0]) # 42 (runtime indexing)
```

```
// C++ tuple
#include <tuple>
std::tuple<int, std::string, double> t{42, "hello", 3.14};

// Access by index -- index must be a compile-time constant:
std::get<0>(t); // 42
std::get<1>(t); // "hello"
std::get<2>(t); // 3.14

// Cannot do: std::get<i>(t) where i is a runtime variable
// The index is resolved at compile time

// C++17 structured bindings (much nicer):
auto [num, str, flt] = t;
std::cout << num << " " << str << " " << flt << "\n";
```

`std::get<N>` is resolved at compile time. Accessing `std::get<5>(t)` on a 3-element tuple is a compile error, not a runtime error.

std::conditional – Type Selection at Compile Time

Choose between two types based on a condition:

```
#include <type_traits>

// If condition is true, type is int; otherwise long long
template <bool IsSmall>
using StorageType = std::conditional_t<IsSmall, int, long long>;

StorageType<true> a = 5;           // int
StorageType<false> b = 5000000000; // long long

// Practical use: choose between fast and accurate computation
template <bool Fast>
using Float = std::conditional_t<Fast, float, double>;
```

The Difference Between TMP and constexpr

Template Metaprogramming:

- Computation done by the template instantiation engine
- Types are first-class citizens
- Syntax is complex (recursive struct templates, enable_if)
- Result is a type or compile-time constant
- Use when you need to manipulate TYPES at compile time

constexpr:

- Computation done by a regular function (or expression) evaluated at compile time
- Normal C++ syntax, easy to read
- Result is a VALUE (int, array, struct...)
- Use when you need to compute VALUES at compile time

C++20 guidance:

- For compile-time values: use constexpr functions
- For type selection: use concepts, if constexpr, std::conditional
- Raw TMP (recursive struct templates) is rarely needed in new code

Common Mistakes in This Chapter

Mistake 1: Trying to Use a Runtime Value as a Template Argument

The bug:

```
int n;
std::cin >> n;
std::array<int, n> arr; // ERROR: n is not a compile-time constant
```

The fix: Use `std::vector<int>(n)` for runtime sizes.

Mistake 2: Recursive TMP Instead of constexpr

The bug:

```
template <int N> struct Pow2 { static constexpr int v = 2 * Pow2<N-1>::v; };
template <> struct Pow2<0> { static constexpr int v = 1; };
```

Better: `constexpr int pow2(int n) { return 1 << n; }`

Exercises

Exercise 26.1 – Compile-time lookup table

Create a `constexpr std::array<int, 16>` of the first 16 powers of 2 (1, 2, 4, 8, ...). Access element 10 and verify it is 1024.

Answer:

```
constexpr std::array<int, 16> powers_of_2 = []() {
    std::array<int, 16> arr{};
    arr[0] = 1;
    for (int i = 1; i < 16; ++i) arr[i] = arr[i-1] * 2;
    return arr;
}();

static_assert(powers_of_2[10] == 1024); // compile-time check
std::cout << powers_of_2[10] << "\n"; // 1024
```

Exercise 26.2 – Type traits

Write a function `type_name<T>()` that returns a `std::string` naming the type as “integral”, “floating_point”, “string”, or “other”:

Answer:

```
#include <concepts>
#include <string>

template <typename T>
std::string type_name() {
    if constexpr (std::integral<T>) return "integral";
    else if constexpr (std::floating_point<T>) return "floating_point";
}
```



```

    else if constexpr (std::same_as<T, std::string>) return "string";
    else
        return "other";
}

type_name<int>();           // "integral"
type_name<double>();        // "floating_point"
type_name<std::string>();   // "string"
type_name<bool>();          // "integral" (bool is integral)
type_name<char*>();         // "other"

```

Exercise 26.3 – Tuple structured binding

Create a function `minmax(std::vector<T>)` that returns a `std::pair<T, T>` containing the minimum and maximum element. Use structured bindings to unpack the result.

Answer:

```

#include <vector>
#include <utility>
#include <algorithm>

template <typename T>
std::pair<T, T> minmax_pair(const std::vector<T>& v) {
    auto [mn, mx] = std::minmax_element(v.begin(), v.end());
    return {*mn, *mx};
}

std::vector<int> v = {5, 2, 8, 1, 9, 3};
auto [lo, hi] = minmax_pair(v);
std::cout << "min=" << lo << " max=" << hi << "\n"; // min=1 max=9

```

Part V is complete. You now understand C++ generic programming: function and class templates, specialization, variadic templates, concepts for type-safe constraints, and the foundations of template metaprogramming.

Part VI covers the C++ Standard Library in depth – containers, iterators, algorithms, lambdas, ranges, and utility types. These are the tools you will use in every real program. Ask to continue.

Part VI – The Standard Library

The C++ Standard Library is not a collection of helper functions bolted on after the fact. It is a carefully designed ecosystem of containers, iterators, algorithms, and utilities that work together through a common interface. Learning to use it well is the difference between writing C++ that feels like C with classes and C++ that is expressive, safe, and fast.

Chapter 27: Containers: vector, map, set, array, and Friends

The Container Taxonomy

All standard containers fall into three categories:

Sequence containers (ordered by position):

<code>std::array<T, N></code>	fixed-size array, stack-allocated
<code>std::vector<T></code>	dynamic array, heap-allocated
<code>std::deque<T></code>	double-ended queue
<code>std::list<T></code>	doubly-linked list
<code>std::forward_list<T></code>	singly-linked list

Associative containers (ordered by key):

<code>std::map<K, V></code>	sorted key-value pairs, unique keys
<code>std::multimap<K, V></code>	sorted key-value pairs, duplicate keys allowed
<code>std::set<K></code>	sorted unique keys
<code>std::multiset<K></code>	sorted keys, duplicates allowed

Unordered associative containers (hash-based):

<code>std::unordered_map<K, V></code>	hash map, unique keys
<code>std::unordered_set<K></code>	hash set, unique keys

Container adapters (built on other containers):

<code>std::stack<T></code>	LIFO adapter (uses deque by default)
<code>std::queue<T></code>	FIFO adapter (uses deque by default)
<code>std::priority_queue<T></code>	max-heap adapter (uses vector by default)

`std::vector<T>` – Your Default Container

Already covered in Chapter 10. Here are the parts you need for real programs:

Reservation and Capacity Management

```
std::vector<int> v;
v.reserve(1000);           // pre-allocate space for 1000 elements
                           // avoids repeated reallocations during push_back

std::cout << v.size()      << "\n"; // 0      (no elements yet)
std::cout << v.capacity() << "\n"; // 1000   (space reserved)

for (int i = 0; i < 1000; ++i)
    v.push_back(i);        // no reallocations happen (capacity was reserved)

v.shrink_to_fit();         // release excess capacity to OS
```

Without reserve (1000 push_backs):

Realloc at 1, 2, 4, 8, 16, ... 512 = ~10 reallocations
 Each realloc: allocate new array, copy all elements, free old array
 Total element copies: $1+2+4+\dots+512 \approx 1000$ extra copies

With reserve(1000):

Zero reallocations. Zero extra copies.

Reserve when you know an upper bound on the number of elements.

emplace_back vs push_back

```
struct Point { double x, y; Point(double x, double y) : x{x}, y{y} {} };

std::vector<Point> pts;

// push_back: constructs a Point, then copies/moves it into the vector
pts.push_back(Point{1.0, 2.0}); // construct temporary, then move-construct into vector

// emplace_back: constructs the Point DIRECTLY inside the vector memory
pts.emplace_back(1.0, 2.0);     // no temporary, arguments forwarded to constructor
```

`emplace_back` is generally preferred for class types – it constructs in-place, avoiding the temporary. For types where the move is cheap (most standard types), the difference is minimal. For non-movable types, `emplace_back` is the only option.

std::deque<T> – Fast at Both Ends

Like vector but also allows $O(1)$ insertion and removal at the front:

```
#include <deque>
std::deque<int> dq = {3, 4, 5};
dq.push_front(2);    // {2, 3, 4, 5}
dq.push_front(1);    // {1, 2, 3, 4, 5}
dq.push_back(6);     // {1, 2, 3, 4, 5, 6}
dq.pop_front();      // {2, 3, 4, 5, 6}
```

Use deque when you need fast insertion at both ends. Use vector otherwise – vector’s cache efficiency is better for sequential access.

std::map<K, V> – Sorted Key-Value Store

```
# Python dict -- hash map
scores = {"Alice": 95, "Bob": 87, "Carol": 92}
scores["Dave"] = 78
print(scores["Alice"]) # 95
```

```
// C++ map -- sorted by key (balanced BST internally)
#include <map>
std::map<std::string, int> scores;
scores["Alice"] = 95;
scores["Bob"] = 87;
scores["Carol"] = 92;
scores["Dave"] = 78;

std::cout << scores["Alice"] << "\n"; // 95
```

Key Difference: std::map vs Python dict

Feature	Python dict	std::map	std::unordered_map
Order	Insertion order (3.7+)	Sorted by key	No guaranteed order
Lookup	O(1) average	O(log n)	O(1) average
Key requirement	Hashable	Comparable (<)	Hashable
Memory	Compact	BST nodes (pointers)	Hash buckets

Use std::map when you need ordered iteration. Use std::unordered_map when you need fast lookups and don’t care about order.

Safe Lookup: at() vs operator[]

```

std::map<std::string, int> m = {"a", 1}, {"b", 2};;

m["c"];           // WARNING: inserts "c" with default value 0 if not present!
                  // m is now {"a":1, "b":2, "c":0}

m.at("d");        // throws std::out_of_range: "d" not in map (no insertion)
m.at("a");        // 1 -- safe read

// Best: check first
if (m.count("a")) { std::cout << m.at("a") << "\n"; }

// Or use find:
auto it = m.find("a");
if (it != m.end()) {
    std::cout << it->first << " -> " << it->second << "\n";
}

```

The operator `[]` creates missing keys with a default-constructed value. This is a very common source of bugs: you check `map[key]` to see if a key exists, and it silently inserts a zero.

Iterating Over a Map

```

std::map<std::string, int> m = {"Alice", 95}, {"Bob", 87}, {"Carol", 92};;

// Iteration is in sorted key order:
for (const auto& [key, value] : m) { // structured binding (C++17)
    std::cout << key << ": " << value << "\n";
}

// Alice: 95
// Bob: 87
// Carol: 92

```

Inserting and Checking Simultaneously

```

// insert_or_assign (C++17): always sets the value
m.insert_or_assign("Dave", 78); // inserts if new, overwrites if exists

// try_emplace (C++17): inserts only if key is absent, does nothing if present
m.try_emplace("Alice", 100);    // Alice already exists: nothing happens (still 95)
m.try_emplace("Eve", 88);       // Eve is new: inserted with 88

// erase:
m.erase("Bob");

```

std::unordered_map<K, V> – Hash Map

Same interface as std::map but uses hashing for O(1) average lookup:

```
#include <unordered_map>
std::unordered_map<std::string, int> umap;
umap["Alice"] = 95;
umap["Bob"]   = 87;

// Same interface as map:
umap.at("Alice");           // 95
umap.count("Carol");        // 0 -- not present
auto it = umap.find("Bob");

// But NO guaranteed iteration order:
for (const auto& [k, v] : umap) { ... } // order is unpredictable
```

Use unordered_map for most cases where you need key-value lookup. Use map when sorted order matters (e.g., printing in alphabetical order, range queries).

std::set<K> – Sorted Unique Keys

```
# Python set
s = {3, 1, 4, 1, 5, 9, 2}
print(s) # {1, 2, 3, 4, 5, 9} (unordered)
```

```
#include <set>
std::set<int> s = {3, 1, 4, 1, 5, 9, 2}; // duplicates silently ignored
// s contains: {1, 2, 3, 4, 5, 9} (sorted, unique)

s.insert(7);           // {1, 2, 3, 4, 5, 7, 9}
s.erase(4);            // {1, 2, 3, 5, 7, 9}
s.count(5);             // 1 (present)
s.count(4);             // 0 (erased)

for (int n : s)
    std::cout << n << " "; // 1 2 3 5 7 9 (sorted order)
```

For hash-based set: std::unordered_set<K> – O(1) lookup, no order.

Choosing the Right Container

I need to store N items and access by position:

- Size known at compile time: std::array<T, N>
- Size varies at runtime: std::vector<T>
- Need fast front insertion: std::deque<T>

I need key-value lookup:

- Order matters / sorted iteration: `std::map<K, V>`
- Maximum speed, order irrelevant: `std::unordered_map<K, V>`

I need to track unique items:

- Order matters: `std::set<K>`
- Maximum speed, no order: `std::unordered_set<K>`

I need LIFO (stack) behavior: `std::stack<T>`

I need FIFO (queue) behavior: `std::queue<T>`

I need the max element always: `std::priority_queue<T>`

Common Mistakes in This Chapter

Mistake 1: `map[key]` to Check Existence

The bug:

```
std::map<std::string, int> m = {{ "a", 1 }};
if (m["b"]) { ... } // inserts "b" with value 0! "b" is now in the map.
```

The fix: `if (m.count("b"))` or `if (m.find("b") != m.end())`

Mistake 2: Invalidating Iterators by Modifying the Container

The bug:

```
std::vector<int> v = {1, 2, 3, 4, 5};
for (auto it = v.begin(); it != v.end(); ++it) {
    if (*it == 3) v.erase(it); // erase invalidates 'it' and all iterators after it!
}
```

The fix:

```
v.erase(std::remove(v.begin(), v.end(), 3), v.end()); // erase-remove idiom
```

Mistake 3: Linear Search on an Unsorted vector When a set Would Be $O(\log n)$

If you are repeatedly calling `std::find` on a vector, consider whether `std::set` or `std::unordered_set` would serve better.

Exercises

Exercise 27.1 – Word frequency count

Read a list of words and count how many times each word appears. Print words and counts in alphabetical order.

Answer:

```
#include <iostream>
#include <map>
#include <string>

int main() {
    std::map<std::string, int> freq;
    std::string word;
    while (std::cin >> word) {
        ++freq[word];    // OK to use [] here: we WANT insertion with 0 if new
    }
    for (const auto& [w, count] : freq) {
        std::cout << w << ": " << count << "\n";
    }
}
```

Exercise 27.2 – Reverse a deque

Push the integers 1 through 10 into a `std::deque`, then repeatedly pop from the front to print them in reverse order (10 down to 1). Do not use `std::reverse`.

Answer:

```
std::deque<int> dq;
for (int i = 1; i <= 10; ++i) dq.push_front(i);
// dq is now: 10 9 8 7 6 5 4 3 2 1

while (!dq.empty()) {
    std::cout << dq.front() << " ";
    dq.pop_front();
} // prints: 10 9 8 7 6 5 4 3 2 1
```

Exercise 27.3 – Unique elements

Given `std::vector<int> v = {5,3,1,4,2,3,5,1,4}`, produce a sorted vector of unique elements without writing a `sort+unique` manually.

Answer:

```
std::vector<int> v = {5,3,1,4,2,3,5,1,4};
std::set<int> s(v.begin(), v.end()); // set removes duplicates, sorts
std::vector<int> unique(s.begin(), s.end()); // back to vector: {1,2,3,4,5}
```


Chapter 28: Iterators

What Is an Iterator?

An iterator is an object that points into a container and can be advanced. It generalizes the concept of a pointer – in fact, raw pointers are valid iterators for arrays.

```
# Python iteration: the iterator protocol (hidden from you)
it = iter([1, 2, 3])
next(it)    # 1
next(it)    # 2
next(it)    # 3
```

```
// C++ iterators: explicit objects you manipulate
std::vector<int> v = {1, 2, 3};
auto it = v.begin();    // iterator to first element
std::cout << *it << "\n"; // 1 -- dereference like a pointer
++it;
std::cout << *it << "\n"; // 2
++it;
std::cout << *it << "\n"; // 3
++it;
// it == v.end() -- one past the last element (do not dereference!)
```

`v.begin()` returns an iterator to the first element. `v.end()` returns an iterator to one-past-the-last element.

The half-open range `[begin, end)` is a C++ convention. `end` itself is never valid to dereference – it is a sentinel.

Iterator Categories

Different containers support different iterator capabilities:

Input iterator: read once, advance once (e.g., reading from a file stream)

Output iterator: write once, advance once

Forward iterator: read/write, advance forward, multi-pass

Bidirectional: forward + go backward (`std::list`, `std::map`)

Random access: any position in $O(1)$ (`std::vector`, `std::array`, raw array)

Contiguous: random access + elements are contiguous in memory (`std::vector`, `std::array`)

```
std::vector<int> v = {1, 2, 3, 4, 5};
auto it = v.begin();

it + 2;           // random access: jump forward by 2
it[2];           // random access: subscript
*(it + 2);        // 3
it += 3;          // advance by 3

std::list<int> l = {1, 2, 3};
auto lit = l.begin();
++lit;           // forward
--lit;           // backward (bidirectional)
// lit + 2;      // ERROR: list iterators are not random access
```

The algorithm functions in `<algorithm>` work on any appropriate iterator category. `std::sort` requires random-access iterators (works on vector, not list). `std::find` requires only forward iterators (works on both).

The Iterator-Pair Pattern

The standard library uses iterator pairs to describe ranges:

```
std::vector<int> v = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

// Operate on a subrange:
std::sort(v.begin() + 2, v.begin() + 7); // sort only elements [2, 7)
// v = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10} (already sorted, but you could sort reversed subrange)

// Find in a subrange:
auto it = std::find(v.begin(), v.begin() + 5, 3); // search only first 5 elements
```

This is both powerful (you can describe any contiguous subrange) and error-prone (easy to get the boundaries wrong).

begin / end Free Functions

C++11 added free function versions that work on arrays too:

```
int arr[5] = {5, 3, 1, 4, 2};
std::sort(std::begin(arr), std::end(arr)); // sorts the C-array!

std::vector<int> v = {5, 3, 1};
std::sort(std::begin(v), std::end(v));    // same syntax
```

`std::begin(arr)` returns a pointer to `arr[0]`. `std::end(arr)` returns a pointer past the last element. Raw pointers are valid random-access iterators.

Reverse Iterators

```
std::vector<int> v = {1, 2, 3, 4, 5};

// Iterate backward:
for (auto it = v.rbegin(); it != v.rend(); ++it) {
    std::cout << *it << " "; // 5 4 3 2 1
}

// rbegin() = reverse iterator to last element
// rend()    = reverse iterator to one-before-first (sentinel)
// ++ on a reverse iterator goes backward
```

Insert Iterators

Output iterators that insert into a container instead of overwriting:

```
#include <iterator>
std::vector<int> src = {1, 2, 3};
std::vector<int> dst;

// back_inserter: calls push_back on each write
std::copy(src.begin(), src.end(), std::back_inserter(dst));
// dst = {1, 2, 3}

// front_inserter: calls push_front (only for deque, list)
std::deque<int> dq;
std::copy(src.begin(), src.end(), std::front_inserter(dq));
// dq = {3, 2, 1} (each is pushed to front)
```

Stream Iterators

Treat input/output streams as iterators:

```
#include <iterator>
#include <fstream>
#include <algorithm>

// Read all ints from stdin into a vector:
std::vector<int> v(
    std::istream_iterator<int>{std::cin}, // begin: reads ints from stdin
    std::istream_iterator<int>{}          // end: default-constructed = EOF
);

// Write a vector to stdout with spaces:
std::copy(v.begin(), v.end(),
    std::ostream_iterator<int>{std::cout, " "});
```

Common Mistakes in This Chapter

Mistake 1: Dereferencing `end()`

The bug:

```
auto it = v.end();
std::cout << *it;    // undefined behavior -- end() is one past the last element
```

The fix: Always check `it != v.end()` before dereferencing.

Mistake 2: Iterator Invalidation

When you modify a container, its iterators may become invalid:

```
std::vector<int> v = {1, 2, 3};
auto it = v.begin();
v.push_back(4);    // may reallocate! it is now dangling
std::cout << *it;    // undefined behavior
```

Rules: - `vector`: any operation that changes size may invalidate ALL iterators - `map/set`: erasing a node invalidates ONLY that node's iterator; other iterators remain valid - `list`: inserting never invalidates iterators; erasing only invalidates the erased node

Exercises

Exercise 28.1 – Manual iteration

Without using a range-based for loop, iterate over `std::vector<int> v = {10, 20, 30, 40, 50}` using `begin/end` iterators and print each element.

Answer:

```
for (auto it = v.begin(); it != v.end(); ++it) {
    std::cout << *it << "\n";
}
```

Exercise 28.2 – Count with iterators

Use `std::count` (takes `begin`, `end`, `value`) to count how many times 3 appears in `std::vector<int> v = {1, 3, 2, 3, 4, 3, 5}`.

Answer:

```
int c = std::count(v.begin(), v.end(), 3); // 3
```


Chapter 29: Algorithms: sort, find, transform, and the Rest

Why Algorithms

The standard library provides ~100 generic algorithms in `<algorithm>` and `<numeric>`. They work on any container via iterators, are highly optimized, and eliminate common hand-written loop bugs.

```
# Python: built-in functions and list methods
nums = [3, 1, 4, 1, 5, 9, 2]
nums.sort()
filtered = [x for x in nums if x > 3]
doubled = [x * 2 for x in nums]
total = sum(nums)
```

```
// C++: algorithms from <algorithm> and <numeric>
#include <algorithm>
#include <numeric>

std::vector<int> nums = {3, 1, 4, 1, 5, 9, 2};

std::sort(nums.begin(), nums.end());

std::vector<int> filtered;
std::copy_if(nums.begin(), nums.end(),
             std::back_inserter(filtered),
             [](int x) { return x > 3; }); // lambda as predicate

std::vector<int> doubled(nums.size());
std::transform(nums.begin(), nums.end(), doubled.begin(),
               [](int x) { return x * 2; });

int total = std::accumulate(nums.begin(), nums.end(), 0);
```

The Most Useful Algorithms

Searching

```

std::vector<int> v = {1, 5, 2, 8, 3};

// Find first element equal to 8:
auto it = std::find(v.begin(), v.end(), 8);
if (it != v.end()) std::cout << "Found at index " << (it - v.begin()) << "\n";

// Find first element satisfying a predicate:
auto it2 = std::find_if(v.begin(), v.end(), [](int x){ return x > 6; });
// *it2 == 8

// Check if any/all/none satisfy a predicate:
bool any = std::any_of(v.begin(), v.end(), [](int x){ return x > 7; }); // true
bool all = std::all_of(v.begin(), v.end(), [](int x){ return x > 0; }); // true
bool none = std::none_of(v.begin(), v.end(), [](int x){ return x > 10; }); // true

// Count elements satisfying predicate:
int count = std::count_if(v.begin(), v.end(), [](int x){ return x % 2 == 0; }); // 1 (just 2)

// Binary search (requires sorted range):
std::vector<int> s = {1, 2, 3, 4, 5};
bool found = std::binary_search(s.begin(), s.end(), 3); // true

```

Sorting and Ordering

```

std::vector<int> v = {3, 1, 4, 1, 5, 9};

// Sort ascending (default):
std::sort(v.begin(), v.end());

// Sort descending (custom comparator):
std::sort(v.begin(), v.end(), std::greater<int>{});

// Sort with lambda comparator:
std::vector<std::string> words = {"banana", "apple", "cherry"};
std::sort(words.begin(), words.end(),
    [](const std::string& a, const std::string& b) {
        return a.size() < b.size(); // sort by length
    });
// {"apple", "banana", "cherry"} (apple=5, banana=6, cherry=6)

// Stable sort: preserves relative order of equal elements
std::stable_sort(v.begin(), v.end());

// Partial sort: get smallest 3 elements in sorted order
std::partial_sort(v.begin(), v.begin() + 3, v.end());

// nth_element: guarantees v[n] is what would be there if sorted
// All elements before v[n] are ≤ v[n]; all after are ≥ v[n]
std::nth_element(v.begin(), v.begin() + 3, v.end());

```

Transforming

```
std::vector<int> v = {1, 2, 3, 4, 5};
std::vector<int> result(v.size());

// Apply function to each element, write to result:
std::transform(v.begin(), v.end(), result.begin(),
    [](int x){ return x * x; });
// result = {1, 4, 9, 16, 25}

// transform with two input ranges:
std::vector<int> a = {1, 2, 3};
std::vector<int> b = {10, 20, 30};
std::vector<int> c(3);
std::transform(a.begin(), a.end(), b.begin(), c.begin(),
    [](int x, int y){ return x + y; });
// c = {11, 22, 33}

// Replace elements:
std::replace(v.begin(), v.end(), 3, 99);    // replace all 3s with 99
std::replace_if(v.begin(), v.end(),
    [](int x){ return x > 3; }, // predicate
    0);                          // replacement

// Fill:
std::fill(v.begin(), v.end(), 42);          // fill all with 42
std::iota(v.begin(), v.end(), 1);           // fill with 1,2,3,4,5 (in <numeric>)
```

The Erase-Remove Idiom

Standard containers do not have a built-in “remove matching elements” operation. The pattern is:

```
std::vector<int> v = {1, 2, 3, 2, 4, 2, 5};

// std::remove moves non-matching elements to the front, returns new end:
auto new_end = std::remove(v.begin(), v.end(), 2);
// v = {1, 3, 4, 5, ?, ?, ?}  <- unspecified values after new_end
// new_end points to the first '?'

// Erase the "removed" elements:
v.erase(new_end, v.end());
// v = {1, 3, 4, 5}

// C++20: std::erase / std::erase_if (cleaner):
std::erase(v, 2); // remove all 2s
std::erase_if(v, [](int x){ return x % 2 == 0; }); // remove all evens
```

Numeric Algorithms (<numeric>)

```
#include <numeric>
std::vector<int> v = {1, 2, 3, 4, 5};

// Sum: accumulate with + (init=0)
int sum = std::accumulate(v.begin(), v.end(), 0);    // 15

// Product: accumulate with *
int product = std::accumulate(v.begin(), v.end(), 1,
                              std::multiplies<int>{}); // 120

// Prefix sums: {1, 3, 6, 10, 15}
std::vector<int> prefix(5);
std::partial_sum(v.begin(), v.end(), prefix.begin());

// Inner product (dot product):
std::vector<int> w = {2, 3, 4, 5, 6};
int dot = std::inner_product(v.begin(), v.end(), w.begin(), 0);
// 1*2 + 2*3 + 3*4 + 4*5 + 5*6 = 2+6+12+20+30 = 70
```

Common Mistakes in This Chapter

Mistake 1: Forgetting That `std::remove` Does Not Actually Remove

The bug:

```
std::remove(v.begin(), v.end(), 3); // returns new_end, but v is unchanged in size
std::cout << v.size(); // still original size -- elements not actually removed
```

The fix: Always pair with `.erase()`: `v.erase(std::remove(...), v.end());`

Mistake 2: Sorting Before a Linear Search (Mismatch of Algorithm)

If you only search once, linear `std::find` ($O(n)$) is fine. If you search many times, sort first and use `std::binary_search` or a set. Sorting and then linear-searching every time is $O(n \log n + n)$ when you could do $O(\log n)$ per search after a one-time sort.

Exercises

Exercise 29.1 – Pipeline

Given `std::vector<int> v = {5, 3, 8, 1, 9, 2, 7, 4, 6}`: 1. Sort it 2. Remove all even numbers (use erase-remove or C++20 `std::erase_if`) 3. Print the result

Answer:

```
std::sort(v.begin(), v.end());
std::erase_if(v, [](int x){ return x % 2 == 0; });
for (int n : v) std::cout << n << " "; // 1 3 5 7 9
```

Exercise 29.2 – Transform and accumulate

Given a vector of prices (doubles), apply a 10% discount to each, then compute the total.

Answer:

```
std::vector<double> prices = {10.0, 25.0, 8.5, 42.0};

std::transform(prices.begin(), prices.end(), prices.begin(),
               [](double p){ return p * 0.9; });

double total = std::accumulate(prices.begin(), prices.end(), 0.0);
std::cout << "Total after discount: " << total << "\n"; // 76.95
```


Chapter 30: Lambdas and Function Objects

What Is a Lambda?

A lambda is an anonymous function defined inline at the point of use. In Python they are limited to single expressions; in C++ they are full functions.

```
# Python lambda (expression only)
square = lambda x: x * x
nums = [1, 2, 3, 4, 5]
squared = list(map(lambda x: x*x, nums))
```

```
// C++ lambda (full function, multiple statements allowed)
auto square = [](int x) { return x * x; };
std::vector<int> nums = {1, 2, 3, 4, 5};
std::transform(nums.begin(), nums.end(), nums.begin(),
               [](int x) { return x * x; });
```

Lambda Syntax

[capture](parameters) -> return_type { body }

[capture] -- what variables from the enclosing scope are accessible
(parameters) -- function parameters (can be omitted if none)
-> return_type -- optional: compiler deduces it if omitted
{ body } -- function body (any C++ code)

Examples:

```
auto greet = []() { std::cout << "Hello!\n"; };        // no capture, no params
auto add = [](int a, int b) { return a + b; };        // two params, deduced return
auto mul = [](int a, int b) -> int { return a*b; };    // explicit return type

greet();        // "Hello!"
add(3, 4);      // 7
mul(3, 4);      // 12
```


Captures: Accessing the Enclosing Scope

The capture clause controls which variables from the surrounding scope are accessible inside the lambda:

```
int x = 10;
int y = 20;

// Capture by value (copy):
auto f1 = [x]() { return x + 1; }; // x is copied into the lambda
x = 99;
f1(); // 11 -- uses the COPY made at capture time, not current x

// Capture by reference:
auto f2 = [&x]() { return x + 1; };
x = 99;
f2(); // 100 -- uses the CURRENT x (reference)

// Capture all by value:
auto f3 = [=]() { return x + y; }; // copies all used local variables

// Capture all by reference:
auto f4 = [&]() { return x + y; }; // references all used local variables

// Mix: capture x by value, everything else by reference:
auto f5 = [x, &]() { return x + y; };

// Capture and modify (mutable lambda):
int count = 0;
auto counter = [count]() mutable { return ++count; };
// 'mutable' allows modifying the captured copy
counter(); // 1
counter(); // 2
// count is still 0 in the enclosing scope (captured by value)
```

Lambda Memory Model

A lambda is syntactic sugar for a **function object** (a struct with operator()):

```
int threshold = 5;
auto above = [threshold](int x) { return x > threshold; };

// The compiler generates something like:
struct Lambda {
    int threshold; // captured variables become data members
    Lambda(int t) : threshold{t} {}
    bool operator()(int x) const { return x > threshold; }
};
Lambda above{threshold}; // lambda = instance of this struct
above(3); // false
above(7); // true
```

This means lambdas are not magic – they are just convenient syntax for creating small, local function objects.

Generic Lambdas (C++14)

```
// 'auto' in parameters makes the lambda a template:
auto square = [](auto x) { return x * x; };

square(3);           // int: 9
square(3.14);        // double: 9.8596
square(3.0f);         // float
```

The compiler generates a different instantiation for each argument type. This is equivalent to a template operator().

Storing Lambdas: std::function

std::function<return_type(param_types)> can store any callable – lambda, function pointer, functor:

```
#include <functional>

// Store different callables in the same variable:
std::function<int(int, int)> op;

op = [](int a, int b) { return a + b; };
op(3, 4);    // 7

op = [](int a, int b) { return a * b; };
op(3, 4);    // 12

// Store in a vector of callbacks:
std::vector<std::function<void()>> callbacks;
callbacks.push_back([]{ std::cout << "first\n"; });
callbacks.push_back([]{ std::cout << "second\n"; });
for (auto& cb : callbacks) cb();
```

Caveat: std::function has overhead (type erasure, heap allocation for large lambdas). For performance-critical code, prefer auto or template parameters when the callable type is known.

Immediately Invoked Lambdas

```
// Call the lambda right away (useful for complex initialization):
const int value = []() {
    int result = 0;
    for (int i = 1; i <= 100; ++i) result += i;
    return result;
}(); // <-- immediately invoked

// value == 5050, computed at runtime (or at compile time if constexpr)
```

Common Mistakes in This Chapter

Mistake 1: Capturing a Local Variable by Reference After It Goes Out of Scope

The bug:

```
std::function<int()> make_counter() {
    int count = 0;
    return [&count]() { return ++count; }; // DANGER: count is a local variable
} // count is destroyed here
auto counter = make_counter();
counter(); // undefined behavior: accesses destroyed 'count'
```

The fix: Capture by value [count] with mutable, or use a shared_ptr<int>:

```
return [count]() mutable { return ++count; }; // captures a copy -- safe
```

Mistake 2: [=] Capturing this by Value

The bug:

```
class Foo {
    int x = 10;
    auto make_lambda() {
        return [=]() { return x; }; // captures 'this' by pointer, not x by value!
        // 'x' inside a member function means 'this->x'
        // [=] captures 'this' by value (the pointer), not the object
    }
};
```

The fix: In C++17+, use [*this] to capture the entire object by value, or [x = this->x] to capture the specific member.

Exercises

Exercise 30.1 – Sort with lambda

Sort `std::vector<std::string>` words by length (shortest first). Break ties alphabetically.

Answer:

```
std::sort(words.begin(), words.end(),
    [](const std::string& a, const std::string& b) {
        if (a.size() != b.size()) return a.size() < b.size();
        return a < b; // alphabetical for same length
    });
```

Exercise 30.2 – Closure counter

Write a function `make_counter(int start)` that returns a lambda. Each call to the lambda returns the next integer, starting from `start`.

Answer:

```
auto make_counter(int start) {
    return [n = start]() mutable { return n++; };
}

auto c = make_counter(5);
c(); // 5
c(); // 6
c(); // 7
```


Chapter 31: Ranges and Views (C++20)

The Problem With Iterator Pairs

The traditional algorithm interface requires two iterators:

```
std::sort(v.begin(), v.end());
std::find(v.begin(), v.end(), 3);
```

This is verbose and error-prone (mismatched iterators from different containers). You also cannot compose operations cleanly:

```
// Sort, then filter, then transform -- with traditional algorithms:
std::sort(v.begin(), v.end());
std::vector<int> filtered;
std::copy_if(v.begin(), v.end(), std::back_inserter(filtered),
    [](int x){ return x % 2 == 0; });
std::transform(filtered.begin(), filtered.end(), filtered.begin(),
    [](int x){ return x * x; });
// Each step creates a new vector -- three allocations
```

Ranges: The C++20 Solution

C++20 introduces `std::ranges`, which lets algorithms take entire containers directly, and `std::views`, which provides lazy composable transformations:

```
#include <ranges>
#include <algorithm>

std::vector<int> v = {5, 3, 1, 4, 2};

// Algorithms take the whole range:
std::ranges::sort(v); // {1, 2, 3, 4, 5}
auto it = std::ranges::find(v, 3); // finds 3
bool any = std::ranges::any_of(v, [](int x){ return x > 4; }); // true
```

No more `v.begin()`, `v.end()`. The algorithm accepts the container directly.

Views: Lazy, Composable Transformations

A **view** is a lightweight, lazy adapter over a range. It does not copy data – it creates a “window” that transforms elements on demand:

```
# Python generators: lazy transformations
nums = range(1, 11)
evens = (x for x in nums if x % 2 == 0)
squared = (x*x for x in evens)
# Nothing computed yet -- all lazy
list(squared) # [4, 16, 36, 64, 100] -- computed here
```

```
// C++20 views: lazy transformations
#include <ranges>

std::vector<int> v = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

// Compose views with | (pipe operator):
auto result = v
    | std::views::filter([](int x){ return x % 2 == 0; }) // keep evens
    | std::views::transform([](int x){ return x * x; }); // square them

for (int n : result) {
    std::cout << n << " "; // 4 16 36 64 100
}
// Nothing computed until the loop iterates! Zero intermediate vectors.
```

The `|` operator chains views. Each view wraps the previous one lazily. When you iterate, each element is pulled through the chain one at a time – no intermediate allocations.

Common Views

```
namespace sv = std::views; // shorthand

std::vector<int> v = {1, 2, 3, 4, 5};

// filter: keep elements satisfying predicate
auto evens = v | sv::filter([](int x){ return x%2 == 0; });

// transform: apply function to each element
auto doubled = v | sv::transform([](int x){ return x * 2; });

// take: first N elements
auto first3 = v | sv::take(3); // {1, 2, 3}

// drop: skip first N elements
auto last3 = v | sv::drop(2); // {3, 4, 5}

// reverse: iterate backward
auto rev = v | sv::reverse; // {5, 4, 3, 2, 1}
```

```
// iota: generate integers
auto nums = sv::iota(1, 6);           // {1, 2, 3, 4, 5} (no vector needed)

// zip (C++23): iterate two ranges together
auto zipped = sv::zip(v, std::vector{10,20,30,40,50});
for (auto [a, b] : zipped)
    std::cout << a << "+" << b << " "; // 1+10 2+20 3+30 4+40 5+50
```

Materializing a View

Since views are lazy, they produce values on demand. To get a concrete container, iterate into one:

```
auto result_view = v
    | sv::filter([](int x){ return x > 2; })
    | sv::transform([](int x){ return x * 10; });

// Materialize into a vector:
std::vector<int> result(result_view.begin(), result_view.end());
// or in C++23:
std::vector<int> result2 = result_view | std::ranges::to<std::vector>();
```

Common Mistakes in This Chapter

Mistake 1: Storing a View to a Destroyed Container

The bug:

```
auto get_view() {
    std::vector<int> v = {1, 2, 3};
    return v | std::views::filter([](int x){ return x > 1; });
} // v is destroyed here!

auto view = get_view();
for (int n : view) { ... } // undefined behavior: iterating view over dead vector
```

The fix: Views are non-owning. The source must outlive the view. Materialize to a vector if you need to return data.

Exercises

Exercise 31.1 – Views pipeline

Using `std::views`, create a pipeline that: takes $\{1..20\}$, keeps multiples of 3, squares them, and takes the first 4 results. Print them.

Answer:

```
auto result = std::views::iota(1, 21)
    | std::views::filter([](int x){ return x % 3 == 0; })
    | std::views::transform([](int x){ return x * x; })
    | std::views::take(4);

for (int n : result) std::cout << n << " ";
// multiples of 3: 3,6,9,12,... -> squared: 9,36,81,144 -> first 4: 9 36 81 144
```

Chapter 32: Utility Types: optional, variant, any, tuple

std::optional<T> – A Value That May Not Exist

optional<T> holds either a T or nothing. It replaces the pattern of using a sentinel value (-1, nullptr, "") to mean “no value.”

```
# Python: None means "no value"
def find_user(id: int) -> dict | None:
    if id in db: return db[id]
    return None

user = find_user(42)
if user is not None:
    print(user["name"])
```

```
// C++: std::optional
#include <optional>

std::optional<std::string> find_user(int id) {
    if (id == 42) return std::string{"Alice"};
    return std::nullopt;    // no value
}

auto user = find_user(42);
if (user) {                // bool conversion: true if has value
    std::cout << *user << "\n";    // dereference like a pointer
    std::cout << user.value() << "\n"; // or .value() (throws if empty)
}

// With default:
std::string name = find_user(99).value_or("Unknown");    // "Unknown"
```

Use optional instead of: - Sentinel values (-1 for “not found”) - bool out-parameter pairs - Pointers just to express “maybe no value”

std::variant<Types...> – Type-Safe Union

variant holds exactly one value from a set of possible types. It is like a tagged union – you always know which type is currently stored.

```
# Python: dynamic typing handles this naturally
result = 42           # could be int
result = "error"      # or str
result = 3.14         # or float
```

```
// C++: variant -- exactly one of the listed types
#include <variant>

std::variant<int, std::string, double> v;

v = 42;
v = "hello";    // now holds string
v = 3.14;       // now holds double

// Check which type:
if (std::holds_alternative<double>(v)) {
    std::cout << "double: " << std::get<double>(v) << "\n";
}

// Visit: apply a callable to whatever is stored
std::visit([](auto& val) {
    std::cout << val << "\n"; // works for all types (generic lambda)
}, v);

// Pattern matching with overloaded visitor:
struct Visitor {
    void operator()(int i)          { std::cout << "int: " << i << "\n"; }
    void operator()(const std::string& s) { std::cout << "str: " << s << "\n"; }
    void operator()(double d)       { std::cout << "dbl: " << d << "\n"; }
};

std::visit(Visitor{}, v);
```

Practical Use: Error Handling

```
// Return either a value or an error message:
std::variant<int, std::string> parse_int(const std::string& s) {
    try {
        return std::stoi(s);           // success: return int
    } catch (...) {
        return std::string{"parse error: "} + s; // failure: return error
    }
}

auto result = parse_int("42");
if (auto val = std::get_if<int>(&result)) {
    std::cout << "Parsed: " << *val << "\n";
} else {
    std::cout << "Error: " << std::get<std::string>(result) << "\n";
}
```

std::any – Type-Erased Storage

std::any can hold a value of any copyable type. Unlike variant, the type is not fixed at compile time:

```
#include <any>

std::any a = 42;
a = std::string{"hello"};
a = 3.14;

// Access requires knowing the type:
std::cout << std::any_cast<double>(a) << "\n";    // 3.14
std::any_cast<int>(a);    // throws std::bad_any_cast: stored type is double

// Safe version:
if (double* p = std::any_cast<double>(&a)) {
    std::cout << *p << "\n";    // 3.14
}
```

Use any when the type is genuinely unknown at compile time (plugin systems, scripting interfaces). Prefer variant when the set of possible types is known.

std::tuple Revisited: Structured Bindings and Helpers

```
// Creating tuples:
auto t = std::make_tuple(42, 3.14, std::string{"hi"});

// Access:
auto& [n, d, s] = t;    // structured binding (C++17)
std::get<0>(t);    // 42

// std::tie: bind tuple elements to existing variables
int x; double y; std::string z;
std::tie(x, y, z) = t;

// Return multiple values from a function:
std::tuple<int, int> div_rem(int a, int b) {
    return {a/b, a%b};
}
auto [quotient, remainder] = div_rem(17, 5);
std::cout << quotient << " r " << remainder << "\n";    // 3 r 2
```

std::pair<A, B> – Two Values

pair is tuple with exactly two elements, more readable names:

```
std::pair<int, std::string> p{42, "hello"};
std::cout << p.first << " " << p.second << "\n";

auto [num, str] = p;    // structured binding

// Common with maps:
for (const auto& [key, val] : my_map) {
    std::cout << key << " -> " << val << "\n";
}
```

Choosing the Right Utility Type

A value that might not exist:

`std::optional<T>` -- simple, clear, fast

A value that is one of several known types:

`std::variant<A,B,C>` -- type-safe, visit with pattern matching

A value of unknown type (runtime):

`std::any` -- flexible, slower, use sparingly

Multiple values of known types, fixed number:

`std::tuple<A,B,C>` -- heterogeneous, structured bindings make it clean

Two values:

`std::pair<A,B>` -- when two is the right number

Common Mistakes in This Chapter

Mistake 1: Accessing Empty `optional`

The bug:

```
std::optional<int> opt;
std::cout << *opt;    // undefined behavior -- opt has no value
std::cout << opt.value(); // throws std::bad_optional_access
```

The fix: Check if (opt) or use `opt.value_or(default)`.

Mistake 2: `std::get` with Wrong Type on variant

The bug:

```
std::variant<int, std::string> v = 42;
std::get<std::string>(v); // throws std::bad_variant_access
```

The fix: Use `std::holds_alternative<T>(v)` to check, or `std::get_if<T>(&v)` which returns `nullptr` on mismatch.

Exercises

Exercise 32.1 – Safe division

Write `safe_divide(int a, int b)` returning `std::optional<double>`. Return `std::nullopt` if `b == 0`.

Answer:

```
std::optional<double> safe_divide(int a, int b) {
    if (b == 0) return std::nullopt;
    return static_cast<double>(a) / b;
}

auto r = safe_divide(10, 3);
std::cout << r.value_or(0.0) << "\n"; // 3.333...

auto r2 = safe_divide(10, 0);
std::cout << r2.value_or(0.0) << "\n"; // 0.0
```

Exercise 32.2 – Shape variant

Use `std::variant<Circle, Rectangle, Triangle>` (define these as simple structs with an `area()` method) and `std::visit` to compute and print the area of each shape in a vector.

Answer:

```
struct Circle { double r; double area() const { return 3.14159*r*r; } };
struct Rectangle { double w, h; double area() const { return w*h; } };
struct Triangle { double b, h; double area() const { return 0.5*b*h; } };

using Shape = std::variant<Circle, Rectangle, Triangle>;

std::vector<Shape> shapes = {
    Circle{5.0},
    Rectangle{3.0, 4.0},
    Triangle{6.0, 8.0}
};

for (const auto& shape : shapes) {
```

```

double area = std::visit([](const auto& s){ return s.area(); }, shape);
std::cout << "Area: " << area << "\n";
}
// Area: 78.5398
// Area: 12
// Area: 24

```

Exercise 32.3 – Multiple return values

Write `parse_date(std::string)` that parses “YYYY-MM-DD” and returns a `std::tuple<int, int, int>` (year, month, day). Use structured bindings to unpack it.

Answer:

```

#include <tuple>
#include <string>

std::tuple<int,int,int> parse_date(const std::string& s) {
    // Format: "YYYY-MM-DD"
    int y = std::stoi(s.substr(0, 4));
    int m = std::stoi(s.substr(5, 2));
    int d = std::stoi(s.substr(8, 2));
    return {y, m, d};
}

auto [year, month, day] = parse_date("2026-06-28");
std::cout << year << "/" << month << "/" << day << "\n"; // 2026/6/28

```

Part VI is complete. You now know the standard library well enough to write real programs: choosing the right container for each job, using iterators correctly, composing algorithms and lambdas, building lazy data pipelines with ranges and views, and using the utility types that make C++ code expressive without sacrificing performance.

Part VII covers modern C++ language features added since C++11 – `auto`, `constexpr`, `std::format`, coroutines, and modules. Ask to continue.

Part VII – Modern C++ (C++11 to C++23)

This part covers language features added after C++03. These are not optional extras – they are the vocabulary of contemporary C++ code. If you read any real C++ codebase written after 2015, you will encounter all of these.

Chapter 33: auto, Type Deduction, and Structured Bindings

Type Deduction: The Full Picture

auto was introduced in Chapter 2 for variable declarations. Here is the complete picture of how the deduction rules work – they matter because surprises here cause bugs.

auto Strips Top-Level Qualifiers

```
const int x = 5;
auto a = x;           // a is int, NOT const int
                      // 'auto' strips top-level const

int arr[3] = {1,2,3};
auto b = arr;         // b is int* (pointer), NOT int[3]
                      // arrays decay to pointers under auto

int& ref = x;
auto c = ref;         // c is int (a copy), NOT int&
                      // 'auto' strips references
```

To preserve qualifiers, add them explicitly:

```
const auto& d = x;    // const int&
auto& e       = x;    // const int& (deduced as const because x is const)
auto* f       = &x;   // const int* (pointer to const, because x is const)
```

decltype – Deduce the Type of an Expression

auto deduces the type of the initializer. decltype deduces the type of any expression without evaluating it:

```
int x = 5;
decltype(x)  a = 10;    // int    (type of x)
decltype(x+1) b = 10;   // int    (type of x+1, an rvalue)
decltype((x)) c = x;    // int&   (type of (x), an lvalue expression -- parentheses
                        // ⇨ matter!)
```

```
// Useful when you need the return type of something complex:
std::vector<int> v = {1,2,3};
decltype(v[0]) front = v[0];    // int& (v[0] returns int&)
```

`decltype((x))` with double parentheses gives you the “lvalue expression type” – always a reference if the expression is an lvalue. `decltype(x)` without double parentheses gives the declared type. This asymmetry trips everyone up once.

`decltype(auto)` – Perfect Return Type Deduction

Used when you want to forward a return type exactly, preserving value category:

```
int x = 5;

auto      f() { return x; }           // returns int (copy -- auto strips &)
decltype(auto) g() { return x; }      // returns int (decltype(x) = int)
decltype(auto) h() { return (x); }    // returns int& (decltype((x)) = int&)
// h() returns a reference to local x -- dangling reference!
```

The main use is perfect-forwarding wrappers that must return exactly what the wrapped function returns:

```
template <typename F, typename... Args>
decltype(auto) call(F&& func, Args&&... args) {
    return std::forward<F>(func)(std::forward<Args>(args)...);
}
```

`auto` for Function Return Types

```
// Deduced return type (C++14):
auto add(int a, int b) { return a + b; }    // returns int

// Trailing return type (C++11, useful for dependent types):
template <typename T, typename U>
auto add(T a, U b) -> decltype(a + b) {    // return type depends on T and U
    return a + b;
}
```

Trailing return types (`-> type` after parameter list) were necessary before C++14 allowed deduced return types. You still see them when the return type cannot be determined until after the parameters are named.

Structured Bindings (C++17)

Structured bindings decompose aggregates (arrays, pairs, tuples, structs) into named variables:

```
# Python: tuple unpacking
x, y = (1, 2)
key, value = ("Alice", 95)
a, b, *rest = [1, 2, 3, 4, 5]
```

```
// C++17: structured bindings
auto [x, y] = std::pair{1, 2};
auto [key, value] = std::pair{"Alice", 95};

// Works on arrays:
int arr[3] = {1, 2, 3};
auto [a, b, c] = arr;

// Works on structs (without user-defined code):
struct Point { double x, y, z; };
Point p{1.0, 2.0, 3.0};
auto [px, py, pz] = p;

// Works on std::tuple:
auto t = std::make_tuple(42, 3.14, std::string{"hi"});
auto [num, flt, str] = t;

// By reference (to avoid copy AND to allow modification):
auto& [rx, ry] = p;
rx = 10.0;    // modifies p.x
```

The most common use is iterating over maps cleanly:

```
std::map<std::string, int> scores = {"Alice", 95}, {"Bob", 87};

// Old style:
for (const std::pair<const std::string, int>& entry : scores) {
    std::cout << entry.first << ": " << entry.second << "\n";
}

// Modern style with structured bindings:
for (const auto& [name, score] : scores) {
    std::cout << name << ": " << score << "\n";
}
```

if and switch With Initializers (C++17)

Declare a variable scoped to the if/switch block:

```
// Pattern: init; condition
if (auto it = map.find("key"); it != map.end()) {
    std::cout << it->second << "\n";
}
// 'it' is NOT accessible here -- scoped to the if/else

// Without initializer (old style): 'it' pollutes the enclosing scope
auto it = map.find("key");
if (it != map.end()) { ... }
// 'it' still accessible here (even though you're done with it)
```

Common Mistakes in This Chapter

Mistake 1: auto Dropping const or Reference

The bug:

```
const std::string& get_name() const { return name; }
auto n = obj.get_name(); // n is std::string (copy), not const std::string&
                        // Large string copied unnecessarily
```

The fix: `const auto& n = obj.get_name();`

Mistake 2: decltype((x)) Returning Reference to Local

```
decltype(auto) bad() {
    int x = 5;
    return (x); // decltype((x)) = int& -- returns reference to local!
}              // x destroyed here, reference is dangling
```

The fix: `return x;` (without extra parentheses) for `decltype(auto)` return.

Exercises

Exercise 33.1 – Deduce the types

For each, what is the type of `x`?

```
int      a = 5;
const int b = 10;
int&     r = a;
int      arr[3] = {1,2,3};

auto x1 = a;      // (a)
auto x2 = b;      // (b)
auto x3 = r;      // (c)
auto x4 = arr;    // (d)
const auto& x5 = a; // (e)
```

Answer: - (a) `int` – plain copy - (b) `int` – `auto` strips `const` - (c) `int` – `auto` strips reference (copy) - (d) `int*` – array decays to pointer - (e) `const int&` – explicitly added back

Exercise 33.2 – Structured bindings in practice

Rewrite this using structured bindings:

```
std::map<int, std::string> months = {{1,"Jan"},{2,"Feb"},{3,"Mar"}};
for (const std::pair<const int, std::string>& p : months) {
    std::cout << p.first << ": " << p.second << "\n";
}
```

Answer:

```
for (const auto& [num, name] : months) {
    std::cout << num << ": " << name << "\n";
}
```


Chapter 34: constexpr and Compile-Time Computation

Beyond Simple Constants

constexpr in C++11 was limited to single-expression functions. C++14 and later removed most restrictions: constexpr functions can now contain loops, local variables, conditionals, and recursion.

```
// C++14 and later: full constexpr functions
constexpr int factorial(int n) {
    int result = 1;
    for (int i = 2; i <= n; ++i)
        result *= i;
    return result;
}

constexpr int f10 = factorial(10); // 3628800, zero runtime cost
```

The rule: a constexpr function is evaluated at compile time when: 1. All its arguments are compile-time constants, AND 2. The result is used in a context requiring a compile-time constant

Otherwise it runs at runtime like a normal function.

```
constexpr int square(int n) { return n * n; }

constexpr int a = square(5); // compile time: a = 25 baked into binary
int          n = 7;
int          b = square(n); // runtime: n is not a compile-time constant
```

constexpr Variables

```
constexpr double PI      = 3.14159265358979323846;
constexpr double TAU     = 2.0 * PI;
constexpr int    MAX_SIZE = 256;

// Use as array size (must be compile-time):
int buffer[MAX_SIZE];

// Use in template arguments (must be compile-time):
std::array<double, MAX_SIZE> arr;
```


constexpr – Must Be Compile-Time (C++20)

constexpr functions are called **immediate functions** – they MUST be evaluated at compile time. Calling them with a runtime argument is a compile error:

```
constexpr int double_it(int n) { return n * 2; }

constexpr int a = double_it(5);    // OK: compile time
int n = 7;
int b = double_it(n);              // COMPILE ERROR: n is not a constant
```

Use constexpr when a function makes no sense at runtime (e.g., generating code, computing type properties).

constexpr – Initialized at Compile Time, Mutable After (C++20)

```
constexpr int counter = 0;    // initialized at compile time (no static init order fiasco)
counter = 5;                  // but can be changed at runtime (unlike constexpr)
```

constexpr solves the **static initialization order fiasco**: global variables may be initialized in an unspecified order. constexpr guarantees initialization happens at compile time (constant initialization), avoiding the dependency problem.

if constexpr – Compile-Time Branch Selection

Already covered in Chapter 24. The key point: only the selected branch is compiled. The other branch can contain code that would fail to compile for this type:

```
template <typename T>
std::string to_str(T val) {
    if constexpr (std::is_same_v<T, std::string>) {
        return val;    // only compiled when T is string
    } else if constexpr (std::is_arithmetic_v<T>) {
        return std::to_string(val);    // only compiled when T is numeric
    } else {
        return "[unknown type]";
    }
}

to_str(42);    // "42"
to_str(3.14);    // "3.140000"
to_str(std::string{"hello"});    // "hello"
```

Compile-Time Strings and Algorithms (C++20)

`std::string` and many algorithms became `constexpr` in C++20:

```
constexpr std::string_view greeting = "Hello, World!";
constexpr auto len = greeting.size();           // 13, compile time
constexpr auto pos = greeting.find(',');        // 5, compile time

constexpr bool starts_with_hello = greeting.starts_with("Hello"); // true

// std::array algorithms are constexpr:
constexpr std::array<int, 5> arr = {5, 3, 1, 4, 2};
constexpr int min_val = *std::min_element(arr.begin(), arr.end()); // 1
constexpr int max_val = *std::max_element(arr.begin(), arr.end()); // 5
```

static_assert – Compile-Time Assertions

```
static_assert(sizeof(int) == 4, "Assumes 32-bit int");
static_assert(sizeof(void*) >= 8, "Requires 64-bit pointers");

template <typename T>
void process(T val) {
    static_assert(std::is_arithmetic_v<T>, "T must be numeric");
    // ...
}
```

`static_assert` is evaluated at compile time. A failing `static_assert` is a compile error with the given message. Use it to document and enforce assumptions.

Common Mistakes in This Chapter

Mistake 1: Calling `constexpr` With a Runtime Argument and Expecting Compile-Time

The bug:

```
constexpr int square(int n) { return n*n; }
std::vector<int> v(square(n), 0); // n is a runtime variable
                                // square(n) runs at runtime -- that's fine
int arr[square(n)];              // ERROR: array size must be compile-time constant
```

The fix: Use `std::vector` for runtime-sized arrays.

Mistake 2: constexpr Function That Cannot Actually Be Evaluated at Compile Time

The bug:

```
constexpr int read_file() {    // ERROR: file I/O cannot be done at compile time
    FILE* f = fopen("data.txt", "r"); // not constexpr
    ...
}
```

The fix: constexpr functions cannot call non-constexpr functions. The compiler will tell you.

Exercises

Exercise 34.1 – Compile-time primes

Write a constexpr function `is_prime(int n)` and use it to build a constexpr `std::array<bool, 20>` marking which numbers 0..19 are prime.

Answer:

```
constexpr bool is_prime(int n) {
    if (n < 2) return false;
    for (int i = 2; i * i <= n; ++i)
        if (n % i == 0) return false;
    return true;
}

constexpr std::array<bool, 20> primes = []() {
    std::array<bool, 20> arr{};
    for (int i = 0; i < 20; ++i) arr[i] = is_prime(i);
    return arr;
}();

// primes[2]=true, primes[4]=false, primes[7]=true, etc.
// All computed at compile time -- zero runtime cost
static_assert(primes[2] == true);
static_assert(primes[4] == false);
static_assert(primes[17] == true);
```

Exercise 34.2 – static_assert contract

Write a template `sum_array<T, N>(std::array<T, N>)` with a `static_assert` that `T` must be arithmetic, and verify it rejects `std::array<std::string, 3>`.

Answer:

```
template <typename T, std::size_t N>
T sum_array(const std::array<T, N>& arr) {
    static_assert(std::is_arithmetic_v<T>, "sum_array requires numeric type");
    T total{};
    for (const T& x : arr) total += x;
    return total;
}

sum_array(std::array{1,2,3,4,5}); // 15
sum_array(std::array{1.0,2.5,0.5}); // 4.0
// sum_array(std::array{std::string{"a"},...}); // Compile error: not arithmetic
```


Chapter 35: `std::format` and Modern String Handling

The String Formatting History in C++

C++ has had three generations of string formatting:

```
// Generation 1: printf (from C) -- fast but unsafe, no type checking
printf("Name: %s, Score: %d\n", name.c_str(), score); // crash if type wrong

// Generation 2: iostream -- type-safe but verbose and stateful
std::cout << "Name: " << name << ", Score: " << score << "\n";

// Generation 3: std::format (C++20) -- type-safe, readable, fast
std::cout << std::format("Name: {}, Score: {}\n", name, score);
```

`std::format` is modeled after Python's `str.format()` and f-strings. If you know Python string formatting, `std::format` will feel immediately familiar.

`std::format` Basics

```
# Python f-string
name, score = "Alice", 95
print(f"Name: {name}, Score: {score}")
print(f"Score: {score:05d}") # "Score: 00095"
print(f"Pi: {3.14159:.2f}") # "Pi: 3.14"
```

```
// C++20 std::format
#include <format>

std::string name = "Alice";
int score = 95;

std::cout << std::format("Name: {}, Score: {}\n", name, score);
std::cout << std::format("Score: {:05d}\n", score); // "Score: 00095"
std::cout << std::format("Pi: {:.2f}\n", 3.14159); // "Pi: 3.14"
```

The `{}` placeholder takes the next argument. `{:format_spec}` applies formatting options.

Format Specifiers

```
// Width and fill:
std::format("{:10}", 42);           // "      42" (right-aligned, width 10)
std::format("{:<10}", 42);          // "42      " (left-aligned)
std::format("{:^10}", 42);          // "      42      " (centered)
std::format("{:0>10}", 42);         // "0000000042" (fill with 0, right-aligned)

// Integer bases:
std::format("{:d}", 255);           // "255" decimal
std::format("{:x}", 255);           // "ff" hex lowercase
std::format("{:X}", 255);           // "FF" hex uppercase
std::format("{:#x}", 255);          // "0xff" hex with prefix
std::format("{:b}", 255);           // "11111111" binary
std::format("{:o}", 255);           // "377" octal

// Floating point:
std::format("{:.2f}", 3.14159);     // "3.14" fixed, 2 decimal places
std::format("{:.4e}", 12345.678);   // "1.2346e+04" scientific
std::format("{:.3g}", 0.000123);    // "0.000123" general (chooses f or e)
std::format("{:10.3f}", 3.14);      // "      3.140" width + precision

// Strings:
std::format("{:>20}", "hello");      // "                  hello" right-aligned
std::format("{:.5}", "hello world"); // "hello" (truncate to 5 chars)

// Named arguments (from a position):
std::format("{0} and {1} and {0}", "A", "B"); // "A and B and A"
```

std::format vs std::to_string vs std::stringstream

```
// std::to_string: simple, no formatting control
std::string s1 = std::to_string(3.14159); // "3.141590" (6 decimal places always)

// std::stringstream: full control but verbose
std::ostringstream oss;
oss << std::fixed << std::setprecision(2) << 3.14159;
std::string s2 = oss.str();                // "3.14"

// std::format: clean, explicit, composable
std::string s3 = std::format("{:.2f}", 3.14159); // "3.14"
```

Use `std::format` for anything where you care about the format. Use `std::to_string` only for quick-and-dirty number-to-string conversion.

std::format for Building Strings

```
// Building a log line:
auto log_line = std::format("[{:H:%M:%S}] {:>8} | {}\\n",
                           std::chrono::system_clock::now(),
                           "INFO",
                           "Server started");

// Building SQL (demonstration -- use a real SQL library for actual queries):
auto query = std::format("SELECT * FROM users WHERE id = {}", user_id);

// Building error messages:
throw std::runtime_error(
    std::format("Index {} out of range [0, {}]", index, size));
```

std::string_view – Non-Owning String Reference

`std::string_view` is a non-owning reference to a string (or substring) – like a read-only `string&` that also works for string literals, substrings, and other char buffers:

```
#include <string_view>

void print(std::string_view sv) { // accepts string, string literal, char array
    std::cout << sv << "\\n";
}

std::string s = "hello world";
print(s); // works: string -> string_view (no copy)
print("hello world"); // works: literal -> string_view (no copy)
print(s.substr(0, 5)); // works: substr returns string, then converts

// Substring WITHOUT copying:
std::string_view first5{s.data(), 5}; // "hello" -- no allocation
```

`string_view` is faster than `const std::string&` for function parameters because it avoids the `std::string` constructor for string literals:

```
void bad(const std::string& s) { ... } // "hello" creates a std::string object
void good(std::string_view sv) { ... } // "hello" is just a pointer+length, no alloc
```

Danger: `string_view` does not own the data. The underlying string must outlive the view:

```
std::string_view dangling() {
    std::string s = "hello";
    return s; // s is destroyed -- returned view is dangling!
}
```


Common Mistakes in This Chapter

Mistake 1: Using `std::format` But Forgetting to `#include <format>`

error: 'format' is not a member of 'std'

Add `#include <format>`.

Mistake 2: `string_view` Outliving the Source String

The bug:

```
std::string_view sv;
{
    std::string s = "hello";
    sv = s;        // sv points to s's internal buffer
}                 // s destroyed -- sv is dangling
std::cout << sv;  // undefined behavior
```

The fix: Store `std::string` if ownership is needed. Use `string_view` only as a parameter type or within a known lifetime scope.

Exercises

Exercise 35.1 – Format table

Format this data as a right-aligned table with columns of width 12:

Name	Score	Grade
Alice	95	A
Bob	72	C
Carol	88	B

Answer:

```
struct Student { std::string name; int score; char grade; };
std::vector<Student> students = {"Alice",95,'A'},{"Bob",72,'C'},{"Carol",88,'B'};

std::cout << std::format("{:<12} {:>8} {:>8}\n", "Name", "Score", "Grade");
for (const auto& s : students) {
    std::cout << std::format("{:<12} {:>8} {:>8}\n", s.name, s.score, s.grade);
}
```

Exercise 35.2 – Hex dump

Given `std::vector<uint8_t> data = {0x48, 0x65, 0x6C, 0x6C, 0x6F}`, print each byte as two-digit uppercase hex separated by spaces, followed by the ASCII interpretation.

Answer:

```
std::string hex_part, ascii_part;
for (uint8_t b : data) {
    hex_part += std::format("{:02X} ", b);
    ascii_part += std::format("{} ", (b >= 32 && b < 127) ? (char)b : '.');
}
std::cout << hex_part << " | " << ascii_part << "\n";
// 48 65 6C 6C 6F | Hello
```


Chapter 36: Coroutines and Generators

What Is a Coroutine?

A coroutine is a function that can **suspend** its execution midway and **resume** later, preserving all its local state. It picks up exactly where it left off.

```
# Python generator: a coroutine
def count_up(start, stop):
    n = start
    while n < stop:
        yield n          # suspend here, return n to caller
        n += 1           # resume here next time

for x in count_up(0, 5):
    print(x)             # 0, 1, 2, 3, 4
```

C++20 introduced coroutines with three keywords: `co_yield`, `co_return`, and `co_await`.

How Coroutines Work (Conceptually)

Normal function:

Caller calls `f()` --> `f` runs to completion --> returns to caller
State: lives on the stack while running, gone when returned

Coroutine:

Caller calls `f()` --> `f` runs until `co_yield` --> suspends
State: moved to the HEAP (coroutine frame), persists
Caller resumes `f()` --> `f` continues from suspension point
This repeats until `co_return`

Coroutine frame (on heap):

+-----+	
local variables	preserved across suspensions
current suspension point	where to resume
promise object	communication with caller
+-----+	

The coroutine's local state lives in a heap-allocated frame. This is what makes it resumable – the stack frame is gone, but the heap frame persists.

Generators With `co_yield`

The most common coroutine use: generating sequences lazily.

C++20 does not provide a ready-made Generator type (it was added in C++23). Here is the C++23 version:

```
#include <generator>    // C++23

std::generator<int> count_up(int start, int stop) {
    for (int i = start; i < stop; ++i) {
        co_yield i;    // suspend, return i to the caller
    }
    // co_return implicit at end of function
}

for (int n : count_up(0, 5)) {
    std::cout << n << " ";    // 0 1 2 3 4
}
```

Each iteration of the for loop resumes the coroutine until the next `co_yield`. When the coroutine function ends, the range is exhausted.

Infinite Sequences

```
std::generator<int> fibonacci() {
    int a = 0, b = 1;
    while (true) {    // infinite -- coroutine suspends each iteration
        co_yield a;
        auto tmp = a + b;
        a = b;
        b = tmp;
    }
}

auto fib = fibonacci();
for (int i = 0; i < 10; ++i) {
    std::cout << *fib.begin() << " ";
    ++fib.begin();    // advance to next
}
// Better: use take view
for (int n : fibonacci() | std::views::take(10)) {
    std::cout << n << " ";    // 0 1 1 2 3 5 8 13 21 34
}
```

`co_await` – Asynchronous Operations

`co_await` suspends the coroutine until an asynchronous operation completes:

```
# Python async/await:
async def fetch_data(url):
    response = await http_client.get(url) # suspend until response arrives
    return response.json()
```

```
// C++20 co_await (with a networking library that provides awaitables):
Task<std::string> fetch_data(std::string url) {
    auto response = co_await http_get(url); // suspend until response
    co_return response.body();
}
```

The machinery for `co_await` requires either a library (Asio, CPP-Coro, etc.) or your own promise type. Chapter 45 covers async patterns in depth.

co_return - Returning a Value

```
Task<int> compute_async(int n) {
    co_await some_async_work();
    co_return n * n; // like 'return', but for coroutines
}
```

A coroutine that uses `co_return` (or any other `co_` keyword) must have a matching **promise type** associated with it. The promise type defines how values are transmitted to the caller. C++23's `std::generator` handles this automatically for generator coroutines.

When to Use Coroutines

Use coroutines for:

- Lazy sequence generation (generators): `co_yield`
- Async I/O without callback hell: `co_await`
- State machines that are simpler to write as coroutines
- Cooperative multitasking (cooperative concurrency)

Do NOT use for:

- CPU-heavy parallel work (use `std::thread` or `std::async`, Chapter 42)
- Simple synchronous code (unnecessary overhead)
- Cases where iterators or callbacks are clearer

Common Mistakes in This Chapter

Mistake 1: Using a Coroutine Return Value as a Regular Value

The bug:

```
std::generator<int> gen = count_up(0, 10);
int first = gen;    // ERROR: generator is not convertible to int
```

The fix: Iterate over the generator, or use `*gen.begin()` for the first element.

Mistake 2: Dangling References in Coroutine Locals

The bug:

```
std::generator<std::string&> bad_gen() {
    std::string s = "hello";
    co_yield s;    // yields a reference to local s
} // s is in the coroutine frame and lives as long as the generator -- this is fine
// BUT: if the generator is destroyed before the reference is used: dangling
```

The fix: Yield values, not references, when in doubt. Or ensure the generator outlives all yielded references.

Exercises

Exercise 36.1 – Range generator

Using `std::generator<int>`, write `range(int start, int stop, int step = 1)` that works like Python's `range()`.

Answer:

```
#include <generator>

std::generator<int> range(int start, int stop, int step = 1) {
    for (int i = start; step > 0 ? i < stop : i > stop; i += step) {
        co_yield i;
    }
}

// Test:
for (int n : range(0, 10, 2)) std::cout << n << " "; // 0 2 4 6 8
for (int n : range(10, 0, -2)) std::cout << n << " "; // 10 8 6 4 2
```

Exercise 36.2 – Powers generator

Write a generator `powers_of(int base)` that yields 1, base, base², base³, ... indefinitely. Use it with `std::views::take(5)` to get the first 5 powers of 3.

Answer:

```
std::generator<long long> powers_of(int base) {
    long long val = 1;
    while (true) {
        co_yield val;
        val *= base;
    }
}

for (long long n : powers_of(3) | std::views::take(5)) {
    std::cout << n << " "; // 1 3 9 27 81
}
```


Chapter 37: Modules (C++20)

The Problem With Headers

The header/source split from Chapter 11 has been C++'s compilation model since the 1970s. It works but has significant problems:

Problems with `#include`:

1. Slow: every `.cpp` re-reads and re-parses every header it includes
A single `#include <iostream>` adds ~45,000 lines to every file
2. Order-dependent: headers must be included in the right order
3. Macro leakage: `#defines` in headers affect all subsequent code
4. ODR violations: accidentally defining something twice across headers
5. No true encapsulation: private impl details still visible in headers

Modules solve all of these problems.

Writing a Module

```
// math_utils.cppm (module implementation unit, .cppm extension by convention)
export module math_utils; // declare this is a module named 'math_utils'

// These are NOT exported -- module-private (not visible to importers)
namespace {
    double pi_approx() { return 3.14159265358979; }
}

// 'export' makes these visible to importers:
export double circle_area(double r) {
    return pi_approx() * r * r;
}

export int factorial(int n) {
    return n <= 1 ? 1 : n * factorial(n - 1);
}

// Export a whole namespace:
export namespace geometry {
    double hypotenuse(double a, double b) {
        return std::sqrt(a*a + b*b);
    }
}
```

```
// main.cpp
import math_utils;           // import the module (not #include)
import <iostream>;           // standard library modules (C++23)

int main() {
    std::cout << circle_area(5.0) << "\n";           // 78.5398
    std::cout << factorial(10) << "\n";               // 3628800
    std::cout << geometry::hypotenuse(3, 4) << "\n"; // 5
}
```

No angle brackets, no header files. `import math_utils` gives `main.cpp` access to everything marked `export` in the module.

Module Partitions

Large modules can be split into partitions:

```
// math_utils-trig.cppm (partition: math_utils:trig)
export module math_utils:trig;
export double sin_deg(double deg) { return std::sin(deg * 3.14159/180); }
export double cos_deg(double deg) { return std::cos(deg * 3.14159/180); }

// math_utils.cppm (primary module interface)
export module math_utils;
export import :trig;           // re-export the trig partition

export double circle_area(double r) { ... }
```

Modules vs Headers: Side-by-Side

Headers (`#include`):

- Parse every time
- Macros leak out
- Textual inclusion (fragile)
- Order-dependent
- Private impl visible in header
- Works with all compilers (C++98+)

Modules (`import`):

- Parsed once, binary interface cached
- No macro leakage
- Semantic interface
- Order-independent
- Private impl truly private
- Requires C++20 + toolchain support

Toolchain Support (as of 2025/2026)

Modules are supported in: - **GCC 14+** with `-std=c++20` - **Clang 16+** with `-std=c++20` - **MSVC** (Visual Studio 2019 v16.8+)

Build system support is still maturing. CMake 3.28+ supports modules natively. For new projects, modules are ready to use. For existing projects, incremental adoption is possible – modules and headers can coexist.

Common Mistakes in This Chapter

Mistake 1: Including Headers Inside a Module That Leaks Macros

The bug:

```
export module mylib;
#include <windows.h>    // defines hundreds of macros (min, max, BOOL, etc.)
                        // These macros DO NOT leak to importers (good)
                        // But they affect everything INSIDE the module
```

Better: Use `import <windows.h>` as a header unit if your toolchain supports it, or wrap the include in a translation unit that doesn't export the macros.

Mistake 2: Exporting Everything

Modules encourage thinking about your API surface. Do not put `export` on internal helper functions. Keep implementation details module-private.

Exercises

Exercise 37.1 – Write a module

Convert this header/source pair into a module:

```
// geometry.h
double circle_area(double r);
double rectangle_area(double w, double h);

// geometry.cpp
double circle_area(double r) { return 3.14159 * r * r; }
double rectangle_area(double w, double h) { return w * h; }
```

Answer:

```
// geometry.cppm
export module geometry;
import <numbers>; // C++20: std::numbers::pi

export double circle_area(double r) {
    return std::numbers::pi * r * r;
}

export double rectangle_area(double w, double h) {
    return w * h;
}

// main.cpp
import geometry;
import <iostream>;

int main() {
    std::cout << circle_area(5.0) << "\n"; // 78.5398
    std::cout << rectangle_area(3.0, 4.0) << "\n"; // 12
}
```

Part VII is complete. You now know the modern C++ language features that distinguish contemporary code from older C++: precise type deduction, full compile-time computation, expressive string formatting, coroutines for generators and async, and the module system that will eventually replace headers.

Part VIII covers performance – value vs reference semantics, memory layout and cache effects, data-oriented design, and profiling. These are what separate C++ programs that are fast from C++ programs that are merely correct. Ask to continue.

Part VIII – Performance

C++ is fast because it gives you direct control over where data lives and how it moves. Python hides these decisions behind an abstraction layer. This part covers the mental model you need to write C++ that is not just correct but genuinely fast.

The central insight: **modern CPUs are not limited by computation speed – they are limited by memory bandwidth.** Code that moves less data, keeps data contiguous, and avoids indirection runs faster regardless of how many instructions it executes.

Chapter 38: Value vs Reference Semantics

The Two Semantic Models

Value semantics: copying an object gives you a fully independent copy. Modifying the copy does not affect the original.

Reference semantics: “copying” an object gives you another reference to the same underlying data. Modifying one reference affects all others.

Python uses reference semantics for all mutable objects:

```
# Python: reference semantics for lists
a = [1, 2, 3]
b = a          # b is a reference to the SAME list
b.append(4)
print(a)       # [1, 2, 3, 4] -- a was affected

# Only immutable types (int, str, tuple) behave like values in Python
x = 5
y = x
y += 1
print(x)       # 5 -- x unchanged (int is immutable)
```

C++ defaults to value semantics for all types:

```
// C++: value semantics by default
std::vector<int> a = {1, 2, 3};
std::vector<int> b = a;          // FULL COPY of the data
b.push_back(4);
std::cout << a.size() << "\n"; // 3 -- a is unchanged

// Reference semantics: explicit opt-in with & or pointer
std::vector<int>& ref = a;       // ref IS a (alias)
ref.push_back(4);
std::cout << a.size() << "\n"; // 4 -- a was affected through ref
```

Why Value Semantics Makes Reasoning Easier


```

void process(std::vector<int> v) { // v is a copy
    v.push_back(99);
    sort(v.begin(), v.end());
}

std::vector<int> data = {5, 3, 1, 4};
process(data);
// data is guaranteed unchanged -- no aliasing possible

```

When you pass by value, a function cannot surprise you by modifying your data. The function receives its own independent copy. This is the reason the standard library returns by value – it is easier to reason about.

When Value Semantics Is Expensive

Copying large objects is expensive. A `std::vector<int>` with a million elements takes ~4 MB to copy. When reading is all you need, pass by `const&` instead:

```

// Expensive: copies the entire vector (4 MB)
double average(std::vector<int> v) {
    double sum = 0;
    for (int x : v) sum += x;
    return sum / v.size();
}

// Free: passes a reference (8 bytes on a 64-bit system)
double average(const std::vector<int>& v) {
    double sum = 0;
    for (int x : v) sum += x;
    return sum / v.size();
}

```

The parameter-passing decision table from Chapter 6, with performance context:

Small trivially-copyable type (int, double, pointer, small struct ≤ 16 bytes):

Pass by value: `T x`

Cost: cheap copy, may allow more compiler optimizations (no aliasing)

Large or non-trivial type (string, vector, custom class):

Read only: `const T& x` -- 8 bytes (pointer), no copy

Must modify: `T& x` -- 8 bytes (pointer), modifies caller's object

Take ownership: `T x (+ std::move)` -- one move (usually free)

Return values:

Always return by value -- NRVO/move makes it free in practice

Never return references to locals

The Slice Problem and Polymorphism

Value semantics and polymorphism conflict. If you have a base-class value, you can only store a base-class object:

```
std::vector<Shape> shapes;           // value semantics
shapes.push_back(Circle{5.0});       // Circle is SLICED to Shape
shapes.push_back(Rectangle{3,4});    // Rectangle is SLICED to Shape

for (const Shape& s : shapes) {
    s.area();                       // always calls Shape::area, never Circle::area
}
```

For polymorphism, you need reference semantics – pointers or references:

```
std::vector<std::unique_ptr<Shape>> shapes; // reference semantics
shapes.push_back(std::make_unique<Circle>(5.0));
shapes.push_back(std::make_unique<Rectangle>(3,4));

for (const auto& s : shapes) {
    s->area();                       // dynamic dispatch -- calls Circle::area, Rectangle::area
}
```

Small Object Optimization (SOO)

Modern standard library types use **small object optimization**: for small values, they store the data inline (in the object itself) instead of on the heap, avoiding a heap allocation.

`std::string` typically stores strings up to 15 characters inline:

Small string (<= 15 chars):	Large string (> 15 chars):
+-----+	+-----+
chars[16]: "hello\0..."	ptr -> [heap: "long..."]
size: 5	size: 100
(no heap allocation)	capacity: 128
+-----+	+-----+

`sizeof(std::string) == 24` in both cases

`std::function` similarly stores small callables inline. `std::any` also has SOO. This means small types often cost no more than raw values even when wrapped in standard library types.

std::span<T> – Non-Owning View Over Contiguous Data

std::span<T> (C++20) is to arrays and vectors what std::string_view is to strings: a non-owning view over a contiguous sequence.

```
#include <span>

void print_first_n(std::span<const int> data, int n) {
    for (int i = 0; i < n && i < (int)data.size(); ++i)
        std::cout << data[i] << " ";
}

int arr[5] = {1, 2, 3, 4, 5};
std::vector<int> v = {10, 20, 30};

print_first_n(arr, 3);    // works: C array -> span
print_first_n(v, 2);      // works: vector -> span
print_first_n({arr+1, 3}, 3); // works: subrange {2,3,4}
```

Use std::span for functions that operate on any contiguous sequence – avoids template proliferation and still has zero overhead.

Common Mistakes in This Chapter

Mistake 1: Passing Large Objects by Value When Only Reading

The bug:

```
void log(std::string message) {           // copies the entire string
    std::cout << message << "\n";
}
log("This is a very long log message that gets copied unnecessarily");
```

The fix: void log(std::string_view message) or void log(const std::string& message)

Mistake 2: Returning const Value (Disables Move)

The bug:

```
const std::vector<int> compute() {        // const return value
    std::vector<int> v = {1,2,3};
    return v;
}
std::vector<int> result = compute(); // cannot move! const prevents moving
```

The fix: Return by non-const value: std::vector<int> compute()

Exercises

Exercise 38.1 – Identify the semantics

For each statement, say whether C++ uses value or reference semantics and whether a copy occurs:

```
std::string s = "hello";  
std::string t = s;           // (a)  
std::string& r = s;          // (b)  
void f(std::string x);    f(s); // (c)  
void g(const std::string& x); g(s); // (d)  
std::string u = std::move(s); // (e)
```

Answer: - (a) Value semantics – copy: `t` is a new independent string - (b) Reference semantics – no copy: `r` is an alias for `s` - (c) Value semantics – copy: `x` in `f` is a full copy of `s` - (d) Reference semantics – no copy: `x` in `g` is a const reference to `s` - (e) Move semantics – no copy: `s` is moved into `u` (`s` is now empty)

Exercise 38.2 – Fix the signature

This function does not modify its argument. It only needs to read it. Fix the signature to avoid an unnecessary copy.

```
int count_vowels(std::string text) {  
    int count = 0;  
    for (char c : text)  
        if (std::string{"aeiouAEIOU"}.find(c) != std::string::npos) ++count;  
    return count;  
}
```

Answer:

```
int count_vowels(std::string_view text) {  
    int count = 0;  
    for (char c : text)  
        if (std::string_view{"aeiouAEIOU"}.find(c) != std::string_view::npos)  
            ++count;  
    return count;  
}
```


Chapter 39: How Memory Layout Affects Speed

The CPU and the Memory Hierarchy

Modern CPUs do not read from RAM one byte at a time. They read in **cache lines** – typically 64 bytes at a time. If you access one byte, the CPU fetches all 64 surrounding bytes and caches them. The next access to nearby data is satisfied from cache (fast); access to distant data causes a **cache miss** and a round-trip to RAM (slow).

Memory hierarchy (approximate latencies on a modern x86-64):

Level	Size	Latency	
L1 cache	32–64 KB	~1 ns	(4–5 CPU cycles)
L2 cache	256 KB–1 MB	~4 ns	(12–15 cycles)
L3 cache	4–32 MB	~15 ns	(40–50 cycles)
RAM	4–64 GB	~60 ns	(200+ cycles)
SSD	1+ TB	~100 µs	(300,000+ cycles)
HDD	1+ TB	~10 ms	(30,000,000+ cycles)

A cache miss costs ~200x the price of a cache hit. Algorithms that access memory randomly (linked lists, hash maps with many collisions, pointer chasing through class hierarchies) pay this penalty constantly. Algorithms that access memory sequentially (arrays, vectors) pre-fetch into cache and pay it rarely.

Contiguous Arrays vs Linked Structures

`std::vector<int>:`

`[1][2][3][4][5][6][7][8]`
sequential, one cache line

`std::list<int>:`

`[1]->[2]->[3]->[4]->[5]`
each node: value + two pointers
scattered across heap, many cache misses

Iterating over a `std::vector<int>` of 1 million elements: the CPU pre-fetches ahead, and most accesses hit the cache. Iterating over a `std::list<int>` of the same elements: each `.next` pointer jump may land anywhere in RAM, causing cache misses for most accesses.

Measured difference: vector iteration is typically **10-50x faster** than list iteration for integer elements on modern hardware.

Array of Structs vs Struct of Arrays

A classic performance pattern. Consider particles with position, velocity, and mass:

Array of Structs (AoS) – the natural OOP layout:

```
struct Particle {
    float x, y, z;      // position
    float vx, vy, vz;   // velocity
    float mass;
};

std::vector<Particle> particles(N);
```

Memory layout:

```
[x0 y0 z0 vx0 vy0 vz0 m0][x1 y1 z1 vx1 vy1 vz1 m1][x2 ...]
<-- particle 0 -->      <-- particle 1 -->
```

Struct of Arrays (SoA) – cache-friendly for SIMD/batch operations:

```
struct Particles {
    std::vector<float> x, y, z;    // all positions
    std::vector<float> vx, vy, vz; // all velocities
    std::vector<float> mass;
};

Particles particles;
particles.x.resize(N);
// ...
```

Memory layout:

```
x:  [x0][x1][x2][x3][x4][x5][x6][x7]...
y:  [y0][y1][y2][y3][y4][y5][y6][y7]...
vx: [vx0][vx1][vx2][vx3]...
```

When does layout matter?

```
// Gravity update: only reads/writes x, y, z, vx, vy, vz -- NOT mass
// AoS: loads a full Particle (28 bytes) but only uses 24 bytes
//      Every 3rd cache line contains mass data you don't need

// SoA: loads x[], y[], z[], vx[], vy[], vz[] sequentially
//      100% of loaded cache lines contain useful data
//      Also: SIMD can process 8 floats at once from contiguous x[] array

// Benchmark: 1M particles, gravity update
// AoS: ~8ms
// SoA: ~1.5ms (5x speedup -- same computation, different layout)
```

Use SoA when you frequently access a subset of fields across all objects. Use AoS when you typically access all fields of one object at a time.

False Sharing: The Multi-Core Cache Problem

When two CPU cores write to different variables that happen to share a cache line, the cores fight over the cache line – even though they are writing to different variables.

```
// These two ints share a cache line (they are adjacent):
struct Counters {
    int counter_a; // core 1 writes here
    int counter_b; // core 2 writes here
};

// Both cores must invalidate and re-fetch the cache line on each other's write
// Performance: similar to a single-threaded program, no parallelism gained
```

Fix: pad to force each variable onto its own cache line:

```
struct alignas(64) PaddedCounter { // 64 = typical cache line size
    int value;
    char padding[60]; // explicit padding, OR use alignas(64)
};

PaddedCounter counter_a; // on its own cache line
PaddedCounter counter_b; // on a different cache line
// cores can now update independently -- true parallelism
```

Memory Alignment

Data types have **alignment requirements** – they must be stored at addresses that are multiples of their size (usually). The CPU fetches aligned data in one operation; misaligned data may require two fetches.

```
struct Packed {
    char a; // 1 byte
    int b; // 4 bytes -- alignment requires padding!
    char c; // 1 byte
    double d; // 8 bytes -- alignment requires padding!
};

// Actual layout:
// a: 1 byte
// padding: 3 bytes (to align b to 4-byte boundary)
// b: 4 bytes
// c: 1 byte
// padding: 7 bytes (to align d to 8-byte boundary)
// d: 8 bytes
// sizeof(Packed) = 24 bytes (not 14!)

struct Reordered {
    double d; // 8 bytes
    int b; // 4 bytes
    char a; // 1 byte
```



```

    char    c;    // 1 byte
    // padding: 2 bytes
};
// sizeof(Reordered) = 16 bytes -- 8 bytes smaller!

```

Rule: order struct members from largest to smallest to minimize padding.

Check with:

```

#include <iostream>
std::cout << sizeof(Packed)    << "\n"; // 24
std::cout << sizeof(Reordered) << "\n"; // 16
static_assert(offsetof(Packed, d) == 16); // d starts at byte 16, not 8

```

Branch Prediction

Modern CPUs execute instructions speculatively, guessing which branch an `if` will take. If the guess is right (branch prediction hit), it is nearly free. If wrong (misprediction), the CPU must flush the pipeline and restart: ~15-20 cycles wasted.

```

// Unpredictable branch: random 50/50 data
// CPU cannot predict: many mispredictions
std::vector<int> v(1'000'000);
std::generate(v.begin(), v.end(), []{ return rand() % 2; });
int sum = 0;
for (int x : v) {
    if (x > 0) sum += x; // branch is 50% unpredictable
}

// Branchless version: no branch to mispredict
for (int x : v) {
    sum += x * (x > 0); // multiply by 0 or 1 -- no branch
}

// Or:
for (int x : v) {
    sum += (x > 0) ? x : 0; // ternary often compiles branchless (cmov instruction)
}

```

Pre-sorting data makes the branch predictable:

```

std::sort(v.begin(), v.end());
// Now: all 0s come first, then all 1s -- branch is perfectly predictable
for (int x : v) {
    if (x > 0) sum += x; // CPU always predicts correctly after the first few
}

```

This is the famous “sorted array is faster” phenomenon.

[[likely]] and [[unlikely]] (C++20)

Hint to the compiler about which branch is more probable:

```
if (cache_hit) [[likely]] {
    return cached_result; // fast path: tell compiler this is common
} else [[unlikely]] {
    return recompute(); // slow path: compiler places this away from hot path
}
```

Common Mistakes in This Chapter

Mistake 1: Using `std::list` When `std::vector` Would Do

The misconception: “I need $O(1)$ insertion in the middle – use `std::list`.” **The reality:** For small numbers of elements (<a few thousand), a `std::vector` with $O(n)$ insertion is faster than a `std::list` with $O(1)$ insertion because the vector’s cache efficiency dominates. Only use `std::list` when: - Elements must not be invalidated by insertions (stable iterators) - The list is very large AND insertions dominate over iteration

Mistake 2: Struct Members Ordered Arbitrarily

Always check `sizeof(YourStruct)`. If it is larger than the sum of member sizes, there is padding. Re-order fields to eliminate waste.

Exercises

Exercise 39.1 – Struct padding

Calculate `sizeof` for each struct without running the code. Assume typical x86-64 alignment rules.

```
struct A { char a; int b; char c; };
struct B { int b; char a; char c; };
struct C { double d; int i; char c; };
struct D { char c; double d; int i; };
```

Answer: - A: $\text{char}(1) + \text{pad}(3) + \text{int}(4) + \text{char}(1) + \text{pad}(3) = 12$ - B: $\text{int}(4) + \text{char}(1) + \text{char}(1) + \text{pad}(2) = 8$ - C: $\text{double}(8) + \text{int}(4) + \text{char}(1) + \text{pad}(3) = 16$ - D: $\text{char}(1) + \text{pad}(7) + \text{double}(8) + \text{int}(4) + \text{pad}(4) = 24$

Lesson: reordering B saves 4 bytes vs A; C saves 8 bytes vs D.

Exercise 39.2 – AoS to SoA conversion

Convert this AoS to SoA and write an `update_positions(float dt)` function that adds `velocity*dt` to each position:

```
struct Particle { float x, y; float vx, vy; };  
std::vector<Particle> particles(1000);
```

Answer:

```
struct Particles {  
    std::vector<float> x, y;    // positions  
    std::vector<float> vx, vy; // velocities  
  
    Particles(int n) : x(n), y(n), vx(n), vy(n) {}  
};  
  
void update_positions(Particles& p, float dt) {  
    for (size_t i = 0; i < p.x.size(); ++i) {  
        p.x[i] += p.vx[i] * dt;  
        p.y[i] += p.vy[i] * dt;  
    }  
    // Cache friendly: reads x[], vx[] sequentially -- full cache line utilization  
    // SIMD-friendly: compiler can vectorize this loop (4 or 8 floats at once)  
}
```

Chapter 40: Data-Oriented Design

What Is Data-Oriented Design?

Object-Oriented Design (OOD) organizes code around objects and their behaviors. The primary question is “what does this thing do?”

Data-Oriented Design (DOD) organizes code around data and transformations. The primary question is “what data do I have, and how does it need to change?”

DOD produces code that is faster because it: 1. Keeps hot data contiguous (cache-friendly) 2. Separates frequently-accessed data from rarely-accessed data 3. Eliminates virtual dispatch and indirection where it is not needed 4. Enables SIMD and auto-vectorization

The game engine industry pioneered DOD because frame-rate performance is non-negotiable. The Entity-Component-System (ECS) architecture is the canonical DOD pattern.

The OOP Game Entity Problem

Classic OOP game design:

```
class Entity {
    std::string name;           // rarely accessed (debug only)
    Transform transform;        // position, rotation, scale -- hot
    Renderer renderer;          // mesh, material -- medium
    Physics physics;            // mass, velocity, collider -- hot
    AIController ai;            // behavior tree -- cold (most entities have none)
    AudioSource audio;          // sound clips -- cold
    HealthComponent health;     // HP, etc -- medium
    // ...
};
std::vector<std::unique_ptr<Entity>> entities;
```

Problems:

1. Large, fat objects: even if you only update position and velocity, you load the entire Entity into cache (including name, AI, audio...)
2. Virtual dispatch everywhere: polymorphic updates walk vtables
3. Heterogeneous data: entities with different components are stored uniformly, wasting space for components they don't have

4. Poor SIMD potential: transform data is scattered, not contiguous

Entity-Component-System (ECS)

ECS separates: - **Entities**: just IDs (integers) - **Components**: plain data (SoA or packed arrays) - **Systems**: functions that transform component data

```
// Entity is just an ID:
using EntityID = uint32_t;

// Components are plain data -- no methods, no virtual:
struct Position { float x, y; };
struct Velocity { float vx, vy; };
struct Health { int current, max; };
struct Renderable { int mesh_id, material_id; };

// Component storage: one contiguous array per component type
struct World {
    std::vector<EntityID> entities;
    std::vector<Position> positions; // dense, contiguous
    std::vector<Velocity> velocities;
    std::vector<Health> healths;
    // ...
};

// Systems: operate on arrays of components -- cache friendly
void physics_system(World& world, float dt) {
    for (size_t i = 0; i < world.positions.size(); ++i) {
        world.positions[i].x += world.velocities[i].vx * dt;
        world.positions[i].y += world.velocities[i].vy * dt;
    }
    // Reads positions[] and velocities[] -- both contiguous
    // CPU prefetcher works perfectly
}

void render_system(const World& world) {
    for (size_t i = 0; i < world.entities.size(); ++i) {
        if (has_component<Renderable>(world, i))
            draw(world.positions[i], world.renderables[i]);
    }
}
```

The physics system touches only `Position` and `Velocity`. In the OOP version, every cache line loaded also contained `name`, `AI`, and `audio` etc. In ECS, every cache line loaded is 100% useful data.

Hot/Cold Data Splitting

Even without full ECS, separating frequently-accessed “hot” data from rarely-accessed “cold” data helps:

```
// OOP (bad cache behavior for the hot loop):
struct Enemy {
    // Hot data (accessed every frame):
    float x, y;
    float hp;
    // Cold data (accessed rarely, e.g., on death):
    std::string name;
    std::string death_sound_path;
    std::vector<std::string> drop_table;
    int sprite_id;
};

// DOD hot/cold split (good cache behavior):
struct EnemyHot {    // 12 bytes -- 5+ fit per cache line
    float x, y;
    float hp;
};
struct EnemyCold {  // large -- accessed rarely
    std::string name;
    std::string death_sound_path;
    std::vector<std::string> drop_table;
    int sprite_id;
};

std::vector<EnemyHot>  enemies_hot;    // main update loop touches this
std::vector<EnemyCold> enemies_cold;  // index matches enemies_hot
```

The Cost of Indirection

Every level of indirection (pointer, virtual call, `shared_ptr`) potentially causes a cache miss:

```
// OOP with virtual + shared_ptr (3 levels of indirection):
std::vector<std::shared_ptr<Shape>> shapes;
for (auto& s : shapes) {
    s->area();    // 1: follow shared_ptr to control block
                 // 2: follow control block to object
                 // 3: follow vptr to vtable, then to function
}

// DOD: process same-type shapes together (no indirection):
std::vector<Circle>    circles;
std::vector<Rectangle> rectangles;
for (const auto& c : circles)    process_circle(c);
for (const auto& r : rectangles) process_rectangle(r);
```

Processing shapes sorted by type eliminates virtual dispatch and pointer chasing. The tradeoff: less flexibility (cannot interleave types), requires knowing types at compile time.

SIMD: Processing Multiple Values at Once

Modern CPUs can operate on 4, 8, or 16 floats simultaneously with a single instruction (SIMD: Single Instruction Multiple Data). The compiler auto-vectorizes tight loops over contiguous data:

```
// This loop is auto-vectorizable:
void add_arrays(float* a, const float* b, const float* c, int n) {
    for (int i = 0; i < n; ++i)
        a[i] = b[i] + c[i];    // 4 or 8 additions per instruction with AVX
}

// This loop is NOT auto-vectorizable (data dependency):
void prefix_sum(float* a, int n) {
    for (int i = 1; i < n; ++i)
        a[i] += a[i-1];    // a[i] depends on a[i-1] -- cannot parallelize
}
```

Enabling auto-vectorization: - Compile with `-O2` or `-O3` - Use `-march=native` to enable CPU-specific instructions - Keep loops simple: no function calls, no complex conditions, contiguous memory access - Use `std::assume_aligned` or `__builtin_assume_aligned` to help the compiler

Common Mistakes in This Chapter

Mistake 1: Premature DOD

DOD adds complexity. Apply it where profiling shows the bottleneck:

Wrong approach: restructure all code as DOD from the start

Right approach: write clear OOP code → profile → find hot path → DOD the hot path

Mistake 2: Pointer-to-Pointer Collections for “Flexibility”

The bug:

```
std::vector<std::vector<int>*> grid;    // vector of pointers to vectors
                                     // two levels of indirection, two heap allocs
```

Better: `std::vector<std::vector<int>> grid;` –or, if the inner size is fixed, `std::vector<std::array<int, N>> grid;`

Exercises

Exercise 40.1 – Hot/cold split

Given this struct used in a game simulation loop (10,000 entities, 60fps):

```
struct NPC {
    float x, y, z;           // position -- updated every frame
    float health;           // checked every frame
    std::string name;        // displayed only on hover
    std::string dialogue;    // played only on interact
    int texture_id;          // used for rendering
    bool visible;            // checked every frame
};
```

Split into hot and cold structs.

Answer:

```
struct NPCHot {              // 17 bytes (+ 1 padding = 20 total)
    float x, y, z;           // 12 bytes -- checked/modified every frame
    float health;           // 4 bytes
    bool visible;            // 1 byte
};

struct NPCCold {             // accessed rarely
    std::string name;
    std::string dialogue;
    int texture_id;
};

std::vector<NPCHot> npcs_hot; // tightly packed, cache friendly
std::vector<NPCCold> npcs_cold; // same index, loaded only when needed
```

Exercise 40.2 – Why is this slow?

Explain why this code is likely to have poor cache performance:

```
std::list<std::unique_ptr<int>> numbers;
for (int i = 0; i < 1'000'000; ++i)
    numbers.push_back(std::make_unique<int>(i));

long long sum = 0;
for (const auto& p : numbers)
    sum += *p;
```

Answer: Two levels of indirection, both cache-unfriendly: 1. `std::list` nodes are scattered across the heap – iterating the list pointer-chases through random memory addresses, causing a cache miss at each node. 2. `std::unique_ptr<int>` each point to a separately heap-allocated `int` – dereferencing `*p` causes another cache miss per element.

Fix: `std::vector<int> numbers(1'000'000); std::iota(numbers.begin(), numbers.end(), 0);` – contiguous, cache-perfect.

Chapter 41: Profiling and Flamegraphs

Measure First, Optimize Second

The cardinal rule of performance optimization:

Never guess where the bottleneck is. Measure.

Human intuition about performance is reliably wrong. The bottleneck is almost always somewhere unexpected. Optimizing the wrong thing wastes time and adds complexity for zero benefit.

The process:

1. Write correct code
2. Profile to find the actual bottleneck
3. Optimize the bottleneck
4. Measure again to confirm improvement
5. Repeat from 2

Never skip step 2.

Compiler Optimization Levels

First, make sure you are profiling optimized code:

```
# Profile build: optimized but with debug info for symbol names
g++ -std=c++23 -O2 -g -o myapp src/*.cpp

# Never profile debug builds (-O0): they are 5-50x slower than optimized
# and the bottleneck is often in the debug overhead, not your code
```

Quick Timing With `std::chrono`

For macro-benchmarks (timing large operations):

```
#include <chrono>

auto start = std::chrono::high_resolution_clock::now();

// ... code to measure ...

auto end = std::chrono::high_resolution_clock::now();
auto duration = std::chrono::duration_cast<std::chrono::microseconds>(end - start);
std::cout << "Time: " << duration.count() << " μs\n";

// Utility wrapper:
template <typename F>
auto time_it(F&& func) {
    auto start = std::chrono::high_resolution_clock::now();
    func();
    auto end = std::chrono::high_resolution_clock::now();
    return std::chrono::duration_cast<std::chrono::nanoseconds>(end - start).count();
}

auto ns = time_it([&]{ sort(v.begin(), v.end()); });
std::cout << ns << " ns\n";
```

Pitfalls: - Run the function multiple times and take the minimum (cold-start effects) - Warm up the cache before measuring (first run is slower) - Prevent dead-code elimination: use the output (e.g., `volatile` or `print` it) - Use `--benchmark` libraries for rigorous micro-benchmarks

Google Benchmark

For micro-benchmarks, use the Google Benchmark library:

```
#include <benchmark/benchmark.h>
#include <vector>
#include <algorithm>

// Benchmark sort on vector of random ints:
static void BM_VectorSort(benchmark::State& state) {
    std::vector<int> v(state.range(0));
    std::iota(v.begin(), v.end(), 0);

    for (auto _ : state) {
        std::shuffle(v.begin(), v.end(), std::mt19937{});
        std::sort(v.begin(), v.end());
        benchmark::DoNotOptimize(v); // prevent dead-code elimination
    }
}

BENCHMARK(BM_VectorSort)->Range(8, 8<<10); // test sizes 8, 64, 512, ...
BENCHMARK_MAIN();
```

Output:

BM_VectorSort/8	96 ns	96 ns	7282143
BM_VectorSort/64	853 ns	853 ns	821055
BM_VectorSort/512	9232 ns	9232 ns	75932
BM_VectorSort/8192	175839 ns	175839 ns	3989

perf – Linux System Profiler

`perf` samples the call stack at high frequency to show where CPU time is spent:

```
# Compile with debug symbols and optimization:
g++ -std=c++23 -O2 -g -o myapp main.cpp

# Profile:
perf record -g ./myapp

# View results:
perf report
```

Output (simplified):

Overhead	Command	Shared Object	Symbol
42.31%	myapp	myapp	[.] update_physics
28.17%	myapp	myapp	[.] render_scene
15.44%	myapp	libc-2.35.so	[.] malloc
8.22%	myapp	myapp	[.] process_ai

`update_physics` takes 42% of CPU time – that is where to focus.

Flamegraphs

A flamegraph is a visualization of call-stack samples. The X-axis is proportion of time spent, the Y-axis is call depth. Each “flame” is a function; wider = more time.

```
# Generate flamegraph (using Brendan Gregg's FlameGraph scripts):
perf record -F 99 -g ./myapp -- your-workload
perf script | stackcollapse-perf.pl | flamegraph.pl > profile.svg
```

Reading a flamegraph: - Wide plateau at the bottom = function that uses most CPU time - Tall narrow spikes = deep call chains (possibly recursive) - Look for unexpectedly wide functions you did not expect to be hot

valgrind --tool=callgrind – Instruction-Level Profiling

```
valgrind --tool=callgrind --callgrind-out-file=callgrind.out ./myapp  
kcachegrind callgrind.out # GUI viewer
```

callgrind counts every instruction executed and every cache miss. More accurate than sampling but 10-100x slower – use for profiling, not for production.

Address Sanitizer, UBSan, and ThreadSanitizer

These are not performance tools – they find bugs. Always run them during development before profiling performance:

```
# Find memory bugs (use-after-free, buffer overflow, leaks):  
g++ -fsanitize=address,undefined -g -o myapp main.cpp  
./myapp  
  
# Find data races:  
g++ -fsanitize=thread -g -o myapp main.cpp  
./myapp
```

ASan adds ~2x runtime overhead. TSan adds ~5-20x. Use them in CI, not in profiling.

The Optimization Checklist

Before profiling:

- [] Build with -O2 (or -O3 for float-heavy code)
- [] Run all sanitizers to ensure correctness first
- [] Make sure the benchmark reflects real workload

After profiling finds the hot path:

- [] Eliminate unnecessary allocations (avoid new in hot loops)
- [] Use contiguous data structures (vector, array over list, map)
- [] Move large objects instead of copying
- [] Reduce virtual dispatch in inner loops
- [] Separate hot/cold data (struct splitting)
- [] Enable auto-vectorization (SoA layout, simple loops)
- [] Reserve containers when size is known
- [] Avoid string construction in hot loops (use string_view)
- [] Profile again to confirm improvement

Common Mistakes in This Chapter

Mistake 1: Optimizing Before Profiling

“I think this function is slow” is almost always wrong. Profile first. Optimize what perf shows.

Mistake 2: Benchmarking Debug Builds

Benchmark results from `-O0` builds are meaningless for production performance. The bottleneck in debug mode is often the debug runtime overhead.

Mistake 3: Letting the Compiler Optimize Away the Benchmark

```
int result = heavy_computation(data);  
// If result is never used, compiler may eliminate heavy_computation entirely!  
// Benchmark shows 0 ns -- not what you wanted.
```

Use `benchmark::DoNotOptimize(result)` or `volatile` to prevent dead-code elimination.

Exercises

Exercise 41.1 – Time a sort

Using `std::chrono`, measure the time to sort a `std::vector<int>` of 1 million random elements. Run it 5 times and print each measurement.

Answer:

```
#include <chrono>  
#include <vector>  
#include <algorithm>  
#include <random>  
#include <iostream>  
  
int main() {  
    std::mt19937 rng{42};  
    for (int run = 0; run < 5; ++run) {  
        std::vector<int> v(1'000'000);  
        std::generate(v.begin(), v.end(), rng);  
  
        auto t0 = std::chrono::high_resolution_clock::now();  
        std::sort(v.begin(), v.end());  
        auto t1 = std::chrono::high_resolution_clock::now();  
  
        auto ms = std::chrono::duration_cast<std::chrono::milliseconds>(t1-t0).count();  
        std::cout << "Run " << run+1 << ": " << ms << " ms\n";  
    }  
}
```

```
}
// Typical output on modern hardware: 60-80 ms per sort of 1M ints
```

Exercise 41.2 – Compare list vs vector

Measure the time to sum 1 million integers stored in `std::vector<int>` vs `std::list<int>`. Explain the difference.

Answer:

```
// Fill both:
std::vector<int> vec(1'000'000);
std::list<int> lst(vec.begin(), vec.end());
std::iota(vec.begin(), vec.end(), 0);

// Time vector sum:
auto t0 = std::chrono::high_resolution_clock::now();
long long sum_v = std::accumulate(vec.begin(), vec.end(), 0LL);
auto t1 = std::chrono::high_resolution_clock::now();

// Time list sum:
auto t2 = std::chrono::high_resolution_clock::now();
long long sum_l = std::accumulate(lst.begin(), lst.end(), 0LL);
auto t3 = std::chrono::high_resolution_clock::now();

// Typical: vector ~0.3ms, list ~10ms (30-50x slower)
```

Explanation: Vector stores integers contiguously – the CPU prefetches entire cache lines of useful data, achieving near-peak memory bandwidth. List nodes are scattered throughout the heap – each `++iterator` dereferences a `.next` pointer that may land anywhere in RAM, causing a cache miss (~60ns each) for most of the 1 million iterations.

Part VIII is complete. You now understand the performance mental model that separates C++ that is fast from C++ that is merely correct: value semantics avoids aliasing overhead; cache-line awareness determines whether loops are fast or slow; data-oriented design keeps hot data contiguous; and profiling tells you where to actually spend optimization effort.

Part IX covers concurrency – threads, mutexes, atomics, and async patterns. Ask to continue.

Part IX – Concurrency

Concurrency is one of the hardest topics in all of programming. C++ gives you the tools to write genuinely parallel code that runs on multiple CPU cores simultaneously. The power comes with responsibility: race conditions, deadlocks, and memory ordering bugs are silent, non-deterministic, and sometimes impossible to reproduce in a debugger.

This part builds the mental model carefully – from how threads work at the hardware level, through the synchronization primitives, to the higher-level abstractions that make concurrency manageable.

Chapter 42: Threads and `std::jthread`

What Is a Thread?

A **thread** is an independent sequence of execution within a process. All threads in a process share the same address space (heap, globals, code), but each thread has its own stack and its own program counter (which instruction it is currently executing).

Process memory:

```
+-----+
| Code (.text)          -- shared by all threads
| Read-only data (.rodata) -- shared
| Global/static data   -- shared (careful!)
| Heap                 -- shared (careful!)
+-----+
| Thread 1 stack        -- private to thread 1
+-----+
| Thread 2 stack        -- private to thread 2
+-----+
```

On a multi-core CPU, threads genuinely run at the same time on different cores. On a single-core CPU, the OS time-slices them – switching rapidly to give the illusion of parallelism.

Python has threads too, but the GIL (Global Interpreter Lock) prevents true parallel execution of Python bytecode. C++ has no GIL – threads run fully in parallel.

`std::thread` – Creating a Thread

```
#include <thread>
#include <iostream>

void worker(int id) {
    std::cout << "Thread " << id << " running\n";
}

int main() {
    std::thread t1{worker, 1}; // launch thread running worker(1)
    std::thread t2{worker, 2}; // launch thread running worker(2)
}
```

```

// Main thread continues here -- three threads running simultaneously

t1.join(); // wait for t1 to finish
t2.join(); // wait for t2 to finish
// If you reach this point, both threads have completed
}

```

join() blocks the calling thread until the target thread finishes.

detach() lets the thread run independently (the `std::thread` object no longer manages it):

```

std::thread t{worker, 3};
t.detach(); // t is now "detached" -- fire and forget
// Do NOT call t.join() after detach()
// The thread continues running even after t is destroyed

```

If a `std::thread` object is destroyed without being joined or detached, `std::terminate()` is called – the program crashes. This is a common beginner mistake.

std::jthread – The Safe Thread (C++20)

`std::jthread` (“joining thread”) automatically joins in its destructor, making it impossible to forget:

```

#include <thread>

void worker(int id) {
    std::cout << "Thread " << id << "\n";
}

int main() {
    std::jthread t1{worker, 1};
    std::jthread t2{worker, 2};
    // When t1 and t2 go out of scope, they automatically join
    // No need to call t1.join() -- cannot forget
}

```

`std::jthread` also supports **cooperative cancellation** via `std::stop_token`:

```

void cancellable_worker(std::stop_token stop, int id) {
    while (!stop.stop_requested()) { // check if cancellation was requested
        std::cout << "Thread " << id << " working\n";
        std::this_thread::sleep_for(std::chrono::milliseconds{100});
    }
    std::cout << "Thread " << id << " cancelled\n";
}

std::jthread t{cancellable_worker, 1};
std::this_thread::sleep_for(std::chrono::seconds{1});
t.request_stop(); // signal the thread to stop
// t's destructor joins -- waits for the thread to actually stop

```

Prefer `std::jthread` over `std::thread` for all new code. The automatic join and stop-token support make it safer and more ergonomic.

Thread Arguments and Return Values

```
// Passing arguments: additional constructor arguments are forwarded to the function
std::jthread t{worker, 42, "hello"};

// Returning values: threads cannot return values directly
// Use a shared variable (protected by mutex -- next chapter)
// or std::future (Chapter 45)

int result = 0;
std::jthread t{[&result]() {
    result = heavy_computation(); // writes to shared result
}};
// After t joins, result is valid
std::cout << result << "\n";
```

Thread Local Storage

Each thread can have its own copy of a variable with `thread_local`:

```
thread_local int error_code = 0; // each thread has its own error_code

void worker() {
    error_code = 42; // only affects THIS thread's copy
    std::cout << error_code; // 42
}

void other_worker() {
    std::cout << error_code; // 0 -- other thread's copy is unmodified
}
```

`thread_local` variables are initialized once per thread (not once per program). Useful for per-thread caches, random number generators, and error codes.

Common Thread Patterns

Parallel For (Manual)

```

#include <thread>
#include <vector>

void parallel_fill(std::vector<int>& v, int val, int start, int end) {
    for (int i = start; i < end; ++i) v[i] = val;
}

std::vector<int> v(1'000'000);
int hardware_threads = std::thread::hardware_concurrency(); // number of CPU cores
int chunk = v.size() / hardware_threads;

std::vector<std::jthread> threads;
for (int i = 0; i < hardware_threads; ++i) {
    int start = i * chunk;
    int end = (i == hardware_threads-1) ? v.size() : start + chunk;
    threads.emplace_back(parallel_fill, std::ref(v), 0, start, end);
}
// All threads join when 'threads' vector goes out of scope (jthread RAI)

```

In practice, use `std::for_each(std::execution::par, ...)` (parallel STL, Chapter 45) or a library like TBB rather than managing threads manually.

Common Mistakes in This Chapter

Mistake 1: Destroying a Joined/Unjoined `std::thread`

The bug:

```

{
    std::thread t{worker};
    // forgot to join or detach!
}
// t destructs without join/detach -> std::terminate() -> program crash

```

The fix: Always join or detach. Or use `std::jthread` which joins automatically.

Mistake 2: Accessing Shared Data Without Synchronization

The bug:

```

int counter = 0;
std::jthread t1{[&]{ for(int i=0; i<1000; ++i) ++counter; }};
std::jthread t2{[&]{ for(int i=0; i<1000; ++i) ++counter; }};
// counter is NOT 2000 -- data race! undefined behavior

```

The fix: Use a mutex (Chapter 43) or atomic (Chapter 44).

Mistake 3: Capturing Local Variables by Reference in a Detached Thread

The bug:

```
void launch() {
    int local = 42;
    std::thread t{[&local]{ std::cout << local; }};
    t.detach();
} // local is destroyed when launch() returns
// the thread may still be running, reading destroyed memory
```

The fix: Capture by value, or pass via `shared_ptr`, or use `jthread` (which joins before the local is destroyed).

Exercises

Exercise 42.1 – Parallel sum

Divide `std::vector<int> v(1'000'000, 1)` (filled with 1s) into two halves and sum each half in a separate `jthread`. Combine the partial sums in the main thread.

Answer:

```
std::vector<int> v(1'000'000, 1);
long long sum1 = 0, sum2 = 0;

{
    std::jthread t1{[&]{
        for (int i = 0; i < 500'000; ++i) sum1 += v[i];
    }};
    std::jthread t2{[&]{
        for (int i = 500'000; i < 1'000'000; ++i) sum2 += v[i];
    }};
} // both jthreads join here

std::cout << sum1 + sum2 << "\n"; // 1000000
// Note: sum1 and sum2 are written by separate threads to separate variables
// (no sharing of the same variable) -- this is safe without a mutex
```

Exercise 42.2 – hardware_concurrency

Write a program that prints the number of hardware threads and launches that many `jthreads`, each printing its thread index.

Answer:

```
int n = std::thread::hardware_concurrency();
std::cout << "Hardware threads: " << n << "\n";

std::vector<std::jthread> threads;
for (int i = 0; i < n; ++i) {
    threads.emplace_back([i]{
        std::cout << "Thread " << i << "\n";
    });
}
// All join when threads vector is destroyed
```

Chapter 43: Mutexes, Locks, and Race Conditions

What Is a Race Condition?

A **race condition** occurs when two threads access the same memory simultaneously and at least one access is a write. The result depends on which thread “wins the race” – it is non-deterministic.

```
int counter = 0;    // shared variable

// Thread 1:           Thread 2:
++counter;           ++counter;
```

`++counter` is NOT atomic. It compiles to three machine instructions:

Thread 1:	Thread 2:
LOAD counter -> reg1	LOAD counter -> reg2
ADD reg1, 1 -> reg1	ADD reg2, 1 -> reg2
STORE reg1 -> counter	STORE reg2 -> counter

Possible interleaving:

```
T1: LOAD counter(0) -> reg1=0
T2: LOAD counter(0) -> reg2=0    <- both read 0!
T1: ADD reg1+1 -> reg1=1
T2: ADD reg2+1 -> reg2=1
T1: STORE reg1(1) -> counter=1
T2: STORE reg2(1) -> counter=1    <- overwrites T1's write!
```

Final counter = 1, expected 2

This is the lost-update problem. In C++, a data race is **undefined behavior** – anything can happen, including corrupted data, crashes, or apparently correct results that mask the bug.

Detection: Compile and run with `-fsanitize=thread` (ThreadSanitizer). It reports data races with a full report of which threads accessed which memory.

std::mutex – Mutual Exclusion

A mutex (mutual exclusion) allows only one thread to hold the lock at a time. Other threads trying to lock it will block until it is released.

```
#include <mutex>

int counter = 0;
std::mutex mtx;

void increment(int n) {
    for (int i = 0; i < n; ++i) {
        mtx.lock();      // acquire the lock (blocks if another thread holds it)
        ++counter;        // critical section: only one thread here at a time
        mtx.unlock();    // release the lock
    }
}

std::jthread t1{increment, 1000};
std::jthread t2{increment, 1000};
// After both join: counter == 2000, guaranteed
```

RAII Mutex Locking: std::lock_guard and std::unique_lock

Never call `mutex.lock()` / `mutex.unlock()` manually – if an exception is thrown between them, the mutex is never released (deadlock).

Use RAII wrappers:

```
// std::lock_guard: simplest -- locks in constructor, unlocks in destructor
void increment_safe(int n) {
    for (int i = 0; i < n; ++i) {
        std::lock_guard<std::mutex> lock{mtx}; // locks here
        ++counter;
    } // lock destructor: unlocks here, even if exception thrown
}

// std::unique_lock: like lock_guard but more flexible (can unlock early, defer, etc.)
void increment_flexible(int n) {
    for (int i = 0; i < n; ++i) {
        std::unique_lock<std::mutex> lock{mtx};
        ++counter;
        lock.unlock(); // can unlock before end of scope
        do_other_work(); // runs without holding the lock
    }
}

// C++17: class template argument deduction -- drop the <std::mutex>:
std::lock_guard lock{mtx}; // type deduced as std::lock_guard<std::mutex>
```

std::shared_mutex – Multiple Readers, One Writer

When multiple threads can safely read simultaneously but writes require exclusive access:

```
#include <shared_mutex>

class ThreadSafeCache {
    std::unordered_map<std::string, int> data;
    mutable std::shared_mutex mtx;    // mutable: can be locked in const methods

public:
    // Multiple threads can read simultaneously:
    int get(const std::string& key) const {
        std::shared_lock lock{mtx};    // shared (read) lock -- non-exclusive
        auto it = data.find(key);
        return it != data.end() ? it->second : -1;
    }

    // Only one thread can write at a time:
    void set(const std::string& key, int value) {
        std::unique_lock lock{mtx};    // exclusive (write) lock
        data[key] = value;
    }
};
```

std::shared_lock is a read lock – many can be held simultaneously. std::unique_lock is a write lock – exclusive.

Deadlocks

A **deadlock** occurs when two (or more) threads each hold a lock that the other needs:

Thread 1:	Thread 2:
lock(mutex_A)	lock(mutex_B)
... waiting for mutex_B ...	... waiting for mutex_A ...
(blocked -- will never proceed)	(blocked -- will never proceed)

```
// Classic deadlock:
std::mutex mtx_a, mtx_b;

void thread1() {
    std::lock_guard lock_a{mtx_a};
    std::this_thread::sleep_for(std::chrono::milliseconds{10}); // let thread2 run
    std::lock_guard lock_b{mtx_b}; // DEADLOCK: thread2 holds mtx_b
}

void thread2() {
    std::lock_guard lock_b{mtx_b};
    std::this_thread::sleep_for(std::chrono::milliseconds{10}); // let thread1 run
    std::lock_guard lock_a{mtx_a}; // DEADLOCK: thread1 holds mtx_a
}
```

Deadlock Prevention

Rule 1: Always acquire multiple locks in the same order.

If both threads lock `mtx_a` then `mtx_b`, no circular dependency can form.

Rule 2: Use `std::lock()` to lock multiple mutexes simultaneously.

```
// std::lock acquires all locks atomically (no deadlock possible):
void transfer(Account& from, Account& to, double amount) {
    std::unique_lock lock_a{from.mtx, std::defer_lock};
    std::unique_lock lock_b{to.mtx, std::defer_lock};
    std::lock(lock_a, lock_b); // locks both without deadlock
    from.balance -= amount;
    to.balance += amount;
}

// C++17: std::scoped_lock does this in one step:
std::scoped_lock lock{from.mtx, to.mtx}; // acquires both, no deadlock
```

Rule 3: Minimize the time you hold a lock.

Do only what is necessary inside the critical section. Never do I/O, network calls, or user interaction while holding a lock.

Condition Variables – Waiting for a Condition

A **condition variable** lets a thread wait until some condition becomes true, without busy-waiting (spinning in a loop):

```
# Python equivalent: threading.Condition
import threading
cond = threading.Condition()
```

```
#include <condition_variable>
#include <queue>

// Classic producer-consumer queue:
std::queue<int> work_queue;
std::mutex queue_mtx;
std::condition_variable cv;

// Producer thread:
void producer() {
    for (int i = 0; i < 100; ++i) {
        {
            std::lock_guard lock{queue_mtx};
            work_queue.push(i);
        }
        cv.notify_one(); // wake one waiting consumer
    }
}
```

```
// Consumer thread:
void consumer() {
    while (true) {
        std::unique_lock lock{queue_mtx};
        cv.wait(lock, []{ return !work_queue.empty(); });
        // cv.wait: atomically releases the lock and waits
        // When notified: re-acquires the lock, checks predicate
        // If predicate is true: continues; if false: waits again
        // (the predicate check handles spurious wakeups)

        int item = work_queue.front();
        work_queue.pop();
        lock.unlock(); // release before processing

        process(item);
    }
}
```

The predicate in `cv.wait(lock, predicate)` handles **spurious wakeups** – condition variables can wake up even when not notified, so always re-check the condition.

Common Mistakes in This Chapter

Mistake 1: Forgetting to Lock Before Checking a Shared Variable

The bug:

```
if (!work_queue.empty()) { // no lock -- race condition!
    std::lock_guard lock{mtx};
    int item = work_queue.front(); // queue might be empty now!
    work_queue.pop();
}
```

The fix: Lock before checking, or use a single `lock_guard` around the entire check-and-use block.

Mistake 2: Deadlock From Recursive Locking

The bug:

```
std::mutex mtx;
void foo() {
    std::lock_guard lock{mtx};
    bar(); // bar also tries to lock mtx -- deadlock!
}
void bar() {
    std::lock_guard lock{mtx}; // DEADLOCK: mtx already held by this thread
    // ...
}
```

The fix: Use `std::recursive_mutex` if you genuinely need recursive locking. Or redesign to avoid calling locked functions from locked functions.

Mistake 3: Condition Variable Without a Predicate

The bug:

```
cv.wait(lock); // no predicate -- susceptible to spurious wakeups
// May wake up when queue is still empty!
```

The fix: Always provide a predicate: `cv.wait(lock, []{ return !queue.empty(); });`

Exercises

Exercise 43.1 – Thread-safe counter

Implement a `ThreadSafeCounter` class with `increment()`, `decrement()`, and `get() const` methods. Verify that incrementing from 2 threads 1000 times each gives 2000.

Answer:

```
class ThreadSafeCounter {
    int value{0};
    mutable std::mutex mtx;
public:
    void increment() { std::lock_guard lock{mtx}; ++value; }
    void decrement() { std::lock_guard lock{mtx}; --value; }
    int get() const { std::lock_guard lock{mtx}; return value; }
};

ThreadSafeCounter c;
std::jthread t1([&]{ for(int i=0; i<1000; ++i) c.increment(); });
std::jthread t2([&]{ for(int i=0; i<1000; ++i) c.increment(); });
// After join: c.get() == 2000, always
```

Exercise 43.2 – Bounded producer-consumer queue

Implement a thread-safe queue with a maximum size. push blocks when full; pop blocks when empty. Use a mutex and condition variable.

Answer:

```
template <typename T>
class BoundedQueue {
    std::queue<T> q;
    std::mutex mtx;
    std::condition_variable not_full, not_empty;
    const int max_size;
public:
```

```
BoundedQueue(int max) : max_size{max} {}

void push(T val) {
    std::unique_lock lock{mtx};
    not_full.wait(lock, [&]{ return (int)q.size() < max_size; });
    q.push(std::move(val));
    not_empty.notify_one();
}

T pop() {
    std::unique_lock lock{mtx};
    not_empty.wait(lock, [&]{ return !q.empty(); });
    T val = std::move(q.front());
    q.pop();
    not_full.notify_one();
    return val;
}
};
```


Chapter 44: Atomics and the C++ Memory Model

When Mutexes Are Too Heavy

A mutex involves OS-level operations – locking, unlocking, and thread scheduling. For simple operations on single values (incrementing a counter, setting a flag), this is overkill. **Atoms** provide lock-free synchronization for single-variable operations.

```
#include <atomic>

std::atomic<int> counter{0};

void increment(int n) {
    for (int i = 0; i < n; ++i) {
        ++counter;           // atomic increment -- indivisible, thread-safe
        // equivalent to: counter.fetch_add(1);
    }
}

std::jthread t1{increment, 1000};
std::jthread t2{increment, 1000};
// After join: counter == 2000, always -- no mutex needed
```

`std::atomic<int>` wraps an `int` and makes its operations **atomic** – indivisible. An atomic operation either completes fully or has not started yet. There is no intermediate state visible to other threads.

What Operations Are Atomic?

```
std::atomic<int> a{0};

a.load();           // read the current value
a.store(5);         // write a new value
a.fetch_add(1);     // add 1, return OLD value
a.fetch_sub(1);     // subtract 1, return OLD value
a.fetch_and(mask);  // bitwise AND
a.fetch_or(mask);   // bitwise OR
a.fetch_xor(mask);  // bitwise XOR
++a;               // increment (like fetch_add(1))
a++;               // post-increment
--a;
a--;
```



```
// Compare-and-swap (CAS): the fundamental building block of lock-free algorithms
int expected = 5;
bool swapped = a.compare_exchange_strong(expected, 10);
// If a == expected: atomically set a = 10, return true
// If a != expected: set expected = current a, return false
```

std::atomic<bool> – Flags and Stop Signals

```
std::atomic<bool> running{true};

void server_loop() {
    while (running.load()) { // read atomically
        handle_request();
    }
    std::cout << "Server stopped\n";
}

std::jthread server{server_loop};

// From another thread or signal handler:
running.store(false); // write atomically -- server loop will see this
```

The C++ Memory Model: Why It Matters

Memory ordering is the most subtle aspect of concurrent programming. Even without data races, modern CPUs and compilers may reorder memory operations for performance. Two threads may see operations in different orders.

Consider:

```
int x = 0, y = 0; // shared variables, not atomic

// Thread 1:           // Thread 2:
x = 1;                y = 1;
int r1 = y;           int r2 = x;
```

Intuitively you might think at least one of `r1` or `r2` must be 1 (if `x=1` happened before `r2=x`, `r2` is 1; if `y=1` happened before `r1=y`, `r1` is 1). But the CPU and compiler may reorder reads and writes, producing `r1 == 0 && r2 == 0`. This seems impossible but is legal.

C++ memory orders control how atomic operations interact with memory reordering:

```
std::atomic<int> a{0};

// Relaxed: no ordering guarantees -- fastest
a.load(std::memory_order_relaxed);
a.store(1, std::memory_order_relaxed);

// Acquire-release: the most common useful pair
// acquire: no reads/writes in this thread can be reordered BEFORE this load
a.load(std::memory_order_acquire);
// release: no reads/writes in this thread can be reordered AFTER this store
a.store(1, std::memory_order_release);

// Sequential consistency (default): strongest, most intuitive, slowest
a.load(std::memory_order_seq_cst); // default
a.store(1, std::memory_order_seq_cst);
a.store(1); // same as seq_cst
```

The Acquire-Release Pattern (Publish-Subscribe)

The most important memory ordering pattern: one thread **publishes** data by writing to an atomic flag with release; another thread **subscribes** by loading the same flag with acquire.

```
std::atomic<bool> ready{false};
int data = 0; // not atomic -- protected by the flag

// Producer thread:
void producer() {
    data = 42; // (1) write data
    ready.store(true, memory_order_release); // (2) publish: "data is ready"
    // Release ensures (1) is visible before (2)
}

// Consumer thread:
void consumer() {
    while (!ready.load(memory_order_acquire)) {} // (3) spin until ready
    // Acquire ensures: once (3) sees true, (1) is also visible
    std::cout << data << "\n"; // (4) safely reads 42
}
```

The acquire-release pair creates a **happens-before** relationship: - All operations before store (release) are visible to the thread that sees the matching load (acquire) return true.

This is the fundamental pattern for lock-free data publication.

std::atomic_flag – The Simplest Atomic

std::atomic_flag is guaranteed to be lock-free (other atomics are usually lock-free but not guaranteed):

```
std::atomic_flag flag = ATOMIC_FLAG_INIT; // cleared

// Spinlock using atomic_flag:
class Spinlock {
    std::atomic_flag flag = ATOMIC_FLAG_INIT;
public:
    void lock()    { while (flag.test_and_set(memory_order_acquire)) {} }
    void unlock() { flag.clear(memory_order_release); }
};

// Spinlocks are efficient when contention is rare and lock time is very short.
// For longer critical sections, use std::mutex (the OS can put waiting threads to sleep).
```

When to Use Atomic vs Mutex

Use `std::atomic` when:

- Single variable (int, bool, pointer)
- Operations are simple (load, store, add, compare-and-swap)
- You need maximum performance with low contention
- Implementing lock-free algorithms

Use `std::mutex` when:

- Multiple variables must be updated together atomically
- Operations are complex (check-then-act patterns)
- Lock-free would require complex CAS loops
- When in doubt (mutex errors are easier to diagnose)

Common Mistakes in This Chapter

Mistake 1: Thinking Non-Atomic Operations on `std::atomic` Are Atomic

The bug:

```
std::atomic<int> a{0};
if (a == 0) a = 1; // NOT atomic! Load and store are two separate atomic ops
                  // Another thread can change a between the load and store
```

The fix: Use `compare_exchange_strong(expected, new_val)` for check-then-set.

Mistake 2: Using `memory_order_relaxed` When Order Matters

```
// WRONG: producer publishes with relaxed -- consumer may not see data before ready
ready.store(true, memory_order_relaxed);
// data may not be visible even after ready == true

// CORRECT: release ensures preceding writes are visible
ready.store(true, memory_order_release);
```

Mistake 3: Busy-Waiting With Atomics in a Long Wait

```
while (!ready.load()) {} // spinlock: burns 100% of one CPU core while waiting
```

For short waits (microseconds), spinning is fine. For long waits (milliseconds or more), use a condition variable instead – it puts the thread to sleep.

Exercises

Exercise 44.1 – Lock-free stack push

Implement a lock-free stack push using `compare_exchange_strong`. The head node pointer should be `std::atomic<Node*>`.

Answer:

```
struct Node {
    int value;
    Node* next;
};

std::atomic<Node*> head{nullptr};

void push(int val) {
    Node* new_node = new Node{val, nullptr};
    new_node->next = head.load(memory_order_relaxed);
    // CAS loop: keep trying until we successfully update head
    while (!head.compare_exchange_weak(new_node->next, new_node,
                                      memory_order_release,
                                      memory_order_relaxed)) {
        // If head changed between our load and CAS attempt,
        // compare_exchange_weak updates new_node->next to current head
        // and returns false -- we loop and try again
    }
}
```

Exercise 44.2 – Atomic statistics

Use `std::atomic<long long>` to count: total operations, total errors, and total bytes processed across multiple threads. Show that you get correct counts even under high contention.

Answer:

```
std::atomic<long long> ops{0}, errors{0}, bytes{0};

void worker(int id) {
    for (int i = 0; i < 10'000; ++i) {
        ops.fetch_add(1, memory_order_relaxed);
        if (i % 100 == 0) errors.fetch_add(1, memory_order_relaxed);
        bytes.fetch_add(512, memory_order_relaxed);
    }
}

std::vector<std::jthread> threads;
for (int i = 0; i < 4; ++i) threads.emplace_back(worker, i);
// After join:
// ops == 40000, errors == 400, bytes == 20480000
```

Chapter 45: Async, Futures, and Tasks

The Problem With Raw Threads for Result Delivery

Getting a result back from a thread is awkward with `std::thread`:

```
int result = 0;
std::jthread t{[&result]{
    result = heavy_computation();
}};
// result is available ONLY after t joins -- but when do we join?
// We might want to do other work in the meantime
```

`std::async`, `std::future`, and `std::promise` provide a higher-level abstraction: **futures** – a handle to a value that will be computed asynchronously.

`std::async` – The Easiest Async Pattern

```
#include <future>

// Launch heavy_computation asynchronously:
auto future = std::async(std::launch::async, heavy_computation, arg1, arg2);

// Do other work here -- heavy_computation runs in a separate thread
do_other_work();

// Get the result (blocks until it is ready):
int result = future.get();
std::cout << result << "\n";
```

`std::async` returns a `std::future<T>`. `future.get()` blocks until the result is available and returns it. If the function threw an exception, `get()` rethrows it.

Launch Policies

```
// std::launch::async: run in a new thread immediately
auto f1 = std::async(std::launch::async, compute);

// std::launch::deferred: run lazily when .get() or .wait() is called (NOT a new thread)
auto f2 = std::async(std::launch::deferred, compute);

// default (policy unspecified): implementation chooses -- avoid for predictability
auto f3 = std::async(compute);
```

Always use `std::launch::async` explicitly if you need parallel execution.

Parallel Async Example

```
#include <future>
#include <numeric>
#include <vector>

long long sum_range(const std::vector<int>& v, int lo, int hi) {
    return std::accumulate(v.begin()+lo, v.begin()+hi, 0LL);
}

std::vector<int> v(1'000'000, 1);

// Launch four parallel computations:
auto f1 = std::async(std::launch::async, sum_range, std::ref(v), 0, 250'000);
auto f2 = std::async(std::launch::async, sum_range, std::ref(v), 250'000, 500'000);
auto f3 = std::async(std::launch::async, sum_range, std::ref(v), 500'000, 750'000);
auto f4 = std::async(std::launch::async, sum_range, std::ref(v), 750'000, 1'000'000);

// Collect results (blocks until each is ready):
long long total = f1.get() + f2.get() + f3.get() + f4.get();
std::cout << total << "\n"; // 1000000
```

std::promise and std::future – Manual Wiring

`std::async` is the convenient wrapper. `std::promise` / `std::future` is the low-level mechanism:

```
std::promise<int> promise;
std::future<int> future = promise.get_future();

// In another thread: set the value when ready
std::jthread worker([&promise]{
    std::this_thread::sleep_for(std::chrono::seconds{1});
    promise.set_value(42); // fulfills the future
    // promise.set_exception(std::make_exception_ptr(std::runtime_error{"oops"}));
});
```

```
// In the main thread: wait for and get the value
int result = future.get(); // blocks until promise.set_value() is called
std::cout << result << "\n"; // 42
```

Use `promise/future` when you need to transfer a result from a callback, event handler, or other non-function context to a waiting thread.

`std::shared_future` – Multiple Waiters

A `std::future` can only be waited on once – calling `get()` twice is an error. `std::shared_future` allows multiple threads to wait on the same result:

```
std::shared_future<int> sf = std::async(std::launch::async, compute).share();

// Multiple threads can wait:
std::jthread t1{[sf]{ std::cout << "Thread 1 got: " << sf.get() << "\n"; }};
std::jthread t2{[sf]{ std::cout << "Thread 2 got: " << sf.get() << "\n"; }};
std::jthread t3{[sf]{ std::cout << "Thread 3 got: " << sf.get() << "\n"; }};
// All three get the same result; the computation runs only once
```

Parallel STL (C++17) – Automatic Parallelism

The simplest way to parallelize standard algorithms: add an execution policy:

```
#include <execution>
#include <algorithm>

std::vector<int> v(1'000'000);
std::iota(v.begin(), v.end(), 0);

// Sequential (default):
std::sort(v.begin(), v.end());

// Parallel (uses thread pool internally):
std::sort(std::execution::par, v.begin(), v.end());

// Parallel + vectorized (SIMD):
std::sort(std::execution::par_unseq, v.begin(), v.end());

// Almost any algorithm works:
std::for_each(std::execution::par, v.begin(), v.end(), [](int& x){ x *= 2; });
long long sum = std::reduce(std::execution::par, v.begin(), v.end(), 0LL);
```

This is the highest-level parallelism tool. For many workloads, adding `std::execution::par` is all you need.

Exception Handling Across Threads

Exceptions in async tasks are captured and rethrown on `future.get()`:

```
auto f = std::async(std::launch::async, []{
    throw std::runtime_error{"something failed"};
    return 42;
});

try {
    int result = f.get(); // rethrows the exception here
} catch (const std::runtime_error& e) {
    std::cout << "Caught: " << e.what() << "\n";
}
```

For `std::thread`, uncaught exceptions call `std::terminate`. Always wrap thread functions in `try-catch` or use `std::async`/`std::future`.

Choosing Concurrency Tools

Level of control needed:

High-level (most cases):

Parallel STL: `std::execution::par` -- one-liner parallelism for algorithms
`std::async`: simple "run this in background and get the result"

Mid-level:

`std::jthread`: full thread control with RAII cleanup
`std::future/promise`: manual value passing between threads

Low-level (for library/framework authors):

`std::mutex`, `std::condition_variable`: classic synchronization
`std::atomic`: lock-free single-value operations

High-performance (when `std::` is not enough):

Intel TBB, OpenMP: production-grade task parallelism
 GPU compute (Chapter 47+): massively parallel workloads

Common Mistakes in This Chapter

Mistake 1: Forgetting to Call `future.get()`

The bug:

```

auto f = std::async(std::launch::async, compute);
// ... forgot to call f.get() ...
// When f is destroyed, its destructor BLOCKS until the task completes!
// Effectively serializes your "parallel" code

```

The fix: Call `f.get()` explicitly when you need the result. If you do not need the result, consider `std::jthread` directly.

Mistake 2: Storing `std::future` in a Container and Not Getting

```

std::vector<std::future<int>> futures;
for (int i = 0; i < 10; ++i)
    futures.push_back(std::async(std::launch::async, compute, i));
// CAREFUL: when the vector is destroyed, each future's destructor blocks
// The futures are destroyed in order -- effectively serialized

```

The fix: Collect all futures first, then call `get()` on each. Or use `std::shared_future`.

Exercises

Exercise 45.1 – Parallel map

Given a vector of strings, use `std::async` to compute the length of each string in parallel, then collect the results.

Answer:

```

std::vector<std::string> words = {"hello", "world", "foo", "bar", "baz"};
std::vector<std::future<int>> futures;

for (const auto& w : words) {
    futures.push_back(std::async(std::launch::async,
        [&w]{ return (int)w.size(); }));
}

std::vector<int> lengths;
for (auto& f : futures) lengths.push_back(f.get());

for (int n : lengths) std::cout << n << " "; // 5 5 3 3 3

```

Exercise 45.2 – Parallel STL sum

Use `std::reduce` with `std::execution::par` to sum 10 million integers. Compare the time with the sequential version.

Answer:

```

#include <execution>
#include <numeric>
#include <vector>
#include <chrono>

std::vector<int> v(10'000'000, 1);

auto t0 = std::chrono::high_resolution_clock::now();
long long seq = std::reduce(v.begin(), v.end(), 0LL);
auto t1 = std::chrono::high_resolution_clock::now();
long long par = std::reduce(std::execution::par, v.begin(), v.end(), 0LL);
auto t2 = std::chrono::high_resolution_clock::now();

auto seq_ms = std::chrono::duration_cast<std::chrono::milliseconds>(t1-t0).count();
auto par_ms = std::chrono::duration_cast<std::chrono::milliseconds>(t2-t1).count();

std::cout << "Sequential: " << seq_ms << " ms, result=" << seq << "\n";
std::cout << "Parallel:   " << par_ms << " ms, result=" << par << "\n";
// Typical: sequential ~10ms, parallel ~3ms on a 4-core machine

```

Part IX is complete. You now understand C++ concurrency at all levels: creating and managing threads with `thread`, protecting shared state with mutexes and condition variables, lock-free synchronization with atomics and the memory model, and the high-level `async/future` interface for result passing.

Part X covers graphics and game development – the math, the GPU pipeline, OpenGL, Vulkan, and game engine architecture. Ask to continue.

Part X – Graphics and Game Development

Graphics programming sits at the intersection of math, hardware, and real-time systems. This part builds the foundation from the bottom up: the linear algebra that describes 3D space, how the GPU processes geometry and pixels, the OpenGL and Vulkan APIs that bridge CPU and GPU, and the engine architecture patterns that organize a real-time game.

Chapter 46: Math for Graphics – Vectors, Matrices, and Quaternions

Why Math Matters

Every object on screen is the result of transforming vertices through a sequence of matrix multiplications. Misunderstand the math and your objects will be in the wrong place, facing the wrong direction, or scaled incorrectly. This chapter gives you the conceptual grounding; Chapter 48 shows how it wires into OpenGL.

Vectors

A **vector** in 3D graphics has two meanings that are always clear from context:

1. **Position:** a point in space (x, y, z)
2. **Direction:** a displacement or orientation (no fixed position)

2D vector (3, 2):

Y
^
| * (3,2)
| /
| / <- vector (arrow from origin)
| /
+-----> X

A vector stores: x=3, y=2

Length (magnitude): $\sqrt{3^2 + 2^2} = \sqrt{13} \approx 3.61$

```
// In C++, implement a simple Vec3:
struct Vec3 {
    float x, y, z;

    Vec3 operator+(const Vec3& o) const { return {x+o.x, y+o.y, z+o.z}; }
    Vec3 operator-(const Vec3& o) const { return {x-o.x, y-o.y, z-o.z}; }
    Vec3 operator*(float s)          const { return {x*s,   y*s,   z*s}; }

    float dot(const Vec3& o) const { return x*o.x + y*o.y + z*o.z; }
```

```

Vec3 cross(const Vec3& o) const {
    return { y*o.z - z*o.y,
            z*o.x - x*o.z,
            x*o.y - y*o.x };
}

float length() const { return std::sqrt(dot(*this)); }
Vec3 normalize() const { float l = length(); return {x/l, y/l, z/l}; }
};

```

Dot product $a \cdot b = |a| |b| \cos(\theta)$: - Two vectors pointing the same direction: $\text{dot} > 0$ - Perpendicular: $\text{dot} == 0$ - Opposite directions: $\text{dot} < 0$ - If both are unit vectors: dot gives $\cos(\text{angle between them})$

Cross product $a \times b$ gives a vector perpendicular to both a and b . Used to compute surface normals.

Cross product right-hand rule:

Point fingers along a , curl toward $b \rightarrow$ thumb points in $a \times b$ direction

```

b
^
|
|
+----> a
(cross product points OUT of the screen, toward you)

```

Matrices and Transformations

A **4x4 matrix** (Mat4) is the workhorse of 3D graphics. Using 4D **homogeneous coordinates** (x, y, z, w) allows translation, rotation, and scaling to all be matrix multiplications.

4x4 identity matrix (does nothing):

```

| 1  0  0  0 |
| 0  1  0  0 |
| 0  0  1  0 |
| 0  0  0  1 |

```

Translation matrix (move by tx, ty, tz):

```

| 1  0  0  tx |
| 0  1  0  ty |
| 0  0  1  tz |
| 0  0  0  1  |

```

Scale matrix (scale by sx, sy, sz):

```

| sx 0  0  0 |
| 0  sy 0  0 |

```

```
| 0  0  sz 0 |
| 0  0  0  1 |
```

Key insight: to transform a point, multiply the 4x4 matrix by the column vector $[x, y, z, 1]$. For directions, use $w=0$ (translation has no effect on directions).

```
struct Vec4 { float x, y, z, w; };

struct Mat4 {
    float m[4][4]; // m[row][col]

    // Transform a point (w=1 means translation applies)
    Vec4 operator*(const Vec4& v) const {
        return {
            m[0][0]*v.x + m[0][1]*v.y + m[0][2]*v.z + m[0][3]*v.w,
            m[1][0]*v.x + m[1][1]*v.y + m[1][2]*v.z + m[1][3]*v.w,
            m[2][0]*v.x + m[2][1]*v.y + m[2][2]*v.z + m[2][3]*v.w,
            m[3][0]*v.x + m[3][1]*v.y + m[3][2]*v.z + m[3][3]*v.w,
        };
    }

    // Matrix multiplication: combine two transforms
    Mat4 operator*(const Mat4& o) const {
        Mat4 result{};
        for (int r = 0; r < 4; ++r)
            for (int c = 0; c < 4; ++c)
                for (int k = 0; k < 4; ++k)
                    result.m[r][c] += m[r][k] * o.m[k][c];
        return result;
    }
};
```

The Transform Pipeline

Every vertex in a 3D scene passes through three spaces:

Object Space World Space View Space Clip Space
 (local to model) $\rightarrow$ (placed in world) $\rightarrow$ (relative to cam) $\rightarrow$
 > (projected to 2D)

```

        Model                View                Projection
vertex -----> world -----> camera -----> NDC
        matrix                matrix                matrix
```

Combined: `clip_pos = Projection * View * Model * object_pos`

In a vertex shader:

```
gl_Position = projection * view * model * vec4(position, 1.0);
```


The **Model matrix** places an object in the world (translation + rotation + scale). The **View matrix** is the inverse of the camera's transform (moves the world around the camera). The **Projection matrix** converts 3D camera space to 2D clip space (perspective divide).

Quaternions – Rotations Without Gimbal Lock

Euler angles (pitch, yaw, roll) seem intuitive but suffer from **gimbal lock**: when two rotation axes align, you lose a degree of freedom. Quaternions avoid this.

A quaternion has four components: $q = (w, x, y, z)$ where w is the scalar part and (x, y, z) is the vector part. A unit quaternion ($|q| = 1$) represents a rotation.

Rotation of angle θ around unit axis (ax, ay, az) :

```
w = cos(θ/2)
x = ax * sin(θ/2)
y = ay * sin(θ/2)
z = az * sin(θ/2)
```

```
struct Quat {
    float w, x, y, z;

    Quat(float angle_rad, Vec3 axis) {
        axis = axis.normalize();
        float half = angle_rad / 2.f;
        w = std::cos(half);
        x = axis.x * std::sin(half);
        y = axis.y * std::sin(half);
        z = axis.z * std::sin(half);
    }

    // Combine two rotations (apply q first, then this):
    Quat operator*(const Quat& q) const {
        return { w*q.w - x*q.x - y*q.y - z*q.z,
                w*q.x + x*q.w + y*q.z - z*q.y,
                w*q.y - x*q.z + y*q.w + z*q.x,
                w*q.z + x*q.y - y*q.x + z*q.w };
    }

    // Convert quaternion to 4x4 rotation matrix for use in shaders
    Mat4 to_mat4() const;
};
```

SLERP (spherical linear interpolation) smoothly interpolates between two rotations:

```
Quat slerp(Quat a, Quat b, float t) {
    float dot = a.w*b.w + a.x*b.x + a.y*b.y + a.z*b.z;
    if (dot < 0) { b.w=-b.w; b.x=-b.x; b.y=-b.y; b.z=-b.z; dot=-dot; }
    if (dot > 0.9995f) { // nearly identical -- linear interpolate
        return { a.w + t*(b.w-a.w), a.x + t*(b.x-a.x),
                a.y + t*(b.y-a.y), a.z + t*(b.z-a.z) };
    }
```

```

    }
    float theta_0 = std::acos(dot);
    float theta = theta_0 * t;
    float sa = std::sin(theta_0), sb = std::sin(theta);
    float s0 = std::cos(theta) - dot * sb / sa;
    float s1 = sb / sa;
    return { s0*a.w + s1*b.w, s0*a.x + s1*b.x,
            s0*a.y + s1*b.y, s0*a.z + s1*b.z };
}

```

In practice, use **GLM** (OpenGL Mathematics library) rather than writing your own:

```

#include <glm/glm.hpp>
#include <glm/gtc/matrix_transform.hpp>
#include <glm/gtc/quaternion.hpp>

glm::vec3 pos{1.0f, 2.0f, 3.0f};
glm::mat4 model = glm::mat4{1.0f};           // identity
model = glm::translate(model, pos);          // move
model = glm::rotate(model, glm::radians(45.f), {0,1,0}); // rotate 45° around Y
model = glm::scale(model, {2.f, 2.f, 2.f});  // scale 2x

glm::quat q = glm::angleAxis(glm::radians(45.f), glm::vec3{0,1,0});
glm::mat4 rot_mat = glm::mat4_cast(q);

```

Common Mistakes in This Chapter

Mistake 1: Column-Major vs Row-Major Confusion

The bug: Writing matrix math assuming row-major order, but OpenGL/GLM is column-major. Your transforms are transposed.

Symptom: Objects appear at wrong positions or with wrong orientations.

Fix: GLM is column-major by default (matching OpenGL). Access elements as `m[col][row]`. Use `glm::value_ptr(matrix)` to pass to OpenGL.

Mistake 2: Normalizing Before Cross Product

The bug:

```

Vec3 normal = a.cross(b);
// normal is NOT a unit vector -- its length depends on |a| and |b|
// Using it directly in lighting calculations gives wrong results

```

Fix: `Vec3 normal = a.cross(b).normalize();`

Mistake 3: Gimbal Lock With Euler Angles

The bug: Accumulating rotations as three Euler angles. When pitch $\approx 90^\circ$, yaw and roll collapse onto the same axis.

Fix: Store rotations as quaternions. Compose them with multiplication. Convert to matrix only at render time.

Exercises

Exercise 46.1 – Look-at matrix

Implement a `look_at(eye, target, up)` function that produces a view matrix. Use GLM's implementation as reference.

Answer:

```
// Manual implementation of glm::lookAt:
Mat4 look_at(Vec3 eye, Vec3 target, Vec3 up) {
    Vec3 f = (target - eye).normalize(); // forward
    Vec3 r = f.cross(up).normalize();    // right
    Vec3 u = r.cross(f);                // true up

    Mat4 m{};
    m.m[0][0]=r.x; m.m[0][1]=r.y; m.m[0][2]=r.z; m.m[0][3]=-r.dot(eye);
    m.m[1][0]=u.x; m.m[1][1]=u.y; m.m[1][2]=u.z; m.m[1][3]=-u.dot(eye);
    m.m[2][0]=-f.x; m.m[2][1]=-f.y; m.m[2][2]=-f.z; m.m[2][3]=f.dot(eye);
    m.m[3][3]=1.f;
    return m;
}
```

Exercise 46.2 – Angle between vectors

Given two 3D vectors, compute the angle between them in degrees using the dot product.

Answer:

```
float angle_between(Vec3 a, Vec3 b) {
    float cos_theta = a.normalize().dot(b.normalize());
    cos_theta = std::clamp(cos_theta, -1.f, 1.f); // guard against float error
    return std::acos(cos_theta) * (180.f / 3.14159265f);
}
// angle_between({1,0,0}, {0,1,0}) == 90.0 degrees
```

Chapter 47: How the GPU Works

CPU vs GPU: Two Different Machines

A CPU has a small number of very powerful cores (4–64) optimized for sequential, branchy code. A GPU has thousands of smaller, simpler cores optimized for running the same operation on many data items simultaneously.

CPU:	GPU:
+-----+-----+-----+-----+	+---+---+---+---+---+---+---+---+
Core Core Core Core	SM SM SM SM SM SM SM SM
(fast)	+---+---+---+---+---+---+---+---+
+-----+-----+-----+-----+	SM SM SM SM SM SM SM SM <--
each SM has	
Large cache / branch prediction	+---+---+---+---+---+---+---+---+ 32-
128 CUDA cores	
Out-of-order execution	... 80+ Streaming
Single thread: very fast	Multiprocessors
+-----+-----+-----+-----+	+---+---+---+---+---+---+---+---+
	Very wide memory bus
	HBM: 900 GB/s bandwidth
	+-----+-----+-----+-----+

CPU: great for 1 task very fast GPU: great for 10,000 tasks simultaneously

The Rendering Pipeline

When you draw a triangle, it travels through a fixed sequence of stages on the GPU:

CPU sends commands and data to GPU over PCIe bus

Vertex Buffer (in GPU VRAM):
position (x,y,z), color, texcoord, normal for each vertex

Pipeline stages:

1. VERTEX SHADER (programmable)

- Input: one vertex (position, attributes)
 Output: clip-space position, any other per-vertex data
 Runs: once per vertex (in parallel for all vertices)
2. PRIMITIVE ASSEMBLY + RASTERIZATION (fixed function)
 Assembles vertices into triangles
 Determines which screen pixels each triangle covers
 Interpolates vertex outputs across the triangle surface
3. FRAGMENT SHADER / PIXEL SHADER (programmable)
 Input: one fragment (screen pixel covered by a triangle)
 + interpolated values from vertex shader
 Output: color (RGBA) for that pixel
 Runs: once per fragment (in parallel for all fragments)
4. OUTPUT MERGER (fixed function)
 Depth test: is this fragment in front of what's already there?
 Blending: transparency / compositing
 Writes final color to the framebuffer

Vertex shader and **fragment shader** are the two stages you write when programming with OpenGL or Vulkan. They run directly on the GPU, written in GLSL (OpenGL) or SPIR-V/GLSL/HLSL (Vulkan).

GLSL Shader Basics

Shaders are written in GLSL (GL Shading Language), which looks like simplified C:

```
// Vertex shader (runs once per vertex):
#version 330 core

layout (location = 0) in vec3 aPosition;    // input from vertex buffer
layout (location = 1) in vec2 aTexCoord;    // texture coordinates

uniform mat4 uModel;                       // set from CPU -- same for all vertices in a draw call
uniform mat4 uView;
uniform mat4 uProjection;

out vec2 vTexCoord;                        // passed to fragment shader (interpolated)

void main() {
    gl_Position = uProjection * uView * uModel * vec4(aPosition, 1.0);
    vTexCoord = aTexCoord;
}
```

```
// Fragment shader (runs once per pixel covered by a triangle):
#version 330 core

in vec2 vTexCoord;
```

```

out vec4 FragColor;           // output: RGBA color for this pixel

uniform sampler2D uTexture;

void main() {
    FragColor = texture(uTexture, vTexCoord);
}

```

GPU Memory: Where Things Live

GPU VRAM (fast, on-chip or HBM):

Framebuffer: color/depth	<- what gets displayed
Vertex Buffers (VBO)	<- geometry data uploaded from CPU
Index Buffers (EBO)	<- triangle connectivity
Textures	<- image data
Uniform Buffers (UBO)	<- small, frequently-changing data (MVP matrices)

CPU RAM (slow path for GPU):

Data must be explicitly transferred CPU -> GPU via DMA

In OpenGL: `glBufferData`, `glTexImage2D`

In Vulkan: staging buffers + `vkCmdCopyBuffer`

The CPU-GPU Handoff

The GPU is a separate processor. The CPU sends **commands** to a **command queue**; the GPU executes them asynchronously. This is the fundamental architecture of modern graphics APIs.

CPU thread:

submit command A	--->	queue	--->	execute A
submit command B	--->		--->	execute B (may overlap with A)
submit command C	--->		--->	execute C

^

GPU pulls commands from queue at its own pace

CPU and GPU run in parallel.

CPU should never block waiting for GPU except at:

- `vkQueueWaitIdle` / `glFinish` (explicit sync)
- End of frame (present swap chain image)

Common Mistakes in This Chapter

Mistake 1: Uploading Data to GPU Every Frame

The bug: Calling `glBufferData` with the same static mesh data every frame. This stalls the GPU pipeline.

Fix: Upload static geometry once at load time. Use `glBufferSubData` or persistent mapped buffers for dynamic data.

Mistake 2: Reading Back Data From GPU to CPU

The bug:

```
glReadPixels(0, 0, width, height, GL_RGBA, GL_UNSIGNED_BYTE, buffer);  
// Forces GPU to finish all pending commands before reading -- stall
```

Fix: Use asynchronous pixel buffer objects (PBOs) to read back without stalling. Or avoid readback entirely by doing computation entirely on the GPU.

Exercises

Exercise 47.1 – Pipeline stage identification

For each operation, identify whether it runs in the vertex shader, fragment shader, or is a fixed-function step:

- a) Clipping triangles to the view frustum
- b) Computing a surface normal from height map data
- c) Sampling a texture for a surface color
- d) Assembling three vertex outputs into a triangle

Answer: a) Fixed-function (after vertex shader), b) Vertex shader, c) Fragment shader, d) Fixed-function (primitive assembly).

Chapter 48: OpenGL Fundamentals

What Is OpenGL?

OpenGL is a cross-platform API for telling the GPU to draw things. It is a state machine: you set state (which shader to use, which vertex buffer is bound, which texture is active), then issue draw calls. The GPU executes those draws using the current state.

OpenGL is older and simpler than Vulkan (Chapter 49). Start here.

Setting Up: GLFW + GLAD

```
// Dependencies:
//   GLFW: creates a window and OpenGL context
//   GLAD: loads OpenGL function pointers (they are not linked at compile time)
// Install: apt install libglfw3-dev or vcpkg install glfw3 glad

#include <glad/glad.h>           // must include before GLFW
#include <GLFW/glfw3.h>
#include <iostream>

int main() {
    glfwInit();
    glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
    glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
    glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);

    GLFWwindow* window = glfwCreateWindow(800, 600, "OpenGL", nullptr, nullptr);
    if (!window) { std::cerr << "Failed to create window\n"; return 1; }
    glfwMakeContextCurrent(window);

    if (!gladLoadGLLoader((GLADloadproc)glfwGetProcAddress)) {
        std::cerr << "Failed to load OpenGL\n"; return 1;
    }
    glViewport(0, 0, 800, 600);

    while (!glfwWindowShouldClose(window)) {
        glfwPollEvents();
        glClearColor(0.1f, 0.1f, 0.2f, 1.0f);
        glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
        // Draw here
        glfwSwapBuffers(window);    // swap front/back buffer
    }
}
```



```

    glfwTerminate();
}

```

Drawing a Triangle: The Full Pipeline

```

// 1. Vertex data: positions for three corners of a triangle
float vertices[] = {
    // x      y      z
    -0.5f, -0.5f,  0.0f, // bottom left
    0.5f, -0.5f,  0.0f, // bottom right
    0.0f,  0.5f,  0.0f, // top center
};

// 2. Create and bind a Vertex Array Object (VAO) -- records the following setup
unsigned int VAO, VBO;
glGenVertexArrays(1, &VAO);
glGenBuffers(1, &VBO);
glBindVertexArray(VAO);

// 3. Upload vertex data to GPU
glBindBuffer(GL_ARRAY_BUFFER, VBO);
glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);

// 4. Tell OpenGL how vertex data is laid out
//     attribute 0 = position, 3 floats, not normalized, stride 12 bytes, offset 0
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float), (void*)0);
glEnableVertexAttribArray(0);

glBindVertexArray(0); // unbind -- setup is recorded in VAO

// 5. Compile shaders
const char* vert_src = R"(
#version 330 core
layout(location = 0) in vec3 aPos;
void main() { gl_Position = vec4(aPos, 1.0); }
)";
const char* frag_src = R"(
#version 330 core
out vec4 FragColor;
void main() { FragColor = vec4(1.0, 0.5, 0.2, 1.0); } // orange
)";

auto compile_shader = [](const char* src, GLenum type) {
    unsigned int shader = glCreateShader(type);
    glShaderSource(shader, 1, &src, nullptr);
    glCompileShader(shader);
    int ok; glGetShaderiv(shader, GL_COMPILE_STATUS, &ok);
    if (!ok) {
        char log[512]; glGetShaderInfoLog(shader, 512, nullptr, log);
        std::cerr << "Shader error: " << log << "\n";
    }
    return shader;
};

```

```

unsigned int vert = compile_shader(vert_src, GL_VERTEX_SHADER);
unsigned int frag = compile_shader(frag_src, GL_FRAGMENT_SHADER);
unsigned int prog = glCreateProgram();
glAttachShader(prog, vert);
glAttachShader(prog, frag);
glLinkProgram(prog);
glDeleteShader(vert);
glDeleteShader(frag);

// 6. Draw loop
while (!glfwWindowShouldClose(window)) {
    glfwPollEvents();
    glClearColor(0.1f, 0.1f, 0.2f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);

    glUseProgram(prog);
    glBindVertexArray(VAO);
    glDrawArrays(GL_TRIANGLES, 0, 3);    // draw 3 vertices as 1 triangle

    glfwSwapBuffers(window);
}

```

Textures

```

// Load image data (using stb_image.h -- single header library):
#define STB_IMAGE_IMPLEMENTATION
#include "stb_image.h"

int w, h, channels;
unsigned char* data = stbi_load("texture.png", &w, &h, &channels, 0);

unsigned int texture;
glGenTextures(1, &texture);
glBindTexture(GL_TEXTURE_2D, texture);

// Filtering: how to sample when the texture is magnified/minified
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR_MIPMAP_LINEAR);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
// Wrapping: what happens at texture edges
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);

// Upload to GPU:
GLenum fmt = (channels == 4) ? GL_RGBA : GL_RGB;
glTexImage2D(GL_TEXTURE_2D, 0, fmt, w, h, 0, fmt, GL_UNSIGNED_BYTE, data);
glGenerateMipmap(GL_TEXTURE_2D);    // generate lower-res versions automatically

stbi_image_free(data);

// In the draw loop:
glActiveTexture(GL_TEXTURE0);

```

```
glBindTexture(GL_TEXTURE_2D, texture);
glUniform1i(glGetUniformLocation(prog, "uTexture"), 0); // texture unit 0
```

Uniforms: Sending Data to Shaders

Uniforms are per-draw-call values set from the CPU:

```
#include <glm/gtc/type_ptr.hpp>

// In vertex shader: uniform mat4 uModel; uniform mat4 uView; uniform mat4 uProjection;

glm::mat4 model = glm::translate(glm::mat4{1.f}, {0.f, 0.f, -3.f});
glm::mat4 view  = glm::lookAt({0,0,5}, {0,0,0}, {0,1,0});
glm::mat4 proj  = glm::perspective(glm::radians(45.f), 800.f/600.f, 0.1f, 100.f);

glUseProgram(prog);
glUniformMatrix4fv(glGetUniformLocation(prog, "uModel"), 1, GL_FALSE,
    ↪ glm::value_ptr(model));
glUniformMatrix4fv(glGetUniformLocation(prog, "uView"), 1, GL_FALSE,
    ↪ glm::value_ptr(view));
glUniformMatrix4fv(glGetUniformLocation(prog, "uProjection"), 1, GL_FALSE,
    ↪ glm::value_ptr(proj));
```

Depth Testing

Without depth testing, objects drawn later overwrite objects drawn earlier regardless of depth:

```
glEnable(GL_DEPTH_TEST); // enable depth testing (compare new fragment Z against depth
    ↪ buffer)

// In the clear call, also clear the depth buffer:
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
```

Common Mistakes in This Chapter

Mistake 1: Wrong Attribute Pointer Stride

The bug:

```
// Vertex data: position (3 floats) + texcoord (2 floats) = 5 floats total
// Stride should be 5 * sizeof(float), but:
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float), (void*)0);
// Wrong stride! GPU reads wrong data for the second vertex onward
```

Fix: Stride is the total size of one vertex: `5 * sizeof(float)`.

Mistake 2: Not Binding VAO Before Drawing

The bug:

```
glDrawArrays(GL_TRIANGLES, 0, 3); // draws nothing or wrong data -- VAO not bound
```

Fix: Always `glBindVertexArray(VAO)` before each draw call.

Mistake 3: Forgetting to Enable Depth Test

Symptom: Objects drawn later appear in front of objects drawn earlier even when they are behind them.

Fix: `glEnable(GL_DEPTH_TEST)` at startup; `glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)` each frame.

Exercises

Exercise 48.1 – Spinning cube

Extend the triangle example to draw a cube (8 vertices, 12 triangles using an index buffer). Apply a model matrix that rotates around Y axis using `glfwGetTime()`.

Answer (key parts):

```
// Index buffer for 12 triangles (36 indices):
unsigned int indices[] = {
    0,1,2, 2,3,0, // front face
    4,5,6, 6,7,4, // back face
    0,4,5, 5,1,0, // left
    3,7,6, 6,2,3, // right
    0,3,7, 7,4,0, // top
    1,2,6, 6,5,1  // bottom
};
unsigned int EBO;
glGenBuffers(1, &EBO);
glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, EBO);
glBufferData(GL_ELEMENT_ARRAY_BUFFER, sizeof(indices), indices, GL_STATIC_DRAW);

// In draw loop:
float angle = (float)glfwGetTime();
glm::mat4 model = glm::rotate(glm::mat4{1.f}, angle, {0.f,1.f,0.f});
glUniformMatrix4fv(glGetUniformLocation(prog,"uModel"),1,GL_FALSE,glm::value_ptr(model));
glDrawElements(GL_TRIANGLES, 36, GL_UNSIGNED_INT, 0);
```


Chapter 49: Vulkan – Explicit GPU Control

Why Vulkan Exists

OpenGL hides complexity: the driver validates your calls, manages synchronization, and compiles shaders at runtime. This has a cost – the driver does work you may not need, and validation overhead can be significant.

Vulkan is explicit: you do everything yourself. More code, but: - Near-zero driver overhead - Explicit control over GPU memory allocation - Multi-threaded command recording - Cross-platform (Windows, Linux, Android, macOS via MoltenVK)

A minimal Vulkan “hello triangle” is ~1000 lines versus ~100 for OpenGL. In a production engine, that complexity pays off. For a first project, OpenGL is fine.

Core Vulkan Concepts

```
VkInstance          -- connection to the Vulkan runtime
VkPhysicalDevice    -- the GPU hardware (can enumerate multiple GPUs)
VkDevice            -- logical device: your connection to a physical device
VkQueue             -- submission queue for commands (graphics/compute/transfer)
VkSwapchainKHR      -- sequence of images to display (triple-buffering etc.)
VkRenderPass        -- describes framebuffer attachments and their usage
VkPipeline           -- compiled shader programs + all fixed-function state
VkCommandBuffer     -- recorded sequence of GPU commands
VkFence/VkSemaphore -- CPU-GPU and GPU-GPU synchronization primitives
```

Vulkan Initialization Sequence

1. Create `VkInstance` (with validation layers in debug builds)
2. Create `VkSurfaceKHR` (platform-specific window surface, via GLFW)
3. Pick `VkPhysicalDevice` (GPU with required queue families + features)
4. Create `VkDevice` + `VkQueues` (graphics queue, present queue)
5. Create `VkSwapchainKHR` (images to render into and display)
6. Create `VkImageViews` for swapchain images
7. Create `VkRenderPass` (what attachments, their load/store ops)

8. Create `VkFramebuffers` (one per swapchain image)
9. Create `VkPipelineLayout` + `VkGraphicsPipeline`
 - Vertex input state
 - Vertex shader (SPIR-V binary)
 - Rasterizer state
 - Fragment shader (SPIR-V binary)
 - Color blending state
 - Depth/stencil state
10. Create `VkCommandPool` + `VkCommandBuffers` (one per frame in flight)
11. Create `VkSemaphores` + `VkFences` (for frame synchronization)

The Vulkan Render Loop

```
// Pseudocode for one frame:
void draw_frame() {
    // 1. Wait for GPU to finish with the previous frame using this frame's slot
    vkWaitForFences(device, 1, &in_flight_fence[current_frame], VK_TRUE, UINT64_MAX);
    vkResetFences(device, 1, &in_flight_fence[current_frame]);

    // 2. Acquire next swapchain image
    uint32_t image_index;
    vkAcquireNextImageKHR(device, swapchain, UINT64_MAX,
                          image_available_sem[current_frame], VK_NULL_HANDLE,
                          &image_index);

    // 3. Record commands into command buffer
    vkResetCommandBuffer(cmd_buf[current_frame], 0);
    record_commands(cmd_buf[current_frame], image_index);

    // 4. Submit command buffer to queue
    VkSubmitInfo submit{VK_STRUCTURE_TYPE_SUBMIT_INFO};
    submit.waitSemaphoreCount = 1;
    submit.pWaitSemaphores = &image_available_sem[current_frame]; // wait for image
    VkPipelineStageFlags wait_stages = VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT;
    submit.pWaitDstStageMask = &wait_stages;
    submit.commandBufferCount = 1;
    submit.pCommandBuffers = &cmd_buf[current_frame];
    submit.signalSemaphoreCount = 1;
    submit.pSignalSemaphores = &render_finished_sem[current_frame]; // signal when done

    vkQueueSubmit(graphics_queue, 1, &submit, in_flight_fence[current_frame]);

    // 5. Present the rendered image
    VkPresentInfoKHR present{VK_STRUCTURE_TYPE_PRESENT_INFO_KHR};
    present.waitSemaphoreCount = 1;
    present.pWaitSemaphores = &render_finished_sem[current_frame]; // wait for render
    present.swapchainCount = 1;
    present.pSwapchains = &swapchain;
    present.pImageIndices = &image_index;
    vkQueuePresentKHR(present_queue, &present);
}
```

```
current_frame = (current_frame + 1) % MAX_FRAMES_IN_FLIGHT;
}
```

Vulkan Memory Management

In OpenGL, the driver allocates GPU memory for you. In Vulkan, you are responsible:

```
// Allocate a buffer (e.g., for vertex data):
VkBufferCreateInfo buf_info{VK_STRUCTURE_TYPE_BUFFER_CREATE_INFO};
buf_info.size = sizeof(vertices);
buf_info.usage = VK_BUFFER_USAGE_VERTEX_BUFFER_BIT;
VkBuffer vertex_buffer;
vkCreateBuffer(device, &buf_info, nullptr, &vertex_buffer);

// Query memory requirements:
VkMemoryRequirements mem_req;
vkGetBufferMemoryRequirements(device, vertex_buffer, &mem_req);

// Find suitable memory type (device-local for GPU-only, host-visible for CPU-writeable):
VkMemoryAllocateInfo alloc{VK_STRUCTURE_TYPE_MEMORY_ALLOCATE_INFO};
alloc.allocationSize = mem_req.size;
alloc.memoryTypeIndex = find_memory_type(mem_req.memoryTypeBits,
    VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT | VK_MEMORY_PROPERTY_HOST_COHERENT_BIT);

VkDeviceMemory buffer_memory;
vkAllocateMemory(device, &alloc, nullptr, &buffer_memory);
vkBindBufferMemory(device, vertex_buffer, buffer_memory, 0);

// Write data via memory mapping (only possible with HOST_VISIBLE memory):
void* mapped;
vkMapMemory(device, buffer_memory, 0, sizeof(vertices), 0, &mapped);
memcpy(mapped, vertices, sizeof(vertices));
vkUnmapMemory(device, buffer_memory);
```

In practice, use the **Vulkan Memory Allocator (VMA)** library which handles memory type selection and suballocation automatically.

OpenGL vs Vulkan: When to Use Each

Use OpenGL when:

- Learning graphics programming for the first time
- Rapid prototyping
- Tool development (not performance critical)
- Target audience: older hardware / drivers

Use Vulkan when:

- Production games or engines

- Need fine-grained control over synchronization
- Multi-threaded command recording (a big OpenGL weakness)
- Cross-platform with best performance
- Modern hardware targets

Common Mistakes in This Chapter

Mistake 1: Not Enabling Validation Layers in Debug

The bug: Writing Vulkan without validation layers and getting a black screen with no error message.

Fix: Always enable `VK_LAYER_KHRONOS_validation` in debug builds. It catches almost every API misuse with a clear error message. Disable only in final release builds.

Mistake 2: Wrong Semaphore Usage

The bug: Submitting a command buffer that signals a semaphore, then waiting on that semaphore in the same `vkQueueSubmit` call.

Symptom: GPU deadlock or validation error.

Fix: Signal semaphores flow from one submit to the next submit (or to present). A submit cannot wait on a semaphore it also signals.

Exercises

Exercise 49.1 – Conceptual: frame-in-flight

Why do production Vulkan engines use 2 or 3 “frames in flight” rather than 1? What would go wrong with only 1?

Answer: With 1 frame in flight, the CPU must wait for the GPU to finish the previous frame before recording the next one. This stalls the CPU. With 2-3 frames in flight, the CPU records frame $N+1$ while the GPU renders frame N , keeping both fully busy. The cost is that GPU memory for framebuffers, command buffers, and semaphores is duplicated per frame.

Chapter 50: Game Loop, ECS, and Engine Design

The Game Loop

Every game runs a loop: read input, update world state, render. The precise structure of this loop determines whether your game runs consistently across different hardware.

Naive loop (DO NOT USE):

```
while (running) {
    handle_input();
    update();      // moves objects by fixed amount
    render();
}
```

Problem: on a fast machine (200 fps), objects move 200 times/second
on a slow machine (30 fps), they move 30 times/second
→ game plays at different speeds on different hardware

Fixed Timestep Loop (The Standard Solution)

```
const double FIXED_DT = 1.0 / 60.0; // 60 physics updates per second, always

double accumulator = 0.0;
double previous_time = glfwGetTime();

while (!glfwWindowShouldClose(window)) {
    double current_time = glfwGetTime();
    double frame_time = current_time - previous_time;
    previous_time = current_time;

    frame_time = std::min(frame_time, 0.25); // spiral-of-death guard
    accumulator += frame_time;

    handle_input();

    while (accumulator >= FIXED_DT) {
        update(FIXED_DT); // physics/game logic at fixed 60 Hz
        accumulator -= FIXED_DT;
    }

    double alpha = accumulator / FIXED_DT; // 0..1: interpolation factor
```

```

    render(alpha); // interpolate between previous and current state for smoothness
    glfwSwapBuffers(window);
    glfwPollEvents();
}

```

Timeline diagram:

```

Physics:  |---P---|---P---|---P---|---P---|
Render:   |-----R-----|-----R-----|

```

P = physics update at fixed 60 Hz

R = render at whatever rate the GPU supports (could be 144 Hz)

Between physics steps, render interpolates positions for smooth motion.

The “spiral of death guard” (`frame_time = std::min(frame_time, 0.25)`) prevents the update loop from running forever when the game falls far behind (e.g., after a freeze).

Entity-Component System (ECS)

OOP designs games as object hierarchies: `Enemy` extends `Character` extends `Actor`. This causes fragmentation: each `Enemy` is a separate heap allocation, accessed through a vtable, with its fields scattered across memory.

ECS separates **identity** (entity), **data** (component), and **behavior** (system):

Entity: just an ID (integer)

Component: plain data, no methods, stored in contiguous arrays

System: functions that operate on entities with specific components

Example:

Entity 1: has Position, Velocity, Sprite

Entity 2: has Position, Velocity, Health

Entity 3: has Position, Sprite (static decoration -- no Velocity)

Physics System: iterates ALL entities with (Position, Velocity)

-> processes Entity 1 and Entity 2 only

-> data is contiguous in memory: cache-friendly

Render System: iterates ALL entities with (Position, Sprite)

-> processes Entity 1 and Entity 3

Simple ECS Implementation

```
using EntityID = uint32_t;

// Component storage: one array per component type
struct Position { float x, y, z; };
struct Velocity { float vx, vy, vz; };
struct Health { int hp, max_hp; };

struct World {
    std::vector<Position> positions; // indexed by EntityID
    std::vector<Velocity> velocities;
    std::vector<Health> healths;
    std::vector<uint32_t> component_mask; // bitfield of which components an entity has

    enum Components : uint32_t {
        HAS_POSITION = 1 << 0,
        HAS_VELOCITY = 1 << 1,
        HAS_HEALTH = 1 << 2,
    };

    EntityID create_entity() {
        EntityID id = positions.size();
        positions.emplace_back();
        velocities.emplace_back();
        healths.emplace_back();
        component_mask.push_back(0);
        return id;
    }

    void add_position(EntityID e, Position p) {
        positions[e] = p;
        component_mask[e] |= HAS_POSITION;
    }

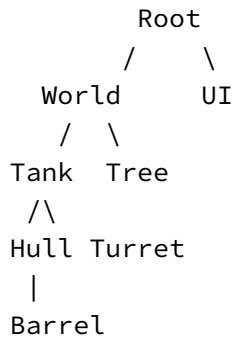
    void add_velocity(EntityID e, Velocity v) {
        velocities[e] = v;
        component_mask[e] |= HAS_VELOCITY;
    }
};

// Physics system: only processes entities with Position AND Velocity
void physics_system(World& w, float dt) {
    for (EntityID e = 0; e < w.positions.size(); ++e) {
        if ((w.component_mask[e] & (World::HAS_POSITION | World::HAS_VELOCITY))
            == (World::HAS_POSITION | World::HAS_VELOCITY)) {
            w.positions[e].x += w.velocities[e].vx * dt;
            w.positions[e].y += w.velocities[e].vy * dt;
            w.positions[e].z += w.velocities[e].vz * dt;
        }
    }
}
```

Production ECS libraries (EnTT, flecs, DOTS) handle this more efficiently with archetype-based storage and sparse sets.

Scene Graph vs ECS

Scene Graph (traditional OOP):



Good for: hierarchical transforms (barrel rotates with turret, turret with hull)

Bad for: performance (random access, pointer chasing, vtable dispatch)

ECS:

Components stored in flat arrays:

positions: [p0, p1, p2, p3, p4, ...]

velocities: [v0, ---, v2, ---, v4, ...] (--- = entity doesn't have this)

Good for: performance (cache-friendly, SIMD-friendly)

Bad for: hierarchical relationships (need explicit parent/child component)

In practice, most engines combine both: ECS for gameplay objects, scene graph for hierarchical transforms.

Resource Management in a Game Engine

```

// Resource handle system: thin ID wrapper, assets loaded asynchronously
using TextureHandle = uint32_t;
using MeshHandle    = uint32_t;

class ResourceManager {
    std::unordered_map<std::string, TextureHandle> texture_cache;
    std::vector<Texture> textures;

public:
    TextureHandle load_texture(const std::string& path) {
        auto it = texture_cache.find(path);
        if (it != texture_cache.end()) return it->second; // cached

        TextureHandle h = textures.size();
        textures.push_back(Texture::from_file(path));
    }
};
  
```

```
        texture_cache[path] = h;  
        return h;  
    }  
  
    const Texture& get(TextureHandle h) const { return textures[h]; }  
};
```

This pattern (handle = index into dense array) avoids dangling pointers, is cache-friendly, and makes serialization trivial.

Common Mistakes in This Chapter

Mistake 1: Variable Timestep for Physics

The bug:

```
update(frame_time); // passes variable delta time directly to physics
```

Symptom: Physics behaves differently at 30 FPS vs 120 FPS. Objects pass through walls at low frame rates.

Fix: Fixed timestep loop as shown above.

Mistake 2: One Component = One Allocation

The bug:

```
struct Enemy {  
    std::unique_ptr<Position> pos; // heap allocation per enemy  
    std::unique_ptr<Health> hp; // heap allocation per enemy  
};
```

Symptom: Cache misses on every component access – poor performance with many entities.

Fix: Store all `Position` components together in a `std::vector<Position>`, all `Health` together in `std::vector<Health>`. Index by `EntityID`.

Exercises

Exercise 50.1 – Frame time cap

In the fixed timestep loop, explain what the “spiral of death” is and why `frame_time = std::min(frame_time, 0.25)` prevents it.

Answer: The spiral of death occurs when `update()` takes longer than `FIXED_DT`. Each iteration of the outer loop adds more to the accumulator than `update` can drain. The next frame has an even larger accumulator, requiring even more updates, which take even longer, until the game freezes. Capping `frame_time` at 0.25 seconds limits how much work one frame can trigger – the game slows down visually (time dilation) but doesn’t freeze.

Exercise 50.2 – Simple ECS

Add a `Render` component (sprite ID) to the `World` class above. Write a `render_system` that iterates all entities with both `Position` and `Render` and prints “drawing entity E at (x, y, z)”.

Answer:

```
struct RenderComp { uint32_t sprite_id; };

struct World {
    // ... existing fields ...
    std::vector<RenderComp> renders;
    enum Components : uint32_t {
        HAS_POSITION = 1 << 0,
        HAS_VELOCITY = 1 << 1,
        HAS_HEALTH   = 1 << 2,
        HAS_RENDER    = 1 << 3,
    };

    void add_render(EntityID e, RenderComp r) {
        renders[e] = r;
        component_mask[e] |= HAS_RENDER;
    }
};

void render_system(const World& w) {
    for (EntityID e = 0; e < w.positions.size(); ++e) {
        if ((w.component_mask[e] & (World::HAS_POSITION | World::HAS_RENDER))
            == (World::HAS_POSITION | World::HAS_RENDER)) {
            auto& p = w.positions[e];
            std::cout << "Drawing entity " << e
                      << " at (" << p.x << ", " << p.y << ", " << p.z << ")\\n";
        }
    }
}
```

Part X is complete. You now understand the full graphics stack: the math that describes 3D space, how the GPU pipeline processes geometry into pixels, how OpenGL gives you a drawable window with a simple API, how Vulkan gives you explicit control at the cost of more code, and how game engines organize their runtime logic with fixed timestep loops and ECS.

Part XI covers systems programming – the machine level: registers, memory, syscalls, Linux APIs, networking from the ground up, and where C++ meets C, eBPF, and Go. Ask to continue.

Part XI – Systems Programming

Systems programming is writing code that talks directly to the operating system, the hardware, and other languages. This is where C++ earns its reputation: you can write code that controls exactly how memory is laid out, how system calls are invoked, and how bytes move across a network socket. Python runs on top of an interpreter that hides all of this. C++ can reach all the way down.

Chapter 51: The Machine – Registers, Memory, and Syscalls

The Execution Model: What the CPU Actually Does

A CPU executes one instruction at a time (ignoring pipelining and out-of-order execution for now). Each instruction reads from and writes to **registers** – small, extremely fast storage inside the CPU itself.

x86-64 general-purpose registers (64-bit):

- RAX -- accumulator: return values, arithmetic
- RBX -- base: general-purpose, callee-saved
- RCX -- counter: loop counts, system call 4th arg
- RDX -- data: I/O, system call 3rd arg
- RSI -- source index: string ops, system call 2nd arg
- RDI -- destination index: string ops, system call 1st arg
- RSP -- stack pointer: top of current stack frame
- RBP -- base pointer: bottom of current stack frame
- R8-R15 -- additional general-purpose

Special registers:

- RIP -- instruction pointer: address of next instruction
- RFLAGS -- condition flags: zero, sign, overflow, carry

The stack pointer RSP points to the top of the call stack. When you call a function, RSP decreases (stack grows downward); when you return, RSP increases.

From C++ to Machine Code

```
int add(int a, int b) {  
    return a + b;  
}
```

Compiles (roughly) to:

```

; System V AMD64 ABI (Linux/macOS calling convention):
; First argument in EDI (lower 32 bits of RDI)
; Second argument in ESI (lower 32 bits of RSI)
; Return value in EAX (lower 32 bits of RAX)

add(int, int):
    lea    eax, [rdi + rsi]    ; eax = edi + esi
    ret                                ; return (RAX holds result)

```

You can see your code's assembly with:

```

g++ -O2 -S -masm=intel program.cpp -o program.s    # emit assembly
# Or online: gcc.godbolt.org (Compiler Explorer)

```

Why does this matter? Understanding what the compiler generates helps you write code that compiles to fewer, faster instructions. It also helps you understand pointer arithmetic, struct layout, and why certain patterns are slow.

Virtual Memory: How the OS Lies About Memory

Every process thinks it has the entire 64-bit address space (16 exabytes) to itself. This is a lie – a useful one called **virtual memory**.

Process virtual address space (64-bit Linux, simplified):

```

0xFFFFFFFFFFFFFFFF ----
                                Kernel space (not accessible from userspace)
0xFFFF800000000000 ----
                                ...
                                (non-canonical, inaccessible)
0x00007FFFFFFFFFFF ----
                                Stack (grows downward)
                                ...
                                Memory-mapped files, shared libs
                                ...
                                Heap (grows upward)
                                BSS segment (zero-initialized globals)
                                Data segment (initialized globals)
                                Text segment (code -- read-only)
0x000000000000400000 ----
                                (nothing -- null pointer trap zone)
0x0000000000000000 ----

```

The OS maintains a **page table** that maps each virtual page (4 KB) to a physical RAM page. When you access a virtual address, the hardware walks the page table; if the page is not present, a page fault fires and the OS loads it from disk or terminates the process.

System Calls: Asking the OS to Do Things

User code cannot directly read files, create sockets, or allocate memory – those are kernel operations. User code makes **system calls** (syscalls) to ask the kernel.

In Linux, a syscall is triggered by the `syscall` instruction. libc functions like `malloc`, `read`, `write`, `open`, and `close` are thin wrappers around syscalls.

```
Python open("file.txt")
  -> CPython C code
    -> libc fopen()
      -> libc internally calls read() which invokes:
        -> syscall(SYS_openat, ...) instruction
          -> kernel handles it
            -> returns file descriptor integer back to user space
```

In C++, you can call libc directly or (on Linux) invoke syscalls yourself:

```
#include <unistd.h>      // POSIX read/write/close
#include <fcntl.h>       // open
#include <sys/syscall.h> // SYS_* constants

// Standard POSIX I/O (preferred):
int fd = open("file.txt", O_RDONLY);
char buf[128];
ssize_t n = read(fd, buf, sizeof(buf));
close(fd);

// Raw syscall (Linux only, rarely needed):
long result = syscall(SYS_write, STDOUT_FILENO, "hello\n", 6);
```

Calling Conventions and ABI

The **ABI** (Application Binary Interface) defines how functions exchange arguments and return values at the machine level. The Linux x86-64 ABI (System V AMD64):

Integer/pointer arguments:	RDI, RSI, RDX, RCX, R8, R9 (first 6)
	Stack for 7th argument onward
Float arguments:	XMM0 – XMM7
Return value:	RAX (integer), XMM0 (float)
Caller-saved (volatile):	RAX, RCX, RDX, RSI, RDI, R8–R11
Callee-saved (preserved):	RBX, RBP, R12–R15

This is why passing more than 6 arguments is slightly slower (7th and beyond go on the stack), and why returning large structs by value is not always free (the caller may pass a hidden pointer in RDI as the return-value destination).

Inspecting the Stack Frame

```
// Stack frame layout during function call:
void callee(int x, int y) {
    // At entry:
    //   RSP points here  <-- top of stack
    //   previous RBP     <- [RBP + 0] (if frame pointer enabled)
    //   return address   <- [RBP + 8] (where to jump when callee returns)
    //   x argument       <- in RDI (not on stack for first 6 args)
    //   y argument       <- in RSI

    int local = x + y;    // local allocated on stack: [RBP - 4]
}
```

You can inspect live stack frames with a debugger:

```
gdb ./program
(gdb) run
(gdb) break callee
(gdb) info registers      # see all register values
(gdb) x/16xg $rsp         # examine 16 8-byte words at stack pointer
(gdb) backtrace           # print call stack
```

mmap – Direct Memory Mapping

mmap maps files or anonymous memory directly into the virtual address space. It is how the OS loader loads executables, how `malloc` gets large chunks from the kernel, and how databases do I/O without copying:

```
#include <sys/mman.h>
#include <fcntl.h>
#include <unistd.h>

// Map a file into memory for read-only access:
int fd = open("large_file.bin", O_RDONLY);
struct stat st; fstat(fd, &st);
void* ptr = mmap(nullptr, st.st_size, PROT_READ, MAP_PRIVATE, fd, 0);
close(fd); // can close fd after mmap -- the mapping persists

if (ptr == MAP_FAILED) { perror("mmap"); return 1; }

// Now access the file contents as if it were an array:
const uint8_t* data = static_cast<const uint8_t*>(ptr);
std::cout << "First byte: " << (int)data[0] << "\n";

munmap(ptr, st.st_size); // unmap when done
```

Reading a 10 GB file with mmap and accessing only a few pages is far faster than `read()` into a buffer – only the accessed pages are loaded from disk.

Common Mistakes in This Chapter

Mistake 1: Reading Uninitialized Stack Variables

The bug:

```
int arr[10];           // on the stack -- NOT zero-initialized
std::cout << arr[5];  // reads whatever happened to be there before
```

Fix: `int arr[10] = {};` (value-initializes all to zero) or use `std::array<int,10> arr{}.`

Mistake 2: Stack Overflow From Large Stack Allocations

The bug:

```
void deep_recursion(int n) {
    int huge_array[1'000'000]; // 4 MB on the stack -- stack is typically 8 MB
    if (n > 0) deep_recursion(n-1);
}
deep_recursion(3); // crash: stack overflow
```

Fix: Large buffers belong on the heap (`std::vector<int> (1'000'000)`), not the stack.

Exercises

Exercise 51.1 – Syscall tracing

Run `strace ./your_program` on a simple C++ program that opens and reads a file. Identify the `open`, `read`, and `close` syscalls. What does `strace` output for a `std::cout << "hello\n"` statement?

Answer: `std::cout << "hello\n"` calls `write(1, "hello\n", 6)` (file descriptor 1 = `stdout`). `strace` will show:

```
write(1, "hello\n", 6) = 6
```

Exercise 51.2 – Memory-mapped file reader

Write a C++ program that uses `mmap` to open a text file and count the number of newline characters (`\n`) without using `fread` or `std::ifstream`.

Answer:


```
#include <sys/mman.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <iostream>

int main(int argc, char* argv[]) {
    int fd = open(argv[1], O_RDONLY);
    struct stat st; fstat(fd, &st);
    const char* data = (const char*)mmap(nullptr, st.st_size, PROT_READ, MAP_PRIVATE, fd,
    ↵ 0);
    close(fd);

    long count = 0;
    for (off_t i = 0; i < st.st_size; ++i)
        if (data[i] == '\n') ++count;

    munmap((void*)data, st.st_size);
    std::cout << count << " lines\n";
}
```

Chapter 52: Working With OS and Linux APIs

POSIX: The Portable Interface

POSIX (Portable Operating System Interface) is a standard for OS APIs. Linux, macOS, and BSDs all implement it. Windows has partial support. POSIX defines: file I/O, processes, threads (pthreads), signals, pipes, sockets, and more.

`std::thread` in C++ is implemented on top of pthreads on Linux. Understanding the underlying POSIX layer explains behavior that the C++ standard doesn't specify.

Processes

A **process** is a running program with its own address space. Unlike threads (which share memory), processes have isolated memory.

```
#include <unistd.h>    // fork, exec, getpid
#include <sys/wait.h>  // waitpid

// fork() creates a child process that is an exact copy of the parent:
pid_t pid = fork();

if (pid < 0) {
    perror("fork failed");
} else if (pid == 0) {
    // We are in the CHILD process
    std::cout << "Child PID: " << getpid() << "\n";
    // Replace child process image with a new program:
    execl("/bin/ls", "ls", "-la", nullptr);
    // If execl returns, it failed:
    perror("exec failed");
    _exit(1);
} else {
    // We are in the PARENT process (pid = child's PID)
    std::cout << "Parent waiting for child " << pid << "\n";
    int status;
    waitpid(pid, &status, 0); // wait for child to finish
    std::cout << "Child exited with status " << WEXITSTATUS(status) << "\n";
}
```

The `fork()/exec()` pattern is how shells work: `fork` creates a child, `exec` replaces it with the new program.

Signals

Signals are asynchronous notifications sent to a process. SIGINT (Ctrl+C), SIGTERM (kill), SIGSEGV (segfault), SIGCHLD (child exited).

```
#include <csignal>

std::atomic<bool> shutdown_requested{false};

void signal_handler(int sig) {
    if (sig == SIGINT || sig == SIGTERM)
        shutdown_requested.store(true, std::memory_order_relaxed);
}

int main() {
    std::signal(SIGINT, signal_handler); // register handler for Ctrl+C
    std::signal(SIGTERM, signal_handler); // register handler for kill

    while (!shutdown_requested) {
        do_work();
    }
    std::cout << "Shutting down cleanly\n";
}
```

Signal handlers run in interrupt context – they can interrupt any instruction. Only **async-signal-safe** functions are permitted inside a handler (e.g., write, not printf or malloc). The pattern above uses an atomic flag and processes the signal in the main loop.

For more complex signal handling, use `signalfd` or `sigaction` with `SA_SIGACTION`.

File Descriptors and POSIX I/O

Everything in Unix is a file: disk files, sockets, pipes, terminals, devices. All are accessed via **file descriptors** (small integers).

```
#include <unistd.h>
#include <fcntl.h>

// Open flags: O_RDONLY, O_WRONLY, O_RDWR, O_CREAT, O_TRUNC, O_APPEND
int fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
// 0644 = permissions: owner rw, group r, other r

const char* msg = "hello from C++\n";
ssize_t written = write(fd, msg, strlen(msg));
if (written < 0) perror("write");

close(fd);

// Duplicating file descriptors (used to redirect stdout):
int saved_stdout = dup(STDOUT_FILENO); // save original stdout fd
int new_fd = open("stdout_capture.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
dup2(new_fd, STDOUT_FILENO); // stdout now writes to the file
```

```
close(new_fd);

std::cout << "This goes to stdout_capture.txt\n";

dup2(saved_stdout, STDOUT_FILENO); // restore stdout
close(saved_stdout);
```

Pipes: Inter-Process Communication

A pipe is a unidirectional byte channel with a read end and a write end:

```
#include <unistd.h>

int pipefd[2]; // pipefd[0] = read end, pipefd[1] = write end
pipe(pipefd);

pid_t pid = fork();
if (pid == 0) {
    // Child: write to pipe
    close(pipefd[0]); // close unused read end
    const char* msg = "message from child";
    write(pipefd[1], msg, strlen(msg));
    close(pipefd[1]);
    _exit(0);
} else {
    // Parent: read from pipe
    close(pipefd[1]); // close unused write end
    char buf[64] = {};
    read(pipefd[0], buf, sizeof(buf)-1);
    std::cout << "Parent received: " << buf << "\n";
    close(pipefd[0]);
    wait(nullptr);
}
```

epoll – Scalable I/O Multiplexing

`select` and `poll` check multiple file descriptors for readiness. `epoll` is the Linux-specific high-performance version used by every production server:

```
#include <sys/epoll.h>

int epfd = epoll_create1(0); // create epoll instance

// Register a socket fd for read events:
epoll_event ev{};
ev.events = EPOLLIN; // notify when data is available to read
ev.data.fd = socket_fd;
epoll_ctl(epfd, EPOLL_CTL_ADD, socket_fd, &ev);
```

```
// Event loop:
epoll_event events[64];
while (true) {
    int n = epoll_wait(epfd, events, 64, -1); // -1 = wait indefinitely
    for (int i = 0; i < n; ++i) {
        if (events[i].events & EPOLLIN) {
            int fd = events[i].data.fd;
            char buf[4096];
            ssize_t bytes = read(fd, buf, sizeof(buf));
            if (bytes <= 0) {
                epoll_ctl(epfd, EPOLL_CTL_DEL, fd, nullptr);
                close(fd);
            } else {
                handle_data(fd, buf, bytes);
            }
        }
    }
}
```

`epoll` is $O(1)$ per event regardless of how many `fds` are registered (unlike `select` which is $O(n)$). It is the foundation of high-performance event loops like `libuv` (Node.js) and `Boost.Asio`.

Common Mistakes in This Chapter

Mistake 1: Not Handling EINTR

The bug:

```
ssize_t n = read(fd, buf, len);
if (n < 0) { perror("read"); return; }
```

Symptom: `read` returns `-1` with `errno == EINTR` when interrupted by a signal (not an error – restart).

Fix:

```
ssize_t n;
do { n = read(fd, buf, len); } while (n < 0 && errno == EINTR);
if (n < 0) { perror("read"); return; }
```

Mistake 2: Forgetting O_CLOEXEC on Forked Programs

The bug: Opening files or sockets without `O_CLOEXEC`. When `fork` + `exec` creates a child process, all open file descriptors are inherited by the child unless marked close-on-exec.

Fix: `open(path, flags | O_CLOEXEC)`, `socket(... | SOCK_CLOEXEC)`.

Exercises

Exercise 52.1 – Process output capture

Fork a child process that runs `ls -la /tmp` using `execl`. Redirect the child's stdout to a pipe, and have the parent read and print the output.

Answer (key structure):

```
int pfd[2]; pipe2(pfd, O_CLOEXEC);
pid_t pid = fork();
if (pid == 0) {
    dup2(pfd[1], STDOUT_FILENO);
    // pfd[0] and pfd[1] auto-close (O_CLOEXEC)
    execl("/bin/ls", "ls", "-la", "/tmp", nullptr);
    _exit(1);
}
close(pfd[1]);
char buf[4096]; ssize_t n;
while ((n = read(pfd[0], buf, sizeof(buf))) > 0)
    write(STDOUT_FILENO, buf, n);
close(pfd[0]);
waitpid(pid, nullptr, 0);
```


Chapter 53: Networking From the Ground Up

The Network Stack

Application layer: HTTP, WebSocket, gRPC -- what your program sends/receives
Transport layer: TCP (reliable, ordered) or UDP (unreliable, fast)
Network layer: IP -- addressing and routing between machines
Link layer: Ethernet, WiFi -- physical transmission

From your C++ code, you interact at the transport layer via the BSD sockets API. The kernel handles everything below.

TCP Sockets: A Server

```
#include <sys/socket.h>
#include <netinet/in.h>
#include <unistd.h>
#include <cstring>
#include <iostream>

int main() {
    // 1. Create a TCP socket (AF_INET = IPv4, SOCK_STREAM = TCP)
    int server_fd = socket(AF_INET, SOCK_STREAM | SOCK_CLOEXEC, 0);

    // 2. Set SO_REUSEADDR to avoid "address already in use" on restart
    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    // 3. Bind to a port
    sockaddr_in addr{};
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = INADDR_ANY; // listen on all interfaces
    addr.sin_port = htons(8080); // host-to-network-short: endian conversion
    bind(server_fd, (sockaddr*)&addr, sizeof(addr));

    // 4. Listen for incoming connections (backlog = 10)
    listen(server_fd, 10);
    std::cout << "Listening on :8080\n";

    while (true) {
        // 5. Accept a connection (blocks until a client connects)
        sockaddr_in client_addr{};
```



```

socklen_t client_len = sizeof(client_addr);
int client_fd = accept(server_fd, (sockaddr*)&client_addr, &client_len);

// 6. Communicate with the client
char buf[1024];
ssize_t n = recv(client_fd, buf, sizeof(buf)-1, 0);
if (n > 0) {
    buf[n] = '\0';
    std::cout << "Received: " << buf;
    const char* response = "HTTP/1.1 200 OK\r\n\r\nHello!\r\n";
    send(client_fd, response, strlen(response), 0);
}
close(client_fd);
}
}

```

TCP Sockets: A Client

```

#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h> // inet_pton
#include <unistd.h>

int main() {
    int fd = socket(AF_INET, SOCK_STREAM, 0);

    sockaddr_in server{};
    server.sin_family = AF_INET;
    server.sin_port = htons(8080);
    inet_pton(AF_INET, "127.0.0.1", &server.sin_addr); // parse IP string

    if (connect(fd, (sockaddr*)&server, sizeof(server)) < 0) {
        perror("connect"); return 1;
    }

    const char* req = "GET / HTTP/1.0\r\nHost: localhost\r\n\r\n";
    send(fd, req, strlen(req), 0);

    char buf[4096];
    ssize_t n = recv(fd, buf, sizeof(buf)-1, 0);
    buf[n] = '\0';
    std::cout << buf;
    close(fd);
}

```

UDP Sockets

UDP sends individual datagrams – no connection, no ordering guarantee, no retransmission. Used for gaming (low latency matters more than reliability), DNS, streaming media.

```
// UDP server:
int fd = socket(AF_INET, SOCK_DGRAM, 0);
sockaddr_in addr{}; addr.sin_family=AF_INET; addr.sin_port=htons(9000);
addr.sin_addr.s_addr = INADDR_ANY;
bind(fd, (sockaddr*)&addr, sizeof(addr));

char buf[1024];
sockaddr_in sender{};
socklen_t sender_len = sizeof(sender);
// recvfrom captures sender's address so we can reply:
ssize_t n = recvfrom(fd, buf, sizeof(buf)-1, 0, (sockaddr*)&sender, &sender_len);
buf[n] = '\0';
std::cout << "UDP received: " << buf << "\n";
sendto(fd, "pong", 4, 0, (sockaddr*)&sender, sender_len);
close(fd);
```

Non-Blocking I/O and `epoll`

A production server handles thousands of connections. One thread per connection wastes memory (each thread stack is ~2-8 MB). Non-blocking I/O + `epoll` (from Chapter 52) handles all connections in one or a few threads:

```
// Make a socket non-blocking:
int flags = fcntl(fd, F_GETFL, 0);
fcntl(fd, F_SETFL, flags | O_NONBLOCK);

// With O_NONBLOCK: read/recv/accept return immediately with EAGAIN/EWOULDBLOCK
// when there is no data, instead of blocking
ssize_t n = recv(fd, buf, len, 0);
if (n < 0 && (errno == EAGAIN || errno == EWOULDBLOCK)) {
    // No data available right now -- come back when epoll says it's ready
}
```

This is the reactor pattern: register fds with `epoll`, wait for readiness events, then process all ready fds. Libraries like **Boost.Asio**, **libuv**, and **io_uring** build on this.

Address Resolution: `getaddrinfo`

Rather than hardcoding IPv4 addresses, use `getaddrinfo` which handles both IPv4 and IPv6, DNS lookups, and service names:

```
#include <netdb.h>

addrinfo hints{};
hints.ai_family = AF_UNSPEC; // accept IPv4 or IPv6
hints.ai_socktype = SOCK_STREAM; // TCP
```

```

addrinfo* result;
int err = getaddrinfo("example.com", "80", &hints, &result);
if (err != 0) {
    std::cerr << gai_strerror(err) << "\n"; return 1;
}

// result is a linked list of addresses -- try each until one connects:
for (addrinfo* rp = result; rp != nullptr; rp = rp->ai_next) {
    int fd = socket(rp->ai_family, rp->ai_socktype, rp->ai_protocol);
    if (connect(fd, rp->ai_addr, rp->ai_addrlen) == 0) {
        // Connected!
        freeaddrinfo(result);
        // use fd ...
    }
    close(fd);
}
freeaddrinfo(result);

```

Common Mistakes in This Chapter

Mistake 1: Assuming recv Returns the Full Message

The bug:

```

char buf[1024];
recv(fd, buf, sizeof(buf), 0);
// Assumes all bytes of the message arrive in one call

```

Symptom: Partial reads. `recv` may return less than the full message.

Fix: Loop until all expected bytes are received (“read exactly N bytes” pattern):

```

size_t total = 0;
while (total < expected) {
    ssize_t n = recv(fd, buf+total, expected-total, 0);
    if (n <= 0) break; // connection closed or error
    total += n;
}

```

Mistake 2: Not Converting Byte Order

The bug:

```

addr.sin_port = 8080; // wrong! network byte order is big-endian

```

Fix: Always use `htons()` for port numbers and `htonl()` for 32-bit addresses: `addr.sin_port = htons(8080)`.

Exercises

Exercise 53.1 – Echo server

Write a TCP echo server that accepts a connection, reads up to 1024 bytes, sends the same bytes back, and closes the connection. Test it with `nc localhost 8080`.

Answer: The server structure is the same as the example above. In the communication step:

```
char buf[1024];
ssize_t n = recv(client_fd, buf, sizeof(buf), 0);
if (n > 0) send(client_fd, buf, n, 0); // echo back exactly what was received
close(client_fd);
```


Chapter 54: Where C++ Meets C, eBPF, and Go

Calling C From C++

C++ is backward-compatible with C at the object-file level. Any C library can be called from C++:

```
// Declare a C function with C linkage (no name mangling):
extern "C" {
    #include <curl/curl.h> // C library header
}

// Or wrap a C function you wrote yourself:
extern "C" int my_c_function(int x);

// C++ calls it like any function:
int result = my_c_function(42);
```

Name mangling: C++ encodes type information into symbol names (foo(int, double) becomes _ZN3foo3dEi or similar). C symbols have no mangling. extern "C" tells the C++ compiler not to mangle the name, making it linkable from C.

```
// A C++ library that exports a C-compatible API:
// header: mylib.h
#ifdef __cplusplus
extern "C" {
#endif

void mylib_init();
int mylib_compute(int x);
void mylib_cleanup();

#ifdef __cplusplus
}
#endif
```

This pattern lets you write the implementation in C++ while exposing a C ABI that any language can call (Python via ctypes, Go via cgo, Rust via bindgen).

Calling C++ From Python (ctypes / cffi)

```
// mylib.cpp -- compile to shared library:
// g++ -O2 -shared -fPIC -o libmylib.so mylib.cpp
extern "C" {
    int add(int a, int b) { return a + b; }
    double sqrt_approx(double x) { return x * 0.5 * (3.0 - x); }
}
```

```
# Python calling the shared library:
import ctypes
lib = ctypes.CDLL("./libmylib.so")
lib.add.argtypes = [ctypes.c_int, ctypes.c_int]
lib.add.restype = ctypes.c_int

result = lib.add(3, 4) # calls C++ function -- result = 7
```

For more ergonomic Python bindings, use **pybind11** (header-only, wraps C++ classes automatically) or **nanobind** (faster, smaller).

Calling C++ From Go (cgo)

```
// main.go -- Go calling C++ via cgo:
package main

/*
#cgo LDFLAGS: -L. -lmylib -lstdc++
#include "mylib.h"
*/
import "C"
import "fmt"

func main() {
    result := C.add(3, 4)
    fmt.Println("Result:", int(result))
}
```

cgo generates glue code that marshals between Go and C types. The overhead is roughly 50-100ns per call – fine for infrequent calls, a problem for tight loops.

eBPF: Running C++ Code in the Linux Kernel

eBPF (extended Berkeley Packet Filter) is a revolutionary Linux feature: you write small programs (in C, compiled to BPF bytecode) that run inside the kernel without kernel module development. They can:

- Trace any kernel function (without modifying the kernel)
- Filter network packets at wire speed

- Collect performance metrics
- Implement custom schedulers and load balancers

```
// eBPF program in C (compiled with clang -target bpf):
// Counts how many times write() is called per PID

#include <linux/bpf.h>
#include <bpf/bpf_helpers.h>

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __uint(max_entries, 1024);
    __type(key, __u32); // PID
    __type(value, __u64); // count
} write_counts SEC(".maps");

SEC("tracepoint/syscalls/sys_enter_write")
int trace_write(struct trace_event_raw_sys_enter* ctx) {
    __u32 pid = bpf_get_current_pid_tgid() >> 32;
    __u64* count = bpf_map_lookup_elem(&write_counts, &pid);
    if (count) {
        __sync_fetch_and_add(count, 1);
    } else {
        __u64 one = 1;
        bpf_map_update_elem(&write_counts, &pid, &one, BPF_ANY);
    }
    return 0;
}

char LICENSE[] SEC("license") = "GPL";
```

```
// C++ user-space loader (using libbpf):
#include <bpf/libbpf.h>

struct bpf_object* obj = bpf_object__open_file("trace_write.bpf.o", nullptr);
bpf_object__load(obj);
struct bpf_program* prog = bpf_object__find_program_by_name(obj, "trace_write");
struct bpf_link* link = bpf_program__attach(prog);

// After running some time, read the map:
int map_fd = bpf_object__find_map_fd_by_name(obj, "write_counts");
__u32 pid; __u64 count;
bpf_map_get_next_key(map_fd, nullptr, &pid);
bpf_map_lookup_elem(map_fd, &pid, &count);
printf("PID %u called write() %llu times\n", pid, count);

bpf_link__destroy(link);
bpf_object__close(obj);
```

eBPF tools you already know: perf, strace (modern versions), tcpdump, and Cilium (Kubernetes networking) are all built on eBPF.

std::bit_cast and Bitwise Reinterpretation (C++20)

When working at the systems level you often need to reinterpret the bits of one type as another. The safe way (avoids undefined behavior):

```
#include <bit>

// Read the IEEE 754 bit pattern of a float as a uint32:
float f = 3.14f;
uint32_t bits = std::bit_cast<uint32_t>(f);
printf("float 3.14 bit pattern: 0x%08X\n", bits);

// Check the sign bit, exponent, mantissa:
bool    sign    = (bits >> 31) & 1;
uint32_t exponent = (bits >> 23) & 0xFF;
uint32_t mantissa = bits    & 0x7FFFFFFF;
printf("sign=%d exponent=%d mantissa=0x%06X\n", sign, exponent-127, mantissa);
```

std::bit_cast requires both types to have the same size and be trivially copyable. It is the correct replacement for the old memcpy trick and the undefined-behavior *(uint32_t*)&f cast.

Inline Assembly

For extreme optimization or hardware-specific operations, C++ allows inline assembly:

```
// Read the CPU's timestamp counter (nanosecond-precision cycle counter):
uint64_t rdtsc() {
    uint32_t lo, hi;
    __asm__ __volatile__(
        "rdtsc"
        : "=a"(lo), "=d"(hi)    // outputs: eax -> lo, edx -> hi
        :                        // no inputs
        : "memory"              // clobbers: forces memory writes to complete
    );
    return ((uint64_t)hi << 32) | lo;
}

uint64_t start = rdtsc();
do_work();
uint64_t elapsed = rdtsc() - start;
printf("Cycles: %llu\n", elapsed);
```

Use inline assembly sparingly – it is non-portable and prevents many compiler optimizations. For most purposes, compiler intrinsics (<immintrin.h> for SIMD) or __builtin_* functions are better.

Common Mistakes in This Chapter

Mistake 1: Name Mangling Mismatch

The bug:

```
// In C++ without extern "C":  
int add(int a, int b); // symbol: _Z3addii (mangled)  
  
// Linked against a C object file where add is compiled as:  
// symbol: add (unmangled)  
  
// Linker error: undefined reference to _Z3addii
```

Fix: Use `extern "C"` when declaring functions that will be linked to C translation units.

Mistake 2: Type Mismatch Across FFI Boundary

The bug:

```
lib.add.argtypes = [ctypes.c_int, ctypes.c_int]  
lib.add(3.14, 4) # passing float where int expected -- silent corruption
```

Fix: Always declare `argtypes` and `restype` in `ctypes`. Add assertions or use a type-safe binding library like `pybind11`.

Exercises

Exercise 54.1 – Python extension via `ctypes`

Write a C++ function `int dot_product(int* a, int* b, int n)` that computes the dot product of two arrays. Compile it as a shared library and call it from Python using `ctypes`.

Answer:

```
// dot.cpp  
extern "C" {  
    int dot_product(const int* a, const int* b, int n) {  
        int sum = 0;  
        for (int i = 0; i < n; ++i) sum += a[i] * b[i];  
        return sum;  
    }  
}  
// Build: g++ -O2 -shared -fPIC -o libdot.so dot.cpp
```

```
import ctypes
lib = ctypes.CDLL("./libdot.so")
lib.dot_product.argtypes = [
    ctypes.POINTER(ctypes.c_int),
    ctypes.POINTER(ctypes.c_int),
    ctypes.c_int
]
lib.dot_product.restype = ctypes.c_int

a = (ctypes.c_int * 3)(1, 2, 3)
b = (ctypes.c_int * 3)(4, 5, 6)
print(lib.dot_product(a, b, 3)) # 1*4 + 2*5 + 3*6 = 32
```

Exercise 54.2 – std::bit_cast exploration

Use `std::bit_cast` to extract the sign, biased exponent, and mantissa bits from `float` values: `1.0f`, `-2.5f`, and `std::numeric_limits<float>::infinity()`. Verify the IEEE 754 structure manually.

Answer:

```
auto inspect = [](float f) {
    uint32_t b = std::bit_cast<uint32_t>(f);
    int sign = b >> 31;
    int exp = ((b >> 23) & 0xFF) - 127;
    uint32_t man = b & 0x7FFFFFFF;
    std::cout << f << ": sign=" << sign << " exp=" << exp
              << " mantissa=0x" << std::hex << man << std::dec << "\n";
};
inspect(1.0f); // sign=0 exp=0 mantissa=0x0000000
inspect(-2.5f); // sign=1 exp=1 mantissa=0x2000000
inspect(std::numeric_limits<float>::infinity()); // sign=0 exp=128(=255-127) mantissa=0x0
```

Part XI is complete. You now understand C++ at the machine level: how registers and stack frames work, how the OS exposes services via syscalls and POSIX APIs, how TCP and UDP sockets are built from first principles, and how C++ interoperates with C, Python, Go, and the Linux kernel via eBPF.

The Appendices (A-D) follow: toolchain setup, compiler flags reference, the Python-to-C++ cheat sheet, and the common mistakes and debugging guide. Ask to continue.

Appendices

Appendix A: Setting Up Your Toolchain

Linux (Ubuntu/Debian)

```
# Compiler and build tools
sudo apt update
sudo apt install build-essential          # gcc, g++, make
sudo apt install clang clang-format clang-tidy lldb

# Install a specific GCC version (for C++23):
sudo apt install gcc-13 g++-13
sudo update-alternatives --install /usr/bin/g++ g++ /usr/bin/g++-13 100

# CMake (build system)
sudo apt install cmake

# Ninja (fast build backend, pairs with CMake)
sudo apt install ninja-build

# vcpkg (C++ package manager)
git clone https://github.com/microsoft/vcpkg.git ~/vcpkg
cd ~/vcpkg && ./bootstrap-vcpkg.sh
echo 'export VCPKG_ROOT=$HOME/vcpkg' >> ~/.bashrc
echo 'export PATH="$VCPKG_ROOT:$PATH"' >> ~/.bashrc

# Conan (alternative package manager)
pip install conan

# Sanitizers (built into gcc/clang -- no install needed)
# AddressSanitizer: -fsanitize=address
# ThreadSanitizer: -fsanitize=thread
# UBSanitizer: -fsanitize=undefined

# Valgrind (memory checker)
sudo apt install valgrind

# perf (CPU profiler)
sudo apt install linux-tools-common linux-tools-generic

# gdb (debugger)
sudo apt install gdb
```

macOS

```
# Xcode Command Line Tools (includes clang, make, git)
xcode-select --install

# Homebrew (package manager)
/bin/bash -c "$(curl -fsSL
↪ https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# GCC (if you want gcc instead of/in addition to Apple clang)
brew install gcc

# CMake, Ninja
brew install cmake ninja

# vcpkg
git clone https://github.com/microsoft/vcpkg.git ~/vcpkg
cd ~/vcpkg && ./bootstrap-vcpkg.sh

# lldb is the default macOS debugger (installed with Xcode tools)

# Instruments (Apple profiler -- comes with Xcode)
# Instruments.app -> Time Profiler for CPU profiling

# Note: ThreadSanitizer and AddressSanitizer work on macOS with clang
```

Windows

```
# Option 1: MSVC (Microsoft Visual C++) -- native Windows compiler
# Install: Visual Studio Community (free)
#   Choose workload: "Desktop development with C++"
#   Or: Build Tools for Visual Studio (command-line only, no IDE)

# Developer Command Prompt (sets up PATH for cl.exe, link.exe, etc.):
# Start Menu -> "x64 Native Tools Command Prompt for VS 2022"

# Compile from command line:
cl /std:c++20 /EHsc /W4 /O2 main.cpp /Fe:main.exe

# Option 2: MSYS2/MinGW (GCC on Windows)
# Download MSYS2 from msys2.org
pacman -S mingw-w64-x86_64-gcc mingw-w64-x86_64-cmake mingw-w64-x86_64-ninja

# Option 3: WSL2 (Windows Subsystem for Linux) -- full Linux toolchain
# In PowerShell:
wsl --install
# Then follow the Linux instructions above inside WSL2

# CMake GUI
# Download from cmake.org -- useful for configuring projects visually
```

```
# vcpkg on Windows:
git clone https://github.com/microsoft/vcpkg.git C:\vcpkg
C:\vcpkg\bootstrap-vcpkg.bat
```

Your First CMake Project

```
project/
├── CMakeLists.txt
├── src/
│   └── main.cpp
└── include/
    └── mylib.hpp
```

```
# CMakeLists.txt
cmake_minimum_required(VERSION 3.20)
project(MyProject LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)           # use -std=c++20 not -std=gnu++20
set(CMAKE_EXPORT_COMPILE_COMMANDS ON)  # for clangd/IDEs

# Build type defaults:
if(NOT CMAKE_BUILD_TYPE)
    set(CMAKE_BUILD_TYPE Debug)
endif()

# Compiler warnings (good practice -- treat warnings as errors in CI):
add_compile_options(
    -Wall -Wextra -Wpedantic
    ${CMAKE_CONFIG:Debug}:-g -O0
    ${CMAKE_CONFIG:Release}:-O3 -DNDEBUG
)

# Your executable:
add_executable(my_app src/main.cpp)
target_include_directories(my_app PRIVATE include)

# Link a library (e.g., pthreads on Linux):
find_package(Threads REQUIRED)
target_link_libraries(my_app PRIVATE Threads::Threads)

# Build:
cmake -B build -G Ninja -DCMAKE_BUILD_TYPE=Release
cmake --build build

# Run:
./build/my_app

# Debug build with sanitizers:
cmake -B build-dbg -G Ninja -DCMAKE_BUILD_TYPE=Debug \
```



```
-DCMAKE_CXX_FLAGS="-fsanitize=address,undefined"  
cmake --build build-dbg  
./build-dbg/my_app
```

IDE Setup

VS Code

```
// .vscode/settings.json  
{  
  "C_Cpp.default.cppStandard": "c++20",  
  "C_Cpp.default.compileCommands": "${workspaceFolder}/build/compile_commands.json",  
  "clangd.arguments": ["--compile-commands-dir=${workspaceFolder}/build"]  
}
```

Extensions: clangd (language server), CodeLLDB (debugger), CMake Tools.

CLion (JetBrains)

Open the folder containing `CMakeLists.txt`. CLion detects it automatically and runs `cmake`. No configuration file needed.

Appendix B: Compiler Flags Reference

GCC / Clang Flags

Warning Flags

```
-Wall           # Enable most common warnings (not all)
-Wextra        # Extra warnings beyond -Wall
-Wpedantic     # Strictly conform to the C++ standard
-Werror        # Treat all warnings as errors (use in CI)
-Wshadow       # Warn when a local variable shadows an outer variable
-Wnon-virtual-dtor # Warn when a class with virtual functions has no virtual dtor
-Wold-style-cast # Warn on C-style casts
-Woverloaded-virtual # Warn when a derived function hides a base virtual function
-Wconversion   # Warn on implicit conversions that may change value
-Wsign-conversion # Warn on signed/unsigned conversion
```

Optimization Flags

```
-O0            # No optimization (default) -- fastest compilation, easiest debugging
-O1            # Basic optimization
-O2            # Standard release optimization (safe)
-O3            # Aggressive optimization (enables vectorization, more inlining)
-Os            # Optimize for size
-Og            # Optimize for debugging experience (some optimization, full debug info)
-Ofast        # O3 + allow non-standard float behavior (breaks IEEE 754 compliance)

# Link-time optimization (whole-program optimization):
-flto         # Enable LTO
-flto=thin    # Clang ThinLTO (faster, parallel)
```

Debug Flags

```
-g            # Generate debug info (DWARF) -- use with -O0 or -Og
-g3          # More debug info (includes macro definitions)
-ggdb        # Extra GDB-specific debug info
-fno-omit-frame-pointer # Keep frame pointer for profilers (perf, gprof)
```

Sanitizer Flags

```
# AddressSanitizer: detects heap/stack overflows, use-after-free, double-free
-fsanitize=address -fno-omit-frame-pointer

# ThreadSanitizer: detects data races
-fsanitize=thread

# UndefinedBehaviorSanitizer: detects UB (integer overflow, null deref, etc.)
-fsanitize=undefined

# MemorySanitizer (clang only): detects uninitialized memory reads
-fsanitize=memory

# Combine ASAN + UBSAN (common in CI):
-fsanitize=address,undefined

# Note: sanitizers add ~2x runtime overhead -- use in debug/test builds only
```

Architecture and Feature Flags

```
-march=native    # Optimize for the CPU you're compiling on (not portable)
-march=x86-64-v3 # Target Intel Haswell and later (AVX2, widely portable)
-mavx2          # Enable AVX2 SIMD instructions
-msse4.2        # Enable SSE4.2 (older but widely available)

# C++ standard:
-std=c++17
-std=c++20
-std=c++23
```

Hardening Flags (Production Security)

```
-D_FORTIFY_SOURCE=2    # Runtime buffer overflow checks (glibc)
-fstack-protector-strong # Stack smashing protection
-fPIE -pie            # Position-independent executable (ASLR support)
-Wl,-z,relro          # Make GOT read-only after dynamic linking
-Wl,-z,now             # Resolve all symbols at startup (no lazy binding)
```

MSVC Flags (Windows)

```
/std:c++20    # C++20 standard
/W4           # Warning level 4 (equivalent to -Wall -Wextra)
/WX           # Treat warnings as errors
```

```
/EHsc          # Standard C++ exception handling
/MD            # Dynamic runtime library (release)
/MDd          # Dynamic runtime library (debug)
/O2           # Optimize for speed
/Od           # No optimization (debug)
/Zi           # Generate debug info (PDB file)
/RTC1         # Runtime checks for uninitialized variables and stack overflows
/analyze      # Static analysis (Clang-based)

# Address sanitizer (MSVC 2019+):
/fsanitize=address

# Whole program optimization:
/GL           # Link-time code generation (like -flto)
/LTCG        # Linker flag for LTO
```

Recommended Flag Sets

```
# Debug build (fast to compile, easy to debug, all checks):
g++ -std=c++20 -g -O0 -Wall -Wextra -Wpedantic \
    -fsanitize=address,undefined -fno-omit-frame-pointer \
    main.cpp -o main_debug

# Release build (maximum performance):
g++ -std=c++20 -O3 -DNDEBUG -march=native -flto \
    -Wall -Wextra -Wpedantic \
    main.cpp -o main_release

# CI build (catches everything, portable):
g++ -std=c++20 -O2 -march=x86-64 -Wall -Wextra -Wpedantic -Werror \
    -fsanitize=address,undefined \
    main.cpp -o main_ci
```


Appendix C: Python to C++ Cheat Sheet

Types

Python	C++	Notes
int	int, long, int64_t	C++ int is 32-bit; use int64_t for Python-sized ints
float	double	Python float is 64-bit double
bool	bool	Same semantics
str	std::string	C++ strings are mutable, not Unicode by default
bytes	std::vector<uint8_t> or std::string	Raw bytes
list	std::vector<T>	Dynamic array
tuple	std::tuple<T,U,V> or std::pair<T,U>	Fixed-size, typed
dict	std::unordered_map<K,V>	Hash map
set	std::unordered_set<T>	Hash set
None	nullptr, std::nullopt, std::monostate	Depends on context
Optional[T]	std::optional<T>	May-or-may-not-have-a-value

Variables and Assignment

```
# Python
x = 42
x = "now a string"  # rebind to different type
y = x               # y is a reference to the same object
```

```
// C++
int x = 42;
// x = "now a string"; // ERROR -- types are fixed
int y = x;           // y is a COPY of x (not a reference)
int& z = x;          // z IS a reference (alias)
```

Functions

```
# Python
def add(a, b):
    return a + b

def greet(name: str = "world") -> str:
    return f"Hello, {name}!"
```

```
// C++
int add(int a, int b) {
    return a + b;
}

std::string greet(const std::string& name = "world") {
    return "Hello, " + name + "!";
    // or: return std::format("Hello, {}", name); // C++20
}
```

Classes

```
# Python
class Animal:
    def __init__(self, name: str):
        self.name = name

    def speak(self) -> str:
        return "..."

class Dog(Animal):
    def speak(self) -> str:
        return f"{self.name} says: Woof!"
```

```
// C++
class Animal {
protected:
    std::string name;
public:
    Animal(std::string n) : name{std::move(n)} {}
}
```

```

    virtual std::string speak() const { return "..."; }
    virtual ~Animal() = default;    // always virtual in polymorphic base classes
};

class Dog : public Animal {
public:
    Dog(std::string n) : Animal{std::move(n)} {}
    std::string speak() const override {
        return name + " says: Woof!";
    }
};

```

Containers

```

# Python list
nums = [1, 2, 3]
nums.append(4)
nums.pop()
for n in nums: print(n)
length = len(nums)

```

```

// C++ vector
std::vector<int> nums = {1, 2, 3};
nums.push_back(4);
nums.pop_back();
for (int n : nums) std::cout << n << "\n";
size_t length = nums.size();

```

```

# Python dict
d = {"key": 42}
d["new"] = 99
v = d.get("missing", 0)
for k, v in d.items(): print(k, v)

```

```

// C++ unordered_map
std::unordered_map<std::string, int> d = {"key", 42};
d["new"] = 99;
int v = d.count("missing") ? d["missing"] : 0;
// or: auto it = d.find("missing"); int v = (it != d.end()) ? it->second : 0;
for (auto& [k, val] : d) std::cout << k << " " << val << "\n";

```

String Formatting


```
# Python
name, age = "Alice", 30
msg = f"Name: {name}, Age: {age}"
pi = f"{3.14159:.2f}"
```

```
// C++20
std::string name = "Alice"; int age = 30;
auto msg = std::format("Name: {}, Age: {}", name, age);
auto pi = std::format("{:.2f}", 3.14159);

// C++17 and earlier: use stringstream or printf-style
std::ostringstream oss;
oss << "Name: " << name << ", Age: " << age;
std::string msg17 = oss.str();
```

Error Handling

```
# Python
try:
    result = int("not_a_number")
except ValueError as e:
    print(f"Error: {e}")
```

```
// C++
try {
    int result = std::stoi("not_a_number");
} catch (const std::invalid_argument& e) {
    std::cerr << "Error: " << e.what() << "\n";
} catch (const std::out_of_range& e) {
    std::cerr << "Out of range: " << e.what() << "\n";
}

// For functions that should not fail: std::optional or std::expected (C++23)
std::optional<int> parse_int(const std::string& s) {
    try { return std::stoi(s); }
    catch (...) { return std::nullopt; }
}
if (auto v = parse_int("42")) std::cout << *v << "\n";
```

Lambdas / Closures

```
# Python
add = lambda a, b: a + b
nums = [3, 1, 4, 1, 5]
nums.sort(key=lambda x: -x) # sort descending
total = sum(x*x for x in nums)
```

```
// C++
auto add = [](int a, int b) { return a + b; };
std::vector<int> nums = {3, 1, 4, 1, 5};
std::sort(nums.begin(), nums.end(), [](int a, int b){ return a > b; }); // descending
int total = 0;
for (int x : nums) total += x*x;
// Or: std::transform_reduce(nums.begin(), nums.end(), 0, std::plus{},
//                             [](int x){ return x*x; });
```

Concurrency

```
# Python (GIL limits true parallelism)
import threading
t = threading.Thread(target=worker)
t.start()
t.join()

import concurrent.futures
with concurrent.futures.ThreadPoolExecutor() as ex:
    result = ex.submit(compute, arg).result()
```

```
// C++ (true parallelism)
std::jthread t{worker}; // joins automatically in destructor

// Equivalent to ThreadPoolExecutor:
auto future = std::async(std::launch::async, compute, arg);
int result = future.get();
```

Memory Model Comparison

Python:	C++:
Every object on the heap	Objects can be on stack OR heap
Garbage collected	Deterministic destruction (RAII)
No manual memory management	new/delete (avoid) or smart pointers
References everywhere	Value semantics by default, & for reference
No pointer arithmetic	Full pointer arithmetic available
GIL limits thread parallelism	True thread parallelism

Common Python Patterns and Their C++ Equivalents

```

# List comprehension
squares = [x*x for x in range(10)]

# Dict comprehension
d = {x: x*x for x in range(5)}

# Generator
def gen():
    for i in range(10):
        yield i*i

# Context manager
with open("file.txt") as f:
    data = f.read()

```

```

// No list comprehension -- use a loop or std::ranges::transform:
std::vector<int> squares;
for (int x = 0; x < 10; ++x) squares.push_back(x*x);
// Or:
auto v = std::views::iota(0, 10)
    | std::views::transform([](int x){ return x*x; });

// No dict comprehension -- use a loop:
std::unordered_map<int,int> d;
for (int x = 0; x < 5; ++x) d[x] = x*x;

// Generator -> coroutine (C++20):
cppcoro::generator<int> gen() {
    for (int i = 0; i < 10; ++i) co_yield i*i;
}

// Context manager -> RAII class with constructor/destructor:
{
    std::ifstream f{"file.txt"};
    std::string data{std::istreambuf_iterator<char>{f}, {}};
} // file automatically closed here

```

Appendix D: Common Mistakes and Debugging Guide

The Thirty Most Common C++ Bugs

Memory Bugs

D.1 Use-after-free

```
int* p = new int{42};  
delete p;  
std::cout << *p;    // UB: reading freed memory
```

Detection: AddressSanitizer (`-fsanitize=address`). Fix: Use `unique_ptr`.

D.2 Double-free

```
int* p = new int{42};  
delete p;  
delete p;    // UB: freeing already-freed memory
```

Detection: AddressSanitizer. Fix: Use `unique_ptr` (its destructor runs exactly once).

D.3 Buffer overflow

```
int arr[5];  
arr[5] = 42;    // UB: one past the end
```

Detection: AddressSanitizer, `-D_FORTIFY_SOURCE=2`. Fix: Use `std::vector` or `std::array` with `.at()`.

D.4 Stack overflow

```
void f() { char buf[10'000'000]; }    // 10 MB on a ~8 MB stack
```

Detection: Crash with SIGSEGV. Fix: Move large data to heap (`std::vector`).

D.5 Dangling reference

```
std::string& get_name() {
    std::string s = "Alice";
    return s;    // returns reference to local -- destroyed on return
}
```

Detection: AddressSanitizer, compiler warning `-Wreturn-local-addr`. Fix: Return by value.

Uninitialized Variables

D.6 Uninitialized local variable

```
int x;
std::cout << x;    // UB: x has garbage value
```

Detection: `-Wuninitialized`, MemorySanitizer, UBSanitizer. Fix: `int x = 0;` or `int x{};`.

D.7 Uninitialized class member

```
struct Foo {
    int value;    // NOT zero-initialized by default
    Foo() {}    // no initialization of value
};
Foo f; std::cout << f.value;    // UB
```

Fix: `int value{0};` or initialize in the member initializer list.

Integer and Arithmetic Bugs

D.8 Signed integer overflow

```
int x = INT_MAX;
x++;    // UB: signed overflow
```

Detection: UBSanitizer (`-fsanitize=undefined`). Fix: Use `int64_t` or check before overflow.

D.9 Signed/unsigned comparison

```
int size = get_size();
for (size_t i = 0; i < size; ++i) {}    // warning: comparing signed and unsigned
// If size is negative, the comparison wraps and the loop runs ~4 billion times
```

Fix: Use consistent types. `for (int i = 0; i < size; ++i)` or `for (size_t i = 0; i < (size_t)size; ++i)`.

D.10 Integer truncation

```
long long big = 10'000'000'000LL;
int small = big;    // truncated -- wrong value
```

Detection: `-Wconversion`. Fix: Use appropriate type throughout.

Object Lifetime Bugs

D.11 Iterator invalidation

```
std::vector<int> v = {1, 2, 3};
auto it = v.begin();
v.push_back(4);    // may reallocate -- it is now dangling!
std::cout << *it; // UB
```

Fix: Do not hold iterators across mutations. Re-acquire the iterator after mutation.

D.12 Accessing destroyed `shared_ptr` object via `weak_ptr`

```
auto weak = std::make_shared<Foo>().weak(); // shared_ptr destroyed immediately!
auto locked = weak.lock();    // nullptr -- object is gone
locked->do_something();        // crash
```

Fix: Always check `if (auto p = weak.lock())` before using.

D.13 Moving from and then using an object

```
auto s = std::string{"hello"};
auto t = std::move(s);    // s is now in a valid but unspecified state
std::cout << s;           // valid but usually prints ""
s.push_back('!');         // valid (moved-from strings can be reused)
```

Rule: After `std::move(x)`, treat `x` as if it were default-constructed. Do not read until you reassign it.

Concurrency Bugs

D.14 Data race

```
int counter = 0;
std::jthread t1([&]{ for(int i=0;i<1000;++i) ++counter; });
std::jthread t2([&]{ for(int i=0;i<1000;++i) ++counter; });
```

Detection: `ThreadSanitizer (-fsanitize=thread)`. Fix: `std::atomic<int>` or mutex.

D.15 Deadlock

```
// Two threads locking mutexes in opposite order -- classic deadlock
```

Detection: Thread sanitizer, Helgrind (valgrind tool). Fix: Always lock in the same order; use `std::scoped_lock` for multiple mutexes.

D.16 Spurious wakeup without predicate

```
cv.wait(lock); // wakes spuriously, condition may not be true
```

Fix: `cv.wait(lock, []{ return condition; });`

Template and Type Bugs

D.17 Most vexing parse

```
Widget w(); // NOT constructing a Widget -- this declares a function!
```

Fix: `Widget w{};` (brace initialization never parses as a function declaration).

D.18 Narrowing conversion

```
int x = 3;
double d{x}; // OK
int y{3.14}; // ERROR: narrowing -- brace init catches this
int z = 3.14; // OK (silently truncates to 3)
```

Use brace initialization to catch narrowing at compile time.

D.19 Shadowed variable

```
int x = 10;
if (true) {
    int x = 20; // shadows outer x -- is this intentional?
    std::cout << x; // 20
}
std::cout << x; // 10
```

Detection: `-Wshadow`. Fix: Rename the inner variable.

Type System and Casting Bugs

D.20 Wrong `dynamic_cast`

```
Base* b = new Derived1{};
Derived2* d = dynamic_cast<Derived2*>(b); // returns nullptr
d->method(); // crash: null pointer dereference
```

Fix: Always check the result of `dynamic_cast`: `if (auto d = dynamic_cast<Derived2*>(b)) { ... }`.

D.21 C-style cast removes `const`

```
const int x = 42;
int* p = (int*)&x; // removes const -- UB to write through p
*p = 99;          // UB
```

Fix: Use `const_cast` only when you know the object is not actually `const` (e.g., a `const` reference to a non-`const` object). Better: redesign to not need `const` removal.

I/O and Format Bugs

D.22 `Printf` format mismatch

```
size_t n = 42;
printf("%d\n", n); // UB: %d is for int, size_t may be 64-bit
```

Fix: Use `%zu` for `size_t`, or prefer `std::cout` / `std::format`.

D.23 `std::cin` leaving newline in buffer

```
int n;
std::cin >> n;
std::string line;
std::getline(std::cin, line); // reads the '\n' left by >> -- line is ""
```

Fix: `std::cin >> std::ws;` before `getline`, or `std::cin.ignore(...)` to discard the new-line.

Build and Linking Bugs

D.24 Multiple definition

error: multiple definition of 'my_global'

Cause: Defining a variable in a header included by multiple translation units. Fix: Declare `extern int my_global;` in the header, define `int my_global = 0;` in exactly one `.cpp` file. Or use `inline int my_global = 0;` (C++17).

D.25 Circular include

```
// a.hpp includes b.hpp; b.hpp includes a.hpp
```

Fix: Use forward declarations where possible. Break the cycle by forward-declaring the class instead of including the header.

Debugging Workflow

Step 1: Read the Error Message

Compiler errors look intimidating but are precise. The key information is: - **File and line number:** `main.cpp:42:10:` - **Category:** `error:` (must fix) vs `warning:` (should fix) - **Message:** the actual problem

Template errors are long because they print the full instantiation chain. Read the **first** error and the **inner-most** line of the chain.

Step 2: Reproduce Minimally

```
# Isolate the bug: comment out sections until the crash goes away.
# The last thing you commented out is the culprit.

# Compile with debug + sanitizers:
g++ -g -O0 -fsanitize=address,undefined main.cpp && ./a.out

# ASAN output shows: type of error, the exact line, and a stack trace
# ==ERROR: AddressSanitizer: heap-use-after-free on address ...
# WRITE of size 4 at 0x... thread T0
#    #0 0x... in main main.cpp:42
```

Step 3: Use the Debugger

```

# Compile with debug info:
g++ -g -O0 main.cpp -o main

# Launch gdb:
gdb ./main

# Key gdb commands:
(gdb) run                # run the program
(gdb) break main.cpp:42  # set breakpoint at line 42
(gdb) break MyClass::method # break on function entry
(gdb) next (or n)        # step over (execute current line)
(gdb) step (or s)        # step into function
(gdb) continue (or c)    # continue to next breakpoint
(gdb) print variable     # print variable value
(gdb) print *ptr         # dereference and print
(gdb) backtrace (or bt)  # print call stack
(gdb) frame 3            # switch to stack frame 3
(gdb) info locals        # print all local variables
(gdb) watch variable     # break when variable changes (watchpoint)
(gdb) list               # show source around current line

```

Step 4: Add Logging

For bugs that disappear under a debugger (timing-sensitive, release-only):

```

// Use std::cerr (unbuffered -- always printed even on crash):
std::cerr << "[DEBUG] value=" << value << " at line " << __LINE__ << "\n";

// Conditional logging macro:
#define LOG(x) std::cerr << "[" << __FILE__ << ":" << __LINE__ << "]" << " " << x << "\n"
LOG("entering function, n=" << n);

```

Step 5: Static Analysis

```

# clang-tidy: automated code linter (catches many common bugs)
clang-tidy main.cpp -- -std=c++20

# cppcheck: another static analyzer
cppcheck --enable=all main.cpp

# Compiler's own analyzer:
g++ -fanalyzer main.cpp      # GCC static analysis
clang++ --analyze main.cpp   # Clang static analysis

```

Quick Diagnostic Decision Tree

Program crashes:

Segfault (SIGSEGV):

Run with AddressSanitizer -> tells you exactly which line and why

Common causes: null/dangling pointer, buffer overflow, stack overflow

Abort (SIGABRT):

Usually: assert failed, or bad_alloc, or terminate from uncaught exception

Run with -fsanitize=address to catch heap corruption before the abort

Wrong output (no crash):

Is it a numeric bug? Check for overflow (-fsanitize=undefined), truncation

Is it a logic bug? Use debugger, add logging, write a unit test

Is it a race condition? Run with ThreadSanitizer (-fsanitize=thread)

Program hangs:

Deadlock: Ctrl+C, then run under a deadlock detector

Linux: gdb, then 'info threads' and 'thread apply all bt'

Infinite loop: gdb, Ctrl+C to interrupt, 'backtrace'

Waiting on I/O: strace to see which syscall it is blocking on

Compilation error:

Read the FIRST error only -- later errors are often cascading effects

Template errors: find the innermost 'note: in instantiation of...' line

Linker errors: 'undefined reference' usually means:

- missing source file in build
- missing extern "C"
- missing library flag (-l...)

The book is complete. You have traveled from the very first C++ compilation to systems programming, graphics, and game development. The concepts build on each other: memory ownership enables performance, performance enables real-time graphics, and real-time graphics enables games. The tools in these appendices – compiler flags, sanitizers, and the debugger – are how you close the loop between writing code and understanding what it actually does.

The best next step is to build something. Pick a project that excites you – a game, a networking tool, a renderer, a data structure library – and start. Every concept in this book will become concrete the first time you debug it at 2am.